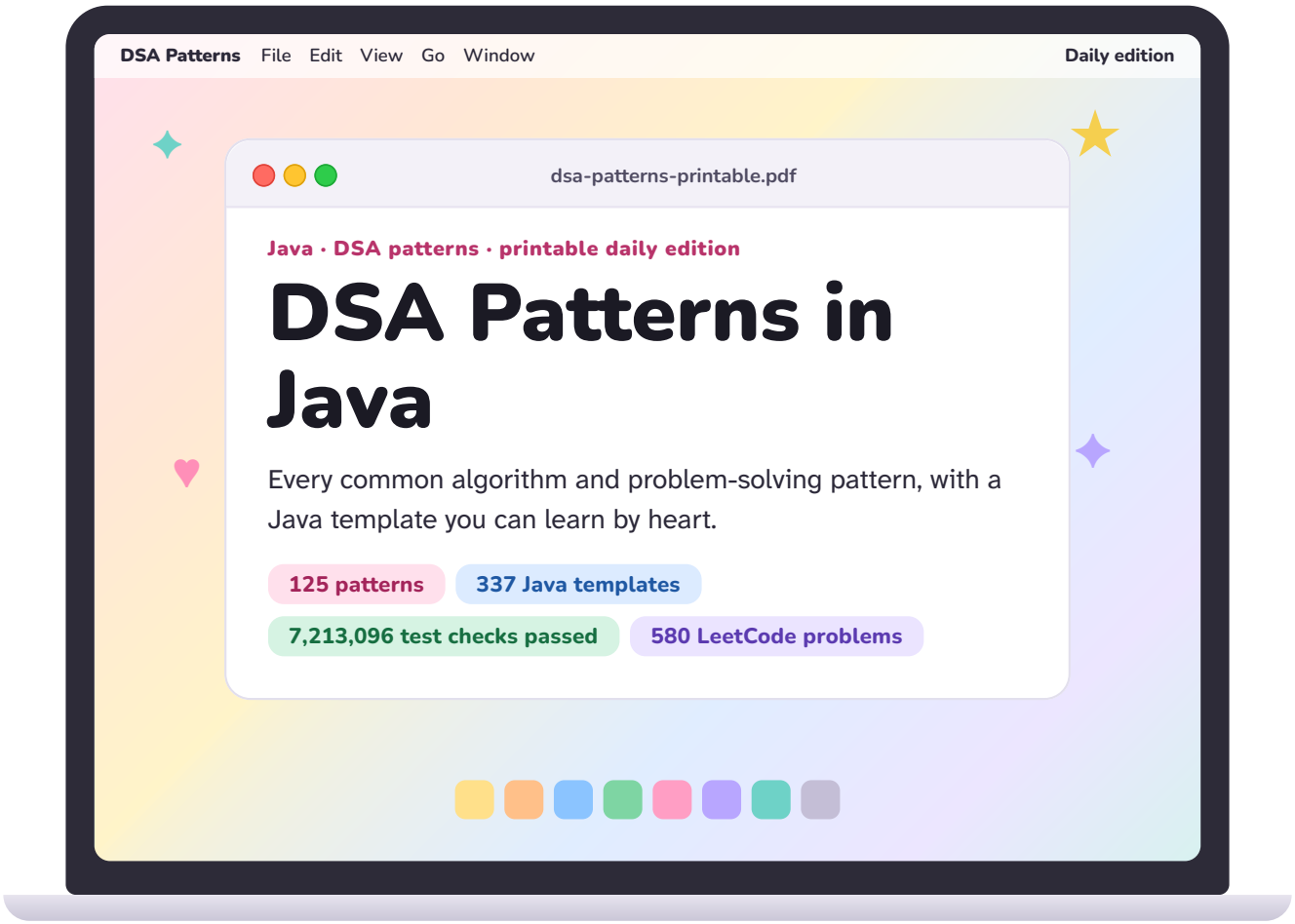


Daily plan page 5

Which pattern? page 8

1. Arrays & hashing page 18

2. Two pointers & windows page 44

3. Search & sort page 68

4. Stacks & heaps page 99

5. Linked lists page 128

6. Intervals & greedy page 143

7. Backtracking page 160

8. Trees page 185

9. Graphs page 222

10. Dynamic programming page 277

11. Strings page 320

12. Math & bits page 339

13. Range queries page 363

Started on _____ **Test or goal date** _____ **Windows done** _____

Prepared for **Purvam Prajapati** · one window a day, a template from memory, one problem solved

How to use this book

This book has one window per pattern: 125 of them, from counting with a hash map to lazy segment trees. Each window teaches you to recognise the pattern from a problem statement, explains why it works, and gives you a Java template to learn by heart. Read one or two windows a day, write the template from memory, then solve a problem with it.

What each window contains

1. **Spot it.** What the problem statement and its limits look like when this pattern is the answer.
2. **How it works.** The idea, the reason it is correct, and the reason for its cost.
3. **Worked example.** A small input traced step by step. Every number in it was printed by the code.
4. **Template.** The Java to learn. The one tagged **memorise** is the one to write from memory; **variant** and **example** blocks show the same idea on other problems.
5. **Cost, Variants, Traps.** Time and space, the small changes that solve related problems, and the bugs people really write.
6. **Practise.** Free LeetCode problems that use the pattern, easiest first. Tick the box when you solve one.

Daily routine about 60–90 minutes

1. **Read the window.** Read *Spot it* and *How it works*. Trace the worked example with a pencil before you read the code.
2. **Learn the template.** Read the **memorise** block line by line until you can say why each line is there.
3. **Write it from memory.** Close the book and write the template on paper or in a blank editor. Compare, fix, and tick **read it** in the box at the bottom of the window.
4. **Solve one problem.** Open the first unticked practice problem and solve it without looking at the book. Look only when you have been stuck for 20 minutes.
5. **Review old windows.** The plan's right-hand column lists windows from 1, 3, 7 and 14 days ago. Rewrite each template from memory and tick its next box (+1 day, +3 days, +7 days, +21 days).

Java

What the code assumes

Every template compiles with `javac --release 8`, so it runs on any judge. It assumes `import java.util.*;` and nothing else. Methods are `static` so you can paste them into LeetCode's `class Solution` as they are. `ListNode` and `TreeNode` are LeetCode's own classes (see the Java kit page).

Tested

Why you can trust the code

All code in the book is cut from Java files that were compiled and run. Each template was compared with a slow brute force on thousands of random inputs and checked on the LeetCode samples: 7,213,096 checks in total, all passing.

Printing

For the best result

Print at **100% / Actual size** (not "fit to page"). Double-sided works well: choose **flip on long edge**. Colour helps but greyscale is fine. The margins leave room for a hole punch or a binder clip.

Contents

How to use this book 2 | The 60-day plan 5 | Which pattern? 8 | Input size → complexity 12
Checklist 13 | Java traps 14 | Java kit 15 | Costs at a glance 379 | Problem index 383

Part 1 · Arrays & hashing 18

1.1 Frequency counting 18
1.2 Seen-before hash map 21
1.3 Prefix sums 23
1.4 Prefix sum + hash map 26
1.5 2D prefix sums 28
1.6 Difference array 31
1.7 Kadane's maximum subarray 34
1.8 Boyer–Moore majority vote 36
1.9 Cyclic sort (index as hash) 38
1.10 In-place matrix moves 40

Part 2 · Two pointers & windows 44

2.1 Opposite-end pointers 44
2.2 K-sum (sort + two pointers) 46
2.3 Read/write pointers (in place) 49
2.4 Three-way partition (Dutch flag) 51
2.5 Merge two sorted sequences 53
2.6 Fixed-size sliding window 55
2.7 Sliding window (variable size) 58
2.8 Exactly k (two at-most counts) 60
2.9 Minimum window substring 62
2.10 Sliding window maximum (deque) 64

Part 3 · Search & sort 68

3.1 Lower bound and upper bound 68
3.2 Search in a rotated sorted array 71
3.3 Binary search on the answer 74
3.4 Binary search on real numbers 76
3.5 Binary search on a slope 79
3.6 K-th smallest by counting 82
3.7 Custom sort and comparators 85
3.8 Merge sort with counting 87
3.9 Quickselect 90
3.10 Counting sort and buckets 93
3.11 Coordinate compression 96

Part 4 · Stacks & heaps 99

4.1 Stack for matching and nesting 99
4.2 Expression evaluation 102
4.3 Monotonic stack: next greater 105
4.4 Largest rectangle in histogram 108
4.5 Contribution counting 111
4.6 Designing with stacks 113
4.7 Top k with a size-k heap 116
4.8 Two heaps for a running median 119
4.9 K-way merge with a heap 121
4.10 Greedy with a heap 124

Part 5 · Linked lists 128

5.1 Dummy head node 128
5.2 In-place reversal (prev, cur, next) 131
5.3 Fast and slow pointers 134
5.4 Gap pointers and head switching 137
5.5 LRU cache (hash map + linked list) 139

Part 6 · Intervals & greedy 143

6.1 Merge overlapping intervals 143
6.2 Insert and intersect intervals 145
6.3 Interval scheduling (sort by end) 147
6.4 Sweep line (start and end events) 149
6.5 Jump game (farthest reach) 152
6.6 Gas station (running balance) 155
6.7 Two passes: left, then right 157

Part 7 · Backtracking 160

7.1 Subsets (include or exclude) 160
7.2 Skip duplicates at the same depth 163
7.3 Permutations (used[] array) 166
7.4 Combinations from a start index 169
7.5 Constraint backtracking (N-Queens) 172
7.6 Grid DFS with mark and restore 175
7.7 Cut a prefix, recurse on the rest 178
7.8 Meet in the middle 181

Part 8 · Trees 185

8.1 Preorder, inorder, postorder 186
8.2 Level order (BFS by levels) 189
8.3 Return a value up the tree 191
8.4 Maximum path sum (one arm up) 194
8.5 Root-to-leaf paths and path sums 196
8.6 Binary search tree operations 199
8.7 Lowest common ancestor 202
8.8 Build and serialize a tree 205
8.9 General tree from an edge list 207
8.10 Binary lifting (k-th ancestor, LCA) 210
8.11 Tree DP (take or skip a node) 213
8.12 Rerooting (every node as root) 216
8.13 Euler tour (subtree = range) 218

Part 9 · Graphs 222

9.1 Graph DFS and connected components 223
9.2 BFS shortest path (unweighted) 226
9.3 Multi-source BFS 229
9.4 BFS over states 232
9.5 Bipartite check (two colours) 235
9.6 Directed cycles with three colours 238
9.7 Topological sort (Kahn's algorithm) 241
9.8 DP on a DAG in topological order 244
9.9 Union-Find (disjoint set union) 247
9.10 Dijkstra's shortest paths 250
9.11 0-1 BFS (deque) 254
9.12 Bellman–Ford and at most k edges 257
9.13 Floyd–Warshall (all pairs) 260
9.14 Kruskal's MST (sort + DSU) 263
9.15 Prim's MST (dense or heap) 265
9.16 Bridges and articulation points 268
9.17 Strongly connected components 271
9.18 Euler path (Hierholzer) 274

Part 10 · Dynamic programming 277

10.1 The DP recipe (memo, table, rolling) 278
10.2 Linear DP over a sequence 280
10.3 Grid DP (one row or in place) 283
10.4 0/1 knapsack (reverse loop) 286
10.5 Unbounded knapsack (forward loop) 289
10.6 Longest increasing subsequence 292
10.7 Longest common subsequence 294
10.8 Edit distance and alignment 297
10.9 Palindrome DP on s[i..j] 300
10.10 Interval DP (last split point) 303
10.11 Stock DP (hold and free states) 306
10.12 Bitmask DP over subsets 309
10.13 Digit DP (count numbers ≤ N) 313
10.14 Partition into k contiguous groups 316

Part 11 · Strings 320

11.1 Expand around the centre 320
11.2 KMP prefix function 323
11.3 Z-function 326
11.4 Rolling hash (Rabin–Karp) 329
11.5 Trie (prefix tree) 332
11.6 Manacher's algorithm 335

Part 12 · Math & bits 339

12.1 Bit tricks: lowest bit, submasks 339
12.2 XOR tricks (pairs cancel) 342
12.3 Binary trie for maximum XOR 345
12.4 GCD, LCM and extended Euclid 348
12.5 Fast power and modular inverse 350
12.6 Prime sieve and factorisation 353
12.7 nCr mod p (factorial tables) 356
12.8 Matrix power for recurrences 359

Part 13 · Range queries 363

13.1 Sparse table (static range min) 363
13.2 Fenwick tree (binary indexed) 366
13.3 Segment tree (point update) 369
13.4 Lazy propagation (range updates) 372
13.5 Ordered maps (TreeMap, TreeSet) 375

The 60-day plan

About two windows a day, 60 to 90 minutes. Write the date in each row and tick **Done** when you finish the day. *Learn* lists the new windows and the page where the first one starts. *Rewrite from memory* lists the windows learned 1, 3, 7 and 14 days earlier: write those templates again without looking. Short of time before a test? Do two rows a day.

Day	Date	Learn (new windows)	Page	Rewrite from memory	Done
1		1.1 Frequency counting 1.2 Seen-before hash map 1.3 Prefix sums	18	—	☐
2		1.4 Prefix sum + hash map 1.5 2D prefix sums	26	d1 1.1–1.3	☐
3		1.6 Difference array 1.7 Kadane's maximum subarray 1.8 Boyer–Moore majority vote	31	d2 1.4–1.5	☐
4		1.9 Cyclic sort (index as hash) 1.10 In-place matrix moves	38	d3 1.6–1.8 · d1 1.1–1.3	☐
5		2.1 Opposite-end pointers 2.2 K-sum (sort + two pointers) 2.3 Read/write pointers (in place)	44	d4 1.9–1.10 · d2 1.4–1.5	☐
6		2.4 Three-way partition (Dutch flag) 2.5 Merge two sorted sequences 2.6 Fixed-size sliding window	51	d5 2.1–2.3 · d3 1.6–1.8	☐
7		2.7 Sliding window (variable size) 2.8 Exactly k (two at-most counts)	58	d6 2.4–2.6 · d4 1.9–1.10	☐
8		2.9 Minimum window substring 2.10 Sliding window maximum (deque) 3.1 Lower bound and upper bound	62	d7 2.7–2.8 · d5 2.1–2.3 · d1 1.1–1.3	☐
9		3.2 Search in a rotated sorted array 3.3 Binary search on the answer	71	d8 2.9–2.10, 3.1 · d6 2.4–2.6 · d2 1.4–1.5	☐
10		3.4 Binary search on real numbers 3.5 Binary search on a slope	76	d9 3.2–3.3 · d7 2.7–2.8 · d3 1.6–1.8	☐
11		3.6 K-th smallest by counting 3.7 Custom sort and comparators	82	d10 3.4–3.5 · d8 2.9–2.10, 3.1 · d4 1.9–1.10	☐
12		3.8 Merge sort with counting 3.9 Quickselect 3.10 Counting sort and buckets	87	d11 3.6–3.7 · d9 3.2–3.3 · d5 2.1–2.3	☐
13		3.11 Coordinate compression 4.1 Stack for matching and nesting	96	d12 3.8–3.10 · d10 3.4–3.5 · d6 2.4–2.6	☐
14		4.2 Expression evaluation 4.3 Monotonic stack: next greater	102	d13 3.11, 4.1 · d11 3.6–3.7 · d7 2.7–2.8	☐
15		4.4 Largest rectangle in histogram 4.5 Contribution counting	108	d14 4.2–4.3 · d12 3.8–3.10 · d8 2.9–2.10, 3.1 · d1 1.1–1.3	☐
16		4.6 Designing with stacks 4.7 Top k with a size-k heap	113	d15 4.4–4.5 · d13 3.11, 4.1 · d9 3.2–3.3 · d2 1.4–1.5	☐
17		4.8 Two heaps for a running median 4.9 K-way merge with a heap 4.10 Greedy with a heap	119	d16 4.6–4.7 · d14 4.2–4.3 · d10 3.4–3.5 · d3 1.6–1.8	☐

Day	Date	Learn (new windows)	Page	Rewrite from memory	Done
18		5.1 Dummy head node 5.2 In-place reversal (prev, cur, next)	128	d17 4.8–4.10 · d15 4.4–4.5 · d11 3.6–3.7 · d4 1.9–1.10	☐
19		5.3 Fast and slow pointers 5.4 Gap pointers and head switching	134	d18 5.1–5.2 · d16 4.6–4.7 · d12 3.8–3.10 · d5 2.1–2.3	☐
20		5.5 LRU cache (hash map + linked list) 6.1 Merge overlapping intervals	139	d19 5.3–5.4 · d17 4.8–4.10 · d13 3.11, 4.1 · d6 2.4–2.6	☐
21		6.2 Insert and intersect intervals 6.3 Interval scheduling (sort by end) 6.4 Sweep line (start and end events)	145	d20 5.5, 6.1 · d18 5.1–5.2 · d14 4.2–4.3 · d7 2.7–2.8	☐
22		6.5 Jump game (farthest reach) 6.6 Gas station (running balance)	152	d21 6.2–6.4 · d19 5.3–5.4 · d15 4.4–4.5 · d8 2.9–2.10, 3.1	☐
23		6.7 Two passes: left, then right 7.1 Subsets (include or exclude)	157	d22 6.5–6.6 · d20 5.5, 6.1 · d16 4.6–4.7 · d9 3.2–3.3	☐
24		7.2 Skip duplicates at the same depth 7.3 Permutations (used[] array) 7.4 Combinations from a start index	163	d23 6.7, 7.1 · d21 6.2–6.4 · d17 4.8–4.10 · d10 3.4–3.5	☐
25		7.5 Constraint backtracking (N-Queens)	172	d24 7.2–7.4 · d22 6.5–6.6 · d18 5.1–5.2 · d11 3.6–3.7	☐
26		7.6 Grid DFS with mark and restore 7.7 Cut a prefix, recurse on the rest	175	d25 7.5 · d23 6.7, 7.1 · d19 5.3–5.4 · d12 3.8–3.10	☐
27		7.8 Meet in the middle 8.1 Preorder, inorder, postorder	181	d26 7.6–7.7 · d24 7.2–7.4 · d20 5.5, 6.1 · d13 3.11, 4.1	☐
28		8.2 Level order (BFS by levels) 8.3 Return a value up the tree	189	d27 7.8, 8.1 · d25 7.5 · d21 6.2–6.4 · d14 4.2–4.3	☐
29		8.4 Maximum path sum (one arm up) 8.5 Root-to-leaf paths and path sums 8.6 Binary search tree operations	194	d28 8.2–8.3 · d26 7.6–7.7 · d22 6.5–6.6 · d15 4.4–4.5	☐
30		8.7 Lowest common ancestor 8.8 Build and serialize a tree	202	d29 8.4–8.6 · d27 7.8, 8.1 · d23 6.7, 7.1 · d16 4.6–4.7	☐
31		8.9 General tree from an edge list 8.10 Binary lifting (k-th ancestor, LCA)	207	d30 8.7–8.8 · d28 8.2–8.3 · d24 7.2–7.4 · d17 4.8–4.10	☐
32		8.11 Tree DP (take or skip a node) 8.12 Rerooting (every node as root) 8.13 Euler tour (subtree = range)	213	d31 8.9–8.10 · d29 8.4–8.6 · d25 7.5 · d18 5.1–5.2	☐
33		9.1 Graph DFS and connected components	223	d32 8.11–8.13 · d30 8.7–8.8 · d26 7.6–7.7 · d19 5.3–5.4	☐
34		9.2 BFS shortest path (unweighted) 9.3 Multi-source BFS	226	d33 9.1 · d31 8.9–8.10 · d27 7.8, 8.1 · d20 5.5, 6.1	☐
35		9.4 BFS over states 9.5 Bipartite check (two colours)	232	d34 9.2–9.3 · d32 8.11–8.13 · d28 8.2–8.3 · d21 6.2–6.4	☐
36		9.6 Directed cycles with three colours 9.7 Topological sort (Kahn's algorithm)	238	d35 9.4–9.5 · d33 9.1 · d29 8.4–8.6 · d22 6.5–6.6	☐
37		9.8 DP on a DAG in topological order 9.9 Union-Find (disjoint set union)	244	d36 9.6–9.7 · d34 9.2–9.3 · d30 8.7–8.8 · d23 6.7, 7.1	☐
38		9.10 Dijkstra's shortest paths 9.11 0-1 BFS (deque)	250	d37 9.8–9.9 · d35 9.4–9.5 · d31 8.9–8.10 · d24 7.2–7.4	☐
39		9.12 Bellman–Ford and at most k edges 9.13 Floyd–Warshall (all pairs)	257	d38 9.10–9.11 · d36 9.6–9.7 · d32 8.11–8.13 · d25 7.5	☐

Day	Date	Learn (new windows)	Page	Rewrite from memory	Done
40		9.14 Kruskal's MST (sort + DSU) 9.15 Prim's MST (dense or heap)	263	d39 9.12–9.13 · d37 9.8–9.9 · d33 9.1 · d26 7.6–7.7	☐
41		9.16 Bridges and articulation points 9.17 Strongly connected components	268	d40 9.14–9.15 · d38 9.10–9.11 · d34 9.2–9.3 · d27 7.8, 8.1	☐
42		9.18 Euler path (Hierholzer) 10.1 The DP recipe (memo, table, rolling)	274	d41 9.16–9.17 · d39 9.12–9.13 · d35 9.4–9.5 · d28 8.2–8.3	☐
43		10.2 Linear DP over a sequence 10.3 Grid DP (one row or in place)	280	d42 9.18, 10.1 · d40 9.14–9.15 · d36 9.6–9.7 · d29 8.4–8.6	☐
44		10.4 0/1 knapsack (reverse loop) 10.5 Unbounded knapsack (forward loop)	286	d43 10.2–10.3 · d41 9.16–9.17 · d37 9.8–9.9 · d30 8.7–8.8	☐
45		10.6 Longest increasing subsequence 10.7 Longest common subsequence	292	d44 10.4–10.5 · d42 9.18, 10.1 · d38 9.10–9.11 · d31 8.9–8.10	☐
46		10.8 Edit distance and alignment 10.9 Palindrome DP on s[i..j]	297	d45 10.6–10.7 · d43 10.2–10.3 · d39 9.12–9.13 · d32 8.11–8.13	☐
47		10.10 Interval DP (last split point) 10.11 Stock DP (hold and free states)	303	d46 10.8–10.9 · d44 10.4–10.5 · d40 9.14–9.15 · d33 9.1	☐
48		10.12 Bitmask DP over subsets 10.13 Digit DP (count numbers ≤ N)	309	d47 10.10–10.11 · d45 10.6–10.7 · d41 9.16–9.17 · d34 9.2–9.3	☐
49		10.14 Partition into k contiguous groups 11.1 Expand around the centre	316	d48 10.12–10.13 · d46 10.8–10.9 · d42 9.18, 10.1 · d35 9.4–9.5	☐
50		11.2 KMP prefix function 11.3 Z-function 11.4 Rolling hash (Rabin–Karp)	323	d49 10.14, 11.1 · d47 10.10–10.11 · d43 10.2–10.3 · d36 9.6–9.7	☐
51		11.5 Trie (prefix tree)	332	d50 11.2–11.4 · d48 10.12–10.13 · d44 10.4–10.5 · d37 9.8–9.9	☐
52		11.6 Manacher's algorithm 12.1 Bit tricks: lowest bit, submasks	335	d51 11.5 · d49 10.14, 11.1 · d45 10.6–10.7 · d38 9.10–9.11	☐
53		12.2 XOR tricks (pairs cancel) 12.3 Binary trie for maximum XOR 12.4 GCD, LCM and extended Euclid	342	d52 11.6, 12.1 · d50 11.2–11.4 · d46 10.8–10.9 · d39 9.12–9.13	☐
54		12.5 Fast power and modular inverse 12.6 Prime sieve and factorisation	350	d53 12.2–12.4 · d51 11.5 · d47 10.10–10.11 · d40 9.14–9.15	☐
55		12.7 nCr mod p (factorial tables) 12.8 Matrix power for recurrences	356	d54 12.5–12.6 · d52 11.6, 12.1 · d48 10.12–10.13 · d41 9.16–9.17	☐
56		13.1 Sparse table (static range min) 13.2 Fenwick tree (binary indexed)	363	d55 12.7–12.8 · d53 12.2–12.4 · d49 10.14, 11.1 · d42 9.18, 10.1	☐
57		13.3 Segment tree (point update) 13.4 Lazy propagation (range updates)	369	d56 13.1–13.2 · d54 12.5–12.6 · d50 11.2–11.4 · d43 10.2–10.3	☐
58		13.5 Ordered maps (TreeMap, TreeSet)	375	d57 13.3–13.4 · d55 12.7–12.8 · d51 11.5 · d44 10.4–10.5	☐
59		**Review day. Cover the right-hand column of the *Which pattern?* table and name the pattern for each clue. Reread every window you could not name.**			☐
60		**Review day. Rewrite the ten templates you ticked least often. Then solve one unticked practice problem from each part.**			☐

Which pattern? Read the problem, then this table

Read the problem and its limits, find the clue that fits best, and start with the first pattern listed. When two clues fit, the limits (next page) usually decide.

Arrays and strings	
Have I seen this value, or its complement, before?	1.2 **Seen-before hash map** (p. 21) 1.1 **Frequency counting** (p. 18)
Anagrams, character counts, "appears more than n/2 times"	1.1 **Frequency counting** (p. 18) 1.8 **Boyer–Moore majority vote** (p. 36)
Many range-sum queries on an array that does not change	1.3 **Prefix sums** (p. 23) 1.5 **2D prefix sums** (p. 28)
Count subarrays with sum k (negative numbers allowed)	1.4 **Prefix sum + hash map** (p. 26)
Many "add v to every element in [l, r]" updates, then read once	1.6 **Difference array** (p. 31)
Maximum-sum subarray	1.7 **Kadane's maximum subarray** (p. 34)
Values are 1..n and one is missing or repeated, O(1) extra space	1.9 **Cyclic sort (index as hash)** (p. 38) 12.2 **XOR tricks (pairs cancel)** (p. 342)
Rotate, spiral or zero a matrix in place	1.10 **In-place matrix moves** (p. 40)
Pairs, windows and in-place moves	
Sorted array, find a pair or triple with a target sum	2.1 **Opposite-end pointers** (p. 44) 2.2 **K-sum (sort + two pointers)** (p. 46)
Longest or shortest substring or subarray with a rule	2.7 **Sliding window (variable size)** (p. 58) 2.9 **Minimum window substring** (p. 62)
Count subarrays with exactly k of something	2.8 **Exactly k (two at-most counts)** (p. 60)
A window of fixed size k slides over the array	2.6 **Fixed-size sliding window** (p. 55) 2.10 **Sliding window maximum (deque)** (p. 64)
Remove, compact or partition in place with O(1) extra space	2.3 **Read/write pointers (in place)** (p. 49) 2.4 **Three-way partition (Dutch flag)** (p. 51)
Two sorted arrays or lists, merge or intersect them	2.5 **Merge two sorted sequences** (p. 53) 5.1 **Dummy head node** (p. 128)
Sorted data and searching	
Sorted array, find a position or the first/last occurrence	3.1 **Lower bound and upper bound** (p. 68) 3.2 **Search in a rotated sorted array** (p. 71)
Minimise the maximum or maximise the minimum, and a bigger answer is always easier	3.3 **Binary search on the answer** (p. 74) 3.4 **Binary search on real numbers** (p. 76)
k-th smallest in a sorted matrix, a table, or among pair distances	3.6 **K-th smallest by counting** (p. 82) 4.9 **K-way merge with a heap** (p. 121)
Top k, k-th largest, k most frequent	4.7 **Top k with a size-k heap** (p. 116) 3.9 **Quickselect** (p. 90) 3.10 **Counting sort and buckets** (p. 93)
Values up to 10^9, but you need them as array indices	3.11 **Coordinate compression** (p. 96)
Count inversions or smaller elements after each position	3.8 **Merge sort with counting** (p. 87) 13.2 **Fenwick tree (binary indexed)** (p. 366)
Order things by a rule of your own (largest number, by several keys)	3.7 **Custom sort and comparators** (p. 85)

Stacks, heaps and ordered sets

Brackets, nesting, decoding, undo the most recent thing	4.1 **Stack for matching and nesting** (p. 99) 4.2 **Expression evaluation** (p. 102)
Next greater or smaller element, spans, temperatures	4.3 **Monotonic stack: next greater** (p. 105)
Largest rectangle, or a sum of min/max over all subarrays	4.4 **Largest rectangle in histogram** (p. 108) 4.5 **Contribution counting** (p. 111)
Running median, or keep two halves balanced	4.8 **Two heaps for a running median** (p. 119)
Repeatedly take the best option still available	4.10 **Greedy with a heap** (p. 124) 4.7 **Top k with a size-k heap** (p. 116)
Nearest value above or below, bookings, a sorted set that changes	13.5 **Ordered maps (TreeMap, TreeSet)** (p. 375)
Design a stack or queue with an extra O(1) operation	4.6 **Designing with stacks** (p. 113) 5.5 **LRU cache (hash map + linked list)** (p. 139)

Intervals and greedy

Overlapping intervals to merge or insert into	6.1 **Merge overlapping intervals** (p. 143) 6.2 **Insert and intersect intervals** (p. 145)
Most non-overlapping intervals, fewest removals or arrows	6.3 **Interval scheduling (sort by end)** (p. 147)
Most things happening at the same time (rooms, capacity, overlap)	6.4 **Sweep line (start and end events)** (p. 149) 1.6 **Difference array** (p. 31)
Can you reach the end, fewest jumps	6.5 **Jump game (farthest reach)** (p. 152)
Circular route or a running balance that must not go negative	6.6 **Gas station (running balance)** (p. 155)
Each answer depends on everything to its left and to its right	6.7 **Two passes: left, then right** (p. 157)

Linked lists

Reverse a list, a part of it, or groups of k	5.2 **In-place reversal (prev, cur, next)** (p. 131)
Cycle, middle, palindrome list	5.3 **Fast and slow pointers** (p. 134)
k-th node from the end, rotate, where two lists meet	5.4 **Gap pointers and head switching** (p. 137)

All combinations (small n)

List all subsets, combinations or permutations	7.1 **Subsets (include or exclude)** (p. 160) 7.4 **Combinations from a start index** (p. 169) 7.3 **Permutations (used[] array)** (p. 166)
The same, but the input has duplicates	7.2 **Skip duplicates at the same depth** (p. 163)
Place items under rules (queens, sudoku)	7.5 **Constraint backtracking (N-Queens)** (p. 172)
Find a word or path in a grid, each cell used once	7.6 **Grid DFS with mark and restore** (p. 175)
Split a string into valid pieces, generate valid strings	7.7 **Cut a prefix, recurse on the rest** (p. 178)
n is about 40 and you need subset sums	7.8 **Meet in the middle** (p. 181)

Trees

Visit every node in some order, or level by level	8.1 **Preorder, inorder, postorder** (p. 186) 8.2 **Level order (BFS by levels)** (p. 189)
The answer comes from the children (height, diameter, balance)	8.3 **Return a value up the tree** (p. 191) 8.4 **Maximum path sum (one arm up)** (p. 194)
Paths from the root, or any downward path, with a sum	8.5 **Root-to-leaf paths and path sums** (p. 196)

Binary search tree to search, validate, change or rank	8.6 **Binary search tree operations** (p. 199)
Lowest common ancestor, k-th ancestor, distance between two nodes	8.7 **Lowest common ancestor** (p. 202) 8.10 **Binary lifting (k-th ancestor, LCA)** (p. 210)
Rebuild a tree from traversals, or serialise it	8.8 **Build and serialize a tree** (p. 205)
Tree given as n nodes and n – 1 edges	8.9 **General tree from an edge list** (p. 207)
Choose nodes with a rule about neighbours	8.11 **Tree DP (take or skip a node)** (p. 213)
The answer for every node taken as the root	8.12 **Rerooting (every node as root)** (p. 216)
Many queries about whole subtrees	8.13 **Euler tour (subtree = range)** (p. 218)

Graphs

Count islands, components or groups; flood fill	9.1 **Graph DFS and connected components** (p. 223) 9.9 **Union-Find (disjoint set union)** (p. 247)
Fewest steps when every move costs 1	9.2 **BFS shortest path (unweighted)** (p. 226) 9.3 **Multi-source BFS** (p. 229)
Fewest moves in a puzzle (locks, words, boards, keys)	9.4 **BFS over states** (p. 232)
Split into two groups so no edge stays inside a group	9.5 **Bipartite check (two colours)** (p. 235)
Prerequisites, an order of tasks, or "is there a cycle?"	9.7 **Topological sort (Kahn's algorithm)** (p. 241) 9.6 **Directed cycles with three colours** (p. 238)
Longest path or number of paths in a DAG	9.8 **DP on a DAG in topological order** (p. 244)
Cheapest path, weights ≥ 0	9.10 **Dijkstra's shortest paths** (p. 250)
Every edge costs 0 or 1	9.11 **0-1 BFS (deque)** (p. 254)
At most k edges, or negative weights	9.12 **Bellman–Ford and at most k edges** (p. 257)
Distances between all pairs, n ≤ 400	9.13 **Floyd–Warshall (all pairs)** (p. 260)
Connect everything at the lowest total cost	9.14 **Kruskal's MST (sort + DSU)** (p. 263) 9.15 **Prim's MST (dense or heap)** (p. 265)
An edge or node whose removal disconnects the graph	9.16 **Bridges and articulation points** (p. 268)
Groups where everyone reaches everyone (directed)	9.17 **Strongly connected components** (p. 271)
Use every edge exactly once	9.18 **Euler path (Hierholzer)** (p. 274)

Dynamic programming

Count the ways or find the best cost, and choices overlap	10.1 **The DP recipe (memo, table, rolling)** (p. 278) 10.2 **Linear DP over a sequence** (p. 280)
Move right or down on a grid	10.3 **Grid DP (one row or in place)** (p. 283)
Pick items to reach a capacity or target, each at most once	10.4 **0/1 knapsack (reverse loop)** (p. 286)
The same with unlimited copies (coins)	10.5 **Unbounded knapsack (forward loop)** (p. 289)
Longest increasing subsequence, chains, envelopes	10.6 **Longest increasing subsequence** (p. 292)
Two strings, common subsequence, or edit operations	10.7 **Longest common subsequence** (p. 294) 10.8 **Edit distance and alignment** (p. 297)
Palindromic subsequence or fewest palindrome cuts	10.9 **Palindrome DP on s[i..j]** (p. 300)
Merge, burst or cut a range; choose the best split point	10.10 **Interval DP (last split point)** (p. 303)
Buy and sell with cooldowns, fees or k transactions	10.11 **Stock DP (hold and free states)** (p. 306)
n ≤ 20 and you must assign or visit every item	10.12 **Bitmask DP over subsets** (p. 309)
Count numbers up to N with a digit rule	10.13 **Digit DP (count numbers ≤ N)** (p. 313)
Split an array into k contiguous parts	10.14 **Partition into k contiguous groups** (p. 316) 3.3 **Binary search on the answer** (p. 74)

Strings

Find a pattern in a text in O(n + m)	11.2 **KMP prefix function** (p. 323) 11.3 **Z-function** (p. 326) 11.4 **Rolling hash (Rabin–Karp)** (p. 329)
Prefix lookups, a dictionary of words	11.5 **Trie (prefix tree)** (p. 332)
Longest palindromic substring, count palindromes	11.1 **Expand around the centre** (p. 320) 11.6 **Manacher's algorithm** (p. 335)

Math, bits and range queries

Bits, subsets of a mask, powers of two	12.1 **Bit tricks: lowest bit, submasks** (p. 339)
Everything appears twice except one; pairs cancel	12.2 **XOR tricks (pairs cancel)** (p. 342)
Maximum XOR of two values	12.3 **Binary trie for maximum XOR** (p. 345)
Answer modulo $10^9 + 7$, huge powers, nCr	12.5 **Fast power and modular inverse** (p. 350) 12.7 **nCr mod p (factorial tables)** (p. 356)
Primes, factors, divisibility	12.6 **Prime sieve and factorisation** (p. 353) 12.4 **GCD, LCM and extended Euclid** (p. 348)
A linear recurrence with n up to 10^{18}	12.8 **Matrix power for recurrences** (p. 359)
Range min or max on an array that does not change	13.1 **Sparse table (static range min)** (p. 363)
Range sums or minimums while values change	13.2 **Fenwick tree (binary indexed)** (p. 366) 13.3 **Segment tree (point update)** (p. 369) 13.4 **Lazy propagation (range updates)** (p. 372)

Read the limits: input size → target complexity

Java does roughly 10^8 simple steps per second, and judges allow 1–2 seconds. Find the largest n in the constraints, then aim for the complexity in its row.

n up to	Aim for	Typical patterns
n ≤ 10	`O(n!)`	All permutations, brute force over orders: 7.3 p. 166
n ≤ 20	`O(2^n · n)`	All subsets 7.1 p. 160, bitmask DP 10.12 p. 309
n ≤ 40	`O(2^(n/2) · n)`	Meet in the middle 7.8 p. 181
n ≤ 500	`O(n^3)`	Floyd–Warshall 9.13 p. 260, interval DP 10.10 p. 303
n ≤ 5 000	`O(n^2)`	Two-string DP 10.7 p. 294 10.8 p. 297, all pairs, the $O(n^2)$ LIS 10.6 p. 292
n ≤ 10^6	`O(n log n)`	Sorting, heaps 4.7 p. 116, binary search 3.1 p. 68, Fenwick and segment trees 13.2 p. 366, Dijkstra 9.10 p. 250
n ≤ 10^8	`O(n)`	Two pointers 2.1 p. 44, sliding windows 2.7 p. 58, prefix sums 1.3 p. 23, BFS/DFS 9.2 p. 226, a sieve 12.6 p. 353
n ≤ 10^{18}	`O(log n) or O(1)`	Binary search on the answer 3.3 p. 74, fast power 12.5 p. 350, matrix power 12.8 p. 359, a formula

With q queries the total is the setup cost plus q times the cost per query, so n, q ≤ 10^5 rules out O(n) per query and asks for O(log n): prefix sums, a Fenwick tree or a segment tree.

Memory. An `int` takes 4 bytes and a `long` 8, so `new int[10_000_000]` is 40 MB and fits the usual 256 MB. A `HashMap<Integer, Integer>` costs about 50–80 bytes per entry, so 10^6 entries are already 50–80 MB; use arrays when the keys are small integers.

Overflow. `int` holds up to about $2.1 \cdot 10^9$. Sums of 10^5 values up to 10^9 need `long`. When the answer is "modulo $10^9 + 7$", take the remainder after every addition and multiplication, and multiply as `long`.

Before you code: a checklist for any problem

Use the same steps on every problem. Most wrong answers come from skipping step 1 or 6.

Every problem in this order

1. **Pin down the output.** What exactly is returned: a count, an index, the list itself? 0- or 1-indexed? Modulo something? What if there is no answer?
2. **Read the limits.** Take the largest n and pick the target complexity from the table on the previous page. Note value ranges that need `long`.
3. **Work two examples by hand.** Use the sample and one edge case of your own: empty, one element, all equal, negatives, the largest values.
4. **Say the brute force and its cost.** Then find the repeated work it does. That repeated work is what the pattern removes. Look up the clue in *Which pattern?*.
5. **Write the template, then adapt it.** Start from the template you know by heart and change only what the problem needs. Keep its names so you can check it against the book.
6. **Test before you submit.** Run the samples and your edge cases. Check overflow, off-by-one at both ends, an empty input, and that every path returns a value.

Stuck? questions that unlock most problems

- Would sorting the input help, and does the question still make sense after sorting?
- Is a bigger answer always easier (or always harder) to achieve? Then binary search the answer (3.3 **Binary search on the answer** (p. 74)).
- Can you fix one element (the right end, the middle, the last step) and solve the rest quickly?
- Can you go backwards, from the target to the start, or think of the last operation first?
- Is there a graph hiding in it? States as nodes, moves as edges (9.4 **BFS over states** (p. 232)).
- Is it counting or optimising over choices that repeat? Write the recursion first, then memoise it (10.1 **The DP recipe (memo, table, rolling)** (p. 278)).

Java traps and collection costs

Java traps each of these has cost people an accepted solution

- `Integer` objects compared with `==` are equal by value only from −128 to 127. Use `.equals`, or unbox to `int` first.
- `a * b` with two `int`s overflows before it is stored in a `long`. Write `(long) a * b`.
- The comparator `(p, q) -> p - q` overflows for large values and then sorts wrongly. Use `Integer.compare(p, q)`.
- `list.remove(i)` with an `int` removes the element at **index** i. To remove the value, write `list.remove(Integer.valueOf(x))`.
- `s += c` in a loop copies the whole string every time, $O(n^2)$. Use a `StringBuilder`.
- `Arrays.fill(grid, new int[m])` puts the *same* row in every slot. Create each row in a loop.
- Recursion deeper than about 10^4 calls can throw `StackOverflowError`. Use an explicit stack or a loop (see **8.9 General tree from an edge list** (p. 207)).
- `static` fields keep their values between LeetCode test cases. Reset them at the start of the method.
- `Math.abs(Integer.MIN_VALUE)` is still negative. Work in `long` when a value can be -2^{31}.

What Java's collections cost

`ArrayList`	
`get(i)`, `set(i, x)`, `size()`	`O(1)`
`add(x)` at the end	`O(1)` amortised
`add(i, x)`, `remove(i)`	`O(n - i)`
`contains(x)`, `indexOf(x)`, `remove(Object)`	`O(n)`

`ArrayDeque` (stack and queue)	
`push`, `pop`, `peek`, `offer`, `poll` at either end	`O(1)` amortised
`contains(x)`, `remove(Object)`	`O(n)`

`HashMap`, `HashSet`	
`get`, `put`, `remove`, `containsKey`, `merge`	`O(1)` average
iterate over all entries (in no fixed order)	`O(n + capacity)`

`TreeMap`, `TreeSet`	
`get`, `put`, `remove`, `containsKey`	`O(log n)`
`floorKey`, `ceilingKey`, `firstKey`, `pollFirst`	`O(log n)`
iterate in sorted order	`O(n)`

`PriorityQueue`	
`offer`, `poll`	`O(log n)`
`peek`, `size()`	`O(1)`
`remove(Object)`, `contains(x)`	`O(n)`
`new PriorityQueue<>(collection)`	`O(n)`

`String`, `StringBuilder`	
`charAt`, `length`; `sb.append(c)`	`O(1)`
`substring(i, j)`, `s + t`, `equals`, `toCharArray`	`O(length)`
`sb.insert(i, x)`, `sb.deleteCharAt(i)`	`O(n - i)`
`sb.reverse()`, `sb.toString()`	`O(n)`

Sorting and arrays	
`Arrays.sort(int[])` (quicksort)	`O(n log n)`
`Arrays.sort(T[], cmp)`, `Collections.sort` (stable)	`O(n log n)`
`Arrays.fill`, `Arrays.copyOfRange`, `Collections.reverse`	`O(n)`

`HashMap` is O(1) only on average; keys with a good `hashCode` keep it there. Sorting an `int[]` uses a quicksort that crafted tests on Codeforces can slow down on older Java versions: shuffle the array first, or sort an `Integer[]` or a `List` (merge sort).

The Java kit every template assumes

The templates use the same few Java tools again and again. These are LeetCode's node classes, the collection idioms, and a full program for judges that read from standard input (Infosys, HackerRank, Codeforces). Everything here is compiled and tested too.

`ListNode` and `TreeNode`: LeetCode's node classes

helper

```
class ListNode {
    int val;
    ListNode next;
    ListNode(int val) { this.val = val; }
    ListNode(int val, ListNode next) { this.val = val; this.next = next; }
}

class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int val) { this.val = val; }
    TreeNode(int val, TreeNode left, TreeNode right) {
        this.val = val; this.left = left; this.right = right;
    }
}
```

A full program for stdin judges

memorise

```
import java.util.*;
import java.io.*;

public class Main {
    public static void main(String[] args) throws IOException {
        FastReader in = new FastReader(System.in);
        PrintWriter out = new PrintWriter(new BufferedOutputStream(System.out));
        int t = in.nextInt();                    // number of test cases
        while (t-- > 0) {
            int n = in.nextInt();
            long[] a = new long[n];
            for (int i = 0; i < n; i++) a[i] = in.nextLong();
            out.println(solve(n, a));
        }
        out.flush();                             // prints nothing without this
    }

    static long solve(int n, long[] a) {         // your algorithm goes here
        long sum = 0;
        for (long x : a) sum += x;
        return sum;
    }

    // static class FastReader { ... }  is the next block; paste it here
```

Most stdin judges want the class to be called `Main`; check the platform's instructions. The input `2 / 3 / 1 2 3 / 2 / -10 5` (one value or line per slash) prints `6` and `-5`.

Collection idioms the templates use

usage

```
// Heaps: min-heap by default; max-heap and custom orders take a comparator
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());
PriorityQueue<int[]> byFirst =
        new PriorityQueue<>((p, q) -> Integer.compare(p[0], q[0]));

// Sort rows by column 0, ties by column 1 descending (never p[0] - q[0])
Arrays.sort(iv, (p, q) -> p[0] != q[0] ? Integer.compare(p[0], q[0])
                                       : Integer.compare(q[1], p[1]));
// iv -> [[1, 5], [2, 9], [2, 1]]

// Sort int[] descending: sort ascending, then reverse in place
Arrays.sort(a);
for (int i = 0, j = a.length - 1; i < j; i++, j--) {
    int t = a[i]; a[i] = a[j]; a[j] = t;
}                                                    // a -> [5, 4, 1]

// Count and group
Map<String, Integer> cnt = new HashMap<>();
for (String w : words) cnt.merge(w, 1, Integer::sum);  // {a=1, b=2}
int bs = cnt.getOrDefault("b", 0);                     // 2
Map<Integer, List<Integer>> groups = new HashMap<>();
groups.computeIfAbsent(7, k -> new ArrayList<>()).add(1);

// Adjacency list for nodes 0..n-1
List<Integer>[] adj = new List[n];
for (int i = 0; i < n; i++) adj[i] = new ArrayList<>();

// ArrayDeque is both the stack and the queue
Deque<Integer> st = new ArrayDeque<>();
st.push(1); st.push(2);                                // top is 2
int top = st.pop();                                    // 2
Deque<Integer> q = new ArrayDeque<>();
q.offer(1); q.offer(2);
int front = q.poll();                                  // 1

// Sorted map: nearest keys in O(log n)
TreeMap<Integer, String> tm = new TreeMap<>();
tm.put(10, "x"); tm.put(20, "y");
Integer fl = tm.floorKey(15), ce = tm.ceilingKey(15);  // 10, 20

// Fill a 2D table, encode a grid cell as one int
for (int[] row : dp) Arrays.fill(row, -1);
int r = 2, c = 3, id = r * cols + c;                   // 13
int r0 = id / cols, c0 = id % cols;                    // 2, 3
```

`FastReader` : **fast input, goes inside** `Main` memorise

```
// Reads numbers and words from a byte buffer: much faster than Scanner.
static class FastReader {
    private final InputStream is;
    private final byte[] buf = new byte[1 << 16];
    private int len = 0, ptr = 0;

    FastReader(InputStream is) { this.is = is; }

    private int read() throws IOException {
        if (ptr == len) {
            len = is.read(buf, 0, buf.length);
            ptr = 0;
            if (len <= 0) return -1;
        }
        return buf[ptr++];
    }

    long nextLong() throws IOException {
        int c = read();
        while (c != '-' && (c < '0' || c > '9')) {
            if (c == -1) throw new EOFException();
            c = read();
        }
        boolean neg = c == '-';
        if (neg) c = read();
        long x = 0;
        while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = read(); }
        return neg ? -x : x;
    }

    int nextInt() throws IOException { return (int) nextLong(); }

    String next() throws IOException {        // one word, skipping spaces
        int c = read();
        while (c != -1 && c <= ' ') c = read();
        StringBuilder sb = new StringBuilder();
        while (c != -1 && c > ' ') { sb.append((char) c); c = read(); }
        return sb.toString();
    }
}
```

Part 1

Arrays, hashing and prefix sums

Most array and string questions come down to a little bookkeeping done in one pass. A hash map or an `int[26]` remembers what you have already seen, a prefix-sum array answers any range sum in O(1), and a single running value (the best sum ending here, a vote count) carries the answer forward. Reach for this part when the brute force checks every pair or every subarray and n is around 10^5, so you need O(n).

1.1	**Frequency counting** Same letters in any order, or how often each value occurs.	O(n)	p. 18
1.2	**Seen-before hash map** For each element, ask in O(1) whether its partner (or a copy) appeared earlier.	O(n)	p. 21
1.3	**Prefix sums** Many range-sum queries on a fixed array, or left sum against right sum.	O(n + q)	p. 23
1.4	**Prefix sum + hash map** Count or measure subarrays with sum k when values can be negative.	O(n)	p. 26
1.5	**2D prefix sums** Many rectangle-sum queries on a fixed grid, or a block sum around every cell.	O(rc + q)	p. 28
1.6	**Difference array** Many updates "add v to a[l..r]", then you read the array once.	O(n + q)	p. 31
1.7	**Kadane's maximum subarray** Largest (or smallest) sum or product of a contiguous subarray, in one pass.	O(n)	p. 34
1.8	**Boyer–Moore majority vote** Find the value that appears more than n/2 (or n/3) times, with O(1) extra space.	O(n)	p. 36
1.9	**Cyclic sort (index as hash)** Values in 1..n; find what is missing or doubled with O(1) extra space.	O(n)	p. 38
1.10	**In-place matrix moves** Rotate, spiral-walk or zero out a matrix in place, with no second grid.	O(r·c)	p. 40

1.1 Frequency counting

O(n)

Spot it when the problem looks like this

- Anagram, permutation, "can be rearranged into", "same characters": compare counts, not order.
- The limits say "lowercase English letters only": use `int[26]` instead of a map.
- First, most or least frequent element, or a character that appears exactly once.
- Group words that are equal up to reordering their letters.

How it works

An anagram ignores order. All that matters is how many times each letter appears, so turn each string into its letter counts and compare those. With `int[26]` you index by `c - 'a'`, and each update is one array write. Counting a string of length n is O(n); comparing two count arrays is O(26).

For valid anagram one array is enough: add one for each letter of `s` and take one away for each letter of `t`. The strings are anagrams exactly when every count ends at 0.

To group anagrams, turn the count array into the map key with `Arrays.toString(cnt)`. Two words get the same key if and only if they have the same counts, so each group collects in one bucket. For values that are not small letters (ints up to 10^9, any Unicode), use a `HashMap` and `merge(x, 1, Integer::sum)`.

Worked example **group anagrams of ["eat", "tea", "tan", "ate", "nat", "bat"]**

```
word  counts (key)   its group so far
eat   a1 e1 t1       [eat]
tea   a1 e1 t1       [eat, tea]
tan   a1 n1 t1       [tan]
ate   a1 e1 t1       [eat, tea, ate]
nat   a1 n1 t1       [tan, nat]
bat   a1 b1 t1       [bat]
result: [[eat, tea, ate], [bat], [tan, nat]]   (3 keys, any order)
```

Count with a map or an int[26] memorise

```java
// Count each value. A missing key counts as 0.
static Map<Integer, Integer> countValues(int[] a) {
    Map<Integer, Integer> cnt = new HashMap<>();
    for (int x : a) {
        cnt.merge(x, 1, Integer::sum);
        // same as: cnt.put(x, cnt.getOrDefault(x, 0) + 1);
    }
    return cnt;
}

// Valid anagram: same letters with the same counts.
// For 'a'..'z' an int[26] is faster and shorter than a map.
static boolean isAnagram(String s, String t) {
    if (s.length() != t.length()) return false;
    int[] cnt = new int[26];
    for (int i = 0; i < s.length(); i++) {
        cnt[s.charAt(i) - 'a']++;  // s adds one
        cnt[t.charAt(i) - 'a']--;  // t takes one away
    }
    for (int c : cnt) if (c != 0) return false;
    return true;
}
```

Group anagrams by a count key

variant

```java
// Group anagrams: words with the same letter counts get the same key.
static List<List<String>> groupAnagrams(String[] words) {
    Map<String, List<String>> groups = new HashMap<>();
    for (String w : words) {
        int[] cnt = new int[26];
        for (char c : w.toCharArray()) cnt[c - 'a']++;
        String key = Arrays.toString(cnt);     // "[1, 0, 0, 0, 1, ...]"
        groups.computeIfAbsent(key, k -> new ArrayList<>()).add(w);
    }
    return new ArrayList<>(groups.values());
}
```

First unique character

example

```java
// First unique character: count everything, then scan in order.
static int firstUniqChar(String s) {
    int[] cnt = new int[26];
    for (char c : s.toCharArray()) cnt[c - 'a']++;
    for (int i = 0; i < s.length(); i++)
        if (cnt[s.charAt(i) - 'a'] == 1) return i;
    return -1;
}
```

Cost Time `O(n)` Space `O(k)`

n = total characters, one array or map write each; k = distinct keys, and `int[26]` is O(1).

Variants same idea, small change

- Take one away: `if (cnt.merge(x, -1, Integer::sum) == 0) cnt.remove(x);` keeps `cnt.size()` equal to the number of distinct values.
- Ransom note: count the magazine, then spend those counts on the note; fail as soon as one drops below 0.
- Sorted key for group anagrams: sort `w.toCharArray()` and use `new String(c)`. It costs O(L log L) per word instead of O(L + 26).
- Close strings: both words use the same set of letters, and their sorted `int[26]` arrays are equal.

Traps bugs people actually write

- `cnt.put(x, cnt.get(x) + 1)` throws `NullPointerException` the first time `x` appears. Use `merge` or `getOrDefault(x, 0)`.
- `map.get(a) == map.get(b)` compares two `Integer` objects. Java caches only -128..127, so two counts of 128 are not `==`. Use `.equals` or unbox to `int`.
- `c - 'a'` assumes lowercase letters. Uppercase letters or digits give a negative index; use `int[128]` for ASCII, or a map.
- An `int[]` as a map key uses identity, so `map.get(new int[] {...})` never finds anything. Convert the counts to a `String` first.

Practise on LeetCode, in this order

- [ ] LC 242 Valid Anagram — Easy
- [ ] LC 383 Ransom Note — Easy
- [ ] LC 387 First Unique Character in a String — Easy
- [ ] LC 49 Group Anagrams — Medium
- [ ] LC 1657 Determine if Two Strings Are Close — Medium

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

1.2 Seen-before hash map

`O(n)`

Spot it when the problem looks like this

- Unsorted array, find two indexes with `a[i] + a[j] == target` (or a given difference).
- "Contains duplicate", "within distance k", "has this appeared before".
- n up to 10^5 rules out the $O(n^2)$ pair loop, and sorting would lose the original indexes.
- Longest run of consecutive *values* in any order, asked for in O(n).

How it works

The brute force checks every pair `i < j`. Turn it around: walk left to right and keep a hash map of everything already passed. Then "is there a partner for `a[i]`?" is one `get`, O(1) on average, and the whole scan is O(n).

For two sum the partner of `a[i]` is `target - a[i]`. Look it up **before** you store `a[i]`, so an element cannot pair with itself. For "duplicate within k", store each value's last index: the most recent copy is the closest one, so older indexes never matter.

For longest consecutive sequence, put every value in a set and start counting only at an `x` whose `x - 1` is missing, because that is where a run begins. Every other value is skipped in O(1), and each value is walked over once by the inner `while`, so the total is O(n), not $O(n^2)$.

Worked example **two sum, a = [3, 8, 4, 6, 2], target = 10**

```
i  a[i] need  seen?      map after (value=index)
0  3    7     no         {3=0}
1  8    2     no         {3=0, 8=1}
2  4    6     no         {3=0, 8=1, 4=2}
3  6    4     yes, at 2  return (2, 3)
---- longest consecutive, a = [100, 4, 200, 1, 3, 2] ----
x = 1    0 not in set:     walk 1, 2, 3, 4 -> len 4
x = 100  99 not in set:    walk 100 -> len 1
x = 200  199 not in set:   walk 200 -> len 1
x = 2, 3, 4: x - 1 is in the set, skip.  best = 4
```

Two sum: look back, then remember

memorise

```java
// Two sum: for each a[i], ask whether its partner was seen earlier.
static int[] twoSum(int[] a, int target) {
    Map<Integer, Integer> indexOf = new HashMap<>();  // value -> index
    for (int i = 0; i < a.length; i++) {
        Integer j = indexOf.get(target - a[i]);       // 1. look back first
        if (j != null) return new int[] {j, i};
        indexOf.put(a[i], i);                         // 2. then remember a[i]
    }
    return new int[] {-1, -1};
}
```

Duplicate within distance k

variant

```java
// Contains duplicate II: two equal values at most k indexes apart.
// Only the last index of each value matters: it is the closest one.
static boolean containsNearbyDuplicate(int[] a, int k) {
    Map<Integer, Integer> last = new HashMap<>();  // value -> last index
    for (int i = 0; i < a.length; i++) {
        Integer j = last.put(a[i], i);             // put returns the old index
        if (j != null && i - j <= k) return true;
    }
    return false;
}
```

Longest consecutive sequence

memorise

```java
// Longest run of consecutive values (any order) in O(n).
// Count only from x where x - 1 is missing, so each run is walked once.
static int longestConsecutive(int[] a) {
    Set<Integer> set = new HashSet<>();
    for (int x : a) set.add(x);
    int best = 0;
    for (int x : set) {                    // the set, not a: no repeat walks
        if (set.contains(x - 1)) continue;  // x is not the start of a run
        int len = 1;
        while (set.contains(x + len)) len++;
        best = Math.max(best, len);
    }
    return best;
}
```

Cost Time `O(n)` Space `O(n)`

One pass of O(1) average hash operations; the map or set holds up to n entries.

Variants same idea, small change

- Count pairs instead of finding one: before storing `a[i]`, add `cnt.getOrDefault(target - a[i], 0)` to the answer.
- 4Sum II: put every `a + b` from the first two arrays in a count map, then look up `-(c + d)` for the other two. $O(n^2)$ instead of $O(n^4)$.
- Contains duplicate: `if (!set.add(x)) return true;`, because `add` returns false when `x` is already there.
- Sorted input, or no indexes needed: two pointers with O(1) space (2.1 **Opposite-end pointers** (p. 44)).

Traps bugs people actually write

- Storing `a[i]` before the lookup lets it pair with itself: `[3, 2, 4]` with target 6 returns `(0, 0)`.
- `i - j < k` instead of `<= k` misses pairs exactly k apart: `[1, 0, 1]` with k = 2 returns false.
- Looping `for (int x : a)` instead of over the set in longest consecutive: with many copies of a run's first value, each copy walks the whole run, which is $O(n^2)$.

Practise on LeetCode, in this order

☐	LC 1	Two Sum	Easy
☐	LC 217	Contains Duplicate	Easy
☐	LC 219	Contains Duplicate II	Easy
☐	LC 128	Longest Consecutive Sequence	Medium
☐	LC 454	4Sum II	Medium

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

1.3 Prefix sums

`O(n + q)`

Spot it when the problem looks like this

- Many queries "sum of `a[l..r]`" on a fixed array.
- Compare the sum left of i with the sum right of i: pivot index, middle index, minimum average difference.
- Product or sum of *everything except* `a[i]`, often with "without using division".

How it works

Store `pre[i]` = the sum of the first i elements, so `pre` has n + 1 entries and `pre[0] = 0`. Every range sum is then a difference of two prefixes: `sum(l..r) = pre[r + 1] - pre[l]`, because `pre[r + 1]` covers `a[0..r]` and subtracting `pre[l]` removes `a[0..l-1]`. Building is O(n) and each query is O(1).

The extra slot is the reason for the n + 1 convention: a range that starts at 0 needs no special case, since `pre[0]` is 0.

The same idea works from both ends. In product of array except self, `ans[i]` is the product of everything left of i times everything right of i. A forward pass writes the left products into `ans`. A backward pass multiplies in a running right product. No division, and nothing extra beyond the output array.

Worked example a = [3, -1, 4, 1, 5]

```
i         0   1   2   3   4   5
a[i]      3  -1   4   1   5
pre[i]    0   3   2   6   7  12
sum(1, 3) = pre[4] - pre[1] = 7 - 3 = 4
sum(0, 4) = pre[5] - pre[0] = 12 - 0 = 12
sum(2, 2) = pre[3] - pre[2] = 6 - 2 = 4
```

Range sum query, pre[n + 1]

memorise

```java
// Range sum query: pre[i] = a[0] + ... + a[i-1], so pre[0] = 0
// and the sum of a[l..r] is pre[r + 1] - pre[l].
static class NumArray {
    private final long[] pre;

    NumArray(int[] a) {
        pre = new long[a.length + 1];
        for (int i = 0; i < a.length; i++) pre[i + 1] = pre[i] + a[i];
    }

    long sumRange(int l, int r) {              // inclusive l..r
        return pre[r + 1] - pre[l];
    }
}
```

LeetCode's `sumRange` returns `int`: cast with `(int)` there.

Pivot index

example

```java
// Pivot index: sum left of i == sum right of i.
// right = total - left - a[i], so one running sum is enough.
static int pivotIndex(int[] a) {
    long total = 0, left = 0;
    for (int x : a) total += x;
    for (int i = 0; i < a.length; i++) {
        if (left == total - left - a[i]) return i;
        left += a[i];
    }
    return -1;
}
```

Product of array except self

variant

```java
// Product of array except self, without division:
// ans[i] = (product of a[0..i-1]) * (product of a[i+1..n-1]).
static int[] productExceptSelf(int[] a) {
    int n = a.length;
    int[] ans = new int[n];
    ans[0] = 1;           // 1. left products
    for (int i = 1; i < n; i++) ans[i] = ans[i - 1] * a[i - 1];
    int right = 1;        // 2. times right products
    for (int i = n - 1; i >= 0; i--) {
        ans[i] *= right;  // right = product of a[i+1..]
        right *= a[i];
    }
    return ans;
}
```

Cost Time `O(n + q)` Space `O(n)`

O(n) to build `pre`, then O(1) for each of q queries. Product except self needs O(1) beyond its output.

Variants same idea, small change

- Running sum of 1d array: this is `pre` without the leading 0.
- Count something in a range: take the prefix of an indicator, e.g. `pre[i + 1] = pre[i] + (a[i] % 2 != 0 ? 1 : 0)` counts odd values in any range.
- Minimum average difference: one pass with `left` and `total - left`, as in pivot index.
- Count subarrays with a given sum: add a hash map of prefixes (1.4 **Prefix sum + hash map** (p. 26)). Rectangles: 1.5 **2D prefix sums** (p. 28).

Traps bugs people actually write

- Off by one: `pre[r] - pre[l]` is the sum of `a[l..r-1]`. The inclusive range needs `pre[r + 1]`.
- `int[] pre` overflows when n = 10^5 and values reach 10^5 (total 10^{10}). Use `long[]`.
- Product except self with `total / a[i]` divides by zero on a 0, and is not allowed anyway. Use the two passes.
- Pivot index compares the sums *without* `a[i]`: `left == total - left` counts `a[i]` on the right side.

Practise on LeetCode, in this order

☐	LC 1480	Running Sum of 1d Array	Easy
☐	LC 303	Range Sum Query - Immutable	Easy
☐	LC 724	Find Pivot Index	Easy
☐	LC 238	Product of Array Except Self	Medium
☐	LC 2256	Minimum Average Difference	Medium

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

1.4 Prefix sum + hash map

O(n)

Spot it when the problem looks like this

- "Number of subarrays with sum k" where values can be negative, so a sliding window breaks.
- "Sum is a multiple of k" or "divisible by k": compare prefix sums mod k.
- "Equal number of 0s and 1s" (or of any two kinds): count one kind as -1 and look for sum 0.
- Longest subarray with such a property, n up to 10^5.

How it works

A subarray's sum is the prefix sum at its end minus the prefix sum that ends right before its start. So each subarray that ends at i with sum k matches one earlier prefix equal to `pre - k`. Keep a map from prefix value to how many times you have seen it. At each i, add `seen[pre - k]` to the answer, then record `pre`. That is one pass with O(1) average work per step.

Put `(0, 1)` in the map before the loop. It stands for the empty prefix, so a subarray that starts at index 0 is counted too.

For *longest* questions, store the **earliest** index of each prefix value and never overwrite it (`putIfAbsent`), because the earliest start gives the longest subarray. Remainders work the same way: if two prefixes leave the same remainder mod k, the part between them is a multiple of k.

Worked example **subarray sum equals k, a = [1, -1, 1, -1, 1], k = 1**

```
start: seen {0=1}  (the empty prefix)
i  x   pre  pre-k  matches  count  seen after
0  1   1    0      1        1      {0=1, 1=1}
1  -1  0    -1     0        1      {0=2, 1=1}
2  1   1    0      2        3      {0=2, 1=2}
3  -1  0    -1     0        3      {0=3, 1=2}
4  1   1    0      3        6      {0=3, 1=3}
```

Count subarrays with sum k memorise

```java
// Number of subarrays with sum k. The subarray after prefix j and up to
// i has sum pre_i - pre_j, so at each i count earlier pre_j == pre_i - k.
static int subarraySum(int[] a, int k) {
    Map<Integer, Integer> seen = new HashMap<>();  // prefix sum -> times seen
    seen.put(0, 1);                                // the empty prefix
    int pre = 0, count = 0;
    for (int x : a) {
        pre += x;
        count += seen.getOrDefault(pre - k, 0);    // 1. count first
        seen.merge(pre, 1, Integer::sum);          // 2. then add pre_i
    }
    return count;
}
```

Multiple of k, earliest index

variant

```
// Continuous subarray sum: length >= 2 and sum a multiple of k (a[i] >= 0).
// Two prefixes with the same remainder enclose a multiple of k.
static boolean checkSubarraySum(int[] a, int k) {
    Map<Integer, Integer> first = new HashMap<>();  // remainder -> first index
    first.put(0, -1);                               // empty prefix "ends" at -1
    int rem = 0;
    for (int i = 0; i < a.length; i++) {
        rem = (int) ((rem + (long) a[i]) % k);      // long: may pass 2^31
        Integer j = first.putIfAbsent(rem, i);      // keeps the earliest index
        if (j != null && i - j >= 2) return true;
    }
    return false;
}
```

Equal 0s and 1s (balance)

variant

```
// Contiguous array: longest subarray with as many 0s as 1s.
// Count 0 as -1; the subarray is balanced when two prefixes are equal.
static int findMaxLength(int[] a) {
    Map<Integer, Integer> first = new HashMap<>();  // balance -> first index
    first.put(0, -1);
    int bal = 0, best = 0;
    for (int i = 0; i < a.length; i++) {
        bal += a[i] == 1 ? 1 : -1;
        Integer j = first.putIfAbsent(bal, i);
        if (j != null) best = Math.max(best, i - j);
    }
    return best;
}
```

Cost Time `O(n)` Space `O(n)`

One pass; the map holds at most n + 1 prefix values (at most k remainders).

Variants same idea, small change

- Subarray sums divisible by k: count equal remainders as in the memorise template, keyed by `((pre % k) + k) % k`.
- Longest subarray with sum k: map each prefix to its earliest index and take `i - first.get(pre - k)`.
- Make sum divisible by p: remove the shortest subarray with sum ≡ total (mod p). For *shortest*, store the latest index of each remainder.
- All values ≥ 0 and you want longest or shortest: a sliding window (2.7 **Sliding window (variable size)** (p. 58)) needs O(1) space.

Traps bugs people actually write

- Forgetting `seen.put(0, 1)` (or `first.put(0, -1)`) misses subarrays that start at index 0: `[3]` with k = 3 returns 0.
- Recording `pre` before the lookup: with k = 0 each prefix matches itself, so `[1, 2]` returns 2 instead of 0.
- Java's `%` keeps the sign, `-1 % 5 == -1`. With negative values use `((pre % k) + k) % k`, or equal remainders look different.
- `put` instead of `putIfAbsent` overwrites the earliest index, so you measure from a later start and get subarrays that are too short.

Practise on LeetCode, in this order

☐	LC 560	Subarray Sum Equals K	Medium
☐	LC 525	Contiguous Array	Medium
☐	LC 974	Subarray Sums Divisible by K	Medium
☐	LC 523	Continuous Subarray Sum	Medium
☐	LC 1590	Make Sum Divisible by P	Medium

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

1.5 2D prefix sums

`O(rc + q)`

Spot it when the problem looks like this

- "Sum of the elements inside the rectangle from (row1, col1) to (row2, col2)", asked many times.
- Every cell needs the sum of the block around it: `|r - i| ≤ k` and `|c - j| ≤ k`.
- A grid up to about 300 × 300 where you try many squares or rectangles, so each sum must be O(1).

How it works

Let `pre[i][j]` be the sum of the rectangle from (0, 0) to (i - 1, j - 1). The extra zero row and column do what `pre[0] = 0` does in one dimension: no edge cases.

Build it by inclusion–exclusion. The rectangle up to a cell is the one above it plus the one to its left, minus their overlap (counted twice), plus the cell itself. A query runs this backwards: take the rectangle to the bottom-right corner, subtract the strip above and the strip to the left, and add back the top-left block you subtracted twice. Building is O(rows · cols) and each query is four lookups.

To count submatrices with a target sum, fix the top and bottom rows. The column sums between them form a 1D array, and the prefix-sum + hash map count from 1.4 **Prefix sum + hash map** (p. 26) finishes the job.

Worked example m is 3 × 3; sum of rows 1..2, columns 1..2

```
m          pre (zero row and column added)
3 0 1        0  0  0  0
5 6 3        0  3  3  4
1 2 0        0  8 14 18
             0  9 17 21
sumRegion(1, 1, 2, 2) = pre[3][3] - pre[1][3] - pre[3][1] + pre[1][1]
  = 21 - 4 - 9 + 3 = 11   (6 + 3 + 2 + 0)
```

Range sum query 2D

memorise

```java
// Range sum query 2D. pre[i][j] = sum of rows 0..i-1, columns 0..j-1.
// The extra row and column of zeros remove every bounds check.
static class NumMatrix {
    private final long[][] pre;

    NumMatrix(int[][] m) {
        int rows = m.length, cols = m[0].length;
        pre = new long[rows + 1][cols + 1];
        for (int i = 0; i < rows; i++)
            for (int j = 0; j < cols; j++)
                pre[i + 1][j + 1] = pre[i][j + 1] + pre[i + 1][j]
                                  - pre[i][j] + m[i][j];
    }

    // Inclusive corners (r1, c1) top-left and (r2, c2) bottom-right.
    long sumRegion(int r1, int c1, int r2, int c2) {
        return pre[r2 + 1][c2 + 1] - pre[r1][c2 + 1]
             - pre[r2 + 1][c1] + pre[r1][c1];
    }
}
```

Matrix block sum

example

```java
// Matrix block sum: clip the (2k+1) x (2k+1) block to the grid,
// then answer each cell with one sumRegion call.
static int[][] matrixBlockSum(int[][] m, int k) {
    int rows = m.length, cols = m[0].length;
    NumMatrix ps = new NumMatrix(m);
    int[][] ans = new int[rows][cols];
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++) {
            int r1 = Math.max(0, i - k), c1 = Math.max(0, j - k);
            int r2 = Math.min(rows - 1, i + k), c2 = Math.min(cols - 1, j + k);
            ans[i][j] = (int) ps.sumRegion(r1, c1, r2, c2);
        }
    return ans;
}
```

Submatrices that sum to target

variant

```
// Submatrices that sum to target in O(rows^2 * cols): fix the top and
// bottom rows, squash them into column sums, then count 1D subarrays
// with sum target using a prefix sum + hash map.
static int numSubmatrixSumTarget(int[][] m, int target) {
    int rows = m.length, cols = m[0].length, count = 0;
    for (int top = 0; top < rows; top++) {
        int[] col = new int[cols];              // col[j] = m[top..bot][j]
        for (int bot = top; bot < rows; bot++) {
            for (int j = 0; j < cols; j++) col[j] += m[bot][j];
            Map<Integer, Integer> seen = new HashMap<>();
            seen.put(0, 1);
            int pre = 0;
            for (int j = 0; j < cols; j++) {
                pre += col[j];
                count += seen.getOrDefault(pre - target, 0);
                seen.merge(pre, 1, Integer::sum);
            }
        }
    }
    return count;
}
```

Cost Time `O(rc + q)` Space `O(rc)`

O(rows · cols) to build, O(1) per query. The target count is O(rows² · cols), so let the smaller side be the rows.

Variants same idea, small change

- Submatrices that contain the top-left cell with sum ≤ k: check `pre[i + 1][j + 1] <= k` for every cell; no queries needed.
- Largest square with sum ≤ threshold: for each corner, test the next side length in O(1); the answer only grows.
- Many rectangle *updates*, then one read: the 2D difference array in **1.6 Difference array** (p. 31).
- Cells change between queries: a 2D Fenwick tree (**13.2 Fenwick tree (binary indexed)** (p. 366)).

Traps bugs people actually write

- Wrong sign on the corner: the query ends with `+ pre[r1][c1]`, which adds back the block subtracted twice.
- Mixed indexes: `pre` is shifted by one, so the query uses `r2 + 1` and `c2 + 1` but plain `r1` and `c1`.
- Large grids pass `int`: 1000 × 1000 cells of 10^4 sum to 10^{10}. The template stores `long`.

Practise on LeetCode, in this order

- [ ] LC 304 Range Sum Query 2D - Immutable — Medium
- [ ] LC 1314 Matrix Block Sum — Medium
- [ ] LC 3070 Count Submatrices with Top-Left Element and Sum Less Than k — Medium
- [ ] LC 1292 Maximum Side Length of a Square with Sum Less than or Equal to Threshold — Medium
- [ ] LC 1074 Number of Submatrices That Sum to Target — Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

1.6 Difference array

`O(n + q)`

Spot it when the problem looks like this

- A list of bookings, trips or shifts, each adding a value to a whole range of positions.
- "Return the array after all operations" or "is the load ever above capacity?"
- Up to 10^5 updates on 10^5 positions: touching every element of every range is 10^{10} steps.

How it works

Instead of changing every element in `[l, r]`, write two marks: `diff[l] += v` and `diff[r + 1] -= v`. The final value at i is the prefix sum `diff[0] + ... + diff[i]`. An update adds `v` to that sum exactly when `l ≤ i ≤ r`: the `+v` is included once i reaches l, and the `-v` cancels it from r + 1 on.

So each update costs O(1), and one prefix-sum pass at the end rebuilds the whole array in O(n). The array needs n + 1 slots so that `r + 1` is always a valid index.

Check where a range really ends. In car pooling, people get out *at* stop `to`, so they ride stops `from..to-1` and the minus goes at `to`, not `to + 1`. The 2D version puts four marks at the corners of the rectangle and rebuilds the grid with a 2D prefix sum.

Worked example n = 5, updates (l, r, v)

```
update       diff[0..5] after it
(1, 3, +2)     0   2   0   0  -2   0
(2, 4, +3)     0   2   3   0  -2  -3
(0, 2, -1)    -1   2   3   1  -2  -3
prefix sums   -1   1   4   5   3   <- a, the final values
```

Range adds, then one pass

memorise

```java
// Many range adds, then read every value once.
// diff[l] += v starts the add at l; diff[r + 1] -= v stops it after r.
static int[] rangeAdds(int n, int[][] updates) {  // each {l, r, v}
    int[] diff = new int[n + 1];                  // +1 so r + 1 fits
    for (int[] u : updates) {
        diff[u[0]] += u[2];
        diff[u[1] + 1] -= u[2];
    }
    int[] a = new int[n];
    int run = 0;
    for (int i = 0; i < n; i++) {
        run += diff[i];                           // prefix sum = a[i]
        a[i] = run;
    }
    return a;
}
```

Car pooling (half-open trips)

variant

```java
// Car pooling: t = {people, from, to}. They ride stops from..to-1,
// so add at from and remove at to (they are out before others get in).
static boolean carPooling(int[][] trips, int capacity) {
    int[] diff = new int[1001];                   // stops 0..1000
    for (int[] t : trips) {
        diff[t[1]] += t[0];
        diff[t[2]] -= t[0];
    }
    int load = 0;
    for (int d : diff) {
        load += d;
        if (load > capacity) return false;
    }
    return true;
}
```

2D difference array

variant

```
// 2D difference: add v to rectangle (r1, c1)..(r2, c2) with four marks,
// then a 2D prefix sum turns the marks back into values.
// Each update is {r1, c1, r2, c2, v}, corners inclusive.
static int[][] rectAdds(int rows, int cols, int[][] ups) {
    int[][] d = new int[rows + 1][cols + 1];
    for (int[] u : ups) {
        d[u[0]][u[1]] += u[4];
        d[u[0]][u[3] + 1] -= u[4];
        d[u[2] + 1][u[1]] -= u[4];
        d[u[2] + 1][u[3] + 1] += u[4];
    }
    int[][] a = new int[rows][cols];
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++) {
            a[i][j] = d[i][j];
            if (i > 0) a[i][j] += a[i - 1][j];
            if (j > 0) a[i][j] += a[i][j - 1];
            if (i > 0 && j > 0) a[i][j] -= a[i - 1][j - 1];
        }
    return a;
}
```

Cost Time `O(n + q)` Space `O(n)`

O(1) per update and one O(n) pass at the end, instead of O(n) per update.

Variants same idea, small change

- Corporate flight bookings: the template with 1-indexed input, so pass `first - 1` and `last - 1`.
- Shifting letters II: the difference array holds +1 or -1 shifts per range; apply `((c - 'a' + shift) % 26 + 26) % 26` at the end.
- Maximum population year: +1 at the birth year, -1 at the death year (not counted), then find the largest prefix sum.
- Positions up to 10^9: sort the start and end events instead of allocating an array (6.4 **Sweep line (start and end events)** (p. 149)).

Traps bugs people actually write

- `new int[n]` instead of `new int[n + 1]`: `diff[r + 1]` throws when `r = n - 1`.
- Half-open ranges: in car pooling `diff[to + 1] -= p` keeps people on board at the stop where they get out. `[[2, 1, 5], [3, 5, 7]]` with capacity 3 then wrongly returns false.
- Reading `diff[i]` as the value: it is only the change at i. Take the prefix sum first.

Practise on LeetCode, in this order

- [ ] LC 1854 Maximum Population Year — Easy
- [ ] LC 1109 Corporate Flight Bookings — Medium
- [ ] LC 1094 Car Pooling — Medium
- [ ] LC 2381 Shifting Letters II — Medium
- [ ] LC 2536 Increment Submatrices by One — Medium

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

1.7 Kadane's maximum subarray

`O(n)`

Spot it when the problem looks like this

- "Maximum sum of a contiguous subarray", with negative numbers present.
- A circular array plus maximum subarray: think total minus the minimum subarray.
- Maximum *product* subarray: the same pass, but carry the minimum too.
- Any best-segment score you can extend one element at a time, such as one buy and one sell.

How it works

Let `cur` be the best sum of a subarray that **ends exactly at i**. Such a subarray is either `a[i]` alone or `a[i]` added to the best one ending at `i - 1`, so `cur = max(a[i], cur + a[i])`. The answer is the largest `cur` you see. Put another way, drop the running part as soon as it is negative, because it can only lower whatever follows. This is a one-variable DP: O(n) time, O(1) space.

In a circular array the best subarray either does not wrap (plain Kadane) or wraps past the end. A wrapping subarray leaves out a middle part, so its sum is `total - (minimum subarray)`. Run Kadane for the max and the min in the same loop. If every number is negative, the minimum subarray is the whole array and `total - min` is 0, an empty subarray, so return the plain maximum.

Worked example a = [-2, 1, -3, 4, -1, 2, 1, -5, 4]

```
i  a[i]  cur = max(a[i], cur + a[i])  best
0  -2    -2  (start)                  -2
1  1     max(1, -1) = 1 restart       1
2  -3    max(-3, -2) = -2 extend      1
3  4     max(4, 2) = 4 restart        4
4  -1    max(-1, 3) = 3 extend        4
5  2     max(2, 5) = 5 extend         5
6  1     max(1, 6) = 6 extend         6
7  -5    max(-5, 1) = 1 extend        6
8  4     max(4, 5) = 5 extend         6
```

Maximum subarray

memorise

```java
// Kadane: largest sum of a non-empty subarray.
// cur = best sum of a subarray that ends exactly at i.
static int maxSubArray(int[] a) {
    int cur = a[0], best = a[0];
    for (int i = 1; i < a.length; i++) {
        cur = Math.max(a[i], cur + a[i]);   // start fresh at i, or extend
        best = Math.max(best, cur);
    }
    return best;
}
```

Maximum circular subarray

variant

```java
// Max circular subarray. The best one either does not wrap (plain
// Kadane) or wraps, and then it is total minus a minimum middle part.
static int maxSubarraySumCircular(int[] a) {
    int total = 0, curMax = 0, bestMax = a[0], curMin = 0, bestMin = a[0];
    for (int x : a) {
        curMax = Math.max(x, curMax + x);
        bestMax = Math.max(bestMax, curMax);
        curMin = Math.min(x, curMin + x);
        bestMin = Math.min(bestMin, curMin);
        total += x;
    }
    // All negative: total - bestMin = 0 is the empty subarray, not allowed.
    return bestMax < 0 ? bestMax : Math.max(bestMax, total - bestMin);
}
```

Maximum product subarray

variant

```java
// Max product subarray. A negative x turns the smallest product into
// the largest, so track both for subarrays ending at i.
static int maxProduct(int[] a) {
    int hi = a[0], lo = a[0], best = a[0];
    for (int i = 1; i < a.length; i++) {
        int x = a[i];
        if (x < 0) { int t = hi; hi = lo; lo = t; }   // swap before use
        hi = Math.max(x, hi * x);
        lo = Math.min(x, lo * x);
        best = Math.max(best, hi);
    }
    return best;
}
```

Cost Time `O(n)` Space `O(1)` One pass that keeps a fixed number of running values.

Variants same idea, small change

- Return the subarray too: set `start = i` whenever you restart, and save it as `bestStart` when `best` improves.
- Maximum absolute sum: run max and min Kadane together and return `max(best, -worst)`.
- Best time to buy and sell stock once: the profit is the maximum subarray of the day-to-day price changes, or 0 if that is negative (or keep the lowest price so far).
- One deletion allowed: two values per i, the best sum ending at i with no deletion and with one. More in 10.2 **Linear DP over a sequence** (p. 280).

Traps bugs people actually write

- Starting from `cur = 0, best = 0` returns 0 for `[-3, -1, -2]`, but the subarray must be non-empty: the answer is -1. Start from `a[0]`.
- Circular without the all-negative check: `max(bestMax, total - bestMin)` gives 0 for `[-3, -2, -3]` instead of -2.
- Max product without the swap on a negative `x`: `[2, -4, 3, -2]` returns 3 instead of 48, because the large product comes from the smallest one.

Practise on LeetCode, in this order

☐	LC 121	Best Time to Buy and Sell Stock	Easy
☐	LC 53	Maximum Subarray	Medium
☐	LC 1749	Maximum Absolute Sum of Any Subarray	Medium
☐	LC 918	Maximum Sum Circular Subarray	Medium
☐	LC 152	Maximum Product Subarray	Medium
☐	LC 1186	Maximum Subarray Sum with One Deletion	Medium

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

1.8 Boyer–Moore majority vote

O(n)

Spot it when the problem looks like this

- "Appears more than ⌊n/2⌋ times", often with "you may assume it always exists".
- "More than ⌊n/3⌋ times": at most two values can, so keep two candidates.
- A follow-up asks for O(1) space, which rules out the counting map.

How it works

Pair each copy of the majority value with a different value and cancel both. The majority has more than half of the elements, so some of its copies are left after any such cancelling. The vote does this in one pass with a candidate and a counter: a match adds one, a mismatch cancels one, and when the counter is 0 the next element becomes the candidate. If a majority exists, it is the candidate at the end.

For "more than n/3", keep two candidates. A value that matches neither cancels one vote from each, which removes three different elements at once. A value with more than n/3 copies cannot lose all of them this way, so it ends as a candidate. The candidates are only *possible* answers, so a second pass counts them and keeps those above n/3.

Worked example a = [2, 2, 1, 1, 1, 2, 2]

```
i  x  cand  votes
0  2  2     1   (new candidate)
1  2  2     2
2  1  2     1
3  1  2     0
4  1  1     1   (new candidate)
5  2  1     0
6  2  2     1   (new candidate)
answer: 2
```

Majority element (more than n/2)

memorise

```java
// Boyer-Moore vote: a value seen more than n/2 times survives
// when every vote for it is cancelled by one vote against it.
static int majorityElement(int[] a) {
    int cand = 0, votes = 0;
    for (int x : a) {
        if (votes == 0) cand = x;       // nothing left: new candidate
        votes += (x == cand) ? 1 : -1;  // agree +1, cancel -1
    }
    return cand;                        // valid only if a majority is guaranteed
}
```

Majority element II (more than n/3)

variant

```java
// Majority element II: every value seen more than n/3 times (at most 2).
// A cancel removes three different values at once, so a value with
// more than n/3 copies cannot be cancelled out completely.
static List<Integer> majorityElementII(int[] a) {
    int c1 = 0, c2 = 0, v1 = 0, v2 = 0;
    for (int x : a) {          // 1. find two candidates
        if (x == c1) v1++;
        else if (x == c2) v2++;
        else if (v1 == 0) { c1 = x; v1 = 1; }
        else if (v2 == 0) { c2 = x; v2 = 1; }
        else { v1--; v2--; }  // x, c1, c2 cancel
    }
    v1 = v2 = 0;
    for (int x : a) {          // 2. verify: count them again
        if (x == c1) v1++;
        else if (x == c2) v2++;
    }
    List<Integer> out = new ArrayList<>();
    if (v1 > a.length / 3) out.add(c1);
    if (v2 > a.length / 3) out.add(c2);
    return out;
}
```

Cost Time `O(n)` Space `O(1)` One pass (two for the n/3 version) with a fixed number of counters.

Variants same idea, small change

- No promise that a majority exists: count the final candidate in a second pass and check `> n / 2`.
- More than n/k times: keep k - 1 candidates and verify them the same way. O(n·k) time, O(k) space.
- Minimum index of a valid split: find the dominant value by vote, then sweep with its running count.

Traps bugs people actually write

- Returning the candidate when a majority is not guaranteed: `[1, 2, 3]` returns 3.
- Skipping the second pass in the n/3 version: `[1, 2, 3]` returns candidates 1 and 2.
- `>= n / 3` instead of `> n / 3`: for n = 7 it accepts a value with only 2 copies.

Practise on LeetCode, in this order

- [ ] `LC 169` Majority Element — Easy
- [ ] `LC 229` Majority Element II — Medium
- [ ] `LC 2780` Minimum Index of a Valid Split — Medium

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

1.9 Cyclic sort (index as hash) `O(n)`

Spot it when the problem looks like this

- "Integers in the range [1, n]" in an array of length n.
- Find the missing number, the duplicates, or the first missing positive.
- "O(n) time and constant extra space": no hash set and no sorting.

How it works

When the values are 1..n and there are n slots, value v has a natural home: index v - 1. The array itself can then serve as the hash set.

Cyclic sort sends values home by swapping. At index i, while `a[i]` is in range and its home does not already hold that value, swap it home. Each swap puts one value at its home for good, so there are at most n swaps in total and the loop is O(n), even with the inner `while`. Out-of-range values and extra copies stay wherever they land. A final scan finds the first i with `a[i] != i + 1`: the value i + 1 is missing, and that cell holds a duplicate or junk.

When you only need to *mark* that v was seen, make `a[v - 1]` negative instead of swapping. Read values with `Math.abs`, because the sign may already be a flag.

Worked example first missing positive, a = [3, 4, -1, 1]

```
start             [3, 4, -1, 1]
i=0   3 -> a[2]   [-1, 4, 3, 1]
i=1   4 -> a[3]   [-1, 1, 3, 4]
i=1   1 -> a[0]   [1, -1, 3, 4]
scan: a[1] = -1, not 2  -> answer 2
```

Cyclic sort, first missing positive

memorise

```java
// Cyclic sort: put each value v in 1..n at index v - 1 by swapping.
// Values outside 1..n and extra copies stay wherever they land.
static void cyclicSort(int[] a) {
    for (int i = 0; i < a.length; i++)
        while (a[i] >= 1 && a[i] <= a.length && a[a[i] - 1] != a[i]) {
            int j = a[i] - 1;                       // home of a[i]
            int t = a[i]; a[i] = a[j]; a[j] = t;  // one more value home
        }
}

// First missing positive in O(n) time, O(1) extra space.
static int firstMissingPositive(int[] a) {
    cyclicSort(a);
    for (int i = 0; i < a.length; i++)
        if (a[i] != i + 1) return i + 1;          // first empty home
    return a.length + 1;                          // 1..n all present
}
```

Set mismatch

variant

```java
// Set mismatch: 1..n with one value doubled and one missing.
// After cyclic sort the only wrong cell is the missing value's home,
// and it holds the extra copy.
static int[] findErrorNums(int[] a) {
    cyclicSort(a);
    for (int i = 0; i < a.length; i++)
        if (a[i] != i + 1) return new int[] {a[i], i + 1};  // {dup, missing}
    return new int[] {-1, -1};
}
```

All duplicates by sign marking

variant

```java
// Find all duplicates (each value 1..n appears once or twice).
// Mark "v seen" by making a[v - 1] negative; the sign is the flag.
static List<Integer> findDuplicates(int[] a) {
    List<Integer> out = new ArrayList<>();
    for (int i = 0; i < a.length; i++) {
        int v = Math.abs(a[i]);        // a[i] may be marked
        if (a[v - 1] < 0) out.add(v);  // second time we see v
        else a[v - 1] = -a[v - 1];
    }
    return out;
}
```

Cost Time `O(n)` Space `O(1)`
Every swap puts one value home for good, so at most n swaps happen in total.

Variants same idea, small change

- Missing number (values 0..n, one missing): `n * (n + 1) / 2 - sum`, or XOR together 0..n and every value.
- All numbers disappeared: sign-mark every value, then report each `i + 1` whose cell stayed positive.
- Values 0..n-1: the home of v is index v, so drop the `- 1`.
- You may not modify the array (find the duplicate number): treat `i -> a[i]` as a linked list and use 5.3 **Fast and slow pointers** (p. 134).

Traps bugs people actually write

- `if` instead of `while`: the value swapped in may also need to go home. `[3, 4, -1, 1]` then returns 1 instead of 2.
- Loop condition `a[i] != i + 1` instead of `a[a[i] - 1] != a[i]`: with two copies of a value, they swap with each other for ever (`[2, 2]`).
- Sign marking with `a[a[i] - 1]` after `a[i]` was made negative gives a negative index. Use `Math.abs(a[i]) - 1`.
- The array is changed. Copy it first if the caller still needs it.

Practise on LeetCode, in this order

- [ ] LC 268 Missing Number — Easy
- [ ] LC 645 Set Mismatch — Easy
- [ ] LC 448 Find All Numbers Disappeared in an Array — Easy
- [ ] LC 442 Find All Duplicates in an Array — Medium
- [ ] LC 41 First Missing Positive — Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

1.10 In-place matrix moves `O(r·c)`

Spot it when the problem looks like this

- "Rotate the image by 90 degrees, in place" on an n × n matrix.
- Return the elements in spiral order, or fill a matrix in spiral order.
- "If an element is 0, set its entire row and column to 0", with an O(1) space follow-up.
- Any grid walk that steps to neighbours (up, down, left, right).

How it works

A 90° clockwise turn sends cell (i, j) to (j, n - 1 - i). Do it in place as two moves. The transpose gives (j, i): for every pair with `j > i`, swap the two cells across the diagonal. Reversing each row then gives (j, n - 1 - i).

Spiral order keeps four boundaries, `top`, `bottom`, `left` and `right`. Walk one side, then move that boundary inward. The loop runs while `top <= bottom && left <= right`, and the two inner checks stop a single leftover row or column from being walked back. Each cell is visited once.

For set zeroes in O(1) space, row 0 and column 0 hold the marks, and two booleans remember whether row 0 and column 0 had a zero of their own. Clear the inner cells first and row 0 and column 0 last; in the other order the marks wipe out the data you still need.

Worked example **rotate a 3 × 3 matrix; spiral order of a 3 × 4 matrix**

```
start      transpose  reverse rows
1 2 3      1 4 7      7 4 1
4 5 6      2 5 8      8 5 2
7 8 9      3 6 9      9 6 3
---- spiral order of [[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]] ----
right along row 0: 1 2 3 4    then top = 1
down along col 3:  8 12       then right = 2
left along row 2:  11 10 9    then bottom = 1
up along col 0:    5          then left = 1
right along row 1: 6 7        then top = 2
down along col 2:  (none)     then right = 1
up along col 1:    (none)     then left = 2
```

Rotate 90° clockwise in place

memorise

```java
// Rotate an n x n matrix 90 degrees clockwise, in place:
// transpose (swap across the main diagonal), then reverse each row.
static void rotate(int[][] m) {
    int n = m.length;
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++) {          // j > i: each pair once
            int t = m[i][j]; m[i][j] = m[j][i]; m[j][i] = t;
        }
    for (int[] row : m)
        for (int l = 0, r = n - 1; l < r; l++, r--) {
            int t = row[l]; row[l] = row[r]; row[r] = t;
        }
}
```

Spiral order with four boundaries

memorise

```java
// Spiral order: walk the outer ring, moving each boundary in after
// its side is done. The ifs stop a leftover row or column being walked back.
static List<Integer> spiralOrder(int[][] m) {
    List<Integer> out = new ArrayList<>();
    int top = 0, bottom = m.length - 1, left = 0, right = m[0].length - 1;
    while (top <= bottom && left <= right) {
        for (int j = left; j <= right; j++) out.add(m[top][j]);         // ->
        top++;
        for (int i = top; i <= bottom; i++) out.add(m[i][right]);       // down
        right--;
        if (top <= bottom) {
            for (int j = right; j >= left; j--) out.add(m[bottom][j]);  // <-
            bottom--;
        }
        if (left <= right) {
            for (int i = bottom; i >= top; i--) out.add(m[i][left]);    // up
            left++;
        }
    }
    return out;
}
```

Set matrix zeroes in O(1) space

variant

```java
// Set matrix zeroes with O(1) extra space: row 0 and column 0 hold the
// marks, and two booleans remember whether they had a zero themselves.
static void setZeroes(int[][] m) {
    int rows = m.length, cols = m[0].length;
    boolean row0 = false, col0 = false;
    for (int j = 0; j < cols; j++) if (m[0][j] == 0) row0 = true;
    for (int i = 0; i < rows; i++) if (m[i][0] == 0) col0 = true;
    for (int i = 1; i < rows; i++)                          // 1. mark
        for (int j = 1; j < cols; j++)
            if (m[i][j] == 0) { m[i][0] = 0; m[0][j] = 0; }
    for (int i = 1; i < rows; i++)                          // 2. clear the inside
        for (int j = 1; j < cols; j++)
            if (m[i][0] == 0 || m[0][j] == 0) m[i][j] = 0;
    if (row0) for (int j = 0; j < cols; j++) m[0][j] = 0;  // 3. marks last
    if (col0) for (int i = 0; i < rows; i++) m[i][0] = 0;
}
```

Cost Time `O(r·c)` Space `O(1)`

Each cell is read or swapped a constant number of times; only a few indexes are stored (plus the spiral's output list).

Variants same idea, small change

- Grid neighbours: declare `int[][] DIRS = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}}` once. For each `d` the neighbour is `(i + d[0], j + d[1])`, if it is inside the grid. The order is clockwise, so a right turn is `k = (k + 1) % 4`.
- Counter-clockwise: transpose, then reverse each column. 180°: reverse each row, then reverse the order of the rows.
- Spiral matrix II (fill 1..n^2 in spiral order): the same four-boundary loop, writing instead of reading.
- Game of life in place: store old and new state in one cell (e.g. 2 = was alive, now dead) and visit the 8 neighbours with an 8-entry DIRS.

Traps bugs people actually write

- Transposing with `j` from 0 instead of `i + 1` swaps every pair twice, so the transpose does nothing and you get a mirror image.
- Dropping the two inner checks in spiral: `[[1, 2, 3]]` gives `[1, 2, 3, 2, 1]`.
- Set zeroes: clearing row 0 before step 2 turns all its cells into marks, so every column is wiped.
- A non-square r × c matrix cannot turn in place: fill `b[j][r - 1 - i] = m[i][j]` in a new c × r array.

Practise on LeetCode, in this order

☐	LC 867	Transpose Matrix	Easy
☐	LC 1886	Determine Whether Matrix Can Be Obtained By Rotation	Easy
☐	LC 48	Rotate Image	Medium
☐	LC 54	Spiral Matrix	Medium
☐	LC 73	Set Matrix Zeroes	Medium
☐	LC 289	Game of Life	Medium

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

Part 2

Two pointers and sliding windows

Two indexes that only move forward (or only move inward) replace a nested loop. Each index moves at most n times, so $O(n^2)$ brute force becomes O(n). Reach for this part whenever the input is an array or string and the question is about a pair, a contiguous range, or an in-place rearrangement.

2.1	**Opposite-end pointers** Sorted array (or one you can sort); find a pair, or the best pair, using both ends.	O(n)	p. 44
2.2	**K-sum (sort + two pointers)** All unique triples or quadruples with a target sum, or the closest triple.	$O(n^2)$	p. 46
2.3	**Read/write pointers (in place)** Remove or compact elements of an array in place and return the new length.	O(n)	p. 49
2.4	**Three-way partition (Dutch flag)** Sort three kinds of values, or split into < p, = p, > p, in one pass.	O(n)	p. 51
2.5	**Merge two sorted sequences** Two sorted arrays or strings: merge, intersect or match them in one pass.	O(m + n)	p. 53
2.6	**Fixed-size sliding window** Every window of exactly k elements: best sum, a count, or an anagram match.	O(n)	p. 55
2.7	**Sliding window (variable size)** Longest or shortest contiguous subarray or substring that satisfies a condition.	O(n)	p. 58
2.8	**Exactly k (two at-most counts)** Count subarrays where a count (distinct values, ones, odds) is exactly k.	O(n)	p. 60
2.9	**Minimum window substring** Shortest substring or subarray that contains every required item.	O(m + n)	p. 62
2.10	**Sliding window maximum (deque)** Max or min of every window, or a DP step that needs the best of the last k.	O(n)	p. 64

2.1 Opposite-end pointers

O(n)

Spot it when the problem looks like this

- The array is sorted, or sorting it does not lose what the question asks for.
- You need a pair with a given sum, difference or product, or the pair that maximises something.
- Moving one end makes the value only bigger or only smaller.

How it works

Start with `l = 0` and `r = n - 1`. Look at the pair, then move the end that cannot be part of a better answer. In a sorted array, if `a[l] + a[r]` is too small, `a[l]` is too small to pair with anything left of `r`, so drop it with `l++`. Each step throws away one element for good, so there are at most n steps.

For *container with most water* the same argument works on heights: the shorter wall limits every container that uses it, so move the shorter wall inward.

Worked example a = [1, 3, 4, 6, 9], target = 10

```
l  r  a[l]+a[r]  move
0  4  1 + 9 = 10 found: (0, 4)
---- target = 13 ----
0  4  1 + 9 = 10 too small -> l++
1  4  3 + 9 = 12 too small -> l++
2  4  4 + 9 = 13 found: (2, 4)
```

Pair with a target sum

memorise

```java
// Sorted array: find indices i < j with a[i] + a[j] == target.
static int[] pairWithSum(int[] a, int target) {
    int l = 0, r = a.length - 1;
    while (l < r) {
        long s = (long) a[l] + a[r];
        if (s == target) return new int[] {l, r};
        if (s < target) l++;          // need a bigger sum
        else r--;                     // need a smaller sum
    }
    return new int[] {-1, -1};
}
```

Container with most water

variant

```java
// Container with most water: always move the shorter wall inward.
static int maxArea(int[] h) {
    int l = 0, r = h.length - 1, best = 0;
    while (l < r) {
        best = Math.max(best, Math.min(h[l], h[r]) * (r - l));
        if (h[l] < h[r]) l++;
        else r--;
    }
    return best;
}
```

Cost Time `O(n)` Space `O(1)` Each step moves l or r once; add O(n log n) if you have to sort first.

Variants same idea, small change

- Count pairs with sum < target: when `a[l] + a[r] < target`, all of `l+1..r` pair with `l`, so add `r - l` and move `l`.
- Valid palindrome: skip non-alphanumerics, then compare `s[l]` and `s[r]` ignoring case.
- Squares of a sorted array: the largest square is at one of the two ends; fill the answer from the back.
- Need indices of an unsorted array? Use a hash map instead (1.2 **Seen-before hash map** (p. 21)); sorting loses the original indices.

Traps bugs people actually write

- Use `l < r`, not `l <= r`; one element cannot pair with itself.
- `a[l] + a[r]` can overflow `int` near $2 \cdot 10^9$; add as `long`.
- Sorting changes the indices. If the answer needs original positions, sort an index array or pairs.

Practise on LeetCode, in this order

☐	LC 125	Valid Palindrome	Easy
☐	LC 977	Squares of a Sorted Array	Easy
☐	LC 167	Two Sum II - Input Array Is Sorted	Medium
☐	LC 11	Container With Most Water	Medium
☐	LC 881	Boats to Save People	Medium

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

2.2 K-sum (sort + two pointers)

$O(n^2)$

Spot it when the problem looks like this

- *Return all unique triplets* (or quadruplets) with sum X; the answer must not contain the same tuple twice.
- n is up to about 3000 for 3Sum and 200 for 4Sum, so $O(n^2)$ and $O(n^3)$ are the targets.
- The answer is values, not indices, so you are free to sort.

How it works

Sort, fix the first element `a[i]`, and run the pair search from 2.1 Opposite-end pointers (p. 44) on `a[i+1..n-1]` for the sum `-a[i]`. The pair search is O(n), so 3Sum costs $O(n^2)$. 4Sum fixes two elements with two loops: $O(n^3)$.

Sorting also makes duplicates easy to skip. Equal values sit next to each other, so a tuple can repeat only if some level picks the same value twice. Skip `a[i]` when it equals `a[i-1]`: every triple it could start was already found from `a[i-1]`. After a hit, move `l` past copies of `a[l]`. `r` needs no skip: with a new, larger `a[l]`, the old `a[r]` makes the sum too big and the search moves it.

For *closest*, keep the best sum seen. When the sum is below the target, every smaller `r` is further below, so `a[l]` has no better partner and `l++` is safe.

Worked example a = [-1, 0, 1, 2, -1, -4], sorted to [-4, -1, -1, 0, 1, 2]

```
i  a[i]  l  r  sum  move
0   -4   1  5   -3  too small -> l++
0   -4   2  5   -3  too small -> l++
0   -4   3  5   -2  too small -> l++
0   -4   4  5   -1  too small -> l++
1   -1   2  5    0  found [-1, -1, 2]; l++, r--
1   -1   3  4    0  found [-1, 0, 1]; l++, r--
2   -1   skip: same value as a[1]
3    0   4  5    3  too big -> r--
```

3Sum without duplicate triples

memorise

```java
// All unique triples with a[i] + a[l] + a[r] == 0 (LC 15).
static List<List<Integer>> threeSum(int[] a) {
    Arrays.sort(a);
    List<List<Integer>> out = new ArrayList<>();
    int n = a.length;
    for (int i = 0; i < n - 2; i++) {
        if (i > 0 && a[i] == a[i - 1]) continue;   // 1. same first value: skip
        if (a[i] > 0) break;                        // all three would be > 0
        int l = i + 1, r = n - 1;
        while (l < r) {                             // 2. pair search on a[i+1..]
            int s = a[i] + a[l] + a[r];
            if (s < 0) l++;
            else if (s > 0) r--;
            else {
                out.add(Arrays.asList(a[i], a[l], a[r]));
                l++;
                r--;
                while (l < r && a[l] == a[l - 1]) l++;  // 3. same second value
            }
        }
    }
    return out;
}
```

3Sum closest

variant

```java
// Sum of three elements closest to target (LC 16).
static int threeSumClosest(int[] a, int target) {
    Arrays.sort(a);
    int best = a[0] + a[1] + a[2];
    for (int i = 0; i < a.length - 2; i++) {
        int l = i + 1, r = a.length - 1;
        while (l < r) {
            int s = a[i] + a[l] + a[r];
            if (Math.abs(s - target) < Math.abs(best - target)) best = s;
            if (s < target) l++;
            else if (s > target) r--;
            else return s;                          // exact: cannot do better
        }
    }
    return best;
}
```

4Sum with long sums

variant

```java
// All unique quadruples with sum == target (LC 18). Values reach 1e9: use long.
static List<List<Integer>> fourSum(int[] a, int target) {
    Arrays.sort(a);
    List<List<Integer>> out = new ArrayList<>();
    int n = a.length;
    for (int i = 0; i < n - 3; i++) {
        if (i > 0 && a[i] == a[i - 1]) continue;
        for (int j = i + 1; j < n - 2; j++) {
            if (j > i + 1 && a[j] == a[j - 1]) continue;
            int l = j + 1, r = n - 1;
            while (l < r) {
                long s = (long) a[i] + a[j] + a[l] + a[r];
                if (s < target) l++;
                else if (s > target) r--;
                else {
                    out.add(Arrays.asList(a[i], a[j], a[l], a[r]));
                    l++;
                    r--;
                    while (l < r && a[l] == a[l - 1]) l++;
                }
            }
        }
    }
    return out;
}
```

Each extra fixed index adds one loop, one factor of n, and the same skip rule.

Cost Time $O(n^2)$ Space $O(1)$

Sorting is O(n log n); each fixed i runs one O(n) pair search. 4Sum is $O(n^3)$. Space excludes the output.

Variants same idea, small change

- Count triples with sum < target: when `a[i] + a[l] + a[r] < target`, every `r' ≤ r` also works with this `l`, so add `r - l` and move `l`.
- Valid triangle number: sort, fix the largest side `a[k]`, and count pairs `l < r < k` with `a[l] + a[r] > a[k]` the same way.
- 3Sum with multiplicity (count index triples): on a hit with `a[l] < a[r]`, multiply the run lengths of the two values; if `a[l] == a[r]`, add C(r – l + 1, 2).
- General k-sum: recurse, fixing one index per level, until k = 2 and the pair search runs.

Traps bugs people actually write

- Skipping with `a[i] == a[i+1]` instead of `a[i] == a[i-1]` throws away triples that use both copies: on the sample it loses `[-1, -1, 2]`.
- In 4Sum, `a[i] + a[j] + a[l] + a[r]` in `int` overflows: four times 10^9 wraps to –294967296 and matches that target. Cast the first term to `long`.
- Copying the 3Sum early exit into 4Sum as `if (a[i] > target) break;` is wrong for negative targets: `[-5, -4, -3, -2]` with target –14 has an answer although –5 > –14.

Practise on LeetCode, in this order

☐	LC 15	3Sum	Medium
☐	LC 16	3Sum Closest	Medium
☐	LC 18	4Sum	Medium
☐	LC 611	Valid Triangle Number	Medium
☐	LC 923	3Sum With Multiplicity	Medium

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

2.3 Read/write pointers (in place) O(n)

Spot it when the problem looks like this

- *In place*, *O(1) extra memory*, *return k*: the judge reads only the first k elements.
- The elements you keep must stay in their original order.
- Remove duplicates, remove a value, move zeroes (or some other class of elements) to the end.

How it works

`r` reads every element once; `w` is where the next kept element goes. When `a[r]` should stay, copy it to `a[w]` and move `w`. The invariant is that `a[0..w)` is the answer for the prefix `a[0..r)`. `w` moves at most once per step of `r`, so `w ≤ r`: you only ever overwrite cells you have already read.

To keep each value at most k times in a sorted array, compare with the **write** side. The kept part is sorted and `a[r]` is at least as big as all of it, so if `a[w-k] == a[r]`, then `a[w-k..w)` are all copies of `a[r]` and there are already k of them. One pass, O(1) space, and k = 1 and k = 2 are the same code.

Move zeroes swaps instead of copying, so the zeros collect in `a[w..r)` and nothing is left to fill in at the end.

Worked example **keepAtMostK(a, 2), a = [0, 0, 1, 1, 1, 1, 2, 3, 3]**

r	a[r]	a[w-2]	keep?	w	a[0..w)
0	0	-	yes	1	[0]
1	0	-	yes	2	[0, 0]
2	1	0	yes	3	[0, 0, 1]
3	1	0	yes	4	[0, 0, 1, 1]
4	1	1	no	4	[0, 0, 1, 1]
5	1	1	no	4	[0, 0, 1, 1]
6	2	1	yes	5	[0, 0, 1, 1, 2]
7	3	1	yes	6	[0, 0, 1, 1, 2, 3]
8	3	2	yes	7	[0, 0, 1, 1, 2, 3, 3]

Keep each value at most k times

memorise

```java
// Sorted array: keep each value at most k times, in place; return the new length.
// k = 1 is LC 26 (remove duplicates), k = 2 is LC 80.
static int keepAtMostK(int[] a, int k) {
    int w = 0;                          // a[0..w) is the answer for a[0..r)
    for (int r = 0; r < a.length; r++) {
        if (w < k || a[w - k] != a[r])  // fewer than k copies of a[r] kept so far
            a[w++] = a[r];
    }
    return w;
}
```

Remove a value; move zeroes

variant

```java
// Remove every val (LC 27); the kept values stay in order.
static int removeElement(int[] a, int val) {
    int w = 0;
    for (int r = 0; r < a.length; r++)
        if (a[r] != val) a[w++] = a[r];
    return w;
}

// Move zeroes to the end, keep the order of the rest (LC 283).
// Invariant: a[0..w) = non-zeros in order, a[w..r) = zeros.
static void moveZeroes(int[] a) {
    int w = 0;
    for (int r = 0; r < a.length; r++) {
        if (a[r] != 0) {
            int t = a[w]; a[w] = a[r]; a[r] = t;
            w++;
        }
    }
}
```

Cost Time `O(n)` Space `O(1)`
One pass; each element is read once and written at most once (move zeroes swaps, two writes).

Variants same idea, small change

- Remove all adjacent duplicates in a string: treat `a[0..w)` as a stack. If `a[r]` equals `a[w-1]`, do `w--`; otherwise write it.
- String compression: `r` runs to the end of each block of equal chars; `w` writes the char, then the digits of the block length.
- Duplicate zeros (fixed length): count the final length first, then write from the back, as in **2.5 Merge two sorted sequences** (p. 53).
- Order of the kept elements does not matter (LC 27 allows it): replace `a[i]` with `a[--n]` and test `a[i]` again. Fewer writes when few elements go.

Traps bugs people actually write

- For at most two copies, comparing on the read side, `a[r] != a[r-2]`, reads cells already overwritten: `[1, 1, 1, 2, 2, 3]` gives `[1, 1, 2, 3]` instead of `[1, 1, 2, 2, 3]`.
- Move zeroes by copying (`a[w++] = a[r]`) needs a second loop that sets `a[w..n)` to 0. Without it the old values stay at the end.
- Return `w` and look only at `a[0..w)`; the cells after `w` hold leftovers, not zeros or blanks.

Practise on LeetCode, in this order

☐	LC 27	Remove Element	Easy
☐	LC 26	Remove Duplicates from Sorted Array	Easy
☐	LC 283	Move Zeroes	Easy
☐	LC 1047	Remove All Adjacent Duplicates In String	Easy
☐	LC 80	Remove Duplicates from Sorted Array II	Medium
☐	LC 443	String Compression	Medium

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

2.4 Three-way partition (Dutch flag) `O(n)`

Spot it when the problem looks like this

- Values are only 0, 1 and 2 (or three classes, like negative, zero, positive) and the fix must be in place.
- *One pass, constant extra space, without the library sort.*
- Partitioning around a pivot when many values repeat (quicksort or quickselect on duplicates).

How it works

Keep four regions: `a[0..low)` holds 0s, `a[low..mid)` 1s, `a[mid..high]` is not looked at yet, and `a(high..n-1]` holds 2s. Each step classifies `a[mid]`, the first unseen value. A 1 is already in place, so `mid` moves on. A 0 swaps with `a[low]`; that cell held a 1 (or `low == mid`), so `low` and `mid` both move on.

A 2 swaps with `a[high]` and only `high` moves. The value that lands on `mid` came from the unseen region, so it has not been classified; `mid++` would skip it. Every step shrinks the unseen region by one cell, so the loop runs exactly n times, and `mid <= high` stops it when nothing is unseen.

With a pivot p the same loop sends `< p` left and `> p` right, and leaves the `== p` block in the middle.

Worked example sortColors(a), a = [2, 0, 2, 1, 1, 0]

```
low mid high  a[mid]  action                   array after
 0   0    5     2     swap(mid, high), high--  [0, 0, 2, 1, 1, 2]
 0   0    4     0     swap(low, mid), both++   [0, 0, 2, 1, 1, 2]
 1   1    4     0     swap(low, mid), both++   [0, 0, 2, 1, 1, 2]
 2   2    4     2     swap(mid, high), high--  [0, 0, 1, 1, 2, 2]
 2   2    3     1     mid++                    [0, 0, 1, 1, 2, 2]
 2   3    3     1     mid++                    [0, 0, 1, 1, 2, 2]
end: low = 2, mid = 4, high = 3; nothing is unseen
```

Sort colors (0, 1, 2)

memorise

```java
// Sort 0s, 1s and 2s in one pass (LC 75).
// Invariant: a[0..low) = 0, a[low..mid) = 1, a[mid..high] unseen, a(high..] = 2.
static void sortColors(int[] a) {
    int low = 0, mid = 0, high = a.length - 1;
    while (mid <= high) {
        if (a[mid] == 0) swap(a, low++, mid++);      // a[low] was a 1: both move
        else if (a[mid] == 2) swap(a, mid, high--);  // new a[mid] unseen: stay
        else mid++;
    }
}

static void swap(int[] a, int i, int j) { int t = a[i]; a[i] = a[j]; a[j] = t; }
```

Partition around a pivot

variant

```java
// Three-way partition around p. Returns {lt, gt} with
// a[0..lt) < p, a[lt..gt] == p, a(gt..] > p. Same loop, p plays the role of 1.
static int[] partition3(int[] a, int p) {
    int lt = 0, i = 0, gt = a.length - 1;
    while (i <= gt) {
        if (a[i] < p) swap(a, lt++, i++);
        else if (a[i] > p) swap(a, i, gt--);
        else i++;
    }
    return new int[] {lt, gt};
}
```

Cost Time `O(n)` Space `O(1)`

Each step moves mid up or high down, so there are n steps; all swaps are in place.

Variants same idea, small change

- Two groups only (sort by parity, negatives first): one write pointer, and `swap(a, w++, i)` when `a[i]` belongs on the left.
- 3-way quicksort: partition around a random pivot, then recurse only into `a[0..lt)` and `a(gt..]`. The `== p` block is finished, so many equal values cost nothing extra.
- Quickselect with this partition (3.9 **Quickselect** (p. 90)): if k falls inside `[lt, gt]`, the answer is p.
- Only three known values and two passes are allowed? Count them and write them back.

Traps bugs people actually write

- `mid++` after swapping with `high` skips the value that came back: `[1, 2, 0]` ends as `[1, 0, 2]`.
- `while (mid < high)` stops with one cell unseen: `[1, 0]` stays `[1, 0]`.
- The partition is not stable. If the order inside each group must be kept (LC 2161), collect the three groups in separate passes.

Practise on LeetCode, in this order

☐	`LC 905`	Sort Array By Parity	Easy
☐	`LC 75`	Sort Colors	Medium
☐	`LC 912`	Sort an Array	Medium
☐	`LC 324`	Wiggle Sort II	Medium

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

2.5 Merge two sorted sequences `O(m + n)`

Spot it when the problem looks like this

- Both inputs are sorted, or the order inside each input is what matters (a subsequence).
- *Merge into nums1*, which has empty slots at the end.
- Common elements of two arrays, or *is s a subsequence of t*, with t up to 10^4 or more.

How it works

Put `i` on one sequence and `j` on the other. Compare `a[i]` with `b[j]` and advance the pointer whose element is now decided. In a merge, the smaller one is the next output. In an intersection, the smaller one is smaller than everything left in the other array, so it has no partner. Each step uses up at least one element: O(m + n).

To merge into `nums1` in place, fill from the back with the larger element. The write index is `k = i + j + 1`, which stays above `i` while `j ≥ 0`, so no unread `nums1` value is overwritten. When `nums2` runs out, what is left of `nums1` is already in place.

Is subsequence is the same walk: match `s[i]` at its earliest place in `t`, which leaves as much of `t` as possible for the rest of `s`.

Worked example **merge(nums1 = [1, 2, 3, 0, 0, 0], m = 3, nums2 = [2, 5, 6], n = 3)**

```
i  j  k  nums1[i]  nums2[j]  write          nums1 after
2  2  5     3         6      nums1[5] = 6   [1, 2, 3, 0, 0, 6]
2  1  4     3         5      nums1[4] = 5   [1, 2, 3, 0, 5, 6]
2  0  3     3         2      nums1[3] = 3   [1, 2, 3, 3, 5, 6]
1  0  2     2         2      nums1[2] = 2   [1, 2, 2, 3, 5, 6]
j = -1: stop. nums1[0..1] = [1, 2] never had to move.
```

Merge sorted array from the back

memorise

```java
// nums1 has length m + n; its last n slots are free (LC 88).
// Fill from the back: k = i + j + 1 > i, so no unread nums1 value is overwritten.
static void merge(int[] nums1, int m, int[] nums2, int n) {
    int i = m - 1, j = n - 1, k = m + n - 1;
    while (j >= 0) {                // once nums2 is used up, nums1 is in place
        if (i >= 0 && nums1[i] > nums2[j]) nums1[k--] = nums1[i--];
        else nums1[k--] = nums2[j--];
    }
}
```

Intersection and is-subsequence

variant

```java
// Common values of two sorted arrays, with repeats (LC 350 on sorted input).
static int[] intersect(int[] a, int[] b) {
    int[] out = new int[Math.min(a.length, b.length)];
    int i = 0, j = 0, k = 0;
    while (i < a.length && j < b.length) {
        if (a[i] < b[j]) i++;           // a[i] is smaller than all of b[j..]
        else if (a[i] > b[j]) j++;
        else { out[k++] = a[i]; i++; j++; }
    }
    return Arrays.copyOf(out, k);
}

// Is s a subsequence of t (LC 392)? Match each char of s as early as possible.
static boolean isSubsequence(String s, String t) {
    int i = 0;
    for (int j = 0; j < t.length() && i < s.length(); j++)
        if (s.charAt(i) == t.charAt(j)) i++;
    return i == s.length();
}
```

Cost Time `O(m + n)` Space `O(1)`

Every step moves i or j (or both). The intersection's output array is the only extra memory.

Variants same idea, small change

- Unique intersection (LC 349): on a match, also skip every copy of that value in both arrays.
- Two sorted linked lists: the same comparison with a dummy head (**5.1 Dummy head node** (p. 128)). k sorted inputs: a heap of the k fronts (**4.9 K-way merge with a heap** (p. 121)).
- Interval list intersections: the overlap of the two current intervals is `[max(starts), min(ends)]`; advance the one that ends first.
- Many queries against one long t: store the positions of each char of t and binary search the next one after the current position (**3.1 Lower bound and upper bound** (p. 68)).

Traps bugs people actually write

- Merging into nums1 from the front overwrites values you have not read yet. Go from the back.
- `while (i >= 0 && j >= 0)` and nothing after it loses leftover nums2 values (leftover nums1 values are fine). `while (j >= 0)` covers both cases.
- With `List<Integer>` inputs, `a.get(i) == b.get(j)` compares objects and can be false for equal values outside -128..127. Unbox to `int` or use `.equals`.

Practise on LeetCode, in this order

☐	LC 88	Merge Sorted Array	Easy
☐	LC 392	Is Subsequence	Easy
☐	LC 350	Intersection of Two Arrays II	Easy
☐	LC 986	Interval List Intersections	Medium
☐	LC 524	Longest Word in Dictionary through Deleting	Medium

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

2.6 Fixed-size sliding window

`O(n)`

Spot it when the problem looks like this

- *Subarray of length k*, *substring of size k*, *any k consecutive days*.
- *Anagram* or *permutation of p* inside s: the window length is `p.length()`.
- n and k up to 10^5, so recomputing each window from scratch (O(nk)) is too slow.

How it works

Keep the window `[r-k+1, r]` and its state (a sum, a count, an `int[26]`). When `r` moves, add `a[r]` and remove `a[r-k]`, the one element that left. Each update is O(1), so all n − k + 1 windows cost O(n) in total. Record only once `r ≥ k - 1`; before that the window is not full.

For anagrams, compare letter counts. Comparing two `int[26]` arrays costs 26 per step; a match counter costs O(1). `match` is the number of letters whose window count equals their count in p. Changing one

letter's count can only change that letter's equality, so each add or remove moves `match` by –1, 0 or +1. The window is an anagram of p exactly when `match == 26`.

Worked example s = "cbaebabacd", p = "abc"; match starts at 23 (letters not in p)

r	in	out	window	match	anagram?
0	c	-	c	24	
1	b	-	cb	25	
2	a	-	cba	26	yes, start 0
3	e	c	bae	24	
4	b	b	aeb	24	
5	a	a	eba	24	
6	b	e	bab	24	
7	a	b	aba	24	
8	c	a	bac	26	yes, start 6
9	d	b	acd	24	

Max sum of k consecutive elements

memorise

```java
// Max sum of k consecutive elements (LC 643 returns this / k as a double).
static long maxSumOfK(int[] a, int k) {
    long sum = 0, best = Long.MIN_VALUE;
    for (int r = 0; r < a.length; r++) {
        sum += a[r];                                  // 1. add the right end
        if (r >= k) sum -= a[r - k];                  // 2. drop the one that left
        if (r >= k - 1) best = Math.max(best, sum);   // 3. window is full
    }
    return best;
}
```

All anagrams with a match counter

memorise

```java
// Start of every anagram of p in s (LC 438). Lowercase letters only.
static List<Integer> findAnagrams(String s, String p) {
    List<Integer> out = new ArrayList<>();
    int k = p.length();
    int[] need = new int[26], have = new int[26];
    for (char c : p.toCharArray()) need[c - 'a']++;
    int match = 0;                        // letters c with have[c] == need[c]
    for (int c = 0; c < 26; c++) if (need[c] == 0) match++;
    for (int r = 0; r < s.length(); r++) {
        match += change(have, need, s.charAt(r) - 'a', +1);
        if (r >= k) match += change(have, need, s.charAt(r - k) - 'a', -1);
        if (r >= k - 1 && match == 26) out.add(r - k + 1);
    }
    return out;
}

// have[c] += d; returns how the number of matching letters changed (-1, 0, +1).
static int change(int[] have, int[] need, int c, int d) {
    int before = have[c] == need[c] ? 1 : 0;
    have[c] += d;
    return (have[c] == need[c] ? 1 : 0) - before;
}
```

Max vowels in a window of k

example

```java
// Most vowels in any substring of length k (LC 1456).
static int maxVowels(String s, int k) {
    int cnt = 0, best = 0;
    for (int r = 0; r < s.length(); r++) {
        if (isVowel(s.charAt(r))) cnt++;
        if (r >= k && isVowel(s.charAt(r - k))) cnt--;
        if (r >= k - 1) best = Math.max(best, cnt);
    }
    return best;
}

static boolean isVowel(char c) { return "aeiou".indexOf(c) >= 0; }
```

Cost Time `O(n)` Space `O(1)`

Each step does one add and one remove; the count arrays have a fixed size of 26.

Variants same idea, small change

- Permutation in string (LC 567): the same loop; return true at the first `match == 26`.
- Grumpy bookstore owner: count the customers who are happy anyway, then slide the window over the ones you could win back.
- Max sum of a window whose k values are all distinct: keep a count map next to the sum; the window counts only when `map.size() == k`.
- Max or min of each window: see **2.10 Sliding window maximum (deque)** (p. 64).

Traps bugs people actually write

- Starting with `best = 0` is wrong when every window is negative: LC 643 with `[-1]`, k = 1 returns 0 instead of −1. Start at `Long.MIN_VALUE`.
- `if (r > k) sum -= a[r - k]` is off by one: `a[0]` is never removed, so every window from then on has k + 1 elements.
- LC 643 wants a `double`, and `sum / k` on integers truncates (−7 / 2 is −3, not −3.5). Divide by `(double) k`.

Practise on LeetCode, in this order

☐	LC 643	Maximum Average Subarray I	Easy
☐	LC 2379	Minimum Recolors to Get K Consecutive Black Blocks	Easy
☐	LC 1456	Maximum Number of Vowels in a Substring of Given Length	Medium
☐	LC 567	Permutation in String	Medium
☐	LC 438	Find All Anagrams in a String	Medium
☐	LC 1052	Grumpy Bookstore Owner	Medium

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

2.7 Sliding window (variable size)

O(n)

Spot it when the problem looks like this

- *Longest* or *shortest* subarray or substring with a rule: at most k distinct, no repeats, sum ≥ X.
- Shrinking a valid window keeps it valid (or growing an invalid one keeps it invalid).
- All numbers are non-negative, if the rule is about a sum.

How it works

Move `r` one step at a time and add `a[r]` to the window's state (a count map, a sum). While `[l, r]` breaks the rule, remove `a[l]` and move `l` right. After that loop, `[l, r]` is the longest valid window that ends at `r`.

For the **shortest** window, flip it: while the window *is* valid, record its length and shrink. Both pointers only move forward, so the total work is O(n) even though there is a loop inside a loop.

Worked example a = [1, 2, 1, 3, 3, 2], at most 2 distinct

```
r  a[r]  window after shrinking        best
0    1   [1]                           1
1    2   [1, 2]                        2
2    1   [1, 2, 1]                     3
3    3   [1, 3]      (l moved 0 -> 2)  3
4    3   [1, 3, 3]                     3
5    2   [3, 3, 2]   (l moved 2 -> 3)  3
```

Longest valid window

memorise

```java
// Longest subarray with at most k distinct values.
// Grow r; while the window breaks the rule, shrink l; then record.
static int longestAtMostKDistinct(int[] a, int k) {
    Map<Integer, Integer> cnt = new HashMap<>();
    int best = 0;
    for (int l = 0, r = 0; r < a.length; r++) {
        cnt.merge(a[r], 1, Integer::sum);              // 1. add a[r]
        while (cnt.size() > k) {                       // 2. shrink while invalid
            if (cnt.merge(a[l], -1, Integer::sum) == 0) cnt.remove(a[l]);
            l++;
        }
        best = Math.max(best, r - l + 1);              // 3. [l, r] is valid
    }
    return best;
}
```

Shortest valid window

memorise

```java
// Shortest subarray with sum >= target (all values positive).
// Record inside the loop, because a valid window gets shorter as l moves.
static int shortestSumAtLeast(int[] a, long target) {
    int best = Integer.MAX_VALUE;
    long sum = 0;
    for (int l = 0, r = 0; r < a.length; r++) {
        sum += a[r];
        while (sum >= target) {
            best = Math.min(best, r - l + 1);
            sum -= a[l++];
        }
    }
    return best == Integer.MAX_VALUE ? 0 : best;
}
```

Cost Time `O(n)` Space `O(k)`

l and r each move at most n times; the map holds at most k + 1 keys.

Variants same idea, small change

- Longest substring without repeats: the rule is "every count ≤ 1"; shrink while `cnt[c] > 1`.
- Max consecutive ones with k flips: the rule is "zeros in window ≤ k".
- Count windows instead of measuring them: add `r - l + 1` for each `r` (every valid window ending at r). See 2.8 **Exactly k (two at-most counts)** (p. 60).
- The window must contain all of some set (minimum window substring): see 2.9 **Minimum window substring** (p. 62).

Traps bugs people actually write

- With negative numbers a sum rule is not monotone, so the window breaks. Use prefix sums (1.4 **Prefix sum + hash map** (p. 26)).
- When a count drops to 0, remove the key; otherwise `map.size()` is wrong.
- Record the answer at the right moment: after the shrink loop for *longest*, inside it for *shortest*.

Practise on LeetCode, in this order

☐	LC 3	Longest Substring Without Repeating Characters	Medium
☐	LC 209	Minimum Size Subarray Sum	Medium
☐	LC 1004	Max Consecutive Ones III	Medium
☐	LC 424	Longest Repeating Character Replacement	Medium
☐	LC 904	Fruit Into Baskets	Medium

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

2.8 Exactly k (two at-most counts) O(n)

Spot it when the problem looks like this

- *Number of subarrays* with *exactly k* distinct integers, or with sum exactly `goal` in a 0/1 array.
- You count windows instead of measuring one, and n is $2 \cdot 10^4$ or more, so trying every subarray is too slow.
- The *at most k* version of the rule is monotone: shrinking a valid window keeps it valid.

How it works

Fix the right end `r`. The starts that give *at most* k form a range `[l, r]`: the window from **2.7 Sliding window (variable size)** (p. 58) finds the smallest such `l`, and all `r - l + 1` starts count. The starts that give *exactly* k form a range in the middle, `[l1, l2]`, and one window cannot track both of its ends.

So count a difference. A window with exactly k is a window with at most k that does not have at most k – 1: `exactly(k) = atMost(k) - atMost(k - 1)`. That is two O(n) passes. The same works for sums of non-negative numbers, and *nice subarrays* (exactly k odd numbers) becomes a 0/1 sum after `x -> x & 1`.

Worked example a = [1, 2, 1, 2, 3], exactly 2 distinct

r	a[r]	at most 2: [l..r]	+r-l+1	at most 1: [l..r]	+r-l+1
0	1	[1]	1	[1]	1
1	2	[1, 2]	2	[2]	1
2	1	[1, 2, 1]	3	[1]	1
3	2	[1, 2, 1, 2]	4	[2]	1
4	3	[2, 3]	2	[3]	1
total			12		5

exactly 2 = 12 - 5 = 7

Exactly k distinct values

memorise

```java
// Subarrays with exactly k distinct values (LC 992).
static long subarraysWithKDistinct(int[] a, int k) {
    return atMostK(a, k) - atMostK(a, k - 1);
}

// At most k distinct: the 2.7 window, but count every window ending at r.
static long atMostK(int[] a, int k) {
    if (k < 0) return 0;
    Map<Integer, Integer> cnt = new HashMap<>();
    long count = 0;
    for (int l = 0, r = 0; r < a.length; r++) {
        cnt.merge(a[r], 1, Integer::sum);
        while (cnt.size() > k) {
            if (cnt.merge(a[l], -1, Integer::sum) == 0) cnt.remove(a[l]);
            l++;
        }
        count += r - l + 1;            // [l..r], [l+1..r], ..., [r..r]
    }
    return count;
}
```

Binary sum and nice subarrays

variant

```java
// Binary array: subarrays with sum exactly goal (LC 930).
static long numSubarraysWithSum(int[] a, int goal) {
    return atMostSum(a, goal) - atMostSum(a, goal - 1);
}

static long atMostSum(int[] a, int s) {          // needs a[i] >= 0
    if (s < 0) return 0;                          // goal = 0 asks for atMost(-1)
    long count = 0, sum = 0;
    for (int l = 0, r = 0; r < a.length; r++) {
        sum += a[r];
        while (sum > s) sum -= a[l++];
        count += r - l + 1;
    }
    return count;
}

// Subarrays with exactly k odd numbers (LC 1248): map x -> x & 1, reuse.
static long numberOfNiceSubarrays(int[] a, int k) {
    int[] odd = new int[a.length];
    for (int i = 0; i < a.length; i++) odd[i] = a[i] & 1;
    return numSubarraysWithSum(odd, k);
}
```

Cost Time `O(n)` Space `O(k)`

Two passes of the at-most window, each moving l and r at most n times; the map holds at most k + 1 keys.

Variants same idea, small change

- *At least k* (substrings containing all three letters): `total - atMost(k - 1)`, or shrink while the window is valid and then add `l`, the number of starts that still work.
- Exactly k plus a second rule (every vowel and exactly k consonants): `atLeast(k) - atLeast(k + 1)`, each a window that also checks the vowels.
- Sum exactly k with negative numbers: no window works; count prefix sums in a hash map (1.4 **Prefix sum + hash map** (p. 26)).
- Count vowel substrings (only vowels, all five present): inside each run of vowels, count windows with exactly 5 distinct letters.

Traps bugs people actually write

- `goal = 0` asks for `atMost(-1)`. Without `if (s < 0) return 0`, the shrink loop runs `l` past `r` and throws `ArrayIndexOutOfBoundsException`.
- Adding 1 per `r` instead of `r - l + 1` counts only the longest window ending at `r`.
- The count can reach $n(n+1)/2$, which is $5 \cdot 10^9$ for $n = 10^5$. Keep it in a `long`; cast at the end if LeetCode wants an `int`.

Practise on LeetCode, in this order

- [ ] LC 2062 Count Vowel Substrings of a String — Easy
- [ ] LC 930 Binary Subarrays With Sum — Medium
- [ ] LC 1248 Count Number of Nice Subarrays — Medium
- [ ] LC 1358 Number of Substrings Containing All Three Characters — Medium
- [ ] LC 3306 Count of Substrings Containing Every Vowel and K Consonants II — Medium
- [ ] LC 992 Subarrays with K Different Integers — Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

2.9 Minimum window substring

`O(m + n)`

Spot it when the problem looks like this

- *Smallest substring of s that contains all characters of t*, duplicates included.
- *Shortest subarray that covers* a set: one element from each of k lists, every distinct value, every letter.
- Growing a valid window keeps it valid, so shrinking from the left is the only question.

How it works

`need[c]` counts c in t, and `required` is the number of distinct chars in t. `formed` counts the chars c of t with `have[c] >= need[c]`. It goes up only when `have[c]` reaches `need[c]` exactly, and down only when it drops below, so extra copies and chars not in t never change it. The window is valid exactly when `formed == required`.

Grow `r`. While the window is valid, **record it first, then shrink**: the current `[l, r]` is a candidate, and removing `s[l]` may break it. When the loop exits, `l - 1` was the last start that worked for this `r`, so the shortest valid window ending at `r` has been recorded. Both pointers only move forward, so the cost is O(m + n) for m = |s| and n = |t|, with two `int[128]` arrays.

Worked example s = "ADOBECODEBANC", t = "ABC" (* = new best)

```
r      s[r]   formed  recorded while valid, then shrunk             l after
0..4   ADOBE  2/3     -                                            0
5      C      3/3     ADOBEC*                                      1
6..9   ODEB   2/3     -                                            1
10     A      3/3     DOBECODEBA OBECODEBA BECODEBA ECODEBA CODEBA  6
11     N      2/3     -                                            6
12     C      3/3     ODEBANC DEBANC EBANC* BANC*                  10
Each shrink stops when a needed char leaves: A at l = 0,
C at l = 5, B at l = 9. Answer: "BANC".
```

Need, have and formed == required

memorise

```java
// Shortest substring of s that contains every char of t, counts included (LC 76).
static String minWindow(String s, String t) {
    int[] need = new int[128], have = new int[128];
    int required = 0;                       // distinct chars in t
    for (char c : t.toCharArray()) if (need[c]++ == 0) required++;
    int formed = 0;                         // chars c with have[c] >= need[c]
    int bestLen = Integer.MAX_VALUE, bestL = 0;
    for (int l = 0, r = 0; r < s.length(); r++) {
        char c = s.charAt(r);
        if (++have[c] == need[c]) formed++;       // 1. c is now covered
        while (formed == required) {              // 2. valid: record, then shrink
            if (r - l + 1 < bestLen) { bestLen = r - l + 1; bestL = l; }
            char d = s.charAt(l++);
            if (have[d]-- == need[d]) formed--;   // d drops below what t needs
        }
    }
    if (bestLen == Integer.MAX_VALUE) return "";
    return s.substring(bestL, bestL + bestLen);
}
```

One array and a missing counter

variant

```java
// Same window with one array: cnt[c] = copies of c still needed (< 0 = spare).
static String minWindowShort(String s, String t) {
    int[] cnt = new int[128];
    for (char c : t.toCharArray()) cnt[c]++;
    int missing = t.length(), bestL = 0, bestLen = Integer.MAX_VALUE;
    for (int l = 0, r = 0; r < s.length(); r++) {
        if (cnt[s.charAt(r)]-- > 0) missing--;       // a needed copy came in
        while (missing == 0) {
            if (r - l + 1 < bestLen) { bestLen = r - l + 1; bestL = l; }
            if (++cnt[s.charAt(l++)] > 0) missing++; // a needed copy went out
        }
    }
    if (bestLen == Integer.MAX_VALUE) return "";
    return s.substring(bestL, bestL + bestLen);
}
```

Cost Time `O(m + n)` Space `O(1)`

With m = |s| and n = |t|. r and l each pass over s once; the count arrays have a fixed size of 128.

Variants same idea, small change

- Every distinct value of the array: `required` = number of distinct values; use a `HashMap`.
- Smallest range covering k sorted lists: tag every value with its list, sort, and find the window holding all k list ids with the smallest max – min.
- Balanced string (LC 1234): the window is valid when every count *outside* it is ≤ n/4.
- Shortest window with sum ≥ target is the same shape with a sum as the state (2.7 **Sliding window (variable size)** (p. 58)).

Traps bugs people actually write

- `if (++have[c] >= need[c]) formed++` counts every extra copy and every char not in t: s = "AA", t = "AB" reports "AA" as valid. Use `==` going up; going down, test before the decrement.
- Recording after the while loop (as for *longest* in **2.7 Sliding window (variable size)** (p. 58)) records a window that is no longer valid: "DOBEC" at r = 5, which has no A.
- `s.substring` inside the loop builds a new string per step: O(m^2) in the worst case. Store `bestL` and `bestLen` and cut once at the end.

Practise on LeetCode, in this order

☐	LC 1234	Replace the Substring for Balanced String	Medium
☐	LC 3097	Shortest Subarray With OR at Least K II	Medium
☐	LC 76	Minimum Window Substring	Hard
☐	LC 632	Smallest Range Covering Elements from K Lists	Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

2.10 Sliding window maximum (deque) O(n)

Spot it when the problem looks like this

- *Maximum of each sliding window of size k*, with n up to 10^5 and k up to n, so O(nk) is too slow.
- A DP such as `dp[i] = a[i] + max(dp[i-k..i-1])`: the max over a range that slides with i.
- Shortest subarray with sum ≥ k when values can be **negative**.

How it works

Keep a deque of indices whose values strictly decrease from front to back. When `a[r]` arrives, pop every back index `j` with `a[j] <= a[r]`: `r` is later and at least as big, so `j` can never again be the max of a window. Pop the front when it falls out of the window. The front is then the window's max. Each index enters and leaves once, so the n steps cost O(n) in total, even with the inner `while`. Store indices, not values: the index tells you when the front has left.

For *shortest subarray with sum ≥ k*, use prefix sums `p` and keep `p` increasing in the deque. Pop the front while `p[j] - p[front] >= k`: that is the best length for this start, since a later `j` is only longer. Pop the back while `p[back] >= p[j]`: `j` starts later with a smaller prefix, so it is better.

Worked example a = [1, 3, -1, -3, 5, 3, 6, 7], k = 3

r	a[r]	popped	deque (values)	max
0	1		[1]	
1	3	back 1	[3]	
2	-1		[3, -1]	3
3	-3		[3, -1, -3]	3
4	5	front 3, back -3, back -1	[5]	5
5	3		[5, 3]	5
6	6	back 3, back 5	[6]	6
7	7	back 6	[7]	7

Max of every window of size k

memorise

```
// Max of every window of size k (LC 239). The deque holds indices of the
// window whose values strictly decrease from front to back: front = max.
static int[] maxSlidingWindow(int[] a, int k) {
    int[] out = new int[a.length - k + 1];
    Deque<Integer> dq = new ArrayDeque<>();
    for (int r = 0; r < a.length; r++) {
        if (!dq.isEmpty() && dq.peekFirst() <= r - k)
            dq.pollFirst();                     // 1. front left the window
        while (!dq.isEmpty() && a[dq.peekLast()] <= a[r])
            dq.pollLast();                      // 2. can never be a max again
        dq.addLast(r);                          // 3. r joins at the back
        if (r >= k - 1) out[r - k + 1] = a[dq.peekFirst()];
    }
    return out;
}
```

Shortest subarray with sum ≥ k (negatives)

variant

```
// Shortest subarray with sum >= k; values may be negative (LC 862).
static int shortestSubarray(int[] a, long k) {
    int n = a.length, best = n + 1;
    long[] p = new long[n + 1];                 // p[i] = a[0] + ... + a[i-1]
    for (int i = 0; i < n; i++) p[i + 1] = p[i] + a[i];
    Deque<Integer> dq = new ArrayDeque<>();     // start indices, p increasing
    for (int j = 0; j <= n; j++) {
        while (!dq.isEmpty() && p[j] - p[dq.peekFirst()] >= k)
            best = Math.min(best, j - dq.pollFirst()); // a later j is only longer
        while (!dq.isEmpty() && p[dq.peekLast()] >= p[j])
            dq.pollLast();                      // j is a later and lower start
        dq.addLast(j);
    }
    return best <= n ? best : -1;
}
```

Constrained subsequence sum (DP + deque)

variant

```java
// Max sum of a non-empty subsequence whose chosen neighbours are at most
// k apart (LC 1425). dp[i] = a[i] + max(0, max of dp[i-k..i-1]).
static int constrainedSubsetSum(int[] a, int k) {
    int[] dp = new int[a.length];
    Deque<Integer> dq = new ArrayDeque<>();       // indices, dp decreasing
    int best = Integer.MIN_VALUE;
    for (int i = 0; i < a.length; i++) {
        if (!dq.isEmpty() && dq.peekFirst() < i - k) dq.pollFirst();
        dp[i] = a[i] + (dq.isEmpty() ? 0 : Math.max(0, dp[dq.peekFirst()]));
        while (!dq.isEmpty() && dp[dq.peekLast()] <= dp[i]) dq.pollLast();
        dq.addLast(i);
        best = Math.max(best, dp[i]);
    }
    return best;
}
```

Cost Time `O(n)` Space `O(k)`

Every index is pushed once and popped at most once. The two variants also keep an O(n) array.

Variants same idea, small change

- Window minimum: pop the back while `a[back] >= a[r]`; values then rise front to back.
- Jump game VI: `dp[i] = a[i] + max(dp[i-k..i-1])` with no `max(0, ·)`, and the answer is `dp[n-1]`.
- Longest subarray with max – min ≤ limit: a max deque and a min deque inside a variable window (2.7 **Sliding window (variable size)** (p. 58)); shrink while the two fronts differ by more than the limit.
- Next greater element is the same monotonic idea with a stack and no left edge (4.3 **Monotonic stack: next greater** (p. 105)).

Traps bugs people actually write

- `if` instead of `while` when popping the back leaves small values behind a bigger one: the sample gives `[3, 3, -1, 5, 5, 7]`.
- The plain 2.7 **Sliding window (variable size)** (p. 58) needs non-negative values. On `[84, -37, 32, 40, 95]` with k = 167 it returns 5; the answer is 3.
- Prefix sums of 10^5 values of size 10^5 reach 10^{10}: keep `p` as `long[]`.

Practise on LeetCode, in this order

- [ ] LC 1438 Longest Continuous Subarray With Absolute Diff Less Than or Equal to Limit — Medium
- [ ] LC 1696 Jump Game VI — Medium
- [ ] LC 239 Sliding Window Maximum — Hard
- [ ] LC 1425 Constrained Subsequence Sum — Hard
- [ ] LC 862 Shortest Subarray with Sum at Least K — Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

Part 3

Binary search and sorting

Binary search needs a yes/no test that is false up to some point and true from there on; about $\log_2 n$ tests then find the switch point. The test can be on an index, a candidate answer, a real number or a slope. Sorting by value or by your own rule turns many problems into a scan or a search, and merge sort, quickselect and buckets also answer counting and k-th questions directly.

3.1	**Lower bound and upper bound** Sorted array; find the first index where a condition starts to hold.	O(log n)	p. 68
3.2	**Search in a rotated sorted array** Sorted array rotated at an unknown point; find a value or the minimum.	O(log n)	p. 71
3.3	**Binary search on the answer** Smallest x that works, when "does x work?" is cheap and monotone.	O(n log R)	p. 74
3.4	**Binary search on real numbers** A real answer within 10^{-5} from a monotone test, or a unimodal minimum.	O(100 · n)	p. 76
3.5	**Binary search on a slope** Values rise and fall; compare a[mid] with a[mid + 1], go uphill.	O(log n)	p. 79
3.6	**K-th smallest by counting** K-th smallest of a set too big to list; search the value, count those ≤ it.	O(n log R)	p. 82
3.7	**Custom sort and comparators** Sort by a rule of your own: several keys, frequency, or a+b against b+a.	O(n log n)	p. 85
3.8	**Merge sort with counting** Count pairs i < j with a[i] > a[j] (or a[i] > 2·a[j]) in O(n log n).	O(n log n)	p. 87
3.9	**Quickselect** K-th smallest or largest, or the k smallest in any order, in expected O(n).	O(n) expected	p. 90
3.10	**Counting sort and buckets** Small integer keys (counts, letters): index an array, skip comparisons.	O(n + K)	p. 93
3.11	**Coordinate compression** Huge values where only the order matters; map them to ranks 0..m − 1.	O(n log n)	p. 96

3.1 Lower bound and upper bound

O(log n)

Spot it when the problem looks like this

- The array is sorted and you get many queries, so each one must cost O(log n), not O(n).
- *First* or *last* position of x, *insert position*, or *how many* values lie in [lo, hi].
- *Smallest element ≥ x* (ceiling) or *largest element ≤ x* (floor).

How it works

Keep a half-open range `[lo, hi)` and one invariant: everything left of `lo` is `< x`, everything from `hi` on is ≥ x. At the start nothing is known, so `lo = 0` and `hi = n`. If `a[mid] < x`, then `mid` and everything left of it are `< x`, so `lo = mid + 1`. Otherwise `mid` may be the answer, and `hi = mid` keeps it. When `lo == hi` the two zones meet: `lo` is the first index with `a[i] >= x`, or `n` if there is none.

Each step at least halves `hi - lo`, so there are about $\log_2 n$ steps. `mid` is always below `hi`, so `a[mid]` never reads past the end. Write `lo + (hi - lo) / 2`: `(lo + hi) / 2` overflows once the sum passes $2^{31} - 1$, which happens when you search big answer ranges (3.3 Binary search on the answer (p. 74)). Upper bound changes only `<` to `<=`. Every other question in this window is one or two of these calls.

Worked example a = [1, 3, 3, 3, 5, 8], x = 3

```
lo hi mid a[mid]  lowerBound: is a[mid] < 3?
 0  6   3    3    no  -> hi = 3
 0  3   1    3    no  -> hi = 1
 0  1   0    1    yes -> lo = 1
 1  1             first index with a[i] >= 3 is 1
lo hi mid a[mid]  upperBound: is a[mid] <= 3?
 0  6   3    3    yes -> lo = 4
 4  6   5    8    no  -> hi = 5
 4  5   4    5    no  -> hi = 4
 4  4             first index with a[i] > 3 is 4; count of 3s = 4 - 1 = 3
```

Lower bound and upper bound

memorise

```java
// First index i with a[i] >= x, or a.length if there is none.
// Invariant: a[0, lo) < x and a[hi, n) >= x, so the answer is in [lo, hi].
static int lowerBound(int[] a, int x) {
    int lo = 0, hi = a.length;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;      // lo <= mid < hi, so a[mid] exists
        if (a[mid] < x) lo = mid + 1;      // mid and all left of it are < x
        else hi = mid;                     // a[mid] >= x: mid may be the answer
    }
    return lo;                             // lo == hi: the boundary
}

// First index i with a[i] > x. Only the test changes: < becomes <=.
static int upperBound(int[] a, int x) {
    int lo = 0, hi = a.length;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] <= x) lo = mid + 1;
        else hi = mid;
    }
    return lo;
}
```

Everything else is one or two calls

usage

```
// LC 34: first and last index of x, or {-1, -1}.
static int[] searchRange(int[] a, int x) {
    int first = lowerBound(a, x);
    if (first == a.length || a[first] != x) return new int[] {-1, -1};
    return new int[] {first, upperBound(a, x) - 1};
}

// LC 35: index of x if present, else the index where it would be inserted.
static int searchInsert(int[] a, int x) {
    return lowerBound(a, x);
}

// How many elements satisfy lo <= a[i] <= hi.
static int countInRange(int[] a, int lo, int hi) {
    return upperBound(a, hi) - lowerBound(a, lo);
}

// Last index with a[i] <= x (the "floor" of x), or -1 if there is none.
static int lastAtMost(int[] a, int x) {
    return upperBound(a, x) - 1;
}
```

Cost Time `O(log n)` Space `O(1)` Each step halves hi − lo, so $\lceil \log_2(n + 1) \rceil$ steps; 20 for $n = 10^6$.

Variants same idea, small change

- Values `< x`: `lowerBound(a, x)` of them. Values `> x`: `n - upperBound(a, x)`.
- Search a 2D matrix whose rows continue each other (LC 74): search the flat index range `[0, rows * cols)` and read `m[mid / cols][mid % cols]`.
- Time-based key-value store (LC 981): each key's timestamps arrive sorted; answer `get(key, t)` with `upperBound(times, t) - 1`.
- The set changes between queries? `TreeSet.ceiling(x)` and `floor(x)` give the same answers in O(log n) (13.5 **Ordered maps (TreeMap, TreeSet)** (p. 375)).

Traps bugs people actually write

- `Arrays.binarySearch` returns `-(insertion point) - 1` when x is missing: on `{1, 3, 5, 6}` with x = 4 it returns -3. With duplicates it returns *some* match: on `{3, 3, 3, 3, 3}` it returns 2, not 0.
- Mixing templates: `while (lo <= hi)` with `hi = mid` loops forever once `lo == hi` (or reads `a[n]`). Half-open means `hi = a.length`, `lo < hi`, `hi = mid`, `lo = mid + 1`.
- Reading `a[first]` before checking `first < a.length` throws when x is bigger than every element.

Practise on LeetCode, in this order

- [ ] LC 704 Binary Search — Easy
- [ ] LC 35 Search Insert Position — Easy
- [ ] LC 744 Find Smallest Letter Greater Than Target — Easy
- [ ] LC 34 Find First and Last Position of Element in Sorted Array — Medium
- [ ] LC 74 Search a 2D Matrix — Medium
- [ ] LC 981 Time Based Key-Value Store — Medium

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

3.2 Search in a rotated sorted array

`O(log n)`

Spot it when the problem looks like this

- *Sorted in ascending order, then rotated* at an unknown pivot, like `[4, 5, 6, 7, 0, 1, 2]`.
- *You must write an algorithm with O(log n) runtime*: a linear scan is not accepted.
- Find a target, the minimum, or how many times the array was rotated (the minimum's index).

How it works

Cut a rotated array at any `mid` and at least one side is sorted. If `a[lo] <= a[mid]`, the drop is not inside `[lo, mid]`, so that half is sorted and one range test, `a[lo] <= target < a[mid]`, says whether the target can be there. Otherwise `[mid, hi]` is sorted and you test that half. Either way half the range goes. You look for an exact value and stop when you see it, so the closed range `[lo, hi]` with `lo <= hi` is the natural shape here.

The minimum is a plain lower bound. Every value of the left run is `> a[n-1]` and every value of the right run is `<= a[n-1]`, so the test `a[i] <= a[n-1]` reads false, ..., false, true, ..., true and its first true index holds the minimum.

With duplicates, `a[lo] == a[mid] == a[hi]` hides which side is sorted. Dropping both ends is safe, since neither is the target, but it removes only two elements, so the worst case is O(n).

Worked example **a = [6, 7, 0, 1, 2, 4, 5]**

```
search(a, 0)
lo hi mid a[mid]  sorted half     is 0 in it?
 0  6   3    1    right [1..5]    no  -> hi = 2
 0  2   1    7    left  [6..7]    no  -> lo = 2
 2  2   2    0    found at index 2
findMin(a): is a[mid] > a[6] = 5?
 0  6   3    1    no  -> hi = 3
 0  3   1    7    yes -> lo = 2
 2  3   2    0    no  -> hi = 2
 2  2             minimum is a[2] = 0
```

Search (distinct values)

memorise

```java
// LC 33: index of target in a rotated sorted array of distinct values, or -1.
// One of [lo, mid] and [mid, hi] is always sorted; test the target on that one.
static int search(int[] a, int target) {
    int lo = 0, hi = a.length - 1;            // closed: target in [lo, hi]
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == target) return mid;
        if (a[lo] <= a[mid]) {                // left half [lo, mid] is sorted
            if (a[lo] <= target && target < a[mid]) hi = mid - 1;
            else lo = mid + 1;
        } else {                              // right half [mid, hi] is sorted
            if (a[mid] < target && target <= a[hi]) lo = mid + 1;
            else hi = mid - 1;
        }
    }
    return -1;
}
```

Find the minimum

memorise

```java
// LC 153: minimum of a rotated sorted array of distinct values.
// The minimum sits at the first index with a[i] <= a[n-1]: a lower bound.
static int findMin(int[] a) {
    int last = a[a.length - 1];
    int lo = 0, hi = a.length - 1;            // index n-1 passes the test
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] > last) lo = mid + 1;      // mid in the upper (left) run
        else hi = mid;                        // mid in the lower run: min <= mid
    }
    return a[lo];
}
```

Search with duplicates

variant

```java
// LC 81: the same search when values repeat. If a[lo], a[mid] and a[hi]
// are equal you cannot tell which half is sorted, so drop both ends.
static boolean searchDup(int[] a, int target) {
    int lo = 0, hi = a.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == target) return true;
        if (a[lo] == a[mid] && a[mid] == a[hi]) { lo++; hi--; continue; }
        if (a[lo] <= a[mid]) {
            if (a[lo] <= target && target < a[mid]) hi = mid - 1;
            else lo = mid + 1;
        } else {
            if (a[mid] < target && target <= a[hi]) lo = mid + 1;
            else hi = mid - 1;
        }
    }
    return false;
}
```

Cost Time `O(log n)` Space `O(1)`

Each step drops half the range; with duplicates the lo++, hi-- step can repeat, so O(n) in the worst case.

Variants same idea, small change

- Minimum with duplicates (LC 154): compare `a[mid]` with the moving `a[hi]`; if `a[mid] > a[hi]` go right, if smaller `hi = mid`, if equal `hi--` (a copy of `a[hi]` remains at `mid`). Worst case O(n).
- Rotation count: the index of the minimum, which is `lo` when `findMin` ends.
- Search without the two-case logic: find the minimum's index `p`, then run **3.1 Lower bound and upper bound** (p. 68) on `[0, p)` if `target >= a[0]` (and p > 0), else on `[p, n)`.

Traps bugs people actually write

- `a[lo] < a[mid]` instead of `<=`: when `lo == mid` the one-element left half counts as unsorted, and `search([3, 1], 1)` returns -1.
- The distinct-value code on repeated values is wrong: on `[1, 0, 1, 1, 1]` it decides the left half `[1, 0, 1]` is sorted and never finds 0.
- `a[lo] < target` instead of `a[lo] <= target` skips a target sitting at `a[lo]`: `search([4, 5, 6, 7, 0, 1, 2], 4)` then returns -1.

Practise on LeetCode, in this order

- [ ] LC 153 Find Minimum in Rotated Sorted Array — Medium
- [ ] LC 33 Search in Rotated Sorted Array — Medium
- [ ] LC 81 Search in Rotated Sorted Array II — Medium
- [ ] LC 154 Find Minimum in Rotated Sorted Array II — Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

3.3 Binary search on the answer

`O(n log R)`

Spot it when the problem looks like this

- *Minimum* speed, capacity or days, *minimise the largest* sum, or *maximise the smallest* distance.
- If x works, every bigger x also works (or every smaller one).
- The answer range is huge (10^9, or sums up to 10^{14}), but checking one x is a greedy O(n) pass.

How it works

Turn the optimisation into a yes/no question `ok(x)`. For Koko, `ok(k)` is "eating k bananas an hour finishes within h hours". If speed k works, any faster speed works too, so `ok` reads false, ..., false, true, ..., true over the answer range, and the answer is its first true: the **3.1 Lower bound and upper bound** (p. 68) loop with `ok(mid)` in place of `a[mid] >= x`.

Choose `lo` as the smallest answer that could make sense and `hi` as a value that surely works (the biggest pile, the total weight). Then the loop needs no "not found" case. Each probe runs the check once, so the cost is $\log_2(\text{hi} - \text{lo})$ checks: 30 for a range of 10^9, 47 for 10^{14}.

The check is usually greedy. To ship with capacity `cap`, load each day until the next package does not fit. That uses the fewest days possible for `cap`, so `days <= d` is exactly the right test.

Worked example LC 875: piles = [3, 6, 7, 11], h = 8

```
k        1   2   3   4   5   6   7   8   9  10  11
hours   27  15  10   8   8   6   5   5   5   5   4    ok(k): hours <= 8
lo hi mid hours(mid)
 1 11   6     6     ok -> hi = 6
 1  6   3    10     no -> lo = 4
 4  6   5     8     ok -> hi = 5
 4  5   4     8     ok -> hi = 4
 4  4               answer k = 4
```

Smallest x with ok(x)

memorise

```
interface Ok { boolean test(long x); }

// Smallest x in [lo, hi] with ok(x) true. ok must be monotone
// (false ... false true ... true) and ok(hi) must be true.
static long smallestOk(long lo, long hi, Ok ok) {
    while (lo < hi) {
        long mid = lo + (hi - lo) / 2;
        if (ok.test(mid)) hi = mid;       // mid works: answer is mid or less
        else lo = mid + 1;                // mid fails, and so does all below
    }
    return lo;
}
```

Koko eating bananas; m bouquets

example

```
// LC 875: slowest speed k that eats every pile within h hours.
// k = biggest pile always works, because h >= number of piles.
static int minEatingSpeed(int[] piles, int h) {
    int max = 0;
    for (int p : piles) max = Math.max(max, p);
    return (int) smallestOk(1, max, k -> hoursAt(piles, k) <= h);
}

static long hoursAt(int[] piles, long k) {
    long hours = 0;
    for (int p : piles) hours += (p + k - 1) / k;   // ceil(p / k) in integers
    return hours;
}

// LC 1482: fewest days until m bouquets of k adjacent bloomed flowers exist.
static int minDays(int[] bloom, int m, int k) {
    if ((long) m * k > bloom.length) return -1;       // not enough flowers ever
    int max = 0;
    for (int b : bloom) max = Math.max(max, b);
    return (int) smallestOk(1, max, day -> bouquets(bloom, day, k) >= m);
}

static int bouquets(int[] bloom, long day, int k) {
    int made = 0, run = 0;
    for (int b : bloom) {
        run = b <= day ? run + 1 : 0;                 // bloomed flowers in a row
        if (run == k) { made++; run = 0; }
    }
    return made;
}
```

Ship packages within d days

example

```
// LC 1011: least capacity that ships the packages, in order, within d days.
// LC 410 (split array largest sum into k parts) is this call with d = k.
static int shipWithinDays(int[] w, int d) {
    long max = 0, sum = 0;
    for (int x : w) { max = Math.max(max, x); sum += x; }
    return (int) smallestOk(max, sum, cap -> daysNeeded(w, cap) <= d);
}

// Greedy check: fill each day until the next package does not fit.
static int daysNeeded(int[] w, long cap) {
    int days = 1;
    long load = 0;
    for (int x : w) {
        if (load + x > cap) { days++; load = 0; }
        load += x;
    }
    return days;
}
```

Split array largest sum (LC 410) is `shipWithinDays(nums, k)`: the same greedy counts parts.

Cost Time `O(n log R)` Space `O(1)`

R = hi – lo is the answer range; each of the $\log_2 R$ probes runs an O(n) check.

Variants same idea, small change

- Largest x with `ok(x)` when ok reads true, ..., true, false, ..., false (magnetic force between balls, LC 1552): find the smallest x where ok *fails* and subtract 1.
- Sqrt(x) on integers (LC 69): the largest m with `m * m <= x`; compute `m * m` as `long`.
- Answer is a real number? The same idea with a fixed number of halvings: 3.4 **Binary search on real numbers** (p. 76).

Traps bugs people actually write

- An `hi` that does not work: if `ok(hi)` is false, the loop still returns `hi`. Pick a value that surely works, or test `ok(lo)` after the loop.
- Starting the shipping search at 1 instead of the heaviest package: with capacity 5 the check counts `[1, 9, 1]` as 3 days, though 9 never fits.
- Overflow in the check: Koko's hours reach $10^4 \cdot 10^9$ at k = 1. An `int` sum wraps around and `hours <= h` can wrongly say yes.
- `Math.ceil(p / k)` with ints divides first and rounds down; write `(p + k - 1) / k`.

Practise on LeetCode, in this order

☐	`LC 69`	Sqrt(x)	Easy
☐	`LC 875`	Koko Eating Bananas	Medium
☐	`LC 1011`	Capacity To Ship Packages Within D Days	Medium
☐	`LC 1482`	Minimum Number of Days to Make m Bouquets	Medium
☐	`LC 1552`	Magnetic Force Between Two Balls	Medium
☐	`LC 410`	Split Array Largest Sum	Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

3.4 Binary search on real numbers `O(100 · n)`

Spot it when the problem looks like this

- *Answers within 10^{-5} of the actual answer will be accepted.*
- A quantity that only rises or only falls as x grows (area below a line, travel time at a speed), and you need the x where it reaches a target.
- A cost that falls and then rises as x moves (a sum of distances): ternary search finds its minimum.

How it works

This is the integer loop with `lo = mid` and `hi = mid` in place of mid ± 1, run a fixed number of times. 100 halvings shrink the range by $2^{100} \approx 10^{30}$, so a range of $2 \cdot 10^9$ ends far below 10^{-5} and the loop always stops. An epsilon loop `while (hi - lo > 1e-9)` can spin forever: near 10^9 neighbouring doubles are $1.2 \cdot 10^{-7}$ apart, so once `mid` rounds to `lo` or `hi` the gap stops shrinking.

The invariant is the one from **3.1 Lower bound and upper bound** (p. 68): the test is false at `lo` and true at `hi`, so the switch point stays in `[lo, hi]`. In LC 3453 the area below a line never decreases as the line rises, so the first y with `2 * below >= total` is the lowest line that halves the area.

Ternary search minimises a function that falls, then rises. Compare `f` at the two third points and drop the outer third next to the larger value, which cannot hold the minimum.

Worked example sqrt(2): lo = 0, hi = 2

```
it  lo        hi        mid       mid*mid   move
 0  0.000000  2.000000  1.000000  1.000000  < 2  -> lo = mid
 1  1.000000  2.000000  1.500000  2.250000  >= 2 -> hi = mid
 2  1.000000  1.500000  1.250000  1.562500  < 2  -> lo = mid
 3  1.250000  1.500000  1.375000  1.890625  < 2  -> lo = mid
 4  1.375000  1.500000  1.437500  2.066406  >= 2 -> hi = mid
 5  1.375000  1.437500  1.406250  1.977539  < 2  -> lo = mid
...
53  lo and hi are neighbouring doubles; mid rounds to one of them
99  hi = 1.4142135623730951, the same double as Math.sqrt(2)
```

Real binary search (square root) memorise

```java
// Square root of x >= 0 by halving. A fixed count of steps replaces an
// epsilon test: 100 halvings make the range 2^100 times smaller.
static double sqrt(double x) {
    double lo = 0, hi = Math.max(1, x);       // sqrt(x) is in [lo, hi]
    for (int it = 0; it < 100; it++) {
        double mid = (lo + hi) / 2;
        if (mid * mid < x) lo = mid;          // keep the answer inside [lo, hi]
        else hi = mid;
    }
    return hi;
}
```

Separate squares I (LC 3453)

example

```
// LC 3453: lowest y where the square area below the line is half the total.
// below(y) never decreases as y grows: find the first y with 2*below >= total.
static double separateSquares(int[][] sq) {
    long total = 0;
    for (int[] s : sq) total += (long) s[2] * s[2];
    double lo = 0, hi = 2e9;                  // all squares lie in y <= 2e9
    for (int it = 0; it < 100; it++) {
        double mid = (lo + hi) / 2;
        if (lessThanHalfBelow(sq, mid, total)) lo = mid;    // go up
        else hi = mid;
    }
    return hi;
}

// Is 2 * (area below y) < total? Squares wholly below y are added exactly
// in a long, so a total near 1e12 does not eat the decimals of the cut part.
static boolean lessThanHalfBelow(int[][] sq, double y, long total) {
    long whole = 0;
    double cut = 0;
    for (int[] s : sq) {                      // s = {x, y, side}
        if (y >= (long) s[1] + s[2]) whole += (long) s[2] * s[2];
        else if (y > s[1]) cut += (y - s[1]) * s[2];
    }
    return 2 * cut < total - 2 * whole;
}
```

Ternary search on a unimodal function

variant

```
interface Fn { double at(double x); }

// Minimum point of f on [lo, hi] when f falls and then rises (unimodal).
// Each step drops the outer third that cannot hold the minimum.
static double ternaryMin(Fn f, double lo, double hi) {
    for (int it = 0; it < 200; it++) {        // (2/3)^200 of the range is left
        double m1 = lo + (hi - lo) / 3, m2 = hi - (hi - lo) / 3;
        if (f.at(m1) < f.at(m2)) hi = m2;     // the minimum is left of m2
        else lo = m1;                         // the minimum is right of m1
    }
    return (lo + hi) / 2;
}
```

Cost Time `O(100 · n)` Space `O(1)`

100 halvings, each running the O(n) test. Ternary search needs about 170 steps (2 calls each) for the same shrink; the code runs 200.

Variants same idea, small change

- K-th smallest prime fraction (LC 786): binary search a real v in (0, 1), count fractions ≤ v with two pointers, keep the largest one seen, and stop when the count is k (3.6 **K-th smallest by counting** (p. 82)).
- Convex cost on integers (LC 2448): binary search the first x with `f(x) <= f(x + 1)`, the slope test of 3.5 **Binary search on a slope** (p. 79).
- Two variables (LC 1515, best position for a service centre): the sum of distances is convex in x and in y, so nest one ternary search inside the other.

Traps bugs people actually write

- `while (hi - lo > 1e-9)` near 10^9 never ends, because neighbouring doubles there are $1.2 \cdot 10^{-7}$ apart. Loop a fixed 100 times.
- One `double` for big totals: near 10^{12} a double keeps about 4 decimal places. Summing every area in one double put a thin square's answer off by $3 \cdot 10^{-5}$ (tolerance 10^{-5}); the code adds whole squares in a `long`.
- Ternary search on a function with a flat stretch away from the minimum: on the plateau `f(m1) == f(m2)` and you may drop the third that holds the minimum. It is safe for strictly unimodal or convex functions.
- Using `float`: 24 bits give only about 7 significant digits.

Practise on LeetCode, in this order

☐	LC 3453	Separate Squares I	Medium
☐	LC 786	K-th Smallest Prime Fraction	Medium
☐	LC 2448	Minimum Cost to Make Array Equal	Hard
☐	LC 1515	Best Position for a Service Centre	Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

3.5 Binary search on a slope

`O(log n)`

Spot it when the problem looks like this

- *Peak element*, *mountain array*, *bitonic*: the values go up and then down, or wiggle.
- Neighbours are never equal (`nums[i] != nums[i + 1]`), and the array is not sorted.
- *O(log n)* is required, or reads are limited (*more than 100 calls to get is Wrong Answer*).

How it works

Comparing `a[mid]` with `a[mid + 1]` tells you which way the ground slopes at `mid`. If it goes up, walk right from `mid + 1`: the values either rise to the end, where a[n] = −∞ makes the last element a peak, or

they fall somewhere, and the top before the fall is a peak. So a peak lies in `[mid + 1, hi]`. If it goes down, the same walk leftwards puts a peak in `[lo, mid]`.

The invariant is "a peak lies in `[lo, hi]`". Each step halves the range, so after $\log_2 n$ steps the one index left is a peak. Only the local slope matters, so the array need not be sorted, and `mid < hi` means `a[mid + 1]` exists.

In a mountain array the test reads up, ..., up, down, ..., down, so the same loop finds the one peak. Both sides of it are sorted: two more binary searches, the falling one with the test reversed, find a value.

Worked example **LC 162: a = [1, 2, 1, 3, 5, 6, 4]**

```
lo hi mid  a[mid] vs a[mid+1]
 0  6   3   3 < 5   uphill   -> lo = 4
 4  6   5   6 > 4   downhill -> hi = 5
 4  5   4   5 < 6   uphill   -> lo = 5
 5  5               peak at index 5 (value 6); index 1 is a peak too
```

Find a peak

memorise

```java
// LC 162: index of any peak (bigger than both neighbours; a[-1] = a[n] = -inf,
// neighbours never equal). LC 852 (peak of a mountain) is the same code.
static int findPeak(int[] a) {
    int lo = 0, hi = a.length - 1;            // a peak exists in [lo, hi]
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;         // mid < hi, so a[mid + 1] exists
        if (a[mid] < a[mid + 1]) lo = mid + 1;    // uphill: a peak to the right
        else hi = mid;                            // downhill: mid or to the left
    }
    return lo;
}
```

Find in mountain array (LC 1095)

example

```java
interface MountainArray { int get(int index); int length(); }   // LeetCode's

// LC 1095: smallest index holding target, or -1, in about 4 log2(n) gets.
static int findInMountainArray(int target, MountainArray m) {
    int n = m.length(), lo = 0, hi = n - 1;
    while (lo < hi) {                          // 1. the peak, as in findPeak
        int mid = lo + (hi - lo) / 2;
        if (m.get(mid) < m.get(mid + 1)) lo = mid + 1;
        else hi = mid;
    }
    int peak = lo;
    int i = firstEqual(m, target, 0, peak + 1, true);   // 2. rising side first
    if (i != -1) return i;
    return firstEqual(m, target, peak + 1, n, false);   // 3. then falling side
}

// Lower bound of x on [from, to), a run sorted up (asc) or down; -1 if absent.
static int firstEqual(MountainArray m, int x, int from, int to, boolean asc) {
    int lo = from, hi = to;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        int v = m.get(mid);
        if (asc ? v < x : v > x) lo = mid + 1;    // x is further on
        else hi = mid;
    }
    return lo < to && m.get(lo) == x ? lo : -1;
}
```

LeetCode supplies `MountainArray`; the interface line is only there so the file compiles.

Cost Time `O(log n)` Space `O(1)`

Halves the range each step. LC 1095 used at most 56 get() calls at n = 10^4 in our tests (limit 100).

Variants same idea, small change

- Peak index in a mountain array (LC 852): exactly the same `findPeak`; there is one peak.
- 2D peak (LC 1901): binary search the row. Take the largest value in row `mid`; if the cell below it is bigger, a peak is below (`lo = mid + 1`), else `hi = mid`.
- Single element in a sorted array (LC 540): before the single element `a[i] == a[i ^ 1]` holds, after it fails; a 3.1 **Lower bound and upper bound** (p. 68) on that test.

Traps bugs people actually write

- Comparing with `a[mid - 1]` when `mid` can be 0 reads `a[-1]`. Compare with `a[mid + 1]`; `mid < hi` makes it safe.
- Equal neighbours break the argument: on `[1, 3, 3, 3, 4, 1]` the loop returns index 1, which is not a peak. LeetCode promises `nums[i] != nums[i + 1]`.
- Searching the falling side with the rising test: there the values go down, so the test flips to `v > x`.

Practise on LeetCode, in this order

- [ ] LC 852 Peak Index in a Mountain Array — Medium
- [ ] LC 162 Find Peak Element — Medium
- [ ] LC 540 Single Element in a Sorted Array — Medium
- [ ] LC 1901 Find a Peak Element II — Medium
- [ ] LC 1095 Find in Mountain Array — Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

3.6 K-th smallest by counting

`O(n log R)`

Spot it when the problem looks like this

- *K-th smallest* (or k-th largest) element, distance, product or fraction.
- Too many candidates to list ($5{\cdot}10^7$ pairs in LC 719, $9{\cdot}10^8$ cells in LC 668), but you *can* count those ≤ v quickly.
- Rows and columns are sorted, or sorting the input turns counting into a two-pointer pass.

How it works

Let `count(v)` be how many candidates are ≤ v. It never decreases as v grows, and the k-th smallest is the smallest v with `count(v) >= k`. That is **3.3 Binary search on the answer** (p. 74) with `ok(v) = count(v) >= k`, searched over values instead of positions.

The result is always a real candidate. `count` only changes at candidate values, so the smallest v that reaches k is one of them; you never need an "is it in the set?" check.

The work is in counting in O(n). In a matrix with sorted rows and columns, start at the bottom-left. If the entry is ≤ v, the whole column above it is too, so add `row + 1` and step right; otherwise step up. Each step leaves a row or a column, so there are at most 2n steps. For pair distances, sort first; for each `j` the close-enough `i` form a window whose left end only moves right (**2.7 Sliding window (variable size)** (p. 58)).

Worked example LC 378: m = [[1, 5, 9], [10, 11, 13], [12, 13, 15]], k = 8

```
lo hi mid count(<= mid)
 1 15   8     2      < 8  -> lo = 9
 9 15  12     6      < 8  -> lo = 13
13 15  14     8      >= 8 -> hi = 14   (14 is not in m; keep going)
13 14  13     8      >= 8 -> hi = 13
13 13                answer 13
staircase for mid = 12, starting at m[2][0] = 12:
  12 <= 12: +3, right    13 > 12: up    11 <= 12: +2, right
  13 > 12: up            9 <= 12: +1, right (off the grid)    count = 6
```

K-th smallest in a sorted matrix

memorise

```java
// LC 378: k-th smallest (1-based) in an n x n matrix with sorted rows and
// columns. Answer = smallest value v with countAtMost(v) >= k.
static int kthSmallest(int[][] m, int k) {
    int n = m.length;
    int lo = m[0][0], hi = m[n - 1][n - 1];   // count(hi) = n*n >= k
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;         // rounds down, also for negatives
        if (countAtMost(m, mid) >= k) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}

// Staircase from the bottom-left corner: O(n) per count.
static int countAtMost(int[][] m, int v) {
    int n = m.length, count = 0, row = n - 1, col = 0;
    while (row >= 0 && col < n) {
        if (m[row][col] <= v) { count += row + 1; col++; }  // rows 0..row of col
        else row--;                                         // too big: go up
    }
    return count;
}
```

K-th smallest pair distance

example

```java
// LC 719: k-th smallest |a[i] - a[j]| over all pairs i < j.
static int smallestDistancePair(int[] a, int k) {
    Arrays.sort(a);
    int lo = 0, hi = a[a.length - 1] - a[0];  // every pair distance is <= hi
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (pairsAtMost(a, mid) >= k) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}

// Pairs with a[j] - a[i] <= d: for each j, i is the first index close enough.
static long pairsAtMost(int[] a, int d) {
    long count = 0;
    for (int i = 0, j = 0; j < a.length; j++) {
        while (a[j] - a[i] > d) i++;          // i only moves right: O(n) total
        count += j - i;
    }
    return count;
}
```

Median of two sorted arrays: search the cut

variant

```java
// LC 4: median of two sorted arrays in O(log min(n, m)). Binary search the cut
// i in the shorter array a; the cut in b is j = half - i. The cut is right
// when a[i-1] <= b[j] and b[j-1] <= a[i].
static double findMedianSortedArrays(int[] a, int[] b) {
    if (a.length > b.length) return findMedianSortedArrays(b, a);
    int n = a.length, m = b.length, half = (n + m + 1) / 2;
    int lo = 0, hi = n;                          // i is in [lo, hi]
    while (true) {
        int i = lo + (hi - lo) / 2, j = half - i;
        int aL = i == 0 ? Integer.MIN_VALUE : a[i - 1];   // left of the cut in a
        int aR = i == n ? Integer.MAX_VALUE : a[i];       // right of the cut in a
        int bL = j == 0 ? Integer.MIN_VALUE : b[j - 1];
        int bR = j == m ? Integer.MAX_VALUE : b[j];
        if (aL > bR) hi = i - 1;                 // took too many from a
        else if (bL > aR) lo = i + 1;            // took too few from a
        else if ((n + m) % 2 == 1) return Math.max(aL, bL);
        else return (Math.max(aL, bL) + (long) Math.min(aR, bR)) / 2.0;
    }
}
```

Cost Time `O(n log R)` Space `O(1)`

R is the value range; $\log_2 R$ counts of O(n) each. Sorting adds O(n log n). The median search is O(log min(n, m)).

Variants same idea, small change

- Multiplication table (LC 668): row i holds i, 2i, ..., n·i, so `count(v)` is the sum of `min(v / i, n)` over rows i = 1..m.
- Find the duplicate number (LC 287): n + 1 values in 1..n. The duplicate is the smallest v with more than v values ≤ v; O(n log n) time, O(1) space, array untouched.
- K-th smallest prime fraction: the same with a real-valued v (3.4 **Binary search on real numbers** (p. 76)).
- The k-th largest of an array you can hold in memory: use 3.9 **Quickselect** (p. 90) or a heap (4.7 **Top k with a size-k heap** (p. 116)).

Traps bugs people actually write

- `(lo + hi) / 2` rounds toward zero: for lo = -3, hi = -2 it gives -2 = hi, and `hi = mid` loops forever. `lo + (hi - lo) / 2` gives -3.
- Returning `mid` as soon as `count(mid) == k`: mid may not be a candidate (in the trace, count(14) = 8 but 14 is not in the matrix). Keep shrinking to the smallest such v.
- In the median search, call it with the shorter array as `a`; otherwise `j = half - i` can go negative.

Practise on LeetCode, in this order

- [ ] LC 287 Find the Duplicate Number — Medium
- [ ] LC 378 Kth Smallest Element in a Sorted Matrix — Medium
- [ ] LC 668 Kth Smallest Number in Multiplication Table — Hard
- [ ] LC 719 Find K-th Smallest Pair Distance — Hard
- [ ] LC 4 Median of Two Sorted Arrays — Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

3.7 Custom sort and comparators

`O(n log n)`

Spot it when the problem looks like this

- *Arrange* the items to form the largest or smallest result, or *sort by X, then by Y*.
- Sort by frequency, by a custom alphabet, by a column of a 2D array, or by distance.
- You still need the original positions after sorting (sort an index array).

How it works

A comparator returns a negative number when `a` goes first, a positive one when `b` goes first, and `0` for a tie. Build it key by key: compare the first key, and only on a tie compare the next.

For *largest number*, use the exchange argument. Swapping two neighbours `a` and `b` changes only their part of the string, so `a` belongs first exactly when `a + b > b + a`. The test is transitive (it orders numbers by the fraction a / (10^len(a) – 1)), so sorting by it is valid. Any other order has a neighbour pair in the wrong order, and swapping it never makes the string smaller, so the sorted order is the best.

In Java, `Arrays.sort(int[])` takes no comparator, so box to `Integer[]` or sort an index array. Sorting objects is stable (TimSort): ties keep their input order. A comparator must be consistent: `compare(a, b)` has the opposite sign of `compare(b, a)`, and a < b, b < c must give a < c.

Worked example LC 179: nums = [3, 30, 34, 5, 9]

```
a    b    a+b    b+a    goes first
3    30   330    303    3
3    34   334    343    34
30   34   3034   3430   34
5    9    59     95     9
34   5    345    534    5
sorted: [9, 5, 34, 3, 30]  ->  "9534330"
```

Largest number

memorise

```java
// LC 179: arrange the numbers to form the largest number.
// Exchange argument: a goes before b exactly when a+b > b+a as strings.
static String largestNumber(int[] nums) {
    String[] s = new String[nums.length];
    for (int i = 0; i < nums.length; i++) s[i] = String.valueOf(nums[i]);
    Arrays.sort(s, (a, b) -> (b + a).compareTo(a + b));  // bigger a+b first
    if (s[0].equals("0")) return "0";                     // all were 0
    StringBuilder sb = new StringBuilder();
    for (String x : s) sb.append(x);
    return sb.toString();
}
```

Two keys (frequency, then value)

memorise

```java
// LC 1636: sort by frequency (rare first), ties by value (big first).
static int[] frequencySort(int[] nums) {
    Map<Integer, Integer> freq = new HashMap<>();
    for (int x : nums) freq.merge(x, 1, Integer::sum);
    Integer[] b = new Integer[nums.length];       // comparators need objects
    for (int i = 0; i < nums.length; i++) b[i] = nums[i];
    Arrays.sort(b, (x, y) -> {
        int fx = freq.get(x), fy = freq.get(y);
        if (fx != fy) return Integer.compare(fx, fy);   // key 1: frequency up
        return Integer.compare(y, x);                   // key 2: value down
    });
    for (int i = 0; i < nums.length; i++) nums[i] = b[i];
    return nums;
}
```

Columns, helpers and index sorts

usage

```java
// int[][] by column 0 up, ties by column 1 down. Compare; never subtract.
static void sortRows(int[][] rows) {
    Arrays.sort(rows, (x, y) -> x[0] != y[0] ? Integer.compare(x[0], y[0])
                                             : Integer.compare(y[1], x[1]));
}

// Comparator helpers: the same row order, and strings by length, then A-Z.
static final Comparator<int[]> BY_COLUMNS =
        Comparator.<int[]>comparingInt(r -> r[0])
                  .thenComparing((x, y) -> Integer.compare(y[1], x[1]));
static final Comparator<String> BY_LENGTH = Comparator
        .comparingInt(String::length).thenComparing(Comparator.naturalOrder());

// Indices of a in sorted order, so the original positions survive.
static Integer[] sortedIndices(int[] a) {
    Integer[] idx = new Integer[a.length];
    for (int k = 0; k < a.length; k++) idx[k] = k;
    Arrays.sort(idx, (i, j) -> Integer.compare(a[i], a[j]));  // ties: i < j
    return idx;
}
```

Cost Time `O(n log n)` Space `O(n)`
O(n log n) comparisons; in LC 179 each one builds two strings of at most 20 characters.

Variants same idea, small change

- Custom alphabet (LC 791): compare letters by their position in the given order.
- Relative sort array (LC 1122): key = index in `arr2` if present, else `1000 + value`.
- Queue reconstruction by height (LC 406): sort by height down, then k up, and insert each person at index k of a list.
- Reorder log files (LC 937): letter-logs by content, then identifier; digit-logs after them in input order, which a stable sort keeps.

Traps bugs people actually write

- `(x, y) -> x - y` overflows: `Integer.MIN_VALUE - 1` wraps to a positive number, so 1 is put before `Integer.MIN_VALUE`. Use `Integer.compare(x, y)`.
- A comparator that never returns 0, such as `(x, y) -> x < y ? -1 : 1`, breaks the contract. TimSort can throw *Comparison method violates its general contract!*; it did in most of our runs on 100 values from 0..4.
- `Arrays.sort(nums, comparator)` on an `int[]` does not compile; box to `Integer[]` first or sort indices.
- Largest number with only zeros: `[0, 0]` must give "0", not "00".

Practise on LeetCode, in this order

☐	LC 1122	Relative Sort Array	Easy
☐	LC 1636	Sort Array by Increasing Frequency	Easy
☐	LC 179	Largest Number	Medium
☐	LC 937	Reorder Data in Log Files	Medium
☐	LC 406	Queue Reconstruction by Height	Medium
☐	LC 1366	Rank Teams by Votes	Medium

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

3.8 Merge sort with counting `O(n log n)`

Spot it when the problem looks like this

- *Count inversions*, *reverse pairs*, *count of smaller numbers after self*: a condition on pairs with i < j.
- n reaches $5 \cdot 10^4$ to 10^5, so the $O(n^2)$ double loop times out.
- You need a guaranteed O(n log n) sort, or you are sorting a linked list (no random access).

How it works

Merge sort splits `[lo, hi)` at `mid`, sorts both halves, then merges them with two pointers. Every pair `i < j` is looked at on exactly one level: the one where `i` and `j` first land in different halves. At that moment both halves are sorted, which makes counting cheap.

When the merge takes a right element `a[j]` before what is left of the left half, `a[j]` is smaller than all of `a[i..mid)`, so it adds `mid - i` inversions at once. Take equal values from the left (`<=`) so equal pairs are not counted.

For count-of-smaller-after-self you need one count per original index, so sort an array of indices by value and add to `cnt[idx[i]]`. For reverse pairs the test `a[i] > 2 * a[j]` does not follow the merge order, so count first with a separate two-pointer pass over the two sorted halves, then merge. There are $\log_2 n$ levels and each does O(n) work.

Worked example **Count inversions of a = [5, 2, 6, 1, 3]**

```
range  left     right      merged           added
[0,2)  [5]      [2]        [2, 5]           +1   2 jumps 5
[3,5)  [1]      [3]        [1, 3]           +0
[2,5)  [6]      [1, 3]     [1, 3, 6]        +2   1 and 3 jump 6
[0,5)  [2, 5]   [1, 3, 6]  [1, 2, 3, 5, 6]  +3   1 jumps 2 and 5; 3 jumps 5
total 1 + 0 + 2 + 3 = 6 inversions
```

Sort and count inversions memorise

```java
// Sorts a[lo, hi) and returns its inversions: pairs i < j with a[i] > a[j].
// Call sortCount(a, new int[a.length], 0, a.length). LC 912 is the sort alone.
static long sortCount(int[] a, int[] tmp, int lo, int hi) {
    if (hi - lo < 2) return 0;
    int mid = lo + (hi - lo) / 2;
    long inv = sortCount(a, tmp, lo, mid) + sortCount(a, tmp, mid, hi);
    int i = lo, j = mid, k = lo;              // both halves are sorted now
    while (i < mid || j < hi) {
        if (j == hi || (i < mid && a[i] <= a[j])) tmp[k++] = a[i++];
        else {
            inv += mid - i;                   // a[j] < every a[i..mid)
            tmp[k++] = a[j++];
        }
    }
    System.arraycopy(tmp, lo, a, lo, hi - lo);
    return inv;
}
```

Count of smaller numbers after self

example

```java
// LC 315: for each i, count j > i with a[j] < a[i]. Merge-sort the indices by
// value; right indices placed before a left one are smaller and to its right.
static List<Integer> countSmaller(int[] a) {
    int n = a.length;
    int[] idx = new int[n], tmp = new int[n], cnt = new int[n];
    for (int i = 0; i < n; i++) idx[i] = i;
    sortIdx(a, idx, tmp, cnt, 0, n);
    List<Integer> out = new ArrayList<>();
    for (int c : cnt) out.add(c);
    return out;
}

static void sortIdx(int[] a, int[] idx, int[] tmp, int[] cnt, int lo, int hi) {
    if (hi - lo < 2) return;
    int mid = lo + (hi - lo) / 2;
    sortIdx(a, idx, tmp, cnt, lo, mid); sortIdx(a, idx, tmp, cnt, mid, hi);
    int i = lo, j = mid, k = lo;
    while (i < mid || j < hi) {
        if (j == hi || (i < mid && a[idx[i]] <= a[idx[j]])) {
            cnt[idx[i]] += j - mid;           // right items already placed
            tmp[k++] = idx[i++];
        } else tmp[k++] = idx[j++];
    }
    System.arraycopy(tmp, lo, idx, lo, hi - lo);
}
```

Reverse pairs: count, then merge

example

```java
// LC 493: pairs i < j with a[i] > 2 * a[j]. Both halves are sorted, so count
// across them with two pointers first, then merge as usual.
static int reversePairs(int[] a, int[] tmp, int lo, int hi) {
    if (hi - lo < 2) return 0;
    int mid = lo + (hi - lo) / 2;
    int count = reversePairs(a, tmp, lo, mid) + reversePairs(a, tmp, mid, hi);
    for (int i = lo, j = mid; i < mid; i++) {
        while (j < hi && a[i] > 2L * a[j]) j++;   // 2L: 2 * a[j] overflows int
        count += j - mid;
    }
    merge(a, tmp, lo, mid, hi);
    return count;
}

static void merge(int[] a, int[] tmp, int lo, int mid, int hi) {
    int i = lo, j = mid, k = lo;
    while (i < mid || j < hi)
        tmp[k++] = (j == hi || (i < mid && a[i] <= a[j])) ? a[i++] : a[j++];
    System.arraycopy(tmp, lo, a, lo, hi - lo);
}
```

Cost Time `O(n log n)` Space `O(n)`

$\log_2 n$ levels, each merging n elements; one shared tmp array of size n; recursion depth $\log_2 n$.

Variants same idea, small change

- Count of range sum (LC 327): merge-sort the prefix sums; for each left prefix `p[i]`, two pointers find the right prefixes with lower ≤ p[j] − p[i] ≤ upper.
- Global and local inversions (LC 775): count all inversions with `sortCount` and compare with the adjacent ones.
- Sort list (LC 148): the same merge on a linked list; find the middle with **5.3 Fast and slow pointers** (p. 134), then merge behind a dummy head (**5.1 Dummy head node** (p. 128)).
- The same counts with a Fenwick tree over compressed values: **3.11 Coordinate compression** (p. 96).

Traps bugs people actually write

- `a[i] < a[j]` instead of `<=` takes an equal right value first and counts equal pairs: `[2, 2]` gets 1 inversion instead of 0.
- `2 * a[j]` overflows `int` once a[j] ≥ 2^{30} or a[j] < -2^{30}, and LC 493 allows values up to $2^{31} - 1$; write `2L * a[j]`.
- A new temp array in every call allocates O(n log n) cells in total; pass one `tmp` down.
- Inversions reach $n^2/2 \approx 2 \cdot 10^{10}$ for $n = 2 \cdot 10^5$; count in a `long`.

Practise on LeetCode, in this order

☐	LC 912	Sort an Array	Medium
☐	LC 148	Sort List	Medium
☐	LC 775	Global and Local Inversions	Medium
☐	LC 315	Count of Smaller Numbers After Self	Hard
☐	LC 493	Reverse Pairs	Hard
☐	LC 327	Count of Range Sum	Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

3.9 Quickselect

`O(n) expected`

Spot it when the problem looks like this

- *K-th largest element*, *k closest points*, the *median*, when the order of the k items does not matter.
- One query on an array you may reorder; n up to 10^5 or 10^6.
- *Can you solve it without sorting?*: the follow-up wants better than O(n log n).

How it works

Pick a random pivot and partition `[lo, hi]` so smaller values go left and larger ones go right. Only one side can hold index k, so continue there and forget the other; with one side per round, a loop replaces the recursion.

The partition is Hoare-style: `i` scans right past values `< pivot`, `j` scans left past values `> pivot`, and the two are swapped. Both scans stop on values *equal* to the pivot, so equal values spread over both sides. Lomuto's one-scan partition is shorter but sends every equal value to one side; on 2000 equal values it took 1.5 million steps to find the middle one.

A random pivot lands in the middle half of the values half the time, and then at most 3/4 of the range is kept. So on average two rounds cut the range to 3/4, and the expected work is at most $2n(1 + 3/4 + (3/4)^2 + \ldots) = 8n$. $O(n^2)$ needs bad pivots round after round, which is very unlikely.

Worked example **LC 215: a = [3, 2, 1, 5, 6, 4], 2nd largest = kthSmallest(a, 4)**

```
lo hi pivot  a after partition     j  i   next
 0  5   3    [1, 2, 3, 5, 6, 4]    1  2   k >= i -> lo = 2
 2  5   4    [1, 2, 3, 4, 6, 5]    3  4   k >= i -> lo = 4
 4  5   6    [1, 2, 3, 4, 5, 6]    4  5   k <= j -> hi = 4
 4  4                                     lo == hi: a[4] = 5
(pivots from one seeded run; another run picks others, same answer)
```

Quickselect (k-th smallest) memorise

```java
static final Random RND = new Random();

// k-th smallest (0-based) in O(n) expected time; reorders a.
// LC 215 (k-th largest, 1-based) is kthSmallest(a, a.length - k).
static int kthSmallest(int[] a, int k) {
    int lo = 0, hi = a.length - 1;            // index k lies in [lo, hi]
    while (lo < hi) {
        int pivot = a[lo + RND.nextInt(hi - lo + 1)];
        int i = lo, j = hi;                   // Hoare-style partition
        while (i <= j) {
            while (a[i] < pivot) i++;         // stops at a value >= pivot
            while (a[j] > pivot) j--;         // stops at a value <= pivot
            if (i <= j) { int t = a[i]; a[i] = a[j]; a[j] = t; i++; j--; }
        }
        // now a[lo..j] <= pivot, a[i..hi] >= pivot, a[j+1..i-1] == pivot
        if (k <= j) hi = j;
        else if (k >= i) lo = i;
        else return a[k];
    }
    return a[k];
}
```

K closest points to the origin

example

```
// LC 973: the k points nearest (0, 0), any order. The same loop, keyed on
// squared distance; afterwards p[0..k-1] are the k smallest.
static int[][] kClosest(int[][] p, int k) {
    int lo = 0, hi = p.length - 1, target = k - 1;
    while (lo < hi) {
        long pivot = dist(p[lo + RND.nextInt(hi - lo + 1)]);
        int i = lo, j = hi;
        while (i <= j) {
            while (dist(p[i]) < pivot) i++;
            while (dist(p[j]) > pivot) j--;
            if (i <= j) { int[] t = p[i]; p[i] = p[j]; p[j] = t; i++; j--; }
        }
        if (target <= j) hi = j;
        else if (target >= i) lo = i;
        else break;
    }
    return Arrays.copyOf(p, k);
}

static long dist(int[] q) { return (long) q[0] * q[0] + (long) q[1] * q[1]; }
```

Cost Time `O(n) expected` Space `O(1)`

Expected O(n): the median of 10^6 values took about 3.6n comparisons. Worst case $O(n^2)$; a size-k heap is O(n log k).

Variants same idea, small change

- Heap instead: a max-heap of size k keeps the k smallest in O(n log k). It works on a stream and has no $O(n^2)$ case; see 4.7 **Top k with a size-k heap** (p. 116).
- Median (LC 462, minimum moves to equal array elements II): select index n/2, then add up |a[i] – median|.
- Wiggle sort II (LC 324): find the median with quickselect, then a three-way partition around it (2.4 **Three-way partition (Dutch flag)** (p. 51)) on virtual indices.
- K-th largest integer as strings (LC 1985): the same loop with a comparator: shorter is smaller, equal lengths compare as strings.

Traps bugs people actually write

- A fixed pivot (`a[hi]` or `a[lo]`) on sorted input removes one element per round: $O(n^2)$. Always pick it at random.
- Lomuto on many equal values is quadratic even with a random pivot; use the two-scan partition above or a three-way partition.
- LC 215's k is 1-based and counts from the largest, so the index to select is `n - k`.
- `while (a[i] <= pivot) i++` (with `<=`) can run past the end when the pivot is the maximum; the scans must stop on equal values.

Practise on LeetCode, in this order

- [ ] LC 215 Kth Largest Element in an Array — Medium
- [ ] LC 973 K Closest Points to Origin — Medium
- [ ] LC 462 Minimum Moves to Equal Array Elements II — Medium
- [ ] LC 1985 Find the Kth Largest Integer in the Array — Medium
- [ ] LC 324 Wiggle Sort II — Medium

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

3.10 Counting sort and buckets

`O(n + K)`

Spot it when the problem looks like this

- Values in a small range (letters, 1..100, a count from 1 to n), so an array indexed by value replaces sorting.
- *Top k frequent*, *sort by frequency*: every frequency lies between 1 and n.
- *Linear time and linear extra space* is required (LC 164), so O(n log n) sorting is not allowed.

How it works

Any comparison sort needs about n $\log_2$ n comparisons at worst. You avoid them when the key can index an array. Counting sort fills `cnt[v]` for each value v in a range of size K, then writes each value `cnt[v]` times in order: O(n + K).

A frequency is a key from 1 to n, so a value can go into `bucket[f]` by how often it appears. Reading the buckets from `f = n` down gives values from most to least frequent in O(n), without sorting the frequencies.

Maximum gap uses the pigeonhole principle. n values in `[min, max]` leave n – 1 gaps that add up to max – min, so the largest gap is at least (max – min) / (n – 1). Make buckets of `width = max(1, (max - min) / (n - 1))`. Two values in one bucket differ by less than `width`, which is at most the answer, so the answer lies between buckets: a non-empty bucket's min minus the previous non-empty bucket's max.

Worked example LC 164: a = [1, 14, 3, 9, 6, 2]

```
n = 6, min = 1, max = 14, width = 13 / 5 = 2, buckets = 13 / 2 + 1 = 7
bucket  range    values  min  max
  0     1..2     1, 2     1    2
  1     3..4     3        3    3
  2     5..6     6        6    6
  3     7..8     -
  4     9..10    9        9    9
  5     11..12   -
  6     13..14   14      14   14
gaps between buckets: 3-2 = 1, 6-3 = 3, 9-6 = 3, 14-9 = 5  -> answer 5
```

Top k frequent: buckets by frequency

memorise

```java
// LC 347: the k most frequent values. A count is at most n, so bucket[f]
// lists the values seen exactly f times; read the buckets from f = n down.
static int[] topKFrequent(int[] nums, int k) {
    Map<Integer, Integer> freq = new HashMap<>();
    for (int x : nums) freq.merge(x, 1, Integer::sum);
    List<List<Integer>> bucket = new ArrayList<>();
    for (int f = 0; f <= nums.length; f++) bucket.add(new ArrayList<>());
    for (Map.Entry<Integer, Integer> e : freq.entrySet())
        bucket.get(e.getValue()).add(e.getKey());
    int[] out = new int[k];
    int n = 0;
    for (int f = nums.length; f >= 1 && n < k; f--)
        for (int x : bucket.get(f))
            if (n < k) out[n++] = x;
    return out;
}
```

Sort characters by frequency

example

```java
// LC 451: characters by frequency, most frequent first.
static String frequencySort(String s) {
    int[] cnt = new int[128];                       // 1. counting sort's count
    for (int i = 0; i < s.length(); i++) cnt[s.charAt(i)]++;
    StringBuilder[] bucket = new StringBuilder[s.length() + 1];
    for (char c = 0; c < 128; c++) {                // 2. bucket by count
        if (cnt[c] == 0) continue;
        if (bucket[cnt[c]] == null) bucket[cnt[c]] = new StringBuilder();
        for (int i = 0; i < cnt[c]; i++) bucket[cnt[c]].append(c);
    }
    StringBuilder out = new StringBuilder();
    for (int f = s.length(); f >= 1; f--)           // 3. read high to low
        if (bucket[f] != null) out.append(bucket[f]);
    return out.toString();
}
```

Maximum gap: pigeonhole buckets

example

```java
// LC 164: largest gap between neighbours in sorted order, in O(n).
// Pigeonhole: n values in [min, max] leave a gap >= (max - min) / (n - 1).
// Buckets narrower than that cannot hold the answer inside them, so only
// each bucket's min and max matter.
static int maximumGap(int[] a) {
    int n = a.length, min = Integer.MAX_VALUE, max = Integer.MIN_VALUE;
    for (int x : a) { min = Math.min(min, x); max = Math.max(max, x); }
    if (n < 2 || min == max) return 0;
    int width = Math.max(1, (max - min) / (n - 1));
    int m = (max - min) / width + 1;                // number of buckets
    int[] lo = new int[m], hi = new int[m];
    Arrays.fill(lo, Integer.MAX_VALUE);
    Arrays.fill(hi, Integer.MIN_VALUE);
    for (int x : a) {
        int b = (x - min) / width;
        lo[b] = Math.min(lo[b], x);
        hi[b] = Math.max(hi[b], x);
    }
    int best = 0, prev = hi[0];                     // bucket 0 holds min
    for (int b = 1; b < m; b++) {
        if (lo[b] == Integer.MAX_VALUE) continue;   // empty bucket
        best = Math.max(best, lo[b] - prev);
        prev = hi[b];
    }
    return best;
}
```

Cost Time `O(n + K)` Space `O(n + K)`

K is the key range; 128 characters, n + 1 frequencies, or fewer than 2n gap buckets.

Variants same idea, small change

- H-index (LC 274): counts in buckets 0..n, with every paper of ≥ n citations in bucket n; walk down from n until the running total reaches the bucket index.
- Height checker (LC 1051): counting sort of heights 1..100, then compare with the input.
- Contains duplicate III (LC 220): buckets of width `valueDiff + 1` over a sliding window of `indexDiff`; a match sits in the same or a neighbouring bucket.
- Top k with a heap instead: O(n log k), see 4.7 **Top k with a size-k heap** (p. 116).

Traps bugs people actually write

- A bucket array sized by the number of distinct values instead of n + 1: a value seen n times indexes past the end.
- `(max - min) / (n - 1)` is 0 when the values are close together; without `Math.max(1, ...)` the bucket index divides by zero.
- Counting sort over values up to 10^9 allocates gigabytes. It pays only when K is about n or less; otherwise sort normally, or index by rank (3.11 **Coordinate compression** (p. 96)).

Practise on LeetCode, in this order

- [ ] LC 1051 Height Checker — Easy
- [ ] LC 451 Sort Characters By Frequency — Medium
- [ ] LC 347 Top K Frequent Elements — Medium
- [ ] LC 274 H-Index — Medium
- [ ] LC 164 Maximum Gap — Medium
- [ ] LC 220 Contains Duplicate III — Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

3.11 Coordinate compression

`O(n log n)`

Spot it when the problem looks like this

- Values up to 10^9 or below 0, n ≤ 10^5, and you want an array or a Fenwick tree indexed by value.
- Only comparisons between values matter, never their size: *rank*, *count smaller*, *how many before*.
- Interval ends or rectangle corners on a huge grid, with only a few hundred distinct coordinates.

How it works

Sort a copy and remove duplicates; a value's position in that list is its rank. Ranks keep the order (`a[i] < a[j]` exactly when `rank[i] < rank[j]`), equal values get equal ranks, and the ranks run from 0 to m − 1 with m ≤ n. Any algorithm that only compares values gives the same answer on the ranks.

This matters because a Fenwick tree (**13.2 Fenwick tree (binary indexed)** (p. 366)) or a count array needs one slot per possible value. For values up to 10^9 that is 4 GB of `int`s; for ranks it is n + 1 slots. To count inversions, walk from the right and ask the tree how many ranks below the current one are already in it.

Each lookup is a binary search in the unique list, O(log m). Every value is in the list, so `Arrays.binarySearch` is safe here. For a value that may be missing (a query point between coordinates), use **3.1 Lower bound and upper bound** (p. 68).

Worked example **a = [1000000000, -7, 42, -7, 500]**

```
sorted = [-7, -7, 42, 500, 1000000000]
unique = [-7, 42, 500, 1000000000]     m = 4
rank   = [3, 0, 1, 0, 2]
inversions, right to left; the tree holds the ranks seen so far:
i  a[i]        rank  ranks below already in tree   total
4  500          2    0                            0
3  -7           0    0                            0
2  42           1    1  (the -7 at i = 3)         1
1  -7           0    0                            1
0  1000000000   3    4                            5
```

Sort, unique, binary search

memorise

```java
// Replace each value by its rank 0..m-1 among the distinct values.
// Order is kept: a[i] < a[j] exactly when rank[i] < rank[j].
static int[] compress(int[] a) {
    int[] vals = a.clone();
    Arrays.sort(vals);                                  // 1. sort
    int m = 0;
    for (int i = 0; i < vals.length; i++)               // 2. unique, in place
        if (i == 0 || vals[i] != vals[i - 1]) vals[m++] = vals[i];
    int[] rank = new int[a.length];
    for (int i = 0; i < a.length; i++)                  // 3. binary search
        rank[i] = Arrays.binarySearch(vals, 0, m, a[i]);   // present, so >= 0
    return rank;
}
```

Inversions with a Fenwick tree over ranks

example

```java
// Inversions with a Fenwick tree indexed by rank. Values may be up to 1e9
// or negative; ranks are < n, so the tree needs n + 1 slots, not 1e9.
static long inversions(int[] a) {
    int[] rank = compress(a);
    int n = a.length;
    int[] bit = new int[n + 1];                 // slot r + 1 counts rank r
    long inv = 0;
    for (int i = n - 1; i >= 0; i--) {          // right to left
        // how many ranks below rank[i] are in the tree; then add rank[i]
        for (int p = rank[i]; p > 0; p -= p & -p) inv += bit[p];
        for (int p = rank[i] + 1; p <= n; p += p & -p) bit[p]++;
    }
    return inv;
}
```

Cost Time `O(n log n)` Space `O(n)`

One sort plus n binary searches; the Fenwick tree then does O(log n) per step on n + 1 slots.

Variants same idea, small change

- Rank transform of an array (LC 1331): the ranks plus 1.
- A `HashMap<Integer, Integer>` from value to rank, filled after the unique step: O(1) lookups, but more memory and boxing.
- 2D: compress x and y separately. Rectangle area II (LC 850) then works on a grid of at most 2n × 2n cells, each with its real width and height.
- Falling squares (LC 699): compress the left and right edges, then use a plain array or a 13.3 **Segment tree (point update)** (p. 369) over the compressed points.

Traps bugs people actually write

- Skipping the unique step: the ranks get gaps. Rank transform (LC 1331) on `[40, 10, 10, 30]` then gives `[4, 2, 2, 3]`, not `[3, 1, 1, 2]`.
- Querying with a value that was not compressed: `binarySearch` returns a negative number. Add every query value before compressing, or use a lower bound.
- Updating Fenwick slot 0: `p += p & -p` adds 0 and loops forever. Rank r goes to slot r + 1.
- Grid problems need the real distances: cell i is `xs[i + 1] - xs[i]` wide, not 1.

Practise on LeetCode, in this order

- [ ] LC 1331 Rank Transform of an Array — Easy
- [ ] LC 1395 Count Number of Teams — Medium
- [ ] LC 2009 Minimum Number of Operations to Make Array Continuous — Hard
- [ ] LC 850 Rectangle Area II — Hard
- [ ] LC 699 Falling Squares — Hard
- [ ] LC 2426 Number of Pairs Satisfying Inequality — Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

Part 4

Stacks, queues and heaps

A stack keeps what is still open, with the most recent item on top; a heap hands you the smallest or largest item in O(log n). Use the stack patterns when the input nests or cancels, or when each element needs its nearest bigger or smaller neighbour. Use the heap patterns when you need the best k, a running median, the next item from k sorted sources, or the best option unlocked so far.

4.1	**Stack for matching and nesting** Nested brackets or encodings, or neighbours that cancel in pairs.	O(n)	p. 99
4.2	**Expression evaluation** Evaluate an arithmetic string or RPN tokens, with precedence.	O(n)	p. 102
4.3	**Monotonic stack: next greater** For each element, the nearest bigger or smaller one on a side.	O(n)	p. 105
4.4	**Largest rectangle in histogram** Best min(range) × width; the largest all-ones rectangle.	O(n)	p. 108
4.5	**Contribution counting** Sum of min, max or range over all subarrays, per element.	O(n)	p. 111
4.6	**Designing with stacks** A stack or queue class with one extra O(1) query.	O(1)	p. 113
4.7	**Top k with a size-k heap** The k largest, most frequent or closest items, or the kth one.	O(n log k)	p. 116
4.8	**Two heaps for a running median** The median of a stream or a sliding window, after every update.	O(log n)	p. 119
4.9	**K-way merge with a heap** Several sorted lists (real or implied) merged into one order.	O(N log k)	p. 121
4.10	**Greedy with a heap** Options unlock over time; take the best unlocked one each step.	O(n log n)	p. 124

4.1 Stack for matching and nesting

O(n)

Spot it when the problem looks like this

- Brackets, tags or markers open and close, and the one opened last must close first.
- A nested encoding such as `k[...]`, or a path where `..` cancels the folder before it.
- Neighbours cancel in pairs, and a removal can bring two new neighbours together (adjacent duplicates, asteroid collision).
- n up to 10^5, and one left-to-right scan is expected.

How it works

Scan left to right and push each opener. A closer must match the top of the stack, because the item opened most recently is the one that has to close first (last in, first out). If the stack is empty or the top does not match, the input is invalid; at the end the stack must be empty. Each character is pushed and popped at most once, so the scan is O(n).

When a level carries state, push the *outer* state as you enter and pop it as you leave. In decode string, `[` saves the repeat count and the text built so far; `]` pops them and appends the inner text `k` times.

Use `ArrayDeque` with `push`, `pop` and `peek`. `java.util.Stack` extends `Vector`: every call is synchronized, it lets you `get(i)` or insert in the middle, and its own Javadoc recommends `Deque` instead.

Worked example **s = "3[a2[c]]", stacks shown bottom to top**

```
c  k  counts  outer      cur
3  3  []      []         ""
[  0  [3]     [""]       ""
a  0  [3]     [""]       "a"
2  2  [3]     [""]       "a"
[  0  [3 2]   ["" "a"]   ""
c  0  [3 2]   ["" "a"]   "c"
]  0  [3]     [""]       "acc"          "a" + 2 x "c"
]  0  []      []         "accaccacc"    "" + 3 x "acc"
```

Valid parentheses

memorise

```
// Valid parentheses: each closer must match the most recent unclosed opener.
static boolean isValid(String s) {
    Deque<Character> st = new ArrayDeque<>();
    for (char c : s.toCharArray()) {
        if (c == '(') st.push(')');              // push the closer we expect
        else if (c == '[') st.push(']');
        else if (c == '{') st.push('}');
        else if (st.isEmpty() || st.pop() != c) return false;
    }
    return st.isEmpty();                         // nothing left open
}
```

Decode string: a count stack and a string stack

memorise

```java
// Decode string: "3[a2[c]]" -> "accaccacc".
// '[' saves the outer level (its count and its text); ']' restores it.
static String decodeString(String s) {
    Deque<Integer> counts = new ArrayDeque<>();
    Deque<StringBuilder> outer = new ArrayDeque<>();
    StringBuilder cur = new StringBuilder();
    int k = 0;
    for (char c : s.toCharArray()) {
        if (Character.isDigit(c)) {
            k = k * 10 + (c - '0');              // counts can have 2+ digits
        } else if (c == '[') {
            counts.push(k);
            outer.push(cur);
            cur = new StringBuilder();
            k = 0;
        } else if (c == ']') {
            StringBuilder prev = outer.pop();
            for (int r = counts.pop(); r > 0; r--) prev.append(cur);
            cur = prev;
        } else {
            cur.append(c);
        }
    }
    return cur.toString();
}
```

Simplify path and remove adjacent duplicates

variant

```java
// Simplify path: a stack of folder names; ".." pops one.
static String simplifyPath(String path) {
    Deque<String> st = new ArrayDeque<>();
    for (String part : path.split("/")) {
        if (part.isEmpty() || part.equals(".")) continue;
        if (!part.equals("..")) st.push(part);
        else if (!st.isEmpty()) st.pop();            // ".." at the root: no-op
    }
    StringBuilder sb = new StringBuilder();
    while (!st.isEmpty()) sb.append('/').append(st.pollLast());  // bottom first
    return sb.length() == 0 ? "/" : sb.toString();
}

// Remove all adjacent duplicates: the StringBuilder is the stack.
static String removeDuplicates(String s) {
    StringBuilder st = new StringBuilder();
    for (char c : s.toCharArray()) {
        int n = st.length();
        if (n > 0 && st.charAt(n - 1) == c) st.setLength(n - 1);  // pop
        else st.append(c);                                        // push
    }
    return st.toString();
}
```

Cost Time `O(n)` Space `O(n)`
Each character is pushed and popped at most once; decode string also pays for the length of its output.

Variants same idea, small change

- Minimum remove to make valid parentheses: push the *index* of each open bracket. A close bracket that meets an empty stack, and every index left at the end, mark characters to delete.
- Remove k adjacent duplicates: the stack holds `{char, run length}`; pop when the run hits k.
- Asteroid collision: push right-movers. A left-mover pops smaller right-movers until it meets a bigger one (it dies) or an equal one (both die).
- One bracket type: a counter replaces the stack, but it must never go below `0`.

Traps bugs people actually write

- Two `isEmpty()` checks are needed: before `pop()` (otherwise `")"` throws `NoSuchElementException`) and at the end (otherwise `"(("` is accepted).
- A one-digit count (`k = c - '0'`) decodes `12[a]` as `aa`; use `k = k * 10 + digit`.
- A for-each over an `ArrayDeque` runs from the top down, so a path built that way comes out backwards. Use `pollLast()`; note that `java.util.Stack` iterates bottom up.
- Rebuilding a `String` on each removal (`s = s.substring(0, i) + s.substring(i + 2)`) is $O(n^2)$, too slow at $n = 10^5$. Use a `StringBuilder` as the stack.

Practise on LeetCode, in this order

☐	`LC 20`	Valid Parentheses	Easy
☐	`LC 1047`	Remove All Adjacent Duplicates In String	Easy
☐	`LC 71`	Simplify Path	Medium
☐	`LC 394`	Decode String	Medium
☐	`LC 735`	Asteroid Collision	Medium
☐	`LC 1249`	Minimum Remove to Make Valid Parentheses	Medium

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

4.2 Expression evaluation `O(n)`

Spot it when the problem looks like this

- Evaluate an arithmetic string or token list, and `eval` or a library parser is not allowed.
- Operators come after their operands (Reverse Polish notation).
- `*` and `/` bind tighter than `+` and `-`, or a `-` in front of a bracket flips everything inside it.
- The statement says "integer division should truncate toward zero".

How it works

Postfix (RPN) is the easy case: push numbers; an operator pops `b` first, then `a`, and pushes `a op b`.

Infix with precedence (calculator II): remember the operator `op` that came *before* the number being read. When the number ends, `+` pushes `num`, `-` pushes `-num`, and `*` or `/` pops the last term, combines it with `num` and pushes the result. Every `*` and `/` is applied as soon as its right operand is known, so the stack only holds terms to add. Java's `/` truncates toward zero, so `-7 / 2` is -3, the same as `-(7 / 2)`. That is why pushing `-num` and dividing it later gives the right answer.

Brackets with only + and – (calculator): a number's final sign is the product of the signs of the brackets around it. Push the current sign on `(`, pop it on `)`, and add each number once with its sign. Every character is read once, so all three are O(n).

Worked example s = "3+5*2-7/2", terms shown bottom to top

```
at   op  num  action                    terms
+    +   3    push 3                    [3]
*    +   5    push 5                    [3 5]
-    *   2    pop 5, push 5 * 2 = 10    [3 10]
/    -   7    push -7                   [3 10 -7]
end  /   2    pop -7, push -7 / 2 = -3  [3 10 -3]
                                       sum = 10
```

Evaluate reverse Polish notation

memorise

```java
// Reverse Polish notation: operands wait on the stack for their operator.
static int evalRPN(String[] tokens) {
    Deque<Integer> st = new ArrayDeque<>();
    for (String t : tokens) {
        if (t.length() == 1 && "+-*/".indexOf(t.charAt(0)) >= 0) {
            int b = st.pop(), a = st.pop();          // b was pushed last
            if (t.equals("+")) st.push(a + b);
            else if (t.equals("-")) st.push(a - b);
            else if (t.equals("*")) st.push(a * b);
            else st.push(a / b);                     // truncates toward zero
        } else {
            st.push(Integer.parseInt(t));            // "-11" is a number
        }
    }
    return st.pop();
}
```

Basic calculator II: a stack of terms

memorise

```java
// Basic calculator II: + - * / with precedence, no brackets.
// The stack holds signed terms; * and / fold into the last term at once.
static int calculate2(String s) {
    Deque<Long> terms = new ArrayDeque<>();
    long num = 0;
    char op = '+';                                  // operator before num
    for (int i = 0; i < s.length(); i++) {
        char c = s.charAt(i);
        if (Character.isDigit(c)) num = num * 10 + (c - '0');
        boolean last = i == s.length() - 1;
        if (c == '+' || c == '-' || c == '*' || c == '/' || last) {
            if (op == '+') terms.push(num);
            else if (op == '-') terms.push(-num);
            else if (op == '*') terms.push(terms.pop() * num);
            else terms.push(terms.pop() / num);     // -7 / 2 == -3
            op = c;
            num = 0;
        }
    }
    long sum = 0;
    for (long t : terms) sum += t;
    return (int) sum;
}
```

Basic calculator: a stack of bracket signs

variant

```java
// Basic calculator: + - and brackets; '-' may be unary, as in "-(2 + 3)".
// A number's sign is the product of the signs of the brackets around it,
// so a stack of bracket signs is all you need.
static int calculate(String s) {
    Deque<Integer> signs = new ArrayDeque<>();
    signs.push(1);                                  // sign of the outside
    int result = 0, num = 0, sign = 1;              // sign of the next number
    for (char c : s.toCharArray()) {
        if (Character.isDigit(c)) {
            num = num * 10 + (c - '0');
        } else if (c == '+' || c == '-') {
            result += sign * num;
            num = 0;
            sign = signs.peek() * (c == '+' ? 1 : -1);
        } else if (c == '(') {
            signs.push(sign);                       // sign of this bracket
        } else if (c == ')') {
            signs.pop();
        }
    }
    return result + sign * num;
}
```

Cost Time `O(n)` Space `O(n)`

One pass over the input; each term or bracket sign is pushed and popped at most once.

Variants same idea, small change

- All four operators and brackets: on `(`, run the calculator II loop recursively from the next index; when it returns at `)`, treat its result as one number.
- Infix to postfix (shunting-yard): before pushing an operator, pop to the output every operator on top with higher or equal precedence.
- Parsing a boolean expression: push every character except `,`. On `)`, pop the operands down to `(`, then the operator, and push `t` or `f`.
- Baseball game: the same "operands wait on a stack" idea, with `+`, `D` and `C` as the operators.

Traps bugs people actually write

- `int a = st.pop(), b = st.pop();` swaps the operands: `["4", "13", "5", "/", "+"]` returns 4 instead of 6. The first pop is the *right* operand.
- `Math.floorDiv(-7, 2)` is -4. These problems want truncation toward zero, which is plain `/` in Java.
- In calculator II the last number has no operator after it. Without the `last` test it is never applied: `"3+2*2"` returns 5 instead of 7.
- In the bracket calculator, `sign = c == '+' ? 1 : -1` forgets the bracket's own sign: `"1-(2+3)"` returns 2 instead of -4. Multiply by `signs.peek()`.

Practise on LeetCode, in this order

☐	LC 682	Baseball Game	Easy
☐	LC 150	Evaluate Reverse Polish Notation	Medium
☐	LC 227	Basic Calculator II	Medium
☐	LC 224	Basic Calculator	Hard
☐	LC 1106	Parsing A Boolean Expression	Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

4.3 Monotonic stack: next greater

O(n)

Spot it when the problem looks like this

- For every element, the first element to its right (or left) that is bigger (or smaller).
- "How many days until a warmer day", "for how many days in a row was the price at most today's".
- n up to 10^5, so scanning right from every element ($O(n^2)$) is too slow.
- The array is circular, so the search wraps past the end.

How it works

Scan left to right and keep a stack of the indices that have not found their answer yet. **Invariant**: their values never increase from bottom to top. When `a[i]` arrives, every index on top with a smaller value

has now met its first bigger value to the right, so pop it and record `i`; then push `i`. An index `j` stays on the stack exactly until its first bigger value, because any bigger value between `j` and `i` would have popped it earlier.

Each index is pushed once and popped at most once, so the inner loop runs at most n times in total: O(n), not $O(n^2)$.

After the pops, the top of the stack is the nearest earlier value `≥ a[i]`, so the same pass also gives the *previous* greater-or-equal element. Flip `<` to `>` for next *smaller*. For a circular array, walk `2n` steps reading `a[i % n]`, and push only during the first lap.

Worked example **daily temperatures, a = [73, 74, 75, 71, 69, 72, 76, 73]**

```
i  a[i]  pops: next[j] = i      stack as values, bottom to top
0   73   -                      [73]
1   74   next[0]=1              [74]
2   75   next[1]=2              [75]
3   71   -                      [75 71]
4   69   -                      [75 71 69]
5   72   next[4]=5 next[3]=5    [75 72]
6   76   next[5]=6 next[2]=6    [76]
7   73   -                      [76 73]
next = [1, 2, 6, 5, 5, 6, -1, -1]
wait = next[i] - i, or 0  ->  [1, 1, 4, 2, 1, 1, 0, 0]
```

Next greater index, and daily temperatures

memorise

```java
// Next greater: for each i, the first j > i with a[j] > a[i], or -1.
// Invariant: the stack holds the indices still waiting for an answer,
// and their values never increase from bottom to top.
static int[] nextGreater(int[] a) {
    int n = a.length;
    int[] next = new int[n];
    Arrays.fill(next, -1);
    Deque<Integer> st = new ArrayDeque<>();
    for (int i = 0; i < n; i++) {
        while (!st.isEmpty() && a[st.peek()] < a[i])
            next[st.pop()] = i;              // a[i] answers every smaller top
        st.push(i);
    }
    return next;
}

// Daily temperatures: days to wait = next[i] - i, or 0 if never warmer.
static int[] dailyTemperatures(int[] t) {
    int[] next = nextGreater(t), wait = new int[t.length];
    for (int i = 0; i < t.length; i++) wait[i] = next[i] < 0 ? 0 : next[i] - i;
    return wait;
}
```

Next greater element II (circular)

variant

```
// Next greater element II (circular): walk the array twice, read a[i % n].
static int[] nextGreaterCircular(int[] a) {
    int n = a.length;
    int[] ans = new int[n];
    Arrays.fill(ans, -1);
    Deque<Integer> st = new ArrayDeque<>();
    for (int i = 0; i < 2 * n; i++) {
        int x = a[i % n];
        while (!st.isEmpty() && a[st.peek()] < x) ans[st.pop()] = x;
        if (i < n) st.push(i);                // the second lap only answers
    }
    return ans;
}
```

Online stock span: previous greater, as prices arrive

variant

```
// Online stock span: days in a row, ending today, with price <= today's.
// Stack of {price, span}; prices strictly decrease from bottom to top.
static class StockSpanner {
    private final Deque<int[]> st = new ArrayDeque<>();

    int next(int price) {
        int span = 1;                         // today
        while (!st.isEmpty() && st.peek()[0] <= price)
            span += st.pop()[1];              // absorb the days it covered
        st.push(new int[] {price, span});
        return span;
    }
}
```

Cost Time `O(n)` Space `O(n)`

Each index is pushed once and popped at most once; the stock spanner is O(1) amortised per call.

Variants same idea, small change

- Next greater element I: run the template on `nums2`, store `value → next greater value` in a map, then look up each `nums1[i]`.
- Final prices with a discount: next smaller-or-equal to the right, so pop while `a[st.peek()] >= a[i]`.
- Remove k digits: pop while k > 0 and the top digit is bigger than the new one. Cut any k still left from the end and strip leading zeros; the stack, read bottom up, is the answer.
- Sliding window maximum adds a second exit: indices also leave from the bottom when they fall out of the window (2.10 **Sliding window maximum (deque)** (p. 64)).

Traps bugs people actually write

- Pushing values instead of indices loses the distance that daily temperatures asks for. Push `i` and read `a[st.peek()]`.
- `<=` instead of `<` in the while also pops equal values, which gives next greater-*or-equal*: `[70, 70, 71]` answers 1 for day 0 instead of 2.
- A circular loop of only `n` steps leaves every answer that wraps at -1: `[1, 2, 1]` gives `[2, -1, -1]` instead of `[2, -1, 2]`.
- Stock span counts days with price `<=` today's. Popping only on `<` stops at an equal price: `[7, 7]` gives spans 1, 1 instead of 1, 2.

Practise on LeetCode, in this order

☐	LC 496	Next Greater Element I	Easy
☐	LC 1475	Final Prices With a Special Discount in a Shop	Easy
☐	LC 739	Daily Temperatures	Medium
☐	LC 503	Next Greater Element II	Medium
☐	LC 901	Online Stock Span	Medium
☐	LC 402	Remove K Digits	Medium

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

4.4 Largest rectangle in histogram

O(n)

Spot it when the problem looks like this

- Largest rectangle, or a score of the form `min(range) × width` or `min(range) × sum`, over all contiguous ranges.
- A 0/1 matrix and the largest rectangle (not square) of ones.
- n up to 10^5, so trying every pair of borders ($O(n^2)$) is too slow.

How it works

The best rectangle has the height of some bar `i` and spreads from it as far as the bars are at least as high: strictly between the previous lower bar `left` and the next lower bar `right`. Its area is `h[i] × (right - left - 1)`, and the answer is the best of these n areas.

One increasing stack gives both bounds. When `h[i]` is not higher than the top bar, `i` is that bar's right bound and the bar below it on the stack is its left bound: pop and measure. A sentinel height 0 at `i = n` pops what is left. Each bar is pushed and popped once: O(n).

With equal heights, `>=` pops a bar early and measures it too short, but the last of the equal bars gets the full width, so the maximum is still right.

Maximal rectangle: for each row, `h[j]` counts the unbroken ones from `(row, j)` upwards. Run the histogram on every row.

Worked example h = [2, 1, 5, 6, 2, 3]; i = 6 is the sentinel 0

i	h[i]	popped: height x width = area	stack (indices)	best
0	2	-	[0]	0
1	1	2 x 1 = 2	[1]	2
2	5	-	[1 2]	2
3	6	-	[1 2 3]	2
4	2	6 x 1 = 6, 5 x 2 = 10	[1 4]	10
5	3	-	[1 4 5]	10
6	0	3 x 1 = 3, 2 x 4 = 8, 1 x 6 = 6	[6]	10

Largest rectangle, one pass with a sentinel

memorise

```
// Largest rectangle in a histogram. The stack holds indices with rising
// heights. When h[i] is not higher than the top bar, that bar cannot reach
// past i: pop it and measure the widest rectangle of its height.
static int largestRectangle(int[] h) {
    int n = h.length, best = 0;
    Deque<Integer> st = new ArrayDeque<>();
    for (int i = 0; i <= n; i++) {
        int cur = i == n ? 0 : h[i];             // sentinel 0 empties the stack
        while (!st.isEmpty() && h[st.peek()] >= cur) {
            int height = h[st.pop()];
            int left = st.isEmpty() ? -1 : st.peek();     // previous smaller
            best = Math.max(best, height * (i - left - 1)); // i: next smaller
        }
        st.push(i);
    }
    return best;
}
```

Previous and next smaller in two passes

variant

```
// The same answer in two clear passes: the nearest lower bar on each side.
static int largestRectangleTwoPass(int[] h) {
    int n = h.length, best = 0;
    int[] left = new int[n], right = new int[n];
    Deque<Integer> st = new ArrayDeque<>();
    for (int i = 0; i < n; i++) {                // previous smaller
        while (!st.isEmpty() && h[st.peek()] >= h[i]) st.pop();
        left[i] = st.isEmpty() ? -1 : st.peek();
        st.push(i);
    }
    st.clear();
    for (int i = n - 1; i >= 0; i--) {           // next smaller
        while (!st.isEmpty() && h[st.peek()] >= h[i]) st.pop();
        right[i] = st.isEmpty() ? n : st.peek();
        st.push(i);
    }
    for (int i = 0; i < n; i++)                  // bar i spans (left, right)
        best = Math.max(best, h[i] * (right[i] - left[i] - 1));
    return best;
}
```

Maximal rectangle in a binary matrix

usage

```java
// Maximal rectangle of '1's: each row is the floor of a histogram.
static int maximalRectangle(char[][] m) {
    if (m.length == 0) return 0;
    int[] h = new int[m[0].length];
    int best = 0;
    for (char[] row : m) {
        for (int j = 0; j < row.length; j++)
            h[j] = row[j] == '1' ? h[j] + 1 : 0;     // ones straight above
        best = Math.max(best, largestRectangle(h));
    }
    return best;
}
```

Cost Time `O(n)` Space `O(n)`

Each bar is pushed and popped once. Maximal rectangle runs it once per row, so O(rows × cols).

Variants same idea, small change

- Maximum subarray min-product: the same bounds, but the score is `a[i] × sum(left+1 .. right-1)` from prefix sums. Use `long`, take the modulo at the end.
- Maximum score of a good subarray (it must contain index k): the same bounds; keep only bars with `left < k < right`.
- Count submatrices with all ones: row heights again, with a stack per row that counts the rectangles ending at each column.

Traps bugs people actually write

- Without the sentinel, bars still on the stack at the end are never measured: `[1, 2, 3]` returns 0 instead of 4.
- The width is `i - left - 1`, where `left` is the new top *after* the pop. Using `i - popped` ignores the taller bars popped before it: `[2, 1, 2]` returns 2 instead of 3.
- In maximal rectangle a `'0'` must reset `h[j] = 0`. Keeping the old height joins ones across a gap.
- `h[i] × width` fits `int` at LeetCode's limits ($10^4 \times 10^5 = 10^9$). For bigger limits, or `min × sum`, use `long`.

Practise on LeetCode, in this order

☐	LC 84	Largest Rectangle in Histogram	Hard
☐	LC 85	Maximal Rectangle	Hard
☐	LC 1856	Maximum Subarray Min-Product	Medium
☐	LC 1504	Count Submatrices With All Ones	Medium
☐	LC 1793	Maximum Score of a Good Subarray	Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

4.5 Contribution counting

O(n)

Spot it when the problem looks like this

- Sum (or count) of `min`, `max` or `max - min` over **all** subarrays.
- n up to $3 \cdot 10^4$ or 10^5, so the $n^2/2$ subarrays are too many to visit.
- The answer is taken modulo $10^9 + 7$: the total is huge, so it has to be counted, not listed.

How it works

Turn the sum around. Instead of finding the minimum of each subarray, ask of each `a[i]`: in how many subarrays is it the minimum? Let `left` be the nearest index to the left with a value `≤ a[i]`, and `right` the nearest index to the right with a value `< a[i]`. Then `a[i]` is the minimum of exactly the subarrays that start in `(left, i]` and end in `[i, right)`: `(i - left) × (right - i)` of them. Add `a[i]` times that count. One monotonic stack pass (4.3 **Monotonic stack: next greater** (p. 105)) finds both bounds, so this is O(n).

Ties decide the comparisons. A subarray whose minimum appears twice must be counted once. With `≤` on the left and `<` on the right, only its leftmost minimum counts it. Strict on both sides counts it twice, and `≤` on both sides counts it zero times.

For maxima, flip the comparisons, or run the same code on `-a[i]`.

Worked example a = [3, 1, 2, 1]; the two 1s tie

```
i  a[i]  left(<=)  right(<)  starts x ends   a[i] x count
0   3      -1        1       1 x 1 = 1       3
1   1      -1        4       2 x 3 = 6       6
2   2       1        3       1 x 1 = 1       2
3   1       1        4       2 x 1 = 2       2
                                             sum = 13
< on both sides: 15 ([1, 2, 1] and [3, 1, 2, 1] counted twice)
<= on both sides: 11 (those two counted zero times)
```

Sum of subarray minimums

memorise

```java
// Sum of min(sub) over all subarrays, modulo 1e9+7.
// a[i] is the minimum of every subarray that starts in (left, i] and ends
// in [i, right): left = previous value <= a[i], right = next value < a[i].
static int sumSubarrayMins(int[] a) {
    final long MOD = 1_000_000_007L;
    int n = a.length;
    int[] left = new int[n], right = new int[n];
    Deque<Integer> st = new ArrayDeque<>();
    for (int i = 0; i < n; i++) {
        while (!st.isEmpty() && a[st.peek()] > a[i])
            right[st.pop()] = i;                // strictly smaller on the right
        left[i] = st.isEmpty() ? -1 : st.peek(); // smaller or equal on the left
        st.push(i);
    }
    while (!st.isEmpty()) right[st.pop()] = n;   // no smaller value after it
    long sum = 0;
    for (int i = 0; i < n; i++)
        sum = (sum + (long) a[i] * (i - left[i]) * (right[i] - i)) % MOD;
    return (int) sum;
}
```

Sum of subarray ranges, counted at pop time

variant

```java
// Sum of subarray ranges = sum of maxima - sum of minima. No modulo here,
// so return long. max(sub) = -min(-sub), so one helper does both.
static long subArrayRanges(int[] a) {
    int[] neg = new int[a.length];
    for (int i = 0; i < a.length; i++) neg[i] = -a[i];
    return -sumMins(neg) - sumMins(a);
}

// Same tie rule as above, but each index is counted when it is popped:
// i is its next smaller, the new top is its previous smaller-or-equal.
static long sumMins(int[] a) {
    int n = a.length;
    long sum = 0;
    Deque<Integer> st = new ArrayDeque<>();
    for (int i = 0; i <= n; i++) {               // i == n pops everything
        while (!st.isEmpty() && (i == n || a[st.peek()] > a[i])) {
            int j = st.pop();
            int left = st.isEmpty() ? -1 : st.peek();
            sum += (long) a[j] * (j - left) * (i - j);
        }
        st.push(i);
    }
    return sum;
}
```

Cost Time `O(n)` Space `O(n)`

One stack pass finds both bounds for every index; one more loop adds the contributions.

Variants same idea, small change

- Sum of subarray maxima: flip `>` to `<` in the while, or negate the array and negate the result.
- Sum of subarray ranges: the sum of maxima minus the sum of minima. There is no modulo, so keep a `long`.
- Number of subarrays with bounded maximum: count `(i - left) × (right - i)` only for the i where `L ≤ a[i] ≤ R`, using the bounds for maxima.
- Sum of total strength of wizards: the score is `min × sum`, so combine contribution with prefix sums of prefix sums.

Traps bugs people actually write

- Strict comparisons on both sides count ties twice: `[3, 1, 2, 1]` gives 15 instead of 13. `<=` on both sides gives 11.
- `a[i] * (i - left) * (right - i)` in `int` overflows: $3 \cdot 10^4 \times 1.5 \cdot 10^4 \times 1.5 \cdot 10^4 \approx 6.75 \cdot 10^{12}$. Cast `a[i]` to `long` first, then take the modulo.
- Indices still on the stack at the end have no smaller value to their right. Set `right = n` for them; if `right` stays at the default 0, their count is 0 or negative.
- A difference of two sums taken modulo M can be negative in Java. Use `((x - y) % M + M) % M`.

Practise on LeetCode, in this order

☐	LC 907	Sum of Subarray Minimums	Medium
☐	LC 2104	Sum of Subarray Ranges	Medium
☐	LC 795	Number of Subarrays with Bounded Maximum	Medium
☐	LC 2281	Sum of Total Strength of Wizards	Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

4.6 Designing with stacks

O(1)

Spot it when the problem looks like this

- "Design a class" with `push`, `pop`, `top` and one more query (`getMin`, most frequent) that must be O(1).
- Build a queue from stacks, or a stack from queues.
- Up to $3 \cdot 10^4$ calls with queries as frequent as updates, so scanning the stack on each query is too slow.

How it works

Each design stores extra state *per entry*, or splits the items over two stacks, so that every call stays O(1).

Min stack: store `{value, minimum of this entry and all below}`. A pop uncovers the entry below, which still holds the minimum of what is left, so `getMin` is a peek.

Queue from two stacks: push onto `in`; pop from `out`, and only when `out` is empty pour all of `in` into it. Pouring reverses the order, so the oldest item ends on top. An item is pushed and popped at most once on each stack, so n calls cost O(n) in total: O(1) amortised, though a single pop can move n items.

Max frequency stack: the c-th copy of `x` goes on stack `group[c]`. The top of the highest group is the most frequent value, and the most recent one on ties. After it leaves, `group[c - 1]` still holds its earlier copy, so nothing else needs fixing.

Worked example **max frequency stack: push 5 7 5 7 4 5, then pop four times**

```
op       group[c] stacks, bottom to top   maxFreq
push 5   1:[5]                            1
push 7   1:[5 7]                          1
push 5   1:[5 7] 2:[5]                    2
push 7   1:[5 7] 2:[5 7]                  2
push 4   1:[5 7 4] 2:[5 7]                2
push 5   1:[5 7 4] 2:[5 7] 3:[5]          3
pop  5   1:[5 7 4] 2:[5 7]                2
pop  7   1:[5 7 4] 2:[5]                  2
pop  5   1:[5 7 4]                        1
pop  4   1:[5 7]                          1
```

Min stack

memorise

```java
// Min stack: each entry stores {value, minimum of it and everything below}.
static class MinStack {
    private final Deque<int[]> st = new ArrayDeque<>();

    void push(int x) {
        int min = st.isEmpty() ? x : Math.min(x, st.peek()[1]);
        st.push(new int[] {x, min});
    }
    void pop() { st.pop(); }                // the entry below kept its own min
    int top() { return st.peek()[0]; }
    int getMin() { return st.peek()[1]; }
}
```

Queue from two stacks

memorise

```java
// Queue from two stacks. New items go on stack in; stack out holds the
// older items in pop order. Refill out from in only when out is empty.
static class MyQueue {
    private final Deque<Integer> in = new ArrayDeque<>();
    private final Deque<Integer> out = new ArrayDeque<>();

    void push(int x) { in.push(x); }
    int pop() { shift(); return out.pop(); }
    int peek() { shift(); return out.peek(); }
    boolean empty() { return in.isEmpty() && out.isEmpty(); }

    private void shift() {                  // each item moves once, ever
        if (out.isEmpty())
            while (!in.isEmpty()) out.push(in.pop());   // reverses the order
    }
}
```

Maximum frequency stack

variant

```java
// Maximum frequency stack: pop the most frequent value, the most recent
// one on ties. The c-th copy of x is pushed on stack group[c].
static class FreqStack {
    private final Map<Integer, Integer> freq = new HashMap<>();
    private final Map<Integer, Deque<Integer>> group = new HashMap<>();
    private int maxFreq = 0;

    void push(int x) {
        int f = freq.merge(x, 1, Integer::sum);
        maxFreq = Math.max(maxFreq, f);
        group.computeIfAbsent(f, k -> new ArrayDeque<>()).push(x);
    }

    int pop() {
        Deque<Integer> top = group.get(maxFreq);
        int x = top.pop();
        freq.merge(x, -1, Integer::sum);     // its earlier copy is still below
        if (top.isEmpty()) maxFreq--;        // group[maxFreq - 1] is not empty
        return x;
    }
}
```

Cost Time `O(1)` Space `O(n)`

Min stack and frequency stack are O(1) per call; the queue is O(1) amortised because each item moves from `in` to `out` once.

Variants same idea, small change

- Min stack with two stacks: push `x` onto `mins` when `x <= mins.peek()` (the `=` keeps duplicates); when the popped value equals `mins.peek()`, pop `mins` too.
- Stack from one queue: after each `offer`, rotate the queue `size - 1` times so the newest item is at the front. Push is O(n), pop is O(1).
- Queue with `getMin` (a sliding window minimum): use two min stacks as `in` and `out`; the answer is the smaller of their two minima.
- Stack with an increment of the bottom k: keep a lazy `inc[i]` per level, and pass it down to level `i - 1` on pop.

Traps bugs people actually write

- Two-stack min stack with `x < mins.peek()` instead of `<=`: push 1, push 1, pop, and `mins` is empty while a 1 is still on the stack.
- `st.pop() == mins.peek()` on two `Deque<Integer>` compares references, so equal values outside –128..127 compare false. Unbox into an `int` first, or use `.equals`.
- Pouring `in` into `out` while `out` still has items puts newer items above older ones, and the queue returns them out of order.
- In `FreqStack.pop`, forgetting to lower the count of x in `freq`: the next push of x lands one level too high.

Practise on LeetCode, in this order

- [ ] LC 232 Implement Queue using Stacks — Easy
- [ ] LC 225 Implement Stack using Queues — Easy
- [ ] LC 155 Min Stack — Medium
- [ ] LC 1381 Design a Stack With Increment Operation — Medium
- [ ] LC 895 Maximum Frequency Stack — Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

4.7 Top k with a size-k heap

`O(n log k)`

Spot it when the problem looks like this

- "k largest", "k most frequent", "k closest", or "the kth largest" after each new value.
- Values arrive one at a time (a stream), or n is much bigger than k.
- "Return the answer in any order", or an explicit tie-break rule for equal scores.

How it works

To keep the k **largest** values, use a **min**-heap of at most k items. Its root is the smallest value you are keeping, which is exactly the one to give up when something bigger arrives: offer the new value, and if the size is now k + 1, poll the root. After every step the heap holds the k largest values so far, and the root is the kth largest. Each value costs one offer and at most one poll on a heap of k + 1 items, so the total is O(n log k) time and O(k) memory, and it works on a stream.

Mirror it for the k **smallest** (k closest points): a max-heap of size k, with the worst point kept on top.

With a tie-break rule, write the comparator so the root is the *worst* candidate under the full rule. At the end, poll everything and reverse to get the best first.

Worked example k = 3, start with [4, 5, 8, 2], then add 3, 5, 10, 9, 4

```
step     min-heap (sorted)  polled  root = 3rd largest
init 4   [4]                -       4
init 5   [4 5]              -       4
init 8   [4 5 8]            -       4
init 2   [4 5 8]            2       4
add 3    [4 5 8]            3       4
add 5    [5 5 8]            4       5
add 10   [5 8 10]           5       5
add 9    [8 9 10]           5       8
add 4    [8 9 10]           4       8
```

Kth largest element in a stream

memorise

```java
// Kth largest in a stream: a MIN-heap holding the k largest values so far.
// Its root is the smallest of them, which is the kth largest overall.
static class KthLargest {
    private final PriorityQueue<Integer> heap = new PriorityQueue<>();
    private final int k;

    KthLargest(int k, int[] nums) {
        this.k = k;
        for (int x : nums) add(x);
    }

    int add(int x) {
        heap.offer(x);
        if (heap.size() > k) heap.poll();    // drop the smallest of k + 1
        return heap.peek();
    }
}
```

K closest points (a max-heap for the k smallest)

variant

```java
// K closest points to the origin: keep the k smallest distances, so use a
// MAX-heap of size k; its root is the worst point kept, the one to evict.
static int[][] kClosest(int[][] points, int k) {
    PriorityQueue<int[]> heap = new PriorityQueue<>(
        (p, q) -> Long.compare(dist(q), dist(p)));   // farthest on top
    for (int[] p : points) {
        heap.offer(p);
        if (heap.size() > k) heap.poll();
    }
    return heap.toArray(new int[0][]);               // any order is fine
}

static long dist(int[] p) { return (long) p[0] * p[0] + (long) p[1] * p[1]; }
```

Top k frequent words, with a tie-break

variant

```
// Top k frequent words, best first; equal counts in alphabetical order.
// The root must be the WORST candidate: lowest count, and on a tie the
// word that comes last alphabetically. Poll all, then reverse.
static List<String> topKFrequent(String[] words, int k) {
    Map<String, Integer> cnt = new HashMap<>();
    for (String w : words) cnt.merge(w, 1, Integer::sum);
    PriorityQueue<String> heap = new PriorityQueue<>((a, b) -> {
        int ca = cnt.get(a), cb = cnt.get(b);
        return ca != cb ? Integer.compare(ca, cb) : b.compareTo(a);
    });
    for (String w : cnt.keySet()) {
        heap.offer(w);
        if (heap.size() > k) heap.poll();
    }
    List<String> out = new ArrayList<>();
    while (!heap.isEmpty()) out.add(heap.poll());    // worst to best
    Collections.reverse(out);
    return out;
}
```

Cost Time `O(n log k)` Space `O(k)`

n offers and at most n polls, each on a heap of at most k + 1 items.

Variants same idea, small change

- Kth largest element in an array (one query): the same heap, or **3.9 Quickselect** (p. 90) in O(n) on average.
- Top k frequent elements: count with a map, then the size-k heap on the counts; or **3.10 Counting sort and buckets** (p. 93) by count in O(n).
- Subsequence of length k with the largest sum: a size-k min-heap of `{value, index}`, then sort the kept indices to restore the order.
- Kth smallest in a sorted matrix or among pair sums: the candidates come in sorted lists, so use **4.9 K-way merge with a heap** (p. 121).

Traps bugs people actually write

- Polling before offering (`if (heap.size() == k) heap.poll(); heap.offer(x);`) evicts the root even when x is smaller: with k = 1, adding 5 then 3 leaves 3.
- Comparator `(a, b) -> b - a` overflows when values are far apart, such as $2 \cdot 10^9$ and $-2 \cdot 10^9$. Use `Integer.compare(b, a)` or `Collections.reverseOrder()`.
- Iterating a `PriorityQueue` (for-each, `toString`, `toArray`) is not in sorted order; only repeated `poll()` is. Fine for "any order" answers, wrong for top k frequent words.
- Tie-break flipped: in the min-heap the *worse* word must be on top, so equal counts compare `b.compareTo(a)`. With `a.compareTo(b)`, "apple" is evicted before "banana".

Practise on LeetCode, in this order

- [ ] LC 703 Kth Largest Element in a Stream — Easy
- [ ] LC 2099 Find Subsequence of Length K With the Largest Sum — Easy
- [ ] LC 215 Kth Largest Element in an Array — Medium
- [ ] LC 973 K Closest Points to Origin — Medium
- [ ] LC 347 Top K Frequent Elements — Medium
- [ ] LC 692 Top K Frequent Words — Medium

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

4.8 Two heaps for a running median

`O(log n)`

Spot it when the problem looks like this

- The median after every insertion into a stream.
- The median of every window of size k.
- Up to $5 \cdot 10^4$ or 10^5 operations, so sorting for each query is too slow.
- A set split into a lower part and an upper part, where you need the values at the boundary.

How it works

Split the values seen so far into two halves. `low` is a max-heap with the smaller half, `high` a min-heap with the larger half. Keep two invariants: every value in `low` is ≤ every value in `high`, and `low` has the same size as `high` or one more. The median is then `low.peek()` when the count is odd, and the mean of the two roots when it is even.

To add `x`, offer it to `low`, move `low`'s largest to `high`, and if `high` is now bigger, move its smallest back. A value only crosses as the largest of the low side or the smallest of the high side, so the order invariant holds. An add is at most five heap operations, O(log n); the median is a peek.

A sliding window must also delete the value that leaves, and `PriorityQueue.remove(Object)` is O(k). Use two `TreeSet`s of indices instead, ordered by value and then index, so removal is O(log k).

Worked example add 5, 15, 1, 3, 8, 7; up = moved to high, back = moved back to low

add	up	back	low (max-heap)	high (min-heap)	median
5	5	5	[5]	[]	5.0
15	15	-	[5]	[15]	10.0
1	5	5	[1 5]	[15]	5.0
3	5	-	[1 3]	[5 15]	4.0
8	8	5	[1 3 5]	[8 15]	5.0
7	7	-	[1 3 5]	[7 8 15]	6.0

Find median from a data stream

memorise

```java
// Running median. low: max-heap, smaller half. high: min-heap, larger half.
// Invariant: every value in low <= every value in high, and
// low.size() == high.size() or low.size() == high.size() + 1.
static class MedianFinder {
    private final PriorityQueue<Integer> low =
        new PriorityQueue<>(Collections.reverseOrder());
    private final PriorityQueue<Integer> high = new PriorityQueue<>();

    void addNum(int x) {
        low.offer(x);                       // 1. x joins the low half
        high.offer(low.poll());             // 2. low's largest moves up
        if (high.size() > low.size())       // 3. low may be one bigger,
            low.offer(high.poll());         //    never smaller
    }

    double findMedian() {
        if (low.size() > high.size()) return low.peek();
        return ((long) low.peek() + high.peek()) / 2.0;
    }
}
```

Sliding window median with two TreeSets

variant

```java
// Sliding window median. A PriorityQueue cannot delete in O(log k), so use
// two TreeSets of INDICES ordered by value, then index (keeps duplicates).
static double[] medianSlidingWindow(int[] a, int k) {
    Comparator<Integer> byValue = (i, j) ->
        a[i] != a[j] ? Integer.compare(a[i], a[j]) : Integer.compare(i, j);
    TreeSet<Integer> low = new TreeSet<>(byValue), high = new TreeSet<>(byValue);
    double[] out = new double[a.length - k + 1];
    for (int i = 0; i < a.length; i++) {
        low.add(i);                                 // the same 3 steps
        high.add(low.pollLast());
        if (high.size() > low.size()) low.add(high.pollFirst());
        if (i < k - 1) continue;
        out[i - k + 1] = k % 2 == 1 ? a[low.last()]
                       : ((long) a[low.last()] + a[high.first()]) / 2.0;
        int old = i - k + 1;                        // leaves the window
        if (!low.remove(old)) high.remove(old);     // next add rebalances
    }
    return out;
}
```

Cost Time `O(log n)` Space `O(n)`

Each add is at most five heap operations, O(log n); a median is at most two peeks. The window version is O(n log k).

Variants same idea, small change

- Sliding window median with lazy deletion: two `PriorityQueue`s and a map of values waiting to be deleted; drop a root when it is marked. Track the real sizes yourself, because the heaps hold dead items.
- Sequentially ordinal rank tracker (the ith `get` returns the ith best): the same split, but `high` holds the best i items instead of half.
- Finding MK average: three ordered multisets (bottom k, middle, top k) and the middle's sum.
- Values in a small range such as 0..100: a count array, O(range) per query, no heaps.

Traps bugs people actually write

- Adding `x` straight to the smaller heap, without passing it through the other one, breaks the order: adding 5, 1, 10 gives `low = [5, 10]`, `high = [1]` and median 10 instead of 5.
- `(low.peek() + high.peek()) / 2` overflows `int` near 2^{31}, and `/ 2` instead of `/ 2.0` turns 3.5 into 3. Add as `long` and divide by `2.0`.
- A `TreeSet` of *values* drops duplicates: the window `[2, 2, 3]` becomes `{2, 3}`. Store indices and compare by value, then index.
- `PriorityQueue.remove(Object)` in the window loop makes it O(n·k): about $2.5 \cdot 10^9$ steps for n = 10^5 and k = $5 \cdot 10^4$.

Practise on LeetCode, in this order

☐	LC 295	Find Median from Data Stream	Hard
☐	LC 480	Sliding Window Median	Hard
☐	LC 2102	Sequentially Ordinal Rank Tracker	Hard
☐	LC 1825	Finding MK Average	Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

4.9 K-way merge with a heap

`O(N log k)`

Spot it when the problem looks like this

- k sorted lists, arrays or streams, and you need one sorted output or its first k items.
- The kth smallest pair sum or matrix cell: each row is a sorted list you can generate.
- "One element from each list" with the smallest spread.
- N up to 10^4–10^5 items in total, spread over up to 10^4 lists.

How it works

Every list is sorted, so the smallest value not yet output is at the front of one of them. Keep exactly one entry per list in a min-heap: its current front, as `(value, list, index)`. Poll the root, output it, and push the next element of the *same* list. The heap never holds more than k entries, so each of the N values

costs O(log k): O(N log k) in total, against O(N log N) for sorting everything and O(N·k) for scanning the k fronts.

The lists do not have to exist. In *k pairs with smallest sums*, row `i` is the sorted sequence `a[i] + b[0], a[i] + b[1], ...`, and you create the next entry of a row only when its current one is polled.

In *smallest range*, the fronts always cover every list, and the range is [heap root, largest front]. Moving any other list forward can only raise the maximum, so advance the list that holds the minimum, and stop when that list runs out.

Worked example **a = [1, 4, 5], b = [1, 3, 4], c = [2, 6]; entries are (value, list, index)**

heap, smallest first	poll	push	output
(1,a,0) (1,b,0) (2,c,0)	(1,a,0)	(4,a,1)	1
(1,b,0) (2,c,0) (4,a,1)	(1,b,0)	(3,b,1)	1 1
(2,c,0) (3,b,1) (4,a,1)	(2,c,0)	(6,c,1)	1 1 2
(3,b,1) (4,a,1) (6,c,1)	(3,b,1)	(4,b,2)	1 1 2 3
(4,a,1) (4,b,2) (6,c,1)	(4,a,1)	(5,a,2)	1 1 2 3 4
(4,b,2) (5,a,2) (6,c,1)	(4,b,2)	-	1 1 2 3 4 4
(5,a,2) (6,c,1)	(5,a,2)	-	1 1 2 3 4 4 5
(6,c,1)	(6,c,1)	-	1 1 2 3 4 4 5 6

Merge k sorted lists

memorise

```java
// Merge k sorted linked lists. The heap holds the current head of each
// list, so its root is the smallest value not yet written out.
static ListNode mergeKLists(ListNode[] lists) {
    PriorityQueue<ListNode> heap =
        new PriorityQueue<>((x, y) -> Integer.compare(x.val, y.val));
    for (ListNode head : lists) if (head != null) heap.offer(head);
    ListNode dummy = new ListNode(0), tail = dummy;
    while (!heap.isEmpty()) {
        ListNode node = heap.poll();                // smallest remaining
        tail.next = node;
        tail = node;
        if (node.next != null) heap.offer(node.next);    // same list, next
    }
    return dummy.next;
}
```

K pairs with the smallest sums

variant

```java
// K pairs with the smallest sums (a and b sorted). Row i is the sorted
// list a[i]+b[0], a[i]+b[1], ...; merge the rows, creating entries lazily.
// Heap entry {i, j}; compare sums as long (values reach +-1e9).
static List<List<Integer>> kSmallestPairs(int[] a, int[] b, int k) {
    PriorityQueue<int[]> heap = new PriorityQueue<>((x, y) ->
        Long.compare((long) a[x[0]] + b[x[1]], (long) a[y[0]] + b[y[1]]));
    for (int i = 0; i < Math.min(a.length, k); i++) heap.offer(new int[] {i, 0});
    List<List<Integer>> out = new ArrayList<>();
    while (out.size() < k && !heap.isEmpty()) {
        int[] e = heap.poll();
        int i = e[0], j = e[1];
        out.add(Arrays.asList(a[i], b[j]));
        if (j + 1 < b.length) heap.offer(new int[] {i, j + 1});
    }
    return out;
}
```

Smallest range covering k lists

variant

```java
// Smallest range that has an element from each of k sorted lists.
// Heap entry {value, list, index}: one per list. The range is
// [heap root, max]; only advancing the root's list can make it narrower.
static int[] smallestRange(List<List<Integer>> lists) {
    PriorityQueue<int[]> heap =
        new PriorityQueue<>((x, y) -> Integer.compare(x[0], y[0]));
    int max = Integer.MIN_VALUE;
    for (int i = 0; i < lists.size(); i++) {
        int v = lists.get(i).get(0);
        heap.offer(new int[] {v, i, 0});
        max = Math.max(max, v);
    }
    int[] best = {heap.peek()[0], max};
    while (true) {
        int[] e = heap.poll();                          // current minimum
        if (max - e[0] < best[1] - best[0]) best = new int[] {e[0], max};
        List<Integer> list = lists.get(e[1]);
        if (e[2] + 1 == list.size()) return best;       // that list is used up
        int v = list.get(e[2] + 1);
        heap.offer(new int[] {v, e[1], e[2] + 1});
        max = Math.max(max, v);
    }
}
```

Cost Time `O(N log k)` Space `O(k)`

N values in total, each pushed and polled once on a heap of at most k entries.

Variants same idea, small change

- Only two lists: no heap, two pointers (**2.5 Merge two sorted sequences** (p. 53), **5.1 Dummy head node** (p. 128)).
- Merge the lists in pairs, round after round: log k rounds of O(N) each, also O(N log k), with no heap.
- Kth smallest element in a sorted matrix: seed the heap with the first column and poll k times, or binary search on the value (**3.3 Binary search on the answer** (p. 74)).
- Super ugly number: list p is `primes[p] × ugly[0], primes[p] × ugly[1], ...`. Heap entry `(value, p, index)`; skip a value equal to the last one output.

Traps bugs people actually write

- `heap.offer(null)` throws `NullPointerException`, and LeetCode does pass empty lists (`[[]]`). Skip `null` heads.
- Seeding k pairs with all `n·m` pairs: 10^{10} entries at the limits. Seed only `(i, 0)` for the first `min(n, k)` rows.
- Pair sums reach $\pm 2 \cdot 10^9$, so `sum1 - sum2` can reach $4 \cdot 10^9$ and overflow `int`. Compare with `Long.compare` on `long` sums.
- Smallest range: stop as soon as one list runs out. Going on with fewer than k entries measures ranges that miss that list, and they look too good.

Practise on LeetCode, in this order

☐	LC 23	Merge k Sorted Lists	Hard
☐	LC 378	Kth Smallest Element in a Sorted Matrix	Medium
☐	LC 373	Find K Pairs with Smallest Sums	Medium
☐	LC 313	Super Ugly Number	Medium
☐	LC 632	Smallest Range Covering Elements from K Lists	Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

4.10 Greedy with a heap

`O(n log n)`

Spot it when the problem looks like this

- Each item has a requirement (capital, deadline, position) and a value (profit, duration, fuel).
- Choose at most k items, as many as possible, or as few stops as possible.
- An earlier choice can be swapped out later at no cost, because only the count or the total matters.
- Arrange letters so no two equal ones touch, or tasks with a cooldown between equal ones.

How it works

These problems have two keys: one says *when* an option becomes available (capital needed, position, deadline), the other says *how good* it is (profit, fuel, duration). Sort by the first key and walk it with a pointer; push each option that has become available into a heap ordered by the second. Each option enters and leaves the heap once: O(n log n).

IPO: taking the biggest affordable profit is never worse, because more capital only unlocks more projects.

Regret: commit first, undo later. In course schedule III, take courses by deadline; when the total time passes the deadline, drop the longest course taken so far. The count stays what it was before this course, and the time is as small as it can be, which leaves the most room for later courses. Refuelling stops work the same way: drive past stations; when the fuel runs out, refuel (in hindsight) at the biggest one passed.

Reorganize string: always place the most frequent letter that differs from the last one placed.

Worked example **IPO: k = 3, w = 1, projects as capital:profit = 0:2 1:1 2:6 5:4 9:3**

round	w	newly affordable	heap, max first	take	w after
1	1	0:2 1:1	[2 1]	2	3
2	3	2:6	[6 1]	6	9
3	9	5:4 9:3	[4 3 1]	4	13

IPO

memorise

```
// IPO: finish at most k projects, starting with capital w.
// Sort by the gate (capital needed); a max-heap ranks what you can afford.
static int findMaximizedCapital(int k, int w, int[] profits, int[] capital) {
    int n = profits.length;
    int[][] p = new int[n][];
    for (int i = 0; i < n; i++) p[i] = new int[] {capital[i], profits[i]};
    Arrays.sort(p, (x, y) -> Integer.compare(x[0], y[0]));
    PriorityQueue<Integer> heap = new PriorityQueue<>(Collections.reverseOrder());
    int next = 0;
    for (int round = 0; round < k; round++) {
        while (next < n && p[next][0] <= w)
            heap.offer(p[next++][1]);            // unlocked: wait in the heap
        if (heap.isEmpty()) break;               // nothing affordable
        w += heap.poll();                        // best profit available now
    }
    return w;
}
```

Regret: course schedule III and refuelling stops

variant

```java
// Course schedule III: {duration, lastDay}. Take courses by deadline; if
// the total time passes it, drop the longest course taken (max-heap).
static int scheduleCourse(int[][] courses) {
    Arrays.sort(courses, (x, y) -> Integer.compare(x[1], y[1]));
    PriorityQueue<Integer> taken =
        new PriorityQueue<>(Collections.reverseOrder());   // durations
    int time = 0;
    for (int[] c : courses) {
        taken.offer(c[0]);
        time += c[0];
        if (time > c[1]) time -= taken.poll();   // same count, less time
    }
    return taken.size();
}

// Minimum refuelling stops: drive past stations and remember their fuel;
// when you cannot go further, refuel (in hindsight) at the biggest one.
static int minRefuelStops(int target, int fuel, int[][] stations) {
    PriorityQueue<Integer> passed =
        new PriorityQueue<>(Collections.reverseOrder());   // fuel
    long reach = fuel;
    int stops = 0, i = 0;
    while (reach < target) {
        while (i < stations.length && stations[i][0] <= reach)
            passed.offer(stations[i++][1]);
        if (passed.isEmpty()) return -1;         // stuck before the target
        reach += passed.poll();
        stops++;
    }
    return stops;
}
```

Reorganize string

variant

```java
// Reorganize string: place the most frequent letter other than the last one.
// The last letter waits outside the heap, where its count can change safely.
static String reorganizeString(String s) {
    int[] cnt = new int[26];
    for (char c : s.toCharArray()) cnt[c - 'a']++;
    PriorityQueue<Integer> heap = new PriorityQueue<>((x, y) -> cnt[y] - cnt[x]);
    for (int c = 0; c < 26; c++) if (cnt[c] > 0) heap.offer(c);
    StringBuilder sb = new StringBuilder();
    int held = -1;                               // letter placed last
    while (!heap.isEmpty()) {
        int c = heap.poll();
        sb.append((char) ('a' + c));
        cnt[c]--;
        if (held >= 0 && cnt[held] > 0) heap.offer(held);
        held = c;
    }
    return sb.length() == s.length() ? sb.toString() : "";
}
```

Cost Time `O(n log n)` Space `O(n)`
Sorting is O(n log n), and each option is pushed and polled at most once. Reorganize string is O(n log 26).

Variants same idea, small change

- Task scheduler: the answer is `max(tasks.length, (f - 1) × (cooldown + 1) + m)`, where f is the highest count and m is how many letters have it. Simulate with a heap and a cooldown queue if you need the order.
- Maximum number of events: sort by start day, keep a min-heap of end days; each day drop expired events and attend the one that ends first.
- Furthest building you can reach: a min-heap of the climbs that used ladders; when it holds more than `ladders` climbs, pay for the smallest with bricks.
- Single-threaded CPU: sort by enqueue time, then a min-heap by `(processing time, index)`.

Traps bugs people actually write

- IPO: `p[next][0] <= w`, not `<`. A project that costs exactly your capital is affordable.
- IPO: stop when nothing is affordable. `w += heap.poll()` on an empty heap unboxes `null` and throws `NullPointerException`.
- Course schedule III sorted by duration instead of deadline: `[[2, 7], [4, 9], [2, 3]]` gives 2 instead of 3. Skipping a course that does not fit, instead of dropping the longest, gives 1 instead of 2 on `[[5, 5], [4, 6], [2, 6]]`.
- Reorganize string: changing a letter's count while it is inside the heap breaks the heap order. Poll it, change it, then offer it again.

Practise on LeetCode, in this order

☐	LC 621	Task Scheduler	Medium
☐	LC 767	Reorganize String	Medium
☐	LC 1353	Maximum Number of Events That Can Be Attended	Medium
☐	LC 502	IPO	Hard
☐	LC 630	Course Schedule III	Hard
☐	LC 871	Minimum Number of Refueling Stops	Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

Part 5

Linked lists

A linked list gives O(1) insert and delete at a node you hold but no indexing, so every pattern here is about which nodes you hold while you walk. A dummy head removes the special case of a changing head, and three pointers reverse links in place. Two pointers at different speeds or with a fixed gap find positions without knowing the length.

5.1	**Dummy head node** You build a result list node by node, or the head itself may be removed.	`O(m + n)`	p. **128**
5.2	**In-place reversal (prev, cur, next)** Reverse a list, a part of it, or every group of k nodes, with O(1) extra space.	`O(n)`	p. **131**
5.3	**Fast and slow pointers** Find the middle, detect a cycle, or find where it starts, in one pass and O(1) space.	`O(n)`	p. **134**
5.4	**Gap pointers and head switching** A position counted from the end, a rotation by k, or the node where two lists join.	`O(n)`	p. **137**
5.5	**LRU cache (hash map + linked list)** Design a cache with O(1) get and put that evicts the least recently used key.	`O(1)`	p. **139**

5.1 Dummy head node

`O(m + n)`

Spot it when the problem looks like this

- *Merge*, *partition*, *remove all nodes with value*, *add two numbers*: the output is a list you append to.
- The first node of the answer is not known in advance, or the old head may be deleted.
- *Return the head of the new list.*

How it works

Create one spare node, `dummy`, and a pointer `tail` to the last node of the result. Appending is always `tail.next = node; tail = node;`, even for the first node, and the answer is `dummy.next`. Without the dummy every loop needs an `if (head == null)` branch, and that branch is where the bugs are.

For deletion, stand on the node *before* the one you test and look at `prev.next`. The dummy is a "before" node for the head, so deleting the head is the same as deleting any other node. After an unlink, stay on `prev`: the new `prev.next` has not been tested yet.

Merging and partitioning relink the existing nodes, so they use O(1) extra space. Each step uses up one input node, so the cost is O(m + n).

Worked example mergeTwoLists(1 -> 2 -> 4, 1 -> 3 -> 4)

```
a            b            take  result so far
1 -> 2 -> 4  1 -> 3 -> 4  a 1   1
2 -> 4       1 -> 3 -> 4  b 1   1 -> 1
2 -> 4       3 -> 4       a 2   1 -> 1 -> 2
4            3 -> 4       b 3   1 -> 1 -> 2 -> 3
4            4            a 4   1 -> 1 -> 2 -> 3 -> 4
null         4            a is used up: tail.next = b
result: 1 -> 1 -> 2 -> 3 -> 4 -> 4  (ties take a, so the merge is stable)
```

Merge two sorted lists

memorise

```java
// Merge two sorted lists (LC 21). With a dummy, "append" never needs an if.
static ListNode mergeTwoLists(ListNode a, ListNode b) {
    ListNode dummy = new ListNode(0), tail = dummy;
    while (a != null && b != null) {
        if (a.val <= b.val) { tail.next = a; a = a.next; }
        else { tail.next = b; b = b.next; }
        tail = tail.next;
    }
    tail.next = (a != null) ? a : b;      // link the rest of the other list whole
    return dummy.next;
}
```

Remove nodes; partition with two dummies

variant

```java
// Remove every node with this val (LC 203). The head itself may go.
static ListNode removeElements(ListNode head, int val) {
    ListNode dummy = new ListNode(0, head), prev = dummy;
    while (prev.next != null) {
        if (prev.next.val == val) prev.next = prev.next.next;  // stay on prev
        else prev = prev.next;
    }
    return dummy.next;
}

// Partition (LC 86): nodes < x first; both groups keep their order.
static ListNode partition(ListNode head, int x) {
    ListNode lessHead = new ListNode(0), less = lessHead;
    ListNode moreHead = new ListNode(0), more = moreHead;
    for (ListNode p = head; p != null; p = p.next) {
        if (p.val < x) { less.next = p; less = p; }
        else { more.next = p; more = p; }
    }
    more.next = null;                     // cut the old link, or you make a cycle
    less.next = moreHead.next;
    return lessHead.next;
}
```

Add two numbers with a carry

variant

```java
// Add two numbers stored least significant digit first (LC 2).
static ListNode addTwoNumbers(ListNode a, ListNode b) {
    ListNode dummy = new ListNode(0), tail = dummy;
    int carry = 0;
    while (a != null || b != null || carry != 0) {   // carry can add a digit
        int s = carry;
        if (a != null) { s += a.val; a = a.next; }
        if (b != null) { s += b.val; b = b.next; }
        tail.next = new ListNode(s % 10);
        tail = tail.next;
        carry = s / 10;
    }
    return dummy.next;
}
```

Cost Time `O(m + n)` Space `O(1)`

One pass over each list. Merge and partition relink existing nodes; add two numbers creates only the output nodes.

Variants same idea, small change

- Remove duplicates from sorted list II (drop every value that repeats): from `prev`, if `prev.next` starts a run of equal values, link `prev.next` past the whole run.
- Odd even linked list: two tails (odd positions and even positions) in one pass, then link the odd tail to the even head.
- Add two numbers II (most significant digit first): reverse both lists (**5.2 In-place reversal (prev, cur, next)** (p. 131)), add, reverse the result. Or push the digits on two stacks.
- Merge k sorted lists: the same dummy and tail with a heap of the k heads (**4.9 K-way merge with a heap** (p. 121)). Sort list is **3.8 Merge sort with counting** (p. 87) with this merge.

Traps bugs people actually write

- Partition without `more.next = null`: the last node of the second group keeps its old `next`. On `[1, 4, 3, 2, 5, 2]`, x = 3, node 5 still points at the last 2, and the result is a cycle.
- Stepping after an unlink (`prev = prev.next` every time) skips a node: `[1, 6, 6, 2]` with val 6 keeps one 6.
- Add two numbers with `while (a != null || b != null)` loses the last carry: `[5] + [5]` gives `[0]` instead of `[0, 1]`.
- Returning `dummy` instead of `dummy.next` puts a stray 0 at the front of the answer.

Practise on LeetCode, in this order

- [ ] LC 21 Merge Two Sorted Lists — Easy
- [ ] LC 203 Remove Linked List Elements — Easy
- [ ] LC 86 Partition List — Medium
- [ ] LC 2 Add Two Numbers — Medium
- [ ] LC 82 Remove Duplicates from Sorted List II — Medium
- [ ] LC 328 Odd Even Linked List — Medium

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

5.2 In-place reversal (prev, cur, next)

O(n)

Spot it when the problem looks like this

- *Reverse the list*, *reverse from position left to right*, *reverse nodes in k-group*, *swap pairs*.
- *In one pass* or *with O(1) memory*: no copying values into an array.
- A step inside another problem, such as the back half of a palindrome check or digits stored most significant first.

How it works

Walk once with three pointers. `prev` is the head of the part already reversed and `cur` is the first node not yet reversed. Each step saves `next = cur.next`, points `cur.next` back at `prev`, and moves both pointers one node on. When `cur` is null, `prev` is the new head. Each arrow is turned once: O(n) time, O(1) space.

To reverse a segment of k nodes, stand on `pre`, the node before it (the dummy if the segment starts at the head), and run the same loop k times from `first = pre.next`. Afterwards `prev` is the segment's new first node and `cur` is the node after the segment, so two links reconnect it: `pre.next = prev` and `first.next = cur`. `first` is now the segment's last node, which makes it the `pre` of the next group in k-group reversal.

Worked example reverseList(1 -> 2 -> 3 -> 4), then the segment versions on 1..5

```
step  prev (reversed part)   cur (rest)
0     null                   1 -> 2 -> 3 -> 4
1     1                      2 -> 3 -> 4
2     2 -> 1                 3 -> 4
3     3 -> 2 -> 1            4
4     4 -> 3 -> 2 -> 1       null           return prev
---- segments of 1 -> 2 -> 3 -> 4 -> 5 ----
reverseBetween(head, 2, 4)   1 -> 4 -> 3 -> 2 -> 5
reverseKGroup(head, 2)       2 -> 1 -> 4 -> 3 -> 5
reverseKGroup(head, 3)       3 -> 2 -> 1 -> 4 -> 5   (short group kept)
```

Reverse the whole list

memorise

```java
// Reverse a whole list (LC 206).
// Invariant: prev = reversed part, cur = first node not yet reversed.
static ListNode reverseList(ListNode head) {
    ListNode prev = null, cur = head;
    while (cur != null) {
        ListNode next = cur.next;   // 1. save the rest
        cur.next = prev;            // 2. turn the arrow
        prev = cur;                 // 3. step both forward
        cur = next;
    }
    return prev;
}
```

Reverse a segment, between, k-group

memorise

```java
// Reverse the k nodes after pre and reconnect both ends.
// Returns the segment's new last node: the pre for the next segment.
static ListNode reverseAfter(ListNode pre, int k) {
    ListNode first = pre.next, prev = null, cur = first;
    for (int i = 0; i < k; i++) {
        ListNode next = cur.next;
        cur.next = prev;
        prev = cur;
        cur = next;
    }
    pre.next = prev;                // pre -> old k-th node
    first.next = cur;               // old first -> node after the segment
    return first;
}

// Reverse positions left..right, 1-based (LC 92).
static ListNode reverseBetween(ListNode head, int left, int right) {
    ListNode dummy = new ListNode(0, head), pre = dummy;
    for (int i = 1; i < left; i++) pre = pre.next;
    reverseAfter(pre, right - left + 1);
    return dummy.next;
}

// Reverse every full group of k; a short last group stays (LC 25).
static ListNode reverseKGroup(ListNode head, int k) {
    ListNode dummy = new ListNode(0, head), pre = dummy;
    while (true) {
        ListNode probe = pre;
        for (int i = 0; i < k && probe != null; i++) probe = probe.next;
        if (probe == null) break;   // fewer than k nodes left
        pre = reverseAfter(pre, k);
    }
    return dummy.next;
}
```

Recursive version

variant

```java
// Same, recursively: reverse the tail, then hang head after its old next.
static ListNode reverseRec(ListNode head) {
    if (head == null || head.next == null) return head;
    ListNode newHead = reverseRec(head.next);
    head.next.next = head;          // old next is now the tail: append head
    head.next = null;
    return newHead;
}
```

Cost Time `O(n)` Space `O(1)`

Each link is turned once. k-group also walks k nodes ahead per group to check that a full group remains, so each node is visited at most twice. The recursive version uses O(n) stack.

Variants same idea, small change

- Swap nodes in pairs (LC 24): k-group with k = 2. The problem forbids swapping values, which is exactly what relinking avoids.
- Reverse nodes in even-length groups: group sizes 1, 2, 3, ...; count each group's real size (the last may be short) and reverse it only if that size is even.
- Head insertion for left..right: keep `pre` fixed and move each following node to right after `pre`. Same cost, no reconnect step at the end.
- Palindrome list and reorder list reverse the back half (5.3 **Fast and slow pointers** (p. 134)).

Traps bugs people actually write

- Writing `cur.next = prev` before saving `next` loses the rest of the list: the loop stops after one node.
- Returning `head` instead of `prev`: the old head is now the tail with `next == null`, so the caller sees a one-node list.
- The recursive version is n calls deep. LC 206 has n ≤ 5000, which is fine, but a default 1 MB JVM stack can overflow from about 10^4 nodes. Use the loop.
- In k-group, reversing before checking that k nodes remain also reverses the short last group: 1..5 with k = 3 must end `4 -> 5`, not `5 -> 4`.

Practise on LeetCode, in this order

- [ ] LC 206 Reverse Linked List — Easy
- [ ] LC 92 Reverse Linked List II — Medium
- [ ] LC 24 Swap Nodes in Pairs — Medium
- [ ] LC 2074 Reverse Nodes in Even Length Groups — Medium
- [ ] LC 25 Reverse Nodes in k-Group — Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

5.3 Fast and slow pointers

O(n)

Spot it when the problem looks like this

- *Middle node*, *does the list have a cycle*, *return the node where the cycle begins*.
- *O(1) extra memory*, which rules out a `HashSet` of visited nodes.
- A value that maps to a value, where a repeat means a loop: the digits of a number, `i -> a[i]`.

How it works

`slow` moves one node per step and `fast` two. When `fast` reaches the end, `slow` has gone half as far: the middle. If there is a cycle, `fast` never reaches the end. Once both are in the cycle, the gap between them shrinks by one node per step, so they meet within one lap.

Where does the cycle start? Let `a` be the distance from the head to the start, `b` from the start to the meeting point, and `c` the cycle length. Slow walked `a + b` and fast `2(a + b)`. The difference is whole laps, so `a + b = m·c` and `a = m·c - b`: walking `a` steps from the meeting point lands on the start. Put `p` at the head and step `p` and `slow` together; they meet at the start.

Any function x → f(x) on a finite set is a list that ends in a cycle, so the same code finds a repeated value without storing what it has seen.

Worked example 1 -> 2 -> ... -> 8 with 8 linked back to 4 (a = 3, c = 5)

```
step  slow  fast
0     1     1
1     2     3
2     3     5
3     4     7
4     5     4
5     6     6     meet: b = 2, a + b = 5 = 1·c
phase 2: p from the head, slow from the meeting point
0     p=1   slow=6
1     p=2   slow=7
2     p=3   slow=8
3     p=4   slow=4  same node after a = 3 steps: the start
```

Middle node and cycle start

memorise

```java
// Middle node (LC 876). Even length: the second of the two middles.
static ListNode middleNode(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    return slow;
}

// First node of the cycle, or null (LC 142). Phase 1 alone answers LC 141.
static ListNode detectCycle(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) {                 // 1. they meet inside the cycle
            ListNode p = head;              // 2. head and meeting point are the
            while (p != slow) {             //    same distance from the start
                p = p.next;
                slow = slow.next;
            }
            return p;
        }
    }
    return null;                            // fast fell off the end: no cycle
}
```

Palindrome and reorder (middle + reverse)

variant

```java
// Palindrome list (LC 234): reverse the back half, walk both halves.
static boolean isPalindrome(ListNode head) {
    ListNode q = reverse(middleNode(head));
    for (ListNode p = head; q != null; p = p.next, q = q.next)
        if (p.val != q.val) return false;
    return true;
}

// Reorder to L0, Ln, L1, Ln-1, ... (LC 143): cut, reverse the back, zip.
static void reorderList(ListNode head) {
    if (head == null) return;
    ListNode mid = middleNode(head);
    ListNode q = reverse(mid.next);           // back half, reversed
    mid.next = null;                          // front half ends at mid
    for (ListNode p = head; q != null; ) {
        ListNode pNext = p.next, qNext = q.next;
        p.next = q; q.next = pNext;           // p -> q -> old p.next
        p = pNext; q = qNext;
    }
}
```

`reverse` is the loop from 5.2 **In-place reversal (prev, cur, next)** (p. 131).

Floyd on a function

variant

```java
// LC 287: walk i -> a[i] from index 0; the cycle starts at the duplicate value.
static int findDuplicate(int[] a) {
    int slow = 0, fast = 0;
    do { slow = a[slow]; fast = a[a[fast]]; } while (slow != fast);
    int p = 0;
    while (p != slow) { p = a[p]; slow = a[slow]; }
    return p;
}

// Happy number (LC 202): f(n) = sum of squared digits; happy if the loop is {1}.
static boolean isHappy(int n) {
    int slow = n, fast = n;
    do { slow = f(slow); fast = f(f(fast)); } while (slow != fast);
    return slow == 1;
}

static int f(int n) {
    int s = 0;
    for (; n > 0; n /= 10) s += (n % 10) * (n % 10);
    return s;
}
```

Cost Time `O(n)` Space `O(1)`

Phase 1 ends within a + c steps and phase 2 takes a steps. Palindrome and reorder are a few linear passes.

Variants same idea, small change

- Linked list cycle (LC 141): phase 1 only; return true when the two meet.
- Delete the middle node: start `fast` at `head.next.next`, so `slow` stops on the node before the middle.
- Length of the cycle: after they meet, walk one pointer around until it comes back, counting steps.
- Find the duplicate when you may change the array: **1.9 Cyclic sort (index as hash)** (p. 38) puts each value at its own index.

Traps bugs people actually write

- `while (fast.next != null)` without `fast != null` throws `NullPointerException` when `fast` steps past the end: `[1, 2]`.
- Compare nodes, `slow == fast`, not values: `1 -> 2 -> 1 -> 2 -> 1` has no cycle, but `slow.val == fast.val` says it has.
- Phase 2 needs both pointers to start at `head`. Starting with `fast = head.next` (fine for LC 141) moves the meeting point, and on the trace list phase 2 never ends.
- `isPalindrome` above leaves the back half reversed. Reverse it again before returning if the caller still uses the list.

Practise on LeetCode, in this order

- [ ] LC 876 Middle of the Linked List — Easy
- [ ] LC 141 Linked List Cycle — Easy
- [ ] LC 234 Palindrome Linked List — Easy
- [ ] LC 142 Linked List Cycle II — Medium
- [ ] LC 143 Reorder List — Medium
- [ ] LC 287 Find the Duplicate Number — Medium

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

5.4 Gap pointers and head switching

`O(n)`

Spot it when the problem looks like this

- *Remove the n-th node from the end, in one pass.*
- *Rotate the list to the right by k*, with k up to $2 \cdot 10^9$.
- *Two lists that may share a tail*: return the first common node with O(1) memory.

How it works

Two pointers a fixed gap apart tell you about the end without knowing the length. Move `fast` n + 1 links ahead of `slow`, both starting at a dummy, then move both until `fast` is null. `slow` is now n + 1 links before the end, the node *before* the n-th from the end, so `slow.next = slow.next.next` removes it. The dummy covers n = length, where the head itself goes. Rotation uses a gap of `k % len`, and `slow` stops on the new tail.

For the intersection, say the lists have private parts of lengths `a` and `b` and a shared tail of length `c`. Each pointer walks its own list, then switches to the other head. After `a + c + 1 + b` steps (the 1 is the step from null) both stand on the first shared node, or earlier if `a == b`. With no shared tail both reach null together, and `p == q` ends the loop with null.

Worked example **removeNthFromEnd(1 -> 2 -> 3 -> 4 -> 5, n = 2), then LC 160 sample 1**

```
step    fast   slow
start   dummy  dummy
gap 1   1      dummy
gap 2   2      dummy
gap 3   3      dummy    gap is n + 1 = 3 links
both    4      1
both    5      2
both    null   3        unlink slow.next (4): 1 -> 2 -> 3 -> 5
---- intersection: A = 4 1 [8 4 5], B = 5 6 1 [8 4 5] ----
p: 4 1 8 4 5 null 5 6 1 [8]
q: 5 6 1 8 4 5 null 4 1 [8]   after 2 + 3 + 1 + 3 = 9 steps both are on 8
```

Remove the n-th node from the end

memorise

```
// Remove the n-th node from the end (LC 19), one pass, 1 <= n <= length.
static ListNode removeNthFromEnd(ListNode head, int n) {
    ListNode dummy = new ListNode(0, head), fast = dummy, slow = dummy;
    for (int i = 0; i <= n; i++) fast = fast.next;   // 1. gap of n + 1 links
    while (fast != null) {                           // 2. keep the gap
        fast = fast.next;
        slow = slow.next;
    }
    slow.next = slow.next.next;                      // 3. slow is just before it
    return dummy.next;
}
```

Rotate right by k

variant

```
// Rotate right by k (LC 61): the last k % len nodes move to the front.
static ListNode rotateRight(ListNode head, int k) {
    if (head == null) return null;
    int len = 0;
    for (ListNode p = head; p != null; p = p.next) len++;
    k %= len;
    if (k == 0) return head;
    ListNode fast = head, slow = head;
    for (int i = 0; i < k; i++) fast = fast.next;    // gap of k
    while (fast.next != null) { fast = fast.next; slow = slow.next; }
    ListNode newHead = slow.next;      // slow = new tail, fast = old tail
    slow.next = null;
    fast.next = head;
    return newHead;
}
```

Intersection by switching heads

memorise

```
// First shared node of two lists, or null (LC 160). Each pointer walks its
// own list, then the other one: both cover a + c + b and meet at the join.
static ListNode getIntersectionNode(ListNode headA, ListNode headB) {
    ListNode p = headA, q = headB;
    while (p != q) {
        p = (p == null) ? headB : p.next;
        q = (q == null) ? headA : q.next;
    }
    return p;                                        // join, or null if none
}
```

Cost Time `O(n)` Space `O(1)`

One pass for the gap; rotation adds one pass to count the length; the head switch takes at most m + n + 1 steps.

Variants same idea, small change

- k-th node from the end: the same walk with a gap of k, starting at `head`. When `fast` is null, `slow` is the k-th node from the end.
- Swapping nodes (k-th from the start and k-th from the end): walk `fast` to the k-th node and keep it; then move `fast` and a new pointer from the head until `fast.next` is null. Swap the two values.
- Intersection by lengths: count both lengths, move the longer list's pointer ahead by the difference, then walk together.
- Rotate left by k is rotate right by `len - k % len`.

Traps bugs people actually write

- A gap of n leaves `slow` *on* the target, and the unlink drops the node after it: 1..5, n = 2 loses 5, not 4.
- Without the dummy there is no node before the head, so n = length needs a special case that returns `head.next`.
- Without `k %= len` the gap walk runs off the list (`NullPointerException`) whenever k ≥ len, and k can be $2 \cdot 10^9$.
- Compare nodes (`p != q`), not values: the LC sample has a 1 in each list, on different nodes.

Practise on LeetCode, in this order

☐	`LC 160`	Intersection of Two Linked Lists	Easy
☐	`LC 19`	Remove Nth Node From End of List	Medium
☐	`LC 61`	Rotate List	Medium
☐	`LC 1721`	Swapping Nodes in a Linked List	Medium

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

5.5 LRU cache (hash map + linked list)

`O(1)`

Spot it when the problem looks like this

- *Design* a class with `get(key)` and `put(key, value)`, both O(1) on average.
- *Evict the least recently used* (or the oldest, or the least frequent) key when the cache is full.
- Up to $2 \cdot 10^5$ calls, so scanning a list on every call (O(capacity)) is too slow.

How it works

No single structure does both jobs. A `HashMap` finds a key in O(1) but keeps no order. A doubly linked list keeps the order of use, front = newest and back = oldest, and can move or remove a node in O(1) *if you already hold it*. So the map stores key → node and the list links the nodes.

`get` looks the node up, unlinks it and adds it at the front. `put` on an existing key updates the value and moves the node the same way. A new key at full capacity first evicts `tail.prev`, the oldest node, from both the list and the map; the node stores its key so the map entry can be found. The sentinels `head` and `tail` give every real node a `prev` and a `next`, so unlink and insert need no null checks.

Worked example **capacity 2, the LC 146 sample**

```
op          returns  list after (front = newest, back = oldest)
put(1, 1)            [1=1]
put(2, 2)            [2=2, 1=1]
get(1)      1        [1=1, 2=2]
put(3, 3)            [3=3, 1=1]        full: evict 2 from the back
get(2)      -1       [3=3, 1=1]
put(4, 4)            [4=4, 3=3]        full: evict 1
get(1)      -1       [4=4, 3=3]
get(3)      3        [3=3, 4=4]
get(4)      4        [4=4, 3=3]
```

HashMap + doubly linked list

memorise

```
// LRU cache (LC 146): map key -> node; list order = recency, front = newest.
static class LRUCache {
    static class Node {
        int key, val;
        Node prev, next;
        Node(int key, int val) { this.key = key; this.val = val; }
    }

    private final int cap;
    private final Map<Integer, Node> map = new HashMap<>();
    private final Node head = new Node(0, 0), tail = new Node(0, 0);  // sentinels

    LRUCache(int capacity) {
        cap = capacity;
        head.next = tail;
        tail.prev = head;
    }

    int get(int key) {
        Node n = map.get(key);
        if (n == null) return -1;
        unlink(n);                          // used now: move to the front
        addFront(n);
        return n.val;
    }

    void put(int key, int value) {
        Node n = map.get(key);
        if (n != null) {                    // update counts as a use
            n.val = value;
            unlink(n);
            addFront(n);
            return;
        }
        if (map.size() == cap) {            // full: evict the oldest, tail.prev
            Node old = tail.prev;
            unlink(old);
            map.remove(old.key);            // this is why a node stores its key
        }
        n = new Node(key, value);
        map.put(key, n);
        addFront(n);
    }

    private void unlink(Node n) { n.prev.next = n.next; n.next.prev = n.prev; }

    private void addFront(Node n) {
        n.prev = head; n.next = head.next;
        head.next.prev = n; head.next = n;
    }
}
```

On LeetCode, write `class LRUCache` at the top level with `public` methods; here it is nested so all the files compile together.

LinkedHashMap version

variant

```java
// accessOrder = true moves an entry to the end on get/put; after each put,
// removeEldestEntry decides whether the first (oldest) entry goes.
static class LRUShort {
    private final Map<Integer, Integer> map;

    LRUShort(final int capacity) {
        map = new LinkedHashMap<Integer, Integer>(16, 0.75f, true) {
            protected boolean removeEldestEntry(Map.Entry<Integer, Integer> e) {
                return size() > capacity;
            }
        };
    }

    int get(int key) { return map.getOrDefault(key, -1); }
    void put(int key, int value) { map.put(key, value); }
}
```

Cost Time `O(1)` Space `O(capacity)`

Each call is one hash lookup and a few pointer changes; at most capacity + 2 nodes.

Variants same idea, small change

- LFU cache (harder): key → node, frequency → its own list, and `minFreq`. A use moves the node to the `freq + 1` list; evict the oldest node of the `minFreq` list.
- All O(1) data structure: a list of count buckets holding their keys, plus key → bucket.
- Browser history: a doubly linked list with a `cur` pointer; a new visit cuts off everything after `cur`.

Traps bugs people actually write

- Evicting before checking whether the key exists: with capacity 2, `put(1,1)`, `put(2,2)`, `put(2,20)` evicts key 1, so `get(1)` returns –1 instead of 1.
- A `get` that does not move the node to the front lets a key you just used be evicted next.
- LinkedHashMap without `accessOrder = true` keeps insertion order, a FIFO cache: the LC sample gives `[1, 2, -1, 3, 4]` instead of `[1, -1, -1, 3, 4]`.
- Java 8 rejects the diamond `<>` on an anonymous class: write `<Integer, Integer>`.

Practise on LeetCode, in this order

☐	LC 1472	Design Browser History	Medium
☐	LC 707	Design Linked List	Medium
☐	LC 146	LRU Cache	Medium
☐	LC 460	LFU Cache	Hard
☐	LC 432	All O`one Data Structure	Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

Part 6

Intervals and greedy

Sort by the right key, then make one pass in which every choice is final. A greedy choice is safe when an argument (often an exchange) shows it never blocks an optimal answer, and each window below gives that argument. Reach for this part when the input is a list of [start, end] pairs, a reach or fuel budget along a line, or an array where each position depends on both of its sides.

6.1	**Merge overlapping intervals** A list of [start, end] pairs; combine every group that overlaps into one interval.	O(n log n)	p. 143
6.2	**Insert and intersect intervals** The intervals are already sorted and disjoint; add one interval, or intersect two such lists, in one O(n) pass.	O(n)	p. 145
6.3	**Interval scheduling (sort by end)** Keep the most intervals with no overlap, or find the fewest points that hit every interval.	O(n log n)	p. 147
6.4	**Sweep line (start and end events)** How many intervals are active at the same moment, the peak load, or the outline of their union.	O(n log n)	p. 149
6.5	**Jump game (farthest reach)** From index i you may move up to a[i] steps (or a tap covers a range); can you reach the end, and in how few moves?	O(n)	p. 152
6.6	**Gas station (running balance)** Gains and costs along a route; find where the running balance never drops below zero, resetting when it does.	O(n)	p. 155
6.7	**Two passes: left, then right** Each position's answer depends on what lies to its left and what lies to its right.	O(n)	p. 157

6.1 Merge overlapping intervals

O(n log n)

Spot it when the problem looks like this

- The input is `int[][] intervals` of `[start, end]` pairs in no particular order.
- The question says *merge*, *union*, *total time covered* or *remove intervals covered by another*.
- n is up to 10^4 or 10^5, so one O(n log n) sort fits the budget.

How it works

Sort by start. The output list stays sorted and disjoint, and only its last interval can overlap the current one `cur`: every earlier output interval ends before `last` starts, and `last` starts no later than `cur`. So there are two cases. If `cur[0] <= last[1]` they overlap, so set `last[1] = max(last[1], cur[1])`. The `max` matters because `cur` may sit entirely inside `last`. Otherwise there is a gap and `cur` starts a new output interval.

The comparison decides what *touching* means. LeetCode 56 uses closed intervals, so `[1,4]` and `[4,5]` share the point 4 and merge: use `<=`. Half-open times such as meetings `[9, 10)` and `[10, 11)` do not overlap: use `<`.

The sort costs O(n log n) and the pass is O(n).

Worked example **intervals = [[8,10], [1,3], [2,6], [15,18], [6,7]]**

```
sorted: [1,3] [2,6] [6,7] [8,10] [15,18]
cur      last    action            out
[1,3]    -       first: new        [1,3]
[2,6]    [1,3]   2 <= 3: extend    [1,6]
[6,7]    [1,6]   6 <= 6: extend    [1,7]
[8,10]   [1,7]   8 > 7: new        [1,7] [8,10]
[15,18]  [8,10]  15 > 10: new      [1,7] [8,10] [15,18]
```

Merge intervals

memorise

```java
// Merge closed intervals [start, end]. Touching ones ([1, 4], [4, 5]) merge.
static int[][] merge(int[][] intervals) {
    Arrays.sort(intervals, (x, y) -> Integer.compare(x[0], y[0]));  // 1. by start
    List<int[]> out = new ArrayList<>();
    for (int[] cur : intervals) {
        int[] last = out.isEmpty() ? null : out.get(out.size() - 1);
        if (last != null && cur[0] <= last[1]) {
            last[1] = Math.max(last[1], cur[1]);     // 2. overlaps: extend
        } else {
            out.add(new int[] {cur[0], cur[1]});     // 3. gap: start a new one
        }
    }
    return out.toArray(new int[0][]);
}
```

Remove covered intervals

variant

```java
// Remove covered intervals: count the ones not inside another interval.
// Equal starts: longer first, so a container is seen before what it covers.
static int removeCoveredIntervals(int[][] iv) {
    Arrays.sort(iv, (x, y) -> x[0] != y[0] ? Integer.compare(x[0], y[0])
                                           : Integer.compare(y[1], x[1]));
    int kept = 0, maxEnd = Integer.MIN_VALUE;
    for (int[] cur : iv) {
        if (cur[1] > maxEnd) {          // no earlier interval reaches this far
            kept++;
            maxEnd = cur[1];
        }
    }
    return kept;
}
```

An interval is covered when some earlier one (in this order) already reaches its end.

Cost Time `O(n log n)` Space `O(n)` The sort dominates; the output can hold all n intervals.

Variants same idea, small change

- Total length covered: merge, then add `end - start` per merged interval, or `end - start + 1` if you count integer points (points that intersect with cars).
- Partition labels: each letter spans from its first to its last occurrence. Merge those spans and print their lengths.
- Count days without meetings: merge the meetings, then subtract the covered days from the total.
- Intervals arrive one at a time and must stay merged: keep them in a `TreeMap<start, end>` (13.5 **Ordered maps (TreeMap, TreeSet)** (p. 375)).

Traps bugs people actually write

- `last[1] = cur[1]` instead of `Math.max(last[1], cur[1])`: merging `[1,10]` with `[2,3]` shrinks it to `[1,3]`.
- Sorting by end: in `[2,3] [4,5] [1,10]` the last interval overlaps both earlier ones, but the loop only compares it with `[4,5]`, so `[2,3]` survives.
- The comparator `(x, y) -> x[0] - y[0]` overflows when starts are near $\pm 2 \cdot 10^9$; use `Integer.compare`.
- `<` versus `<=` in the overlap test: check whether touching intervals count as overlapping before you write it.

Practise on LeetCode, in this order

☐	LC 2848	Points That Intersect With Cars	Easy
☐	LC 56	Merge Intervals	Medium
☐	LC 1288	Remove Covered Intervals	Medium
☐	LC 763	Partition Labels	Medium
☐	LC 3169	Count Days Without Meetings	Medium

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

6.2 Insert and intersect intervals

O(n)

Spot it when the problem looks like this

- The statement says the intervals are *sorted by start* and *non-overlapping*.
- You insert one new interval (merging if needed), or intersect two lists.
- There is no need to sort, so the expected cost is O(n) or O(m + n).

How it works

In a sorted, disjoint list the intervals that overlap the new one `[s, e]` form one block in the middle. So walk once, in three phases. Phase 1 copies every interval that ends before `s`. Phase 2 absorbs every

interval that starts at or before `e`: each one overlaps, so widen `s` and `e` to cover it. Then push `[s, e]`. Phase 3 copies the rest, which all start after `e`. Each interval is read once, so the cost is O(n).

To intersect two sorted lists `a` and `b`, keep one pointer in each. The common part of `a[i]` and `b[j]` is `[max of the starts, min of the ends]` when that is not empty. Then drop whichever interval ends first. If `a[i]` ends before `b[j]`, it cannot reach `b[j+1]`, which starts after `b[j]` ends.

Worked example **intervals = [[1,2], [3,5], [6,7], [8,10], [12,16]], insert [4,8]**

```
i  a[i]     phase                       s, e
0  [1,2]    1. ends 2 < 4: copy         4, 8
1  [3,5]    2. starts 3 <= 8: absorb    3, 8
2  [6,7]    2. starts 6 <= 8: absorb    3, 8
3  [8,10]   2. starts 8 <= 8: absorb    3, 10
            push [3,10]
4  [12,16]  3. starts 12 > 10: copy
result: [1,2] [3,10] [12,16]
```

Insert interval: three phases

memorise

```java
// intervals: sorted by start and disjoint. Insert [s, e], merging overlaps.
static int[][] insert(int[][] intervals, int[] add) {
    List<int[]> out = new ArrayList<>();
    int i = 0, n = intervals.length, s = add[0], e = add[1];
    while (i < n && intervals[i][1] < s)        // 1. ends before the new one
        out.add(intervals[i++]);
    while (i < n && intervals[i][0] <= e) {     // 2. overlaps: absorb it
        s = Math.min(s, intervals[i][0]);
        e = Math.max(e, intervals[i][1]);
        i++;
    }
    out.add(new int[] {s, e});
    while (i < n)                               // 3. starts after the new one
        out.add(intervals[i++]);
    return out.toArray(new int[0][]);
}
```

Interval list intersections

memorise

```java
// Intersections of two sorted, disjoint lists of closed intervals.
static int[][] intervalIntersection(int[][] a, int[][] b) {
    List<int[]> out = new ArrayList<>();
    int i = 0, j = 0;
    while (i < a.length && j < b.length) {
        int lo = Math.max(a[i][0], b[j][0]);
        int hi = Math.min(a[i][1], b[j][1]);
        if (lo <= hi) out.add(new int[] {lo, hi});
        if (a[i][1] < b[j][1]) i++;    // a[i] ends first: it cannot meet b[j+1]
        else j++;
    }
    return out.toArray(new int[0][]);
}
```

Cost Time `O(n)` Space `O(n)`

Each interval is read once; the output list is the only extra memory. Intersections cost O(m + n).

Variants same idea, small change

- Remove a range `[s, e)`: phases 1 and 3 copy as before; in phase 2 keep only the parts that stick out to the left (`[a, s)`) and to the right (`[e, b)`).
- Many inserts and queries (range module, count integers in intervals): store merged intervals in a `TreeMap<start, end>` and run phase 2 with `floorEntry` and `higherEntry` (13.5 **Ordered maps (TreeMap, TreeSet)** (p. 375)).
- My calendar I: a booking `[s, e)` fits if it overlaps no stored booking; two half-open intervals overlap when `max(starts) < min(ends)`.

Traps bugs people actually write

- Phase 1 tests the *end* (`intervals[i][1] < s`), phase 2 the *start* (`intervals[i][0] <= e`). Writing `intervals[i][0] < s` in phase 1 copies an interval that overlaps the new one.
- Pushing `[s, e]` inside the phase 2 loop, or only when phase 2 ran: an empty list, or a new interval that overlaps nothing, then loses it.
- Intersections: moving the pointer whose interval ends *later* loses overlaps. With `a = [[0,10]]` and `b = [[1,2],[5,6]]`, dropping `a[0]` first misses `[5,6]`.

Practise on LeetCode, in this order

☐	LC 57	Insert Interval	Medium
☐	LC 986	Interval List Intersections	Medium
☐	LC 729	My Calendar I	Medium
☐	LC 2276	Count Integers in Intervals	Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

6.3 Interval scheduling (sort by end)

`O(n log n)`

Spot it when the problem looks like this

- *Minimum number of intervals to remove* so the rest do not overlap: n minus the most you can keep.
- Fewest arrows, points or stabs so that every interval contains one.
- The longest chain in which each pair must start after the previous one ends.

How it works

Sort by **end** and walk once. Keep an interval if it does not overlap the last kept one; otherwise drop it. The exchange argument says why the earliest end is safe. Take any best selection. The greedy first pick ends no later than its first interval, so swapping it in keeps every other chosen interval compatible.

Repeat for the second pick, and so on: greedy is never behind, so it keeps as many. Sorting by start fails: `[1,100]` starts first and blocks `[2,3]`, `[4,5]`, `[6,7]`.

The same pass counts arrows. Shoot at the end of each kept interval. Kept intervals are pairwise disjoint, so each needs its own arrow. Each dropped one starts at or before the current arrow and, being later in end order, ends at or after it, so it is already burst. Only the tie rule changes: `>=` when touching intervals may both stay, `>` when a shared point bursts both.

Worked example **intervals = [[1,2], [2,3], [3,4], [1,3]], touching allowed (LC 435)**

```
sorted by end: [1,2] [2,3] [1,3] [3,4]
cur    lastEnd  decision         kept
[1,2]  -inf     first: keep      1
[2,3]  2        2 >= 2: keep     2
[1,3]  3        1 < 3: remove    2
[3,4]  3        3 >= 3: keep     3
removed = 4 - 3 = 1
```

Non-overlapping intervals (remove fewest)

memorise

```java
// Fewest intervals to remove so the rest do not overlap (LC 435).
// Here [1, 2] and [2, 3] do not overlap, so a start equal to lastEnd is fine.
static int eraseOverlapIntervals(int[][] iv) {
    Arrays.sort(iv, (x, y) -> Integer.compare(x[1], y[1]));   // by END
    int kept = 0;
    long lastEnd = Long.MIN_VALUE;
    for (int[] cur : iv) {
        if (cur[0] >= lastEnd) {     // compatible with everything kept: keep it
            kept++;
            lastEnd = cur[1];
        }                            // else drop it: it ends no earlier
    }
    return iv.length - kept;
}
```

Minimum arrows to burst balloons

memorise

```java
// Fewest arrows so every balloon [s, e] contains one (LC 452).
// Balloons that touch share a point, so a new arrow is needed only if s > arrow.
static int findMinArrowShots(int[][] b) {
    Arrays.sort(b, (x, y) -> Integer.compare(x[1], y[1]));    // not x[1] - y[1]
    int arrows = 0;
    long arrow = Long.MIN_VALUE;
    for (int[] cur : b) {
        if (cur[0] > arrow) {        // not burst yet: shoot at its end
            arrows++;
            arrow = cur[1];
        }
    }
    return arrows;
}
```

Cost Time `O(n log n)` Space `O(1)`

Sorting dominates; the pass keeps one number besides the sort's own memory.

Variants same idea, small change

- Maximum length of pair chain: `[c, d]` may follow `[a, b]` only if `c > b`. Run the arrows loop and return the count it keeps.
- Activity selection (most meetings in one room): return the kept count itself.
- Every interval must contain at least *two* chosen points (set intersection size at least two): sort by end, and by start descending on ties; add `end - 1` and `end` when an interval has no chosen point, only `end` when it has one.
- Intervals with weights (maximise total profit) break the greedy: use DP over intervals sorted by end, with a binary search for the last compatible one (3.1 **Lower bound and upper bound** (p. 68)).

Traps bugs people actually write

- The comparator `x[1] - y[1]` overflows for balloons near $\pm 2^{31}$ (LeetCode 452 tests this) and sorts wrongly; use `Integer.compare`.
- Starting `arrow` at `Integer.MIN_VALUE` with the test `s > arrow`: a balloon `[-2³¹, -2³¹]` is never counted. Start from `Long.MIN_VALUE`, or take the first interval before the loop.
- Sorting by start or by length and keeping whatever fits keeps too few; see `[1,100]` above.
- Mixing up `>=` and `>`: LeetCode 435 keeps both `[1,2]` and `[2,3]`; LeetCode 452 bursts both with one arrow at 2.

Practise on LeetCode, in this order

☐	LC 646	Maximum Length of Pair Chain	Medium
☐	LC 435	Non-overlapping Intervals	Medium
☐	LC 452	Minimum Number of Arrows to Burst Balloons	Medium
☐	LC 757	Set Intersection Size At Least Two	Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

6.4 Sweep line (start and end events)

`O(n log n)`

Spot it when the problem looks like this

- Most intervals overlapping at one point; fewest rooms, groups or platforms.
- A capacity limit along a line: never more than `capacity` passengers in the car.
- The answer is a shape over time: a skyline, a coverage count, the year with the most people alive.

How it works

Turn every interval into two events: `+1` at its start and `-1` at its end. Sort the events by position and add them up in that order. After each event the running sum is the number of intervals covering that position,

so the peak load is the largest sum. The peak is also the fewest groups: the intervals at the peak share a point, so each needs its own group, and that many groups always suffice.

Equal positions need a tie rule. With closed intervals `[1,5]` and `[5,8]`, point 5 is in both, so `+1` must come before `-1`. With half-open ones (a passenger leaves at `to`), `-1` comes first. Adding up all changes at one position in a `TreeMap` gives the half-open rule for free.

For the skyline, keep the live heights in a `TreeMap` used as a multiset. If the maximum height differs after all events at x, `[x, max]` is a key point. Sorting 2n events costs O(n log n).

Worked example **intervals = [[5,10], [6,8], [1,5], [2,3], [1,10]], closed (LC 2406)**

```
pos      1   1   2   3   5   5   6   8  10  10
event   +1  +1  +1  -1  +1  -1  +1  -1  -1  -1
open     1   2   3   2   3   2   3   2   1   0
best     1   2   3   3   3   3   3   3   3   3
At 5 the +1 of [5,10] comes before the -1 of [1,5]: they share point 5.
Answer: 3 groups.
```

Most overlapping intervals (fewest groups)

memorise

```java
// Most closed intervals [s, e] covering one point = fewest groups (LC 2406).
static int maxOverlap(int[][] iv) {
    int n = iv.length;
    int[][] ev = new int[2 * n][];
    for (int i = 0; i < n; i++) {
        ev[2 * i] = new int[] {iv[i][0], +1};       // starts here
        ev[2 * i + 1] = new int[] {iv[i][1], -1};   // ends here
    }
    // By position. Ties: +1 first, because [1, 5] and [5, 8] share point 5.
    // For half-open [s, e) put -1 first instead.
    Arrays.sort(ev, (x, y) -> x[0] != y[0] ? Integer.compare(x[0], y[0])
                                           : Integer.compare(y[1], x[1]));
    int open = 0, best = 0;
    for (int[] e : ev) {
        open += e[1];                               // intervals covering e[0]
        best = Math.max(best, open);
    }
    return best;
}
```

Car pooling: net change per stop

variant

```java
// Car pooling: trip = {people, from, to}; they leave the car at `to`.
// Summing all changes at one stop puts drop-offs before pick-ups for free.
static boolean carPooling(int[][] trips, int capacity) {
    TreeMap<Integer, Integer> change = new TreeMap<>();
    for (int[] t : trips) {
        change.merge(t[1], t[0], Integer::sum);
        change.merge(t[2], -t[0], Integer::sum);
    }
    int load = 0;
    for (int d : change.values()) {                // stops in increasing order
        load += d;
        if (load > capacity) return false;
    }
    return true;
}
```

The skyline problem

variant

```java
// Skyline: building = {left, right, height}. Live heights in a TreeMap
// (height -> count). After all events at x, the top changed => key point.
static List<List<Integer>> getSkyline(int[][] buildings) {
    List<int[]> ev = new ArrayList<>();
    for (int[] b : buildings) {
        ev.add(new int[] {b[0], b[2], +1});        // height starts at left
        ev.add(new int[] {b[1], b[2], -1});        // and stops at right
    }
    ev.sort((p, q) -> Integer.compare(p[0], q[0]));
    TreeMap<Integer, Integer> live = new TreeMap<>();
    live.put(0, 1);                                // the ground, never removed
    List<List<Integer>> out = new ArrayList<>();
    int prev = 0;
    for (int i = 0; i < ev.size(); ) {
        int x = ev.get(i)[0];
        for (; i < ev.size() && ev.get(i)[0] == x; i++) {    // every event at x
            int h = ev.get(i)[1];
            if (ev.get(i)[2] == 1) live.merge(h, 1, Integer::sum);
            else if (live.merge(h, -1, Integer::sum) == 0) live.remove(h);
        }
        int top = live.lastKey();
        if (top != prev) {
            out.add(Arrays.asList(x, top));
            prev = top;
        }
    }
    return out;
}
```

Cost Time `O(n log n)` Space `O(n)`

Sorting 2n events dominates; the skyline's TreeMap adds O(log n) per event.

Variants same idea, small change

- Small coordinates (car pooling stops ≤ 1000, years 1950 to 2050): use a difference array and a prefix sum instead of sorting (1.6 **Difference array** (p. 31)).
- Fewest rooms for half-open meetings: sort starts and ends separately and walk both; a room frees up when `end[j] <= start[i]`.
- My calendar II: keep a `TreeMap` of net changes, add the new booking, sweep, and undo the booking if the count ever reaches 3.
- Total length of the union: sweep and add `x - prevX` whenever the count before x was above 0.

Traps bugs people actually write

- Wrong tie order: with `-1` first, `[1,5]` and `[5,10]` look disjoint, so LeetCode 2406 answers 1 group for them instead of 2.
- Skyline: emitting a point per event instead of per position prints pairs like `[2,10], [2,15]`. Finish every event at x, then compare with the previous height.
- Skyline: `live.remove(h)` when two buildings share height h removes both. Keep a count per height.
- Skyline: without the ground `live.put(0, 1)`, `lastKey()` throws once the last building ends.

Practise on LeetCode, in this order

☐	LC 1854	Maximum Population Year	Easy
☐	LC 1094	Car Pooling	Medium
☐	LC 2406	Divide Intervals Into Minimum Number of Groups	Medium
☐	LC 731	My Calendar II	Medium
☐	LC 218	The Skyline Problem	Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

6.5 Jump game (farthest reach) O(n)

Spot it when the problem looks like this

- `a[i]` is the *maximum* jump length from index i.
- Fewest jumps, video clips or taps needed to cover `[0, n]` or `[0, time]`.
- n is up to 10^4 or 10^5, so the $O(n^2)$ DP over every jump is too slow or close to it.

How it works

Can reach. Scan left to right keeping `far`, the farthest index reachable from any index seen so far. Invariant: every index up to `far` is reachable, because each reachable `i` adds the whole range `i..i+a[i]` and these ranges join up. So the first `i > far` proves the end is out of reach.

Fewest jumps. This is BFS where each layer is a range of indices. Layer 0 is index 0. While scanning the layer that ends at `end`, track `far`; the next layer is `end+1..far`. When `i` reaches `end`, count a jump and set `end = far`. Each index is scanned once, so the cost is O(n), not O(n^2) edges.

Covering with ranges (taps, video clips). Store `reach[x]`, the farthest point a range starting at x covers, then run the same layered scan over positions. If `far` has not passed x when a layer ends, a gap is left: return -1.

Worked example a = [2, 3, 1, 1, 4], fewest jumps

```
i  a[i]  far  i == end?   jumps, end
0  2     2    yes: jump   1, 2
1  3     4    no          1, 2
2  1     4    yes: jump   2, 4
3  1     4    no          2, 4
answer 2 (0 -> 1 -> 4)
```

Jump game: can you reach the end?

memorise

```java
// Can you reach the last index? Invariant: every index <= far is reachable.
static boolean canJump(int[] a) {
    int far = 0;
    for (int i = 0; i < a.length; i++) {
        if (i > far) return false;          // i is past every reachable index
        far = Math.max(far, i + a[i]);
    }
    return true;
}
```

Jump game II: fewest jumps

memorise

```java
// Fewest jumps to the last index (LC 45 promises it is reachable).
// Indices up to `end` need at most `jumps` jumps; `far` is the next layer's end.
static int jump(int[] a) {
    int jumps = 0, end = 0, far = 0;
    for (int i = 0; i < a.length - 1; i++) {   // never jump from the last index
        far = Math.max(far, i + a[i]);
        if (i == end) {                        // layer finished: one more jump
            jumps++;
            end = far;
        }
    }
    return jumps;
}
```

Fewest taps to water a garden

variant

```java
// Fewest taps to water [0, n]; tap i covers [i - r[i], i + r[i]] (LC 1326).
// reach[x] = farthest point a tap starting at x covers. Then layer as in jump.
static int minTaps(int n, int[] r) {
    int[] reach = new int[n + 1];
    for (int i = 0; i <= n; i++) {
        int lo = Math.max(0, i - r[i]);
        reach[lo] = Math.max(reach[lo], Math.min(n, i + r[i]));
    }
    int taps = 0, end = 0, far = 0;
    for (int x = 0; x < n; x++) {
        far = Math.max(far, reach[x]);
        if (x == end) {                 // [0, end] is watered; extend it
            if (far <= x) return -1;    // nothing covers [x, x + 1]
            taps++;
            end = far;
        }
    }
    return taps;
}
```

Cost Time `O(n)` Space `O(1)` One pass; the taps version adds the `reach` array, O(n).

Variants same idea, small change

- Video stitching: clip `[s, e]` sets `reach[s] = max(reach[s], e)` for `s ≤ time`; then run the taps loop up to `time`.
- Jump game III and IV allow backward moves or jumps to equal values, so the reachable set is no longer a prefix: use BFS (9.2 **BFS shortest path (unweighted)** (p. 226)).
- Fewest refuelling stops: you decide which passed stations to use, which is a max-heap greedy (4.10 **Greedy with a heap** (p. 124)).

Traps bugs people actually write

- Jump game II looping to `i < n` instead of `i < n - 1`: when `end` lands on `n - 1`, the extra step counts one jump too many (3 for `[2, 3, 1, 1, 4]`).
- Updating `far` before testing `i > far`: then `far ≥ i` always holds, and `[0, 1]` reports true.
- Jumping as far as possible each time is wrong. From `[2, 3, 1, 1, 4]` it lands on index 2 and needs 3 jumps; the `far` scan picks the best landing spot in the layer.
- Taps: not clamping `i - r[i]` at 0 gives a negative index; leaving out the `far <= x` check carries on past a gap.

Practise on LeetCode, in this order

- [ ] LC 55 Jump Game — Medium
- [ ] LC 45 Jump Game II — Medium
- [ ] LC 1024 Video Stitching — Medium
- [ ] LC 1326 Minimum Number of Taps to Open to Water a Garden — Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

6.6 Gas station (running balance) O(n)

Spot it when the problem looks like this

- A circular route with `gas[i]` gained and `cost[i]` spent per step; travel around once.
- Return the start index or -1; the answer is unique if it exists.
- Fewest fixes so a running count never goes negative, such as unmatched brackets.
- n is up to 10^5, so trying every start ($O(n^2)$) is too slow.

How it works

Let `d[i] = gas[i] - cost[i]`. If the total of `d` is negative, no start works. Otherwise scan once with `tank`, the fuel gathered since `start`. When `tank` drops below 0 at station i, no start in `start..i` can pass i: a later start s only skips `start..s-1`, whose sum is ≥ 0 (or you would have reset there), so it reaches i with no more fuel. Set `start = i + 1` and `tank = 0`.

Why the last `start` works when the total is ≥ 0: each reset block has a negative sum but no negative prefix, so every suffix of a block is negative. Up to the last station the fuel is `tank`, which stayed ≥ 0. After the wrap, at station k, it is the total minus the stations still ahead, `k+1..start-1`: a block suffix plus whole blocks, all negative. So the fuel is at least the total, hence ≥ 0.

Brackets reset the same way: `+1` for `(`, `-1` for `)`. At -1 that `)` needs an added `(`: count it and reset to 0.

Worked example gas = [3, 1, 1, 5, 1], cost = [1, 4, 2, 1, 2]

```
i  d = gas - cost  tank  total  action
0  3 - 1 = 2          2      2
1  1 - 4 = -3        -1     -1  tank < 0: start = 2, tank = 0
2  1 - 2 = -1        -1     -2  tank < 0: start = 3, tank = 0
3  5 - 1 = 4          4      2
4  1 - 2 = -1         3      1
total = 1 >= 0 -> answer 3
start 0 reaches station 1 with 2 and still fails there; start 1 fails at once
```

Gas station

memorise

```java
// Index to start one full circle from, or -1. tank = fuel since `start`.
static int canCompleteCircuit(int[] gas, int[] cost) {
    long total = 0, tank = 0;
    int start = 0;
    for (int i = 0; i < gas.length; i++) {
        int d = gas[i] - cost[i];
        total += d;
        tank += d;
        if (tank < 0) {          // no start in start..i can pass station i
            start = i + 1;
            tank = 0;
        }
    }
    return total >= 0 ? start : -1;   // total >= 0: the last start works
}
```

Minimum add to make parentheses valid

variant

```java
// Fewest '(' or ')' to add so s is valid (LC 921). Same reset at balance < 0.
static int minAddToMakeValid(String s) {
    int balance = 0, added = 0;
    for (int i = 0; i < s.length(); i++) {
        balance += s.charAt(i) == '(' ? 1 : -1;
        if (balance < 0) {       // this ')' has no partner: add a '(' for it
            added++;
            balance = 0;
        }
    }
    return added + balance;      // every '(' still open needs a ')'
}
```

Cost Time `O(n)` Space `O(1)` One pass with two running sums.

Variants same idea, small change

- Kadane's maximum subarray is the same loop: reset the running sum when it goes negative and keep the best value seen (1.7 Kadane's maximum subarray (p. 34)).
- Each `(` needs `))` (minimum insertions): keep `need`, the `)` still owed. `(` adds 2, inserting one `)` first if `need` is odd; `)` subtracts 1, and at -1 insert a `(` and set `need = 1`.
- Minimum swaps to balance `[` and `]`: when a `]` would take the balance below 0, swap it with the last `[`; count a swap and set the balance to 1.

Traps bugs people actually write

- Returning `start` without the `total >= 0` check: for `gas = [2, 3, 4]`, `cost = [3, 4, 3]` the loop ends with `start = 2`, but the answer is -1.
- Resetting with `start = i` instead of `i + 1`: station i is the one that made the tank negative, so starting there fails at once.
- Brackets: returning only `added` forgets the `(` still open at the end; `"((("` needs 3.

Practise on LeetCode, in this order

☐	`LC 921`	Minimum Add to Make Parentheses Valid	Medium
☐	`LC 134`	Gas Station	Medium
☐	`LC 1963`	Minimum Number of Swaps to Make the String Balanced	Medium
☐	`LC 1541`	Minimum Insertions to Balance a Parentheses String	Medium

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

6.7 Two passes: left, then right

`O(n)`

Spot it when the problem looks like this

- A rule about both neighbours: a child rated higher than a neighbour gets more candy.
- Water above each bar, the distance to the nearest X on either side, a falling run before a day and a rising run after it.
- The naive answer scans both ways from every index, $O(n^2)$; the target is O(n).

How it works

Split the two-sided rule into two one-sided rules. Pass 1 runs left to right and computes `L[i]` from `L[i-1]`; pass 2 runs right to left and computes `R[i]` from `R[i+1]`. Then combine each position with `max`, `min` or `+`.

Candy. Pass 1 sets `c[i] = c[i-1] + 1` on every rise. Pass 2 does the same from the right and keeps the larger value. Each pass gives a lower bound (a rising run of length k forces k candies at its top), and their max obeys both rules, so the total is the minimum.

Trapping rain water. Bar i holds `min(maxL[i], maxR[i]) - h[i]`, set by the lower of the tallest walls on each side. Two passes fill `maxL` and `maxR`. To drop the arrays, move two pointers inward with running `lMax` and `rMax`. If `lMax < rMax`, the true right maximum of `l` is at least `rMax`, so bar `l` holds exactly `lMax - h[l]`: settle it and move `l`.

Worked example h = [0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1]

```
i       0  1  2  3  4  5  6  7  8  9 10 11
h       0  1  0  2  1  0  1  3  2  1  2  1
maxL    0  1  1  2  2  2  2  3  3  3  3  3
maxR    3  3  3  3  3  3  3  3  2  2  2  1
water   0  0  1  0  1  2  1  0  0  1  0  0   total 6
```

Candy

memorise

```java
// Every child gets >= 1; a higher rating than a neighbour => more candy.
static int candy(int[] rating) {
    int n = rating.length;
    int[] c = new int[n];
    Arrays.fill(c, 1);
    for (int i = 1; i < n; i++)             // pass 1: left neighbour rule
        if (rating[i] > rating[i - 1])
            c[i] = c[i - 1] + 1;
    for (int i = n - 2; i >= 0; i--)        // pass 2: right neighbour rule
        if (rating[i] > rating[i + 1])
            c[i] = Math.max(c[i], c[i + 1] + 1);    // keep pass 1's need too
    int sum = 0;
    for (int x : c) sum += x;
    return sum;
}
```

Trapping rain water: two passes

memorise

```java
// Water above bar i = min(tallest bar on its left, tallest on its right) - h[i].
static int trap(int[] h) {
    int n = h.length, water = 0;
    int[] maxL = new int[n], maxR = new int[n];
    for (int i = 0; i < n; i++)             // pass 1: left to right
        maxL[i] = Math.max(i > 0 ? maxL[i - 1] : 0, h[i]);
    for (int i = n - 1; i >= 0; i--)        // pass 2: right to left
        maxR[i] = Math.max(i < n - 1 ? maxR[i + 1] : 0, h[i]);
    for (int i = 0; i < n; i++)
        water += Math.min(maxL[i], maxR[i]) - h[i];
    return water;
}
```

Trapping rain water: two pointers

variant

```java
// O(1) space. If lMax < rMax, the true right max of l is >= rMax > lMax,
// so bar l holds exactly lMax - h[l]. Settle the smaller side and move it.
static int trapTwoPointers(int[] h) {
    int l = 0, r = h.length - 1, lMax = 0, rMax = 0, water = 0;
    while (l < r) {
        lMax = Math.max(lMax, h[l]);
        rMax = Math.max(rMax, h[r]);
        if (lMax < rMax) water += lMax - h[l++];
        else water += rMax - h[r--];
    }
    return water;
}
```

Cost Time `O(n)` Space `O(n)`

A few linear passes; the arrays cost O(n), the two-pointer version O(1).

Variants same idea, small change

- Shortest distance to a character: pass 1 stores `i - (last c on the left)`, pass 2 takes the min with `(next c on the right) - i`.
- Good days to rob the bank: `dec[i]` counts the days right before i that keep falling or stay level (left pass), `inc[i]` the days right after i that keep rising or stay level (right pass); day i is good if both are ≥ time.
- Longest mountain: `up[i]` from the left, `down[i]` from the right; a peak with both above `0` gives a mountain of `up + down + 1`.

Traps bugs people actually write

- Candy pass 2 writing `c[i] = c[i + 1] + 1` without `max` throws away pass 1: ratings `[1, 2, 3, 1]` give index 2 only 2 candies, the same as index 1.
- Equal ratings need no relation: `[1, 2, 2]` gets `1, 2, 1`. Testing `>=` hands out extra candy.
- Two pointers: adding `lMax - h[l]` before updating `lMax` makes the water negative when `h[l]` is a new maximum.

Practise on LeetCode, in this order

- [ ] `LC 821` Shortest Distance to a Character — Easy
- [ ] `LC 2100` Find Good Days to Rob the Bank — Medium
- [ ] `LC 845` Longest Mountain in Array — Medium
- [ ] `LC 135` Candy — Hard
- [ ] `LC 42` Trapping Rain Water — Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

Part 7

Recursion and backtracking

A backtracking function builds one answer a piece at a time: choose, recurse, undo. The recursion tree has a leaf for every candidate, so the cost is the number of candidates times the work per leaf, and limits like n ≤ 10, n ≤ 16 or n ≤ 20 are the clue. Pruning cuts branches that cannot succeed, and meet in the middle stretches the reach to n ≈ 40.

7.1	**Subsets (include or exclude)** List every subset, or every yes/no choice per element; n ≤ 20.	$O(n \cdot 2^n)$	p. 160
7.2	**Skip duplicates at the same depth** The input has repeated values, and the answer must not list the same subset or combination twice.	$O(n \cdot 2^n)$	p. 163
7.3	**Permutations (used[] array)** Every ordering of the input; n ≤ 8 to 10 (8! = 40,320 and 10! ≈ $3.6 \cdot 10^6$).	$O(n \cdot n!)$	p. 166
7.4	**Combinations from a start index** Choose k items, or items that add up to a target, where the order inside an answer does not matter.	$O(k \cdot C(n, k))$	p. 169
7.5	**Constraint backtracking (N-Queens)** Place items under rules that forbid conflicts (rows, columns, diagonals, boxes); list or count every valid board.	$O(n!)$	p. 172
7.6	**Grid DFS with mark and restore** Find or count paths on a small grid that use each cell at most once.	$O(mn \cdot 3^L)$	p. 175
7.7	**Cut a prefix, recurse on the rest** Split a string into pieces that each pass a test, listing every split; or build strings under counting rules.	$O(n \cdot 2^n)$	p. 178
7.8	**Meet in the middle** A subset question with n ≈ 30 to 40, where 2^n is too slow but $2^{n/2} \approx 10^6$ fits.	$O(2^{n/2} \cdot n)$	p. 181

7.1 Subsets (include or exclude) $O(n \cdot 2^n)$

Spot it when the problem looks like this

- *Return all possible subsets (the power set)*; every way to change the case of the letters.
- n is 10 to 20: 2^n fits ($2^{20} \approx 10^6$ subsets).
- Each element is in or out on its own, or has a small fixed number of choices.

How it works

`pick(i)` tries element i both ways: first without it, then with it (add to `path`, recurse, remove). When `i == n` every element is decided, so `path` is one subset. Store a **copy**: `path` is one list shared by the

whole search and keeps changing after you store it. The tree is binary with depth n, so it has 2^n leaves, and copying each costs up to n: $O(n \cdot 2^n)$. The recursion is only n deep.

Read the choices at a leaf as binary digits and the leaves come out in counting order: leaf 5 is in, out, in, which is `[1, 3]`. The bitmask loop uses this directly. `mask` runs from 0 to $2^n - 1$, and bit i set means `a[i]` is in. It needs no recursion, and it is the form to use when you test every subset, as in 7.8 **Meet in the middle** (p. 181) and 10.12 **Bitmask DP over subsets** (p. 309).

Letter case permutation is the same tree, with one branch at a digit and two at a letter.

Worked example **a = [1, 2, 3]: leaves in the order pick() reaches them**

leaf	a[0]	a[1]	a[2]	path
0	out	out	out	[]
1	out	out	in	[3]
2	out	in	out	[2]
3	out	in	in	[2, 3]
4	in	out	out	[1]
5	in	out	in	[1, 3]
6	in	in	out	[1, 2]
7	in	in	in	[1, 2, 3]

Subsets: include or exclude

memorise

```java
static List<List<Integer>> subsets(int[] a) {
    List<List<Integer>> out = new ArrayList<>();
    pick(a, 0, new ArrayList<>(), out);
    return out;
}

// Decide a[i]: leave it out, or put it in. At i == n, path is one subset.
static void pick(int[] a, int i, List<Integer> path, List<List<Integer>> out) {
    if (i == a.length) {
        out.add(new ArrayList<>(path));        // copy: path keeps changing
        return;
    }
    pick(a, i + 1, path, out);                 // 1. without a[i]
    path.add(a[i]);                            // 2. with a[i] ...
    pick(a, i + 1, path, out);
    path.remove(path.size() - 1);              // ... then undo the choice
}
```

Subsets: bitmask loop

memorise

```java
// The same 2^n subsets without recursion: bit i of mask means "a[i] is in".
static List<List<Integer>> subsetsBitmask(int[] a) {
    int n = a.length;
    List<List<Integer>> out = new ArrayList<>();
    for (int mask = 0; mask < (1 << n); mask++) {
        List<Integer> s = new ArrayList<>();
        for (int i = 0; i < n; i++)
            if ((mask >> i & 1) == 1) s.add(a[i]);
        out.add(s);
    }
    return out;
}
```

Letter case permutation

variant

```java
// Letter case permutation: a digit has one choice, a letter has two.
static List<String> letterCasePermutation(String s) {
    List<String> out = new ArrayList<>();
    flip(s.toCharArray(), 0, out);
    return out;
}

static void flip(char[] c, int i, List<String> out) {
    if (i == c.length) {
        out.add(new String(c));
        return;
    }
    flip(c, i + 1, out);                          // keep c[i] as it is
    if (Character.isLetter(c[i])) {
        c[i] ^= 32;                               // swap case: 'a' (97) <-> 'A' (65)
        flip(c, i + 1, out);
        c[i] ^= 32;                               // restore
    }
}
```

Cost Time $O(n \cdot 2^n)$ Space $O(n)$

2^n leaves, each copied in O(n); besides the output, the stack and the path hold n items.

Variants same idea, small change

- Sum of all subset XOR totals: pass the running XOR down instead of a path and add it at each leaf.
- Count subsets with a property (maximum bitwise OR, beautiful subsets): return a count from the recursion instead of storing lists, and prune a branch as soon as a choice breaks the rule.
- Subsets of a list with repeated values must not repeat: see 7.2 **Skip duplicates at the same depth** (p. 163). Subsets of one size k: see 7.4 **Combinations from a start index** (p. 169).

Traps bugs people actually write

- `out.add(path)` without a copy: every stored subset is the same list, and at the end they all print as `[]`.
- `path.remove(a[i])` on a `List<Integer>` calls `remove(int index)`: it removes the element at *position* `a[i]`, or throws. Use `path.remove(path.size() - 1)`.
- `1 << n` overflows from n = 31, and above n ≈ 20 the 2^n subsets are too many anyway: try 7.8 **Meet in the middle** (p. 181) or DP.
- `c[i] ^= 32` swaps case only for ASCII letters. Without the `Character.isLetter` check, `'1'` becomes a control character.

Practise on LeetCode, in this order

☐	LC 1863	Sum of All Subset XOR Totals	Easy
☐	LC 78	Subsets	Medium
☐	LC 784	Letter Case Permutation	Medium
☐	LC 2044	Count Number of Maximum Bitwise-OR Subsets	Medium
☐	LC 2597	The Number of Beautiful Subsets	Medium

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

7.2 Skip duplicates at the same depth

$O(n \cdot 2^n)$

Spot it when the problem looks like this

- The array *may contain duplicates*, and the solution set *must not contain duplicate* subsets or combinations.
- Each number may be used once (combination sum II), and the answers are multisets.
- A `HashSet<List<Integer>>` of answers would remove repeats, but only after wasting time on every repeated branch.

How it works

Use the loop form. `build(start)` records `path` as a subset, then tries each `i` in `start..n-1` as the **next** element and recurses with `i + 1`. Sort first, so equal values sit side by side. At one depth, taking the first `2` or the second `2` as the next element leads to identical subtrees, so skip `a[i]` when `i > start && a[i] == a[i-1]`.

The `i > start` part keeps the right answers. The first copy at each depth is always allowed, so the path `[2]` can still take the second `2` one level deeper and make `[2, 2]`. Each distinct multiset is built exactly once, as the sequence that uses the leftmost copies of each value.

Combination sum II adds a target. Because `a` is sorted, `break` as soon as `a[i] > remain`: every later value is at least as big.

Worked example a = [1, 2, 2], subsets II

```
path       start  loop over i
[]         0      i=0 take 1, i=1 take 2, i=2 skip
[1]        1      i=1 take 2, i=2 skip
[1, 2]     2      i=2 take 2
[1, 2, 2]  3      -
[2]        2      i=2 take 2
[2, 2]     3      -
output: [[], [1], [1, 2], [1, 2, 2], [2], [2, 2]]
```

Subsets II

memorise

```java
static List<List<Integer>> subsetsWithDup(int[] a) {
    Arrays.sort(a);                                  // equal values side by side
    List<List<Integer>> out = new ArrayList<>();
    build(a, 0, new ArrayList<>(), out);
    return out;
}

// Every path is a subset. The loop picks the NEXT element from a[start..].
static void build(int[] a, int start, List<Integer> path,
                  List<List<Integer>> out) {
    out.add(new ArrayList<>(path));
    for (int i = start; i < a.length; i++) {
        if (i > start && a[i] == a[i - 1]) continue;   // same value, same depth
        path.add(a[i]);
        build(a, i + 1, path, out);
        path.remove(path.size() - 1);
    }
}
```

Combination sum II

memorise

```java
// Combination sum II: each a[i] used at most once, no repeated combination.
static List<List<Integer>> combinationSum2(int[] a, int target) {
    Arrays.sort(a);
    List<List<Integer>> out = new ArrayList<>();
    choose(a, 0, target, new ArrayList<>(), out);
    return out;
}

static void choose(int[] a, int start, int remain, List<Integer> path,
                   List<List<Integer>> out) {
    if (remain == 0) {
        out.add(new ArrayList<>(path));
        return;
    }
    for (int i = start; i < a.length; i++) {
        if (a[i] > remain) break;                       // sorted: rest too big
        if (i > start && a[i] == a[i - 1]) continue;    // same value, same depth
        path.add(a[i]);
        choose(a, i + 1, remain - a[i], path, out);     // i + 1: no reuse
        path.remove(path.size() - 1);
    }
}
```

Cost Time $O(n \cdot 2^n)$ Space $O(n)$

At most 2^n distinct subsets, each copied in O(n); the sort adds O(n log n).

Variants same idea, small change

- Non-decreasing subsequences: order matters, so you cannot sort. Keep a `HashSet<Integer>` in each call and skip a value already tried at this depth.
- Letter tile possibilities: count the letters into `int[26]`, and at each depth try each letter whose count is above 0. Equal tiles never make two branches.
- Permutations of a list with repeats use a different skip rule: see **7.3 Permutations (used[] array)** (p. 166).

Traps bugs people actually write

- `i > 0` instead of `i > start`: the second `2` is skipped even one level deeper, and `[2, 2]` is lost.
- Forgetting `Arrays.sort(a)`: equal values are not side by side, so `a[i] == a[i-1]` misses the repeat in `[2, 1, 2]`.
- `break` instead of `continue` on the duplicate test ends the loop: in `[1, 2, 2, 3]` the path `[1]` never tries 3, and `[1, 3]` is lost.

Practise on LeetCode, in this order

- [ ] LC 90 Subsets II — Medium
- [ ] LC 40 Combination Sum II — Medium
- [ ] LC 1079 Letter Tile Possibilities — Medium
- [ ] LC 491 Non-decreasing Subsequences — Medium

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

7.3 Permutations (used[] array)

`O(n·n!)`

Spot it when the problem looks like this

- *Return all possible permutations*, arrangements or orderings.
- n is about 8 at most, since the output alone has n! entries.
- The next arrangement in dictionary order, in place with O(1) extra memory (next permutation).

How it works

Fill positions left to right. `used[i]` marks the input elements already in `path`. At each depth, try every unused i: mark it, add it, recurse, then remove and unmark it. Depth k has n – k choices, so there are n! leaves, each copied in O(n): O(n·n!).

Duplicates. Sort, and also skip i when `a[i] == a[i-1] && !used[i-1]`. This forces equal values into the path left to right: the second `1` may only be placed after the first one. Any other order of the copies is skipped, so each distinct permutation appears once.

Next permutation. From the right, find the pivot: the last i with `a[i] < a[i+1]`. Everything after it is descending, the largest order of those values. Swap `a[i]` with the rightmost larger value, then reverse the suffix to get its smallest order. `[1,3,5,4,2]` swaps 3 and 4 to `[1,4,5,3,2]`, then becomes `[1,4,2,3,5]`. With no pivot, reverse the whole array.

Worked example a = [1, 1, 2], permutations II (indices already used are not shown)

```
path       i  a[i]  decision
[]         0  1     take
[1]        1  1     take
[1, 1]     2  2     take  -> [1, 1, 2]
[1]        2  2     take
[1, 2]     1  1     take  -> [1, 2, 1]
[]         1  1     skip (a[0] equal and unused)
[]         2  2     take
[2]        0  1     take
[2, 1]     1  1     take  -> [2, 1, 1]
[2]        1  1     skip (a[0] equal and unused)
```

Permutations

memorise

```java
static List<List<Integer>> permute(int[] a) {
    List<List<Integer>> out = new ArrayList<>();
    arrange(a, new boolean[a.length], new ArrayList<>(), out);
    return out;
}

// Fill the next position with every element not used yet.
static void arrange(int[] a, boolean[] used, List<Integer> path,
                    List<List<Integer>> out) {
    if (path.size() == a.length) {
        out.add(new ArrayList<>(path));
        return;
    }
    for (int i = 0; i < a.length; i++) {
        if (used[i]) continue;
        used[i] = true; path.add(a[i]);                 // choose
        arrange(a, used, path, out);                    // explore
        used[i] = false; path.remove(path.size() - 1);  // undo
    }
}
```

Permutations II (with duplicates)

memorise

```java
// Permutations II: sort, then use the copies of a value only left to right.
static List<List<Integer>> permuteUnique(int[] a) {
    Arrays.sort(a);
    List<List<Integer>> out = new ArrayList<>();
    arrangeUnique(a, new boolean[a.length], new ArrayList<>(), out);
    return out;
}

static void arrangeUnique(int[] a, boolean[] used, List<Integer> path,
                          List<List<Integer>> out) {
    if (path.size() == a.length) {
        out.add(new ArrayList<>(path));
        return;
    }
    for (int i = 0; i < a.length; i++) {
        if (used[i]) continue;
        // left copy of this value still free: taking this copy first would
        // repeat a permutation that starts with the left copy
        if (i > 0 && a[i] == a[i - 1] && !used[i - 1]) continue;
        used[i] = true; path.add(a[i]);
        arrangeUnique(a, used, path, out);
        used[i] = false; path.remove(path.size() - 1);
    }
}
```

Next permutation

variant

```java
// Next permutation in lexicographic order, in place (the largest wraps round).
static void nextPermutation(int[] a) {
    int i = a.length - 2;
    while (i >= 0 && a[i] >= a[i + 1]) i--;     // 1. pivot: last a[i] < a[i+1]
    if (i >= 0) {
        int j = a.length - 1;
        while (a[j] <= a[i]) j--;               // 2. rightmost a[j] > a[i]
        swap(a, i, j);
    }
    reverse(a, i + 1, a.length - 1);            // 3. suffix was descending
}

static void swap(int[] a, int i, int j) {
    int t = a[i]; a[i] = a[j]; a[j] = t;
}

static void reverse(int[] a, int l, int r) {
    while (l < r) swap(a, l++, r--);
}
```

Cost Time `O(n·n!)` Space `O(n)`

n! leaves, each copied in O(n). Next permutation alone is O(n) time and O(1) space.

Variants same idea, small change

- Beautiful arrangement: fill position p only with an unused number that divides p or is divided by it; the check cuts most of the n! tree.
- Squareful arrays: permutations II plus a check that each new number and the previous one add up to a perfect square.
- Swap version: swap `a[k]` with each `a[i]` for `i ≥ k`, recurse on `k + 1`, swap back. It needs no `used[]`, but the output is not in sorted order.

Traps bugs people actually write

- Skipping every `a[i] == a[i-1]` without the `!used[i-1]` part: the second `1` can never be placed, and `[1, 1, 2]` returns nothing.
- Next permutation with `a[i] > a[i+1]` in step 1: an equal pair such as `[1, 1]` becomes a pivot, and step 2 runs `j` off the front of the array.
- Next permutation without the final reverse gives a later permutation, not the next: `[1,3,5,4,2]` becomes `[1,4,5,3,2]`.

Practise on LeetCode, in this order

- [ ] LC 46 Permutations — Medium
- [ ] LC 47 Permutations II — Medium
- [ ] LC 31 Next Permutation — Medium
- [ ] LC 526 Beautiful Arrangement — Medium
- [ ] LC 996 Number of Squareful Arrays — Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

7.4 Combinations from a start index

`O(k·C(n, k))`

Spot it when the problem looks like this

- *Combinations, choose k numbers from 1..n, unique combinations that sum to target.*
- *The same number may be chosen an unlimited number of times* (reuse), or each number is used once.
- Small limits: n ≤ 20 or 30, target ≤ 40, k ≤ 9, and the number of answers is promised to be small.

How it works

`grow(start, remain)` picks the next element from index `start` onward. Every combination is built in increasing index order, so `[2, 3]` is made once and `[3, 2]` never. The recursive call decides reuse: pass `i` to allow `a[i]` again, or `i + 1` to use each element at most once.

Prune the loop, not only the leaves. With the candidates sorted, stop when `a[i] > remain`, because every later one is bigger. When choosing k of 1..n with `need` slots left, the next number can be at most `n - need + 1`, or too few numbers remain after it. Then every branch the search enters has an answer below it, so n choose k costs O(k·C(n, k)).

Letter combinations of a phone number is the same tree with one digit per depth: the choices at depth i are the letters on key `digits[i]`, up to 4^n leaves.

Worked example a = [2, 3, 6, 7], target = 7, reuse allowed

```
path       remain  loop
[]         7       i from 0
[2]        5       i from 0   stop at 6 > 5
[2, 2]     3       i from 0   stop at 6 > 3
[2, 2, 2]  1       i from 0   stop at 2 > 1
[2, 2, 3]  0       found
[2, 3]     2       i from 1   stop at 3 > 2
[3]        4       i from 1   stop at 6 > 4
[3, 3]     1       i from 1   stop at 3 > 1
[6]        1       i from 2   stop at 6 > 1
[7]        0       found
```

Combination sum (reuse allowed)

memorise

```java
// Combination sum: distinct positive candidates, each usable any number of times.
static List<List<Integer>> combinationSum(int[] a, int target) {
    Arrays.sort(a);                                  // lets the loop stop early
    List<List<Integer>> out = new ArrayList<>();
    grow(a, 0, target, new ArrayList<>(), out);
    return out;
}

static void grow(int[] a, int start, int remain, List<Integer> path,
                 List<List<Integer>> out) {
    if (remain == 0) {
        out.add(new ArrayList<>(path));
        return;
    }
    for (int i = start; i < a.length && a[i] <= remain; i++) {
        path.add(a[i]);
        grow(a, i, remain - a[i], path, out);        // i, not i + 1: reuse a[i]
        path.remove(path.size() - 1);
    }
}
```

Combinations: n choose k

memorise

```java
// Combinations: every k-element subset of 1..n, each in increasing order.
static List<List<Integer>> combine(int n, int k) {
    List<List<Integer>> out = new ArrayList<>();
    pickK(1, n, k, new ArrayList<>(), out);
    return out;
}

static void pickK(int start, int n, int k, List<Integer> path,
                  List<List<Integer>> out) {
    if (path.size() == k) {
        out.add(new ArrayList<>(path));
        return;
    }
    int need = k - path.size();
    for (int x = start; x <= n - need + 1; x++) {    // prune: leave `need` values
        path.add(x);
        pickK(x + 1, n, k, path, out);
        path.remove(path.size() - 1);
    }
}
```

Letter combinations of a phone number

variant

```
static final String[] KEYS =
    {"", "", "abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"};

// One digit per depth; the choices at depth i are the letters of digit i.
static List<String> letterCombinations(String digits) {
    List<String> out = new ArrayList<>();
    if (digits.isEmpty()) return out;                // [] for "", not [""]
    spell(digits, 0, new StringBuilder(), out);
    return out;
}

static void spell(String d, int i, StringBuilder sb, List<String> out) {
    if (i == d.length()) {
        out.add(sb.toString());
        return;
    }
    for (char c : KEYS[d.charAt(i) - '0'].toCharArray()) {
        sb.append(c);
        spell(d, i + 1, sb, out);
        sb.setLength(sb.length() - 1);               // undo
    }
}
```

Cost Time `O(k·C(n, k))` Space `O(k)`

For n choose k: C(n, k) leaves, each copied in O(k). Combination sum has no neat formula; the sorted break keeps dead branches short.

Variants same idea, small change

- Combination sum III (k numbers from 1..9 that add up to n): `pickK` over 1..9 with a `remain`, stopping when `x > remain`.
- Binary watch: choose k of the 10 LEDs (the bitmask loop from **7.1 Subsets (include or exclude)** (p. 160) also works) and keep times with hours < 12 and minutes < 60.
- Candidates with repeats, each used once: add the skip rule from **7.2 Skip duplicates at the same depth** (p. 163).
- Only the *number* of combinations is asked: that is a knapsack count (**10.5 Unbounded knapsack (forward loop)** (p. 289)), not a search.

Traps bugs people actually write

- Recursing with `0` instead of `i` as the new start: `[2, 2, 3]`, `[2, 3, 2]` and `[3, 2, 2]` all appear.
- Recursing with `i + 1` in combination sum forbids reuse, so `[2, 2, 3]` for target 7 is missed.
- Letter combinations of `""`: LeetCode expects `[]`, but without the early return the recursion gives `[""]`.
- The `a[i] <= remain` loop test only works on sorted input; unsorted, it stops before smaller candidates further right.

Practise on LeetCode, in this order

- [ ] LC 401 Binary Watch — Easy
- [ ] LC 17 Letter Combinations of a Phone Number — Medium
- [ ] LC 77 Combinations — Medium
- [ ] LC 39 Combination Sum — Medium
- [ ] LC 216 Combination Sum III — Medium

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

7.5 Constraint backtracking (N-Queens) O(n!)

Spot it when the problem looks like this

- *No two queens attack each other*; *each digit once per row, column and 3×3 box.*
- The board is tiny, n ≤ 9 for queens and a fixed 9×9 grid for sudoku.
- Return *all distinct solutions*, *the number of solutions*, or fill the one solution in place.

How it works

Put one queen in each row, row by row, so rows never clash. Before placing at (r, c), check three boolean arrays in O(1): the column `c`, the `\` diagonal where `r - c` is constant (shifted to `r - c + n - 1` so the index runs 0..2n−2), and the `/` diagonal where `r + c` is constant. Mark all three, recurse to row `r + 1`, then clear them.

The checks cut branches early. For n = 8, `place` runs 2,057 times, against 40,320 column orders and $8^8 \approx 1.7 \cdot 10^7$ blind placements. The cost is bounded by O(n!), since each row has fewer free columns than the one before.

Sudoku is the same shape on cells. Take the next empty cell, try each digit not yet in its row, column or box (`r / 3 * 3 + c / 3`), and undo on failure. `fill` returns `true` at the first full board, so the search stops there and the board keeps the answer.

Worked example n = 4 until the first solution (undo steps not shown)

```
r=0 c=0  diag 3, anti 0  place
  r=1 c=2  diag 2, anti 3  place
    r=2: every column attacked -> back
  r=1 c=3  diag 1, anti 4  place
    r=2 c=1  diag 4, anti 3  place
      r=3: every column attacked -> back
r=0 c=1  diag 2, anti 1  place
  r=1 c=3  diag 1, anti 4  place
    r=2 c=0  diag 5, anti 2  place
      r=3 c=2  diag 4, anti 5  place
        r=4: all rows filled -> solution .Q.. ...Q Q... ..Q.
```

N-Queens

memorise

```
static int n;
static int[] queenAt;                    // queenAt[r] = column of row r's queen
static boolean[] cols, diag, anti;       // diag: r - c + n - 1, anti: r + c
static List<List<String>> boards;

static List<List<String>> solveNQueens(int size) {
    n = size;                            // reset every field on each call
    queenAt = new int[n];
    cols = new boolean[n];
    diag = new boolean[2 * n - 1];
    anti = new boolean[2 * n - 1];
    boards = new ArrayList<>();
    place(0);
    return boards;
}

// One queen per row, so rows never clash; check the column and both diagonals.
static void place(int r) {
    if (r == n) { boards.add(draw()); return; }
    for (int c = 0; c < n; c++) {
        if (cols[c] || diag[r - c + n - 1] || anti[r + c]) continue;
        queenAt[r] = c;
        cols[c] = diag[r - c + n - 1] = anti[r + c] = true;     // mark
        place(r + 1);
        cols[c] = diag[r - c + n - 1] = anti[r + c] = false;    // restore
    }
}

static List<String> draw() {
    List<String> rows = new ArrayList<>();
    for (int r = 0; r < n; r++) {
        char[] row = new char[n];
        Arrays.fill(row, '.');
        row[queenAt[r]] = 'Q';
        rows.add(new String(row));
    }
    return rows;
}
```

Sudoku solver

variant

```java
static boolean[][] rowHas, colHas, boxHas;     // [row, col or box][digit 1..9]

static void solveSudoku(char[][] b) {
    rowHas = new boolean[9][10];
    colHas = new boolean[9][10];
    boxHas = new boolean[9][10];
    for (int r = 0; r < 9; r++)
        for (int c = 0; c < 9; c++)
            if (b[r][c] != '.') mark(r, c, b[r][c] - '0', true);
    fill(b, 0);
}

// Visit cells in row-major order; true as soon as the board is full.
static boolean fill(char[][] b, int cell) {
    if (cell == 81) return true;
    int r = cell / 9, c = cell % 9, box = r / 3 * 3 + c / 3;
    if (b[r][c] != '.') return fill(b, cell + 1);
    for (int d = 1; d <= 9; d++) {
        if (rowHas[r][d] || colHas[c][d] || boxHas[box][d]) continue;
        b[r][c] = (char) ('0' + d);
        mark(r, c, d, true);
        if (fill(b, cell + 1)) return true;    // keep it: stop searching
        mark(r, c, d, false);
        b[r][c] = '.';
    }
    return false;                              // no digit fits: back up
}

static void mark(int r, int c, int d, boolean on) {
    rowHas[r][d] = colHas[c][d] = boxHas[r / 3 * 3 + c / 3][d] = on;
}
```

Cost Time `O(n!)` Space `O(n)`

Each row has fewer free columns, so at most n! placements; the diagonal checks prune far below that.

Variants same idea, small change

- N-Queens II: the same `place`, with `count++` instead of `draw()`.
- Bitmasks instead of arrays: the free columns in a row are `~(cols | d | a) & ((1 << n) - 1)`, take them one at a time with `free & -free`, and shift `d` left and `a` right for the next row.
- Matchsticks to square: put each stick on one of 4 sides of length sum / 4. Sort longest first, and skip a side whose length equals an earlier side's, which removes mirror-image branches.

Traps bugs people actually write

- On LeetCode, `static` fields keep their values from one test case to the next. Reset every array in the entry method, as `solveNQueens` does.
- Indexing the `\` diagonal with `r - c` alone gives negative indices; add `n - 1`, and size both diagonal arrays `2n - 1`.
- Sudoku: if `fill` returns `void`, the undo steps wipe the solution on the way back. Return `true` and stop at the first full board.
- Sudoku: forgetting to mark the given digits before the search lets it place a digit that the row already has.

Practise on LeetCode, in this order

☐	`LC 52`	N-Queens II	Hard
☐	`LC 51`	N-Queens	Hard
☐	`LC 473`	Matchsticks to Square	Medium
☐	`LC 37`	Sudoku Solver	Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

7.6 Grid DFS with mark and restore

`O(mn·3^L)`

Spot it when the problem looks like this

- *The same cell may not be used more than once*; *walk over every empty square exactly once*.
- Tiny grids: up to 6×6 for word search, at most 25 cells with gold, m·n ≤ 20 for unique paths III.
- You need any path, the best path or every path, not the shortest one (shortest is BFS, **9.2 BFS shortest path (unweighted)** (p. 226)).

How it works

Run a DFS from each possible start. On entering a cell, mark it as in use by overwriting it (`'#'`, `0` or `-1`), try the four neighbours, then write the original value back before returning. The restore is what makes it backtracking: a cell blocked for the current path is free again for every other path. A flood fill never restores, which is why it is O(mn) but cannot count or compare paths.

Check the cheap things first. Word search rejects a cell as soon as its letter differs from `word[k]`, and a marked `'#'` never equals a letter, so "in use" needs no extra array. After the first step each cell has at most 3 new directions, so a word of length L costs O(m·n·3^L) in the worst case.

For unique paths III, count the open cells first. Reaching the end counts only if it is the last open cell, `left == 1`.

Worked example word ABCB on [ABCE / SFCS / ADEE] from (0,0); off-board moves not shown

```
k  cell   letter  result
0  (0,0)  A       ok, mark '#'
1  (1,0)  S       != B
1  (0,1)  B       ok, mark '#'
2  (1,1)  F       != C
2  (0,0)  #       != C (in use)
2  (0,2)  C       ok, mark '#'
3  (1,2)  C       != B
3  (0,1)  #       != B (in use)
3  (0,3)  E       != B
   restore (0,2), (0,1), (0,0): no path from (0,0)
```

Word search

memorise

```java
static final int[] DR = {-1, 1, 0, 0}, DC = {0, 0, -1, 1};

static boolean exist(char[][] board, String word) {
    for (int r = 0; r < board.length; r++)
        for (int c = 0; c < board[0].length; c++)
            if (match(board, word, 0, r, c)) return true;
    return false;
}

// Can word[k..] be spelled starting at (r, c)? The cell is marked while in use.
static boolean match(char[][] b, String w, int k, int r, int c) {
    if (r < 0 || c < 0 || r >= b.length || c >= b[0].length) return false;
    if (b[r][c] != w.charAt(k)) return false;   // a '#' (in use) never matches
    if (k == w.length() - 1) return true;
    char keep = b[r][c];
    b[r][c] = '#';                              // mark: on the current path
    boolean found = false;
    for (int d = 0; d < 4 && !found; d++)
        found = match(b, w, k + 1, r + DR[d], c + DC[d]);
    b[r][c] = keep;                             // restore for other paths
    return found;
}
```

Path with maximum gold

variant

```java
// Path with maximum gold: start anywhere, never step on 0, never reuse a cell.
static int getMaximumGold(int[][] g) {
    int best = 0;
    for (int r = 0; r < g.length; r++)
        for (int c = 0; c < g[0].length; c++)
            if (g[r][c] > 0) best = Math.max(best, collect(g, r, c));
    return best;
}

static int collect(int[][] g, int r, int c) {
    if (r < 0 || c < 0 || r >= g.length || c >= g[0].length || g[r][c] == 0)
        return 0;
    int gold = g[r][c], more = 0;
    g[r][c] = 0;                                  // mark by emptying the cell
    for (int d = 0; d < 4; d++)
        more = Math.max(more, collect(g, r + DR[d], c + DC[d]));
    g[r][c] = gold;                               // restore
    return gold + more;
}
```

Unique paths III

variant

```java
// Unique paths III: walk from 1 to 2 over every 0 exactly once; -1 is a wall.
static int uniquePathsIII(int[][] g) {
    int sr = 0, sc = 0, open = 0;
    for (int r = 0; r < g.length; r++)
        for (int c = 0; c < g[0].length; c++) {
            if (g[r][c] != -1) open++;            // cells the path must cover
            if (g[r][c] == 1) { sr = r; sc = c; }
        }
    return walk(g, sr, sc, open);
}

// left = cells still to visit, counting (r, c) itself
static int walk(int[][] g, int r, int c, int left) {
    if (r < 0 || c < 0 || r >= g.length || c >= g[0].length || g[r][c] == -1)
        return 0;
    if (g[r][c] == 2) return left == 1 ? 1 : 0; // at the end: all covered?
    int keep = g[r][c], ways = 0;
    g[r][c] = -1;                                 // mark: a wall while on the path
    for (int d = 0; d < 4; d++)
        ways += walk(g, r + DR[d], c + DC[d], left - 1);
    g[r][c] = keep;
    return ways;
}
```

Cost Time `O(mn·3^L)` Space `O(L)`

Each start tries at most 4·3^(L−1) paths of length L; the recursion is as deep as the path.

Variants same idea, small change

- Word search II (many words): walk the board once with a trie of all the words, and remove a word from the trie once found (**11.5 Trie (prefix tree)** (p. 332)).
- Path with maximum gold: start from every gold cell; a cell is worth its gold plus the best of its neighbours.
- Only reachability matters, with no per-path rule: mark without restoring. That is a flood fill (**9.1 Graph DFS and connected components** (p. 223)).

Traps bugs people actually write

- Forgetting the restore, or a `visited[][]` that is set but never cleared: the first path through a cell blocks it for every later path, and counts come out too low.
- Testing `k == word.length()` after the bounds check: on a 1×1 board with a one-letter word every neighbour is off the board, so the match is never reported. Test `k == len - 1` once the letter matches.
- The recursion is as deep as the path, up to m·n. That is fine for these limits, but the method is exponential, so it only suits tiny grids.

Practise on LeetCode, in this order

☐	`LC 79`	Word Search	Medium
☐	`LC 1219`	Path with Maximum Gold	Medium
☐	`LC 980`	Unique Paths III	Hard
☐	`LC 212`	Word Search II	Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

7.7 Cut a prefix, recurse on the rest

`O(n·2ⁿ)`

Spot it when the problem looks like this

- *Partition s so that every substring is a palindrome; return all possible partitions.*
- Return all valid IP addresses made by inserting dots; split into the most unique pieces; all sentences from a dictionary.
- Short input (length ≤ 16 or 20, n ≤ 8 pairs of brackets), because the output itself grows exponentially.

How it works

`cut(start)` chooses the first piece. It tries every end from `start` to n – 1. If `s[start..end]` passes the test, add it to `path`, recurse on `cut(end + 1)`, then remove it. When `start == n` the pieces cover the string, so `path` is one answer. A string of length n has 2^(n–1) ways to cut, so with the copies the worst case is O(n·2ⁿ).

Make the test O(1). Before the search, fill `isPal[i][j]`: `s[i] == s[j]` and the inside `s[i+1..j-1]` is a palindrome or shorter than 2. Fill i from n – 1 down so the inside is ready when you read it.

Restore IP addresses has four pieces of 1 to 3 digits, each ≤ 255 with no leading zero; prune when the digits left cannot fill the pieces left. Generate parentheses builds instead of cutting: add `(` while `open < n` and `)` while `close < open`, so every prefix stays valid.

Worked example s = "aab"

```
start  piece  isPal  path
0      "a"    yes    [a]
1      "a"    yes    [a, a]
2      "b"    yes    [a, a, b]  -> answer
1      "ab"   no
0      "aa"   yes    [aa]
2      "b"    yes    [aa, b]  -> answer
0      "aab"  no
```

Palindrome partitioning

memorise

```
static List<List<String>> partition(String s) {
    int n = s.length();
    boolean[][] isPal = new boolean[n][n];      // s[i..j] is a palindrome
    for (int i = n - 1; i >= 0; i--)
        for (int j = i; j < n; j++)
            isPal[i][j] = s.charAt(i) == s.charAt(j)
                          && (j - i < 2 || isPal[i + 1][j - 1]);
    List<List<String>> out = new ArrayList<>();
    cut(s, 0, isPal, new ArrayList<>(), out);
    return out;
}

// Choose the first piece s[start..end], then partition the rest the same way.
static void cut(String s, int start, boolean[][] isPal, List<String> path,
                List<List<String>> out) {
    if (start == s.length()) { out.add(new ArrayList<>(path)); return; }
    for (int end = start; end < s.length(); end++) {
        if (!isPal[start][end]) continue;
        path.add(s.substring(start, end + 1));
        cut(s, end + 1, isPal, path, out);
        path.remove(path.size() - 1);
    }
}
```

Restore IP addresses

variant

```java
// Restore IP addresses: 4 pieces, each 0..255, no leading zeros.
static List<String> restoreIpAddresses(String s) {
    List<String> out = new ArrayList<>();
    dot(s, 0, new ArrayList<>(), out);
    return out;
}

static void dot(String s, int start, List<String> parts, List<String> out) {
    int left = 4 - parts.size(), rest = s.length() - start;
    if (rest < left || rest > 3 * left) return;          // prune: cannot fit
    if (left == 0) { out.add(String.join(".", parts)); return; }
    for (int len = 1; len <= 3 && start + len <= s.length(); len++) {
        String p = s.substring(start, start + len);
        if (len > 1 && p.charAt(0) == '0') break;         // "01" is not allowed
        if (Integer.parseInt(p) > 255) break;
        parts.add(p);
        dot(s, start + len, parts, out);
        parts.remove(parts.size() - 1);
    }
}
```

Generate parentheses

variant

```java
// Generate parentheses: '(' while open < n, ')' while close < open.
static List<String> generateParenthesis(int n) {
    List<String> out = new ArrayList<>();
    brackets(n, 0, 0, new StringBuilder(), out);
    return out;
}

static void brackets(int n, int open, int close, StringBuilder sb,
                     List<String> out) {
    if (sb.length() == 2 * n) { out.add(sb.toString()); return; }
    if (open < n) {
        sb.append('(');
        brackets(n, open + 1, close, sb, out);
        sb.setLength(sb.length() - 1);
    }
    if (close < open) {
        sb.append(')');
        brackets(n, open, close + 1, sb, out);
        sb.setLength(sb.length() - 1);
    }
}
```

Cost Time $O(n \cdot 2^n)$ Space $O(n^2)$

Up to 2^(n–1) partitions, each copied in O(n); the isPal table is n^2.

Variants same idea, small change

- Word break II: the test is "the piece is in the dictionary" (a `HashSet`); memoise the answers for each `start` if the same suffix comes up often.
- Most unique substrings: keep a `HashSet` of the pieces in use, add and remove like `path`, and track the largest size.
- Only the *fewest* cuts or the *number* of partitions is asked: that is DP over `start` (10.9 **Palindrome DP on s[i..j]** (p. 300)), not a listing.

Traps bugs people actually write

- IP: `Integer.parseInt("012")` is 12, so test for a leading zero on the string first.
- IP: without the length check the search tries every split of a long string; return when `rest > 3 * left` or `rest < left`.
- Filling `isPal` with i going up: `isPal[i+1][j-1]` is still false when you read it, so `"aba"` is missed.
- Generate parentheses: allowing `)` while `close < n` instead of `close < open` builds invalid strings like `)(`.

Practise on LeetCode, in this order

☐	LC 22	Generate Parentheses	Medium
☐	LC 131	Palindrome Partitioning	Medium
☐	LC 93	Restore IP Addresses	Medium
☐	LC 1593	Split a String Into the Max Number of Unique Substrings	Medium
☐	LC 140	Word Break II	Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

7.8 Meet in the middle

`O(2^(n/2)·n)`

Spot it when the problem looks like this

- $n \le 40$ (or $2n \le 30$): $2^{40} \approx 10^{12}$ is far too many, $2^{20} \approx 10^{6}$ is fine.
- Values up to 10^{7} or 10^{9}, possibly negative, so a DP over sums is impossible.
- *Closest to goal, minimise the difference, count subsets with sum ≤ X.*

How it works

Split the array into a left and a right half. Every subset of the whole is a subset of the left plus a subset of the right, so its sum is `x + y` with x a left sum and y a right sum. Each half has only 2^(n/2) subsets.

List each half's sums in O(1) per sum: `sums[mask]` is the sum of `mask` without its lowest bit, plus that element. Sort both lists. For the closest sum, run two pointers as in **2.1 Opposite-end pointers** (p. 44): i climbs the left list, j comes down the right list, and you move the end that brings `x + y` toward the goal.

Or, for each x, binary search the right list for `goal - x` and check its neighbours. Either way the cost is O(2^(n/2)·n).

When both groups must have exactly n elements, group the half sums by how many elements they use: k from the left pairs only with n – k from the right.

Worked example a = [7, -9, 15, -2], goal = -5 (LC 1755)

```
L = sums of [7, -9]  = [0, 7, -9, -2]   sorted [-9, -2, 0, 7]
R = sums of [15, -2] = [0, 15, -2, 13]  sorted [-2, 0, 13, 15]
i  j  L[i] + R[j]   best  move
0  3  -9 + 15 = 6   11    6 > -5: j--
0  2  -9 + 13 = 4   9     4 > -5: j--
0  1  -9 + 0 = -9   4     -9 < -5: i++
1  1  -2 + 0 = -2   3     -2 > -5: j--
1  0  -2 + -2 = -4  1     -4 > -5: j--
j < 0: stop. Answer 1, from {7, -9} + {-2} = -4
```

All subset sums of a range

helper

```
// All 2^k subset sums of a[from..to). sums[mask] extends the sum of mask
// without its lowest bit, so each sum costs O(1).
static long[] subsetSums(int[] a, int from, int to) {
    int k = to - from;
    long[] sums = new long[1 << k];
    for (int mask = 1; mask < (1 << k); mask++) {
        int low = Integer.numberOfTrailingZeros(mask);
        sums[mask] = sums[mask & (mask - 1)] + a[from + low];
    }
    return sums;
}
```

Closest subsequence sum

memorise

```
// Closest subsequence sum (n <= 40): smallest |sum - goal| over all subsets.
static int minAbsDifference(int[] a, int goal) {
    int h = a.length / 2;
    long[] left = subsetSums(a, 0, h), right = subsetSums(a, h, a.length);
    Arrays.sort(left);
    Arrays.sort(right);
    long best = Long.MAX_VALUE;
    int i = 0, j = right.length - 1;        // left goes up, right comes down
    while (i < left.length && j >= 0) {
        long sum = left[i] + right[j];
        best = Math.min(best, Math.abs(sum - goal));
        if (sum < goal) i++;                // need a bigger sum
        else if (sum > goal) j--;           // need a smaller sum
        else return 0;
    }
    return (int) best;
}
```

Two groups of n with the closest sums

variant

```java
// Split 2n numbers into two groups of n with the smallest |sumA - sumB|.
// Group A takes k numbers from the left half and n - k from the right half.
static int minimumDifference(int[] a) {
    int n = a.length / 2;
    long total = 0;
    for (int v : a) total += v;
    List<List<Long>> left = byCount(a, 0, n), right = byCount(a, n, n);
    long best = Long.MAX_VALUE;                 // |sumA - sumB| = |total - 2 sumA|
    for (int k = 0; k <= n; k++) {
        List<Long> r = right.get(n - k);
        for (long x : left.get(k)) {
            int lo = 0, hi = r.size();          // first y with 2(x + y) >= total
            while (lo < hi) {
                int mid = (lo + hi) >>> 1;
                if (2 * (x + r.get(mid)) >= total) hi = mid;
                else lo = mid + 1;
            }
            if (lo < r.size()) best = Math.min(best, 2 * (x + r.get(lo)) - total);
            if (lo > 0) best = Math.min(best, total - 2 * (x + r.get(lo - 1)));
        }
    }
    return (int) best;
}

// Sums of a[from..from+k), grouped by how many numbers they use, each sorted.
static List<List<Long>> byCount(int[] a, int from, int k) {
    long[] sums = subsetSums(a, from, from + k);
    List<List<Long>> g = new ArrayList<>();
    for (int c = 0; c <= k; c++) g.add(new ArrayList<>());
    for (int mask = 0; mask < sums.length; mask++)
        g.get(Integer.bitCount(mask)).add(sums[mask]);
    for (List<Long> list : g) Collections.sort(list);
    return g;
}
```

Cost Time `O(2^(n/2)·n)` Space `O(2^(n/2))`

Each half has 2^(n/2) sums, and sorting them adds the factor n; for n = 40 that is about 10^6 sums per half.

Variants same idea, small change

- 4Sum II: put every `a + b` sum of the first two arrays in a `HashMap` of counts, then look up `-(c + d)`. Same split, with hashing instead of sorting.
- Count subsets with sum ≤ X: sort the right sums, and for each left sum x add the number of right sums ≤ X − x (3.1 **Lower bound and upper bound** (p. 68)).
- Split array with same average: a group of size k works if its sum is `k · total / len`. Group the half sums by count and look for a left sum of k_1 numbers and a right sum of $k - k_1$ numbers that add up to it.

Traps bugs people actually write

- Storing the sums in `int`: 20 values of 10^9 overflow. Use `long`.
- Pairing any left sum with any right sum in the equal-size version ignores the size rule; k from the left needs exactly n − k from the right.
- Building 2^{20} boxed `Long` values is slow and memory-heavy; keep `long[]` when you do not need the groups (the grouped version can afford lists because 2n ≤ 30).

Practise on LeetCode, in this order

- [ ] LC 454 4Sum II — Medium
- [ ] LC 1755 Closest Subsequence Sum — Hard
- [ ] LC 2035 Partition Array Into Two Arrays to Minimize Sum Difference — Hard
- [ ] LC 805 Split Array With Same Average — Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

Part 8

Trees

Every subtree is a tree, so most tree answers are built by combining what the two children report, or by passing state down the path from the root. The first eight windows use LeetCode's `TreeNode`; the last five mostly take a general tree as n and an edge list, walk it once without recursion, and then answer subtree, ancestor and every-root questions in O(n) or O(log n) each. Reach for this part when the input is a `TreeNode root`, or n nodes joined by exactly n – 1 edges.

8.1	**Preorder, inorder, postorder** Visit every node in root-left-right, left-root-right or left-right-root order, by recursion or with your own stack.	O(n)	p. 186
8.2	**Level order (BFS by levels)** Anything per depth of a binary tree, such as each level as a list, the last node of each level, zigzag order or the widest level.	O(n)	p. 189
8.3	**Return a value up the tree** A node's answer comes from its children's answers, as with height, balance, diameter, or subtree sums and counts.	O(n)	p. 191
8.4	**Maximum path sum (one arm up)** The best path anywhere in the tree, with any start and any end, scored by its values or by its length under a rule.	O(n)	p. 194
8.5	**Root-to-leaf paths and path sums** Paths that go down the tree; list the root-to-leaf paths with a property, build a number along them, or count downward paths with a given sum.	O(n)	p. 196
8.6	**Binary search tree operations** A binary search tree; find, insert or delete a key, check the BST rule, or read the values in sorted order.	O(h)	p. 199
8.7	**Lowest common ancestor** The deepest node that has both p and q in its subtree; also the path, distance or directions between two nodes.	O(n)	p. 202
8.8	**Build and serialize a tree** Rebuild a binary tree from two traversals, or turn a tree into a string and back.	O(n)	p. 205
8.9	**General tree from an edge list** A tree given as n and n – 1 edges; you need parents, depths, subtree sizes or a children-first pass, with n up to 10^5.	O(n)	p. 207
8.10	**Binary lifting (k-th ancestor, LCA)** Many queries on a fixed tree, such as the k-th ancestor of v or the LCA of a pair, each answered in O(log n).	O(log n) each	p. 210
8.11	**Tree DP (take or skip a node)** Optimise a choice made at every node, such as rob it or not, or put a camera there or not, when neighbours restrict each other.	O(n)	p. 213
8.12	**Rerooting (every node as root)** The same tree question asked once for every possible root, such as the sum of distances, edges to flip, correct guesses or height.	O(n)	p. 216

8.13 **Euler tour (subtree = range)** O(n) p. 218
Turn every subtree into one contiguous block of positions, so the ancestor test is O(1) and subtree queries become range queries.

8.1 Preorder, inorder, postorder O(n)

Spot it when the problem looks like this

- *Return the inorder (preorder, postorder) traversal*, often with the follow-up *recursion is trivial, could you do it iteratively?*
- A BST question that needs the values in sorted order. Inorder visits them smallest first.
- The tree can be a chain, so its height can reach n, and n can be 10^4 or 10^5.

How it works

All three orders come from one recursive walk. The only difference is where the node is recorded: before both subtrees (pre), between them (in) or after both (post). Each node is entered once and the call stack holds one root-to-node path, so the walk costs O(n) time and O(h) space for a tree of height h.

The iterative versions keep that stack themselves. Inorder pushes the whole left spine, pops the deepest node, records it and moves to its right child. The invariant is that the stack holds exactly the ancestors whose left side is still being visited. Preorder pops a node, records it, and pushes the right child before the left, so the left comes off first. Postorder (left, right, root) is root-right-left read backwards: run preorder with the two pushes swapped and prepend each value. The last-visited version records a node only when its right child is missing or is the node it recorded last.

Worked example **inorder(root), root = [4, 2, 6, 1, 3, 5]; stack bottom to top**

push	pop	out	stack	cur = right child
4 2 1	1	[1]	[4, 2]	null
-	2	[1, 2]	[4]	3
3	3	[1, 2, 3]	[4]	null
-	4	[1, 2, 3, 4]	[]	6
6 5	5	[1, 2, 3, 4, 5]	[6]	null
-	6	[1, 2, 3, 4, 5, 6]	[]	null

One recursive walk, three orders memorise

```
// One walk, three orders: only the position of the visit line changes.
static void walk(TreeNode t, List<Integer> pre, List<Integer> in,
                 List<Integer> post) {
    if (t == null) return;
    pre.add(t.val);                       // before both subtrees
    walk(t.left, pre, in, post);
    in.add(t.val);                        // between them
    walk(t.right, pre, in, post);
    post.add(t.val);                      // after both
}
```

Iterative inorder (push the left spine)

memorise

```java
// Iterative inorder: push the whole left spine, pop, visit, go right.
// The stack holds the ancestors whose left side is still being visited.
static List<Integer> inorder(TreeNode root) {
    List<Integer> out = new ArrayList<>();
    Deque<TreeNode> st = new ArrayDeque<>();
    TreeNode cur = root;
    while (cur != null || !st.isEmpty()) {
        while (cur != null) {              // 1. go left as far as possible
            st.push(cur);
            cur = cur.left;
        }
        cur = st.pop();                    // 2. leftmost unvisited node
        out.add(cur.val);
        cur = cur.right;                   // 3. then its right subtree
    }
    return out;
}
```

Iterative preorder and postorder

variant

```java
// Preorder: pop, visit, push right then left, so left is popped first.
static List<Integer> preorder(TreeNode root) {
    List<Integer> out = new ArrayList<>();
    Deque<TreeNode> st = new ArrayDeque<>();
    if (root != null) st.push(root);
    while (!st.isEmpty()) {
        TreeNode t = st.pop();
        out.add(t.val);
        if (t.right != null) st.push(t.right);
        if (t.left != null) st.push(t.left);
    }
    return out;
}

// Postorder: walk like inorder, but record a node only after its right
// subtree is done. last = the node recorded most recently.
static List<Integer> postorderLast(TreeNode root) {
    List<Integer> out = new ArrayList<>();
    Deque<TreeNode> st = new ArrayDeque<>();
    TreeNode cur = root, last = null;
    while (cur != null || !st.isEmpty()) {
        while (cur != null) { st.push(cur); cur = cur.left; }
        TreeNode top = st.peek();
        if (top.right != null && top.right != last) {
            cur = top.right;               // right subtree not done yet
        } else {
            out.add(top.val);              // both subtrees done
            last = st.pop();
        }
    }
    return out;
}
```

Cost Time `O(n)` Space `O(h)`

Every node is pushed and popped once. The stack holds one root-to-node path, so h is about log n when balanced and n for a chain.

Variants same idea, small change

- Morris inorder, O(1) extra space: before going left, point the rightmost node of the left subtree back at the current node. When that link brings you back to the node, remove it, record the node and go right.
- Recover a BST with two swapped values: inorder with a `prev` pointer. The swapped pair shows up where `prev.val > cur.val`, once if they are adjacent in inorder, twice otherwise.
- Flatten to a linked list in preorder: visit right, left, root (preorder backwards) and set each node's `right` to the node visited before it and its `left` to null.
- Postorder the short way: the preorder loop with `st.push(t.left)` before `st.push(t.right)`, and `out.addFirst(t.val)` into a `LinkedList`. Root-right-left read backwards is left-right-root.

Traps bugs people actually write

- Preorder that pushes the left child before the right pops the right one first: `[1, 2, 3]` comes out as `[1, 3, 2]`.
- Inorder with `while (!st.isEmpty())` and no `cur != null ||` stops as soon as popping the root empties the stack. If you push the root first, `[4, 2, 6, 1, 3, 5]` returns `[1, 2, 3, 4]` and the right subtree is lost.
- Recursion depth is the height. On a 10^4-node chain the recursive walk threw `StackOverflowError` under Java's default 1 MB thread stack in testing; the iterative versions handled 10^5.
- Reversed postorder with `ArrayList.add(0, x)` shifts the whole list each time: $O(n^2)$. Use `LinkedList.addFirst`, or append and reverse once.

Practise on LeetCode, in this order

☐	LC 94	Binary Tree Inorder Traversal	Easy
☐	LC 144	Binary Tree Preorder Traversal	Easy
☐	LC 145	Binary Tree Postorder Traversal	Easy
☐	LC 114	Flatten Binary Tree to Linked List	Medium
☐	LC 99	Recover Binary Search Tree	Medium

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

8.2 Level order (BFS by levels) O(n)

Spot it when the problem looks like this

- *Level by level, from left to right, zigzag, right side view, average* or *largest value in each row.*
- The answer is a `List<List<Integer>>` with one inner list per depth, or one number per depth.
- *Width* counts the gaps between the two end nodes, as if the level were part of a full binary tree.

How it works

A queue hands out nodes in order of depth: each node is queued after its parent's whole level, so depth d comes out before depth d + 1. Read `q.size()` when a level starts. At that moment the queue holds exactly that level, so polling `size` nodes takes the level and nothing else, while their children wait behind it. Each node is queued and polled once: O(n). Right side view and zigzag only post-process the levels: take the last value of each, or reverse every second one.

For width, number the nodes as in a heap: the children of `i` are `2i + 1` and `2i + 2`, and a level's width is `last - first + 1`, gaps included. The numbers double on each level and would need hundreds of bits on a deep, thin tree. Subtracting the level's first index before numbering the children keeps each number at most twice the width of the level above.

Worked example **widthOfBinaryTree on [1, 3, 2, 5, null, null, 9, 6, null, 7]**

```
level  size  values   index -> rebased    last  width  best
0      1     [1]      0->0                 0     1      1
1      2     [3,2]    1->0  2->1           1     2      2
2      2     [5,9]    1->0  4->3           3     4      4
3      2     [6,7]    1->0  7->6           6     7      7
The values column is levelOrder: [[1], [3, 2], [5, 9], [6, 7]].
```

Level order with the level size

memorise

```java
// BFS one level at a time. When a level starts, the queue holds exactly
// that level, so its size is fixed before any child is added.
static List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> out = new ArrayList<>();
    Deque<TreeNode> q = new ArrayDeque<>();
    if (root != null) q.add(root);
    while (!q.isEmpty()) {
        int size = q.size();                    // read once, before the loop
        List<Integer> level = new ArrayList<>();
        for (int i = 0; i < size; i++) {
            TreeNode t = q.poll();
            level.add(t.val);
            if (t.left != null) q.add(t.left);
            if (t.right != null) q.add(t.right);
        }
        out.add(level);
    }
    return out;
}
```

Right side view; zigzag

variant

```java
// Right side view: the last value of each level.
static List<Integer> rightSideView(TreeNode root) {
    List<Integer> out = new ArrayList<>();
    for (List<Integer> lv : levelOrder(root)) out.add(lv.get(lv.size() - 1));
    return out;
}

// Zigzag: the BFS order never changes; every second level is reversed.
static List<List<Integer>> zigzagLevelOrder(TreeNode root) {
    List<List<Integer>> levels = levelOrder(root);
    for (int d = 1; d < levels.size(); d += 2) Collections.reverse(levels.get(d));
    return levels;
}
```

Maximum width with rebased indices

variant

```java
// Maximum width: number the nodes like a heap (children 2i+1 and 2i+2).
// Rebase every level to start at 0, so an index is below 2 * width.
static int widthOfBinaryTree(TreeNode root) {
    if (root == null) return 0;
    Deque<TreeNode> q = new ArrayDeque<>();
    Deque<Long> idx = new ArrayDeque<>();
    q.add(root);
    idx.add(0L);
    long best = 0;
    while (!q.isEmpty()) {
        long first = idx.peek(), last = 0;
        for (int i = 0, size = q.size(); i < size; i++) {
            TreeNode t = q.poll();
            last = idx.poll() - first;              // rebased index
            if (t.left != null) { q.add(t.left); idx.add(2 * last + 1); }
            if (t.right != null) { q.add(t.right); idx.add(2 * last + 2); }
        }
        best = Math.max(best, last + 1);            // first is 0 after rebase
    }
    return (int) best;
}
```

Cost Time `O(n)` Space `O(w)`

Each node is queued and polled once. The queue holds at most one level plus its children; w, the widest level, can be about n/2. The returned lists hold n values.

Variants same idea, small change

- Average, maximum or sum of each level (LC 637, 515, 1161): the same loop with a running total or maximum instead of a list.
- Minimum depth: return the depth of the first leaf polled. BFS reaches the shallowest leaf before any deeper node.
- Populating next right pointers: inside the level loop, link each polled node to the next one; the last node of the level keeps `null`.
- Bottom-up level order (LC 107): build the same list and reverse it once at the end.

Traps bugs people actually write

- `for (int i = 0; i < q.size(); i++)` reads the size again while children are being added: on [3, 9, 20, null, null, 15, 7] the levels come out `[[3, 9], [20, 15], [7]]`.
- Zigzag by changing which child is queued first breaks the next level: on [1, 2, 3, 4, 5, 6, 7] the last level comes out `[6, 7, 4, 5]` instead of `[4, 5, 6, 7]`. Keep the queue order; reverse only what you write.
- Width is not the number of nodes on a level: in the trace the last level has 2 nodes but width 7.
- Right side view is not the right spine: on [1, 2, 3, 4], following `.right` gives `[1, 3]`, but 4 sticks out on the left, so the answer is `[1, 3, 4]`.

Practise on LeetCode, in this order

☐	LC 637	Average of Levels in Binary Tree	Easy
☐	LC 102	Binary Tree Level Order Traversal	Medium
☐	LC 199	Binary Tree Right Side View	Medium
☐	LC 103	Binary Tree Zigzag Level Order Traversal	Medium
☐	LC 1161	Maximum Level Sum of a Binary Tree	Medium
☐	LC 662	Maximum Width of Binary Tree	Medium

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

8.3 Return a value up the tree

O(n)

Spot it when the problem looks like this

- *Height*, *maximum depth*, *balanced*, *diameter*, *tilt*, or *count the nodes whose subtree* has some property.
- Each node needs one or two numbers from each child and nothing from its parent.
- The best answer need not pass through the root, as with the diameter, which can bend at any node.

How it works

Write a function that returns one fact about a subtree and computes it from the facts of the two children; height is `1 + max(left, right)`. This is a postorder: a node finishes after both children, so their values are ready when it needs them. Each node does O(1) work, so the whole tree costs O(n).

Balance needs more than the height: a tree is balanced only if every node is. So `height` returns –1 as soon as one subtree fails, and every caller passes the –1 straight up. One int carries both the height and the verdict.

The diameter is the longest path between any two nodes. Its highest node is where it bends, and there its length is left height + right height. So `depth` returns the height, since only one side can continue up to the parent, and records `l + r` in a field at every node. The largest value recorded is the answer.

Worked example **diameterOfBinaryTree on [1, 2, 3, 4, 5], nodes in finishing order**

```
node  l   r   l + r  returns  best
4     0   0   0      1        0
5     0   0   0      1        0
2     1   1   2      2        2
3     0   0   0      1        2
1     2   1   3      3        3
The longest path, 4-2-1-3 or 5-2-1-3, has 3 edges.
```

Height; balanced with –1 memorise

```java
// Each call returns one number about its subtree; the parent combines two.
static int maxDepth(TreeNode t) {
    if (t == null) return 0;
    return 1 + Math.max(maxDepth(t.left), maxDepth(t.right));
}

// Balanced: return the height, or -1 as soon as any subtree is unbalanced.
static boolean isBalanced(TreeNode root) {
    return height(root) != -1;
}

static int height(TreeNode t) {
    if (t == null) return 0;
    int l = height(t.left);
    if (l == -1) return -1;                    // pass the failure up
    int r = height(t.right);
    if (r == -1 || Math.abs(l - r) > 1) return -1;
    return 1 + Math.max(l, r);
}
```

Diameter: return height, record l + r

memorise

```java
// Diameter (in edges): return the height up, and at every node record
// the longest path that bends there: left height + right height.
static int best;

static int diameterOfBinaryTree(TreeNode root) {
    best = 0;                                   // reset: static outlives a call
    depth(root);
    return best;
}

static int depth(TreeNode t) {
    if (t == null) return 0;
    int l = depth(t.left), r = depth(t.right);
    best = Math.max(best, l + r);               // path through t
    return 1 + Math.max(l, r);                  // only one side can go up
}
```

Cost Time `O(n)` Space `O(h)`

One postorder pass with O(1) work per node; the recursion holds one root-to-leaf path.

Variants same idea, small change

- Return two values when one is not enough: `int[] {sum, count}` for *count nodes equal to the average of their subtree*.
- Distribute coins: return a subtree's coins minus its size. That many coins cross the edge to the parent, so add `|left| + |right|` moves at each node.
- Tilt: return the subtree sum and add `|left sum - right sum|` to a field.
- Arms with negative values that may be dropped, and paths scored by sums: 8.4 **Maximum path sum (one arm up)** (p. 194).

Traps bugs people actually write

- Checking balance only at the root: [1, 2, 2, 3, null, null, 3, 4, null, null, 4] has equal heights under the root, so that check says true; the tree is not balanced.
- `maxDepth(root.left) + maxDepth(root.right)` measures only paths through the root: [1, 2, null, 3, 4, 5, null, null, 6] gives 3, but the diameter is 4 and bends at node 2.
- LeetCode runs all test cases in one JVM, so a `static` field keeps its value from the previous case. Reset `best` at the start of every call.
- Recursion depth is the height. In testing, `maxDepth` handled a 10^4-node chain on Java's default 1 MB thread stack but overflowed at $5{\cdot}10^4$; for deeper trees, use the iterative postorder of 8.1 **Preorder, inorder, postorder** (p. 186).

Practise on LeetCode, in this order

- [ ] LC 104 Maximum Depth of Binary Tree — Easy
- [ ] LC 110 Balanced Binary Tree — Easy
- [ ] LC 543 Diameter of Binary Tree — Easy
- [ ] LC 563 Binary Tree Tilt — Easy
- [ ] LC 2265 Count Nodes Equal to Average of Subtree — Medium
- [ ] LC 979 Distribute Coins in Binary Tree — Medium

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

8.4 Maximum path sum (one arm up) `O(n)`

Spot it when the problem looks like this

- *The path may start and end at any node* and *does not need to pass through the root*.
- Values can be negative, so the best path may leave out an arm, or be a single node.
- *Longest path* in which neighbours must be equal (univalue) or different (adjacent characters).

How it works

Every path has one highest node, where it bends, and from there at most two downward arms. So compute `gain(t)` for every node: the best downward path that starts at t. It is `t.val` plus the better child gain, where a negative gain becomes 0 (leave that arm off).

Each node then gives two numbers. The best path bending at t is `t.val + l + r`, which goes into the global `best`. The parent gets `t.val + max(l, r)`, because it can extend only one arm; a path that kept both would fork. Every node takes its turn as the bend, so one O(n) postorder finds the answer. Longest univalue path works the same way; an arm only continues into a child with an equal value.

Worked example **maxPathSum on [1, -2, 3, 4, -5, -6, 2]: 8, the path 4, -2, 1, 3, 2**

node	l	r	t + l + r	returns	best
4	-	-	4 + 0 + 0 = 4	4	4
-5	-	-	-5 + 0 + 0 = -5	-5	4
-2	4	-5 -> 0	-2 + 4 + 0 = 2	2	4
-6	-	-	-6 + 0 + 0 = -6	-6	4
2	-	-	2 + 0 + 0 = 2	2	4
3	-6 -> 0	2	3 + 0 + 2 = 5	5	5
1	2	5	1 + 2 + 5 = 8	6	8

Binary tree maximum path sum

memorise

```
// gain(t) = best downward path from t; each node is tried as the bend.
static int best;

static int maxPathSum(TreeNode root) {
    best = Integer.MIN_VALUE;                  // a path has at least 1 node
    gain(root);
    return best;
}

static int gain(TreeNode t) {
    if (t == null) return 0;
    int l = Math.max(0, gain(t.left));         // a negative arm is dropped
    int r = Math.max(0, gain(t.right));
    best = Math.max(best, t.val + l + r);      // both arms meet at t
    return t.val + Math.max(l, r);             // only one arm goes up
}
```

Longest univalue path

variant

```
// Longest univalue path (edges): an arm goes on only into an equal child.
static int longest;

static int longestUnivaluePath(TreeNode root) {
    longest = 0;
    arm(root);
    return longest;
}

static int arm(TreeNode t) {
    if (t == null) return 0;
    int l = arm(t.left), r = arm(t.right);     // always visit both children
    int dl = t.left != null && t.left.val == t.val ? l + 1 : 0;
    int dr = t.right != null && t.right.val == t.val ? r + 1 : 0;
    longest = Math.max(longest, dl + dr);
    return Math.max(dl, dr);
}
```

Cost Time `O(n)` Space `O(h)`

One postorder pass with O(1) work per node; the call stack holds one root-to-leaf path.

Variants same idea, small change

- General tree, longest path with different adjacent characters (LC 2246): keep the two longest child arms that pass the rule; `best = max(best, 1 + top1 + top2)` and return `1 + top1`. Fill them children-first over the order of 8.9 **General tree from an edge list** (p. 207).
- Longest zigzag path: return two arms, one that starts by going left and one that starts by going right. `goLeft = 1 + left.goRight`, `goRight = 1 + right.goLeft`.
- The diameter is this pattern with every value 1, minus 1 because it counts edges (8.3 **Return a value up the tree** (p. 191)).
- Count paths with a given sum instead of finding the best: prefix sums on the root path (8.5 **Root-to-leaf paths and path sums** (p. 196)).

Traps bugs people actually write

- Starting `best` at 0: a tree of only negative values, such as [-3], returns 0, but a path has at least one node, so the answer is −3.
- Returning `t.val + l + r` to the parent: on [1, 2, 3, 4, 5] node 2 hands up the forked path 4-2-5 and the root reports 15; the answer is 11.
- In univalue, recursing only into children with the same value skips paths further down: on [1, 2, null, 2, null, 2] the chain of 2s is never reached and the answer is 0 instead of 2.
- `gain` recurses as deep as the tree. A $3 \cdot 10^4$-node chain, which LC 124 allows, overflowed Java's default 1 MB thread stack in testing; if that happens, compute the gains in the iterative postorder of 8.1 **Preorder, inorder, postorder** (p. 186).

Practise on LeetCode, in this order

☐	LC 687	Longest Univalue Path	Medium
☐	LC 1372	Longest ZigZag Path in a Binary Tree	Medium
☐	LC 124	Binary Tree Maximum Path Sum	Hard
☐	LC 2246	Longest Path With Different Adjacent Characters	Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

8.5 Root-to-leaf paths and path sums

O(n)

Spot it when the problem looks like this

- *Root-to-leaf path*: a leaf has no children, and the path must end at one.
- *Return all paths* as a list of lists, or *the sum of all root-to-leaf numbers.*
- *The path does not need to start at the root or end at a leaf, but it must go downwards*: count, do not list.

How it works

Pass state down as parameters: the sum still needed, or the number built so far, `sofar * 10 + t.val`. To list paths, share one `path` list. Add t on the way in and remove it on the way out, so `path` always holds the root-to-t path. At a leaf with the right sum, store a copy. This is backtracking (**7.1 Subsets (include or exclude)** (p. 160)) on a tree.

For downward paths that may start anywhere, use prefix sums along the root path (**1.4 Prefix sum + hash map** (p. 26)). Let `cur` be the sum from the root to t. A path that ends at t and starts right below ancestor a has sum `cur - prefix(a)`, so the paths ending at t with sum target are counted by how many ancestors have prefix `cur - target`. The map must hold t's ancestors only: add `cur` before visiting the children and take it out after. Each node then does O(1) map work, O(n) in total, instead of trying every start in O(n·h).

Worked example **pathSumAny(root, 8), root = [10, 5, -3, 3, 2, null, 11, 3, -2, null, 1]**

```
node  root path      cur  cur - 8  count  path found
10    [10]           10   2        0
5     [10,5]         15   7        0
3     [10,5,3]       18   10       1      [5, 3]
3     [10,5,3,3]     21   13       0
-2    [10,5,3,-2]    16   8        0
2     [10,5,2]       17   9        0
1     [10,5,2,1]     18   10       1      [5, 2, 1]
-3    [10,-3]        7    -1       0
11    [10,-3,11]     18   10       1      [-3, 11]
total 3; count = how many ancestors have prefix cur - 8
```

Path sum II with backtracking

memorise

```java
// Path sum II: path is the root-to-t path; add t going down, remove after.
static List<List<Integer>> pathSum(TreeNode root, int target) {
    List<List<Integer>> out = new ArrayList<>();
    walk(root, target, new ArrayList<>(), out);
    return out;
}

static void walk(TreeNode t, int remain, List<Integer> path,
                 List<List<Integer>> out) {
    if (t == null) return;
    path.add(t.val);
    remain -= t.val;
    if (t.left == null && t.right == null && remain == 0)
        out.add(new ArrayList<>(path));        // copy: path keeps changing
    walk(t.left, remain, path, out);
    walk(t.right, remain, path, out);
    path.remove(path.size() - 1);              // undo before going back up
}
```

Sum root to leaf numbers

variant

```java
// Sum root to leaf numbers: carry the number so far down; add it at leaves.
static int sumNumbers(TreeNode root) { return sumNumbers(root, 0); }

static int sumNumbers(TreeNode t, int sofar) {
    if (t == null) return 0;
    sofar = sofar * 10 + t.val;
    if (t.left == null && t.right == null) return sofar;
    return sumNumbers(t.left, sofar) + sumNumbers(t.right, sofar);
}
```

Path sum III: prefix sums on the path

memorise

```java
// Path sum III: count downward paths (any start, any end) with sum target.
// seen holds the prefix sums of the ancestors of t (root path only).
static int pathSumAny(TreeNode root, int target) {
    Map<Long, Integer> seen = new HashMap<>();
    seen.put(0L, 1);                            // empty prefix: start at root
    return count(root, 0L, target, seen);
}

static int count(TreeNode t, long cur, int target, Map<Long, Integer> seen) {
    if (t == null) return 0;
    cur += t.val;                               // sum from root to t
    int res = seen.getOrDefault(cur - target, 0);   // paths ending at t
    seen.merge(cur, 1, Integer::sum);           // visible to t's subtree
    res += count(t.left, cur, target, seen);
    res += count(t.right, cur, target, seen);
    seen.merge(cur, -1, Integer::sum);          // remove on the way back
    return res;
}
```

Cost Time `O(n)` Space `O(n)`

Path sum III does O(1) map work per node; a key whose count drops to 0 stays, so the map can reach n keys. Path sum II also copies every matching path, O(h) per copy.

Variants same idea, small change

- Path sum I (is there such a path?): return `true` at the first leaf where `remain == 0`.
- Binary numbers on the paths (LC 1022): `sofar * 2 + t.val`.
- Pseudo-palindromic paths: pass a bit mask of digit parities down. At a leaf the path can be rearranged into a palindrome iff `(mask & (mask - 1)) == 0`.
- Binary tree paths as strings: the same backtracking with a `StringBuilder`; undo with `setLength`.

Traps bugs people actually write

- `out.add(path)` without a copy stores the same list every time. It is empty when the walk ends, so LC 113 sample 1 returns `[[], []]`.
- Testing `remain == 0` at a `null` child instead of at a leaf accepts nodes with one child: [1, 2] with target 1 reports the path [1], but 1 is not a leaf.
- Without `seen.put(0L, 1)` no path can start at the root: LC 437 sample 2 gives 1 instead of 3. Keep the sums `long` too; in `int`, LC 437's chain of 10^9 values wraps and gives 2 instead of 0.
- Without the decrement on the way back, a sibling's prefix stays in the map: on [-2, 1, 0] with target −1 the count is 2 (a bogus 1 → 0 across the root) instead of 1.

Practise on LeetCode, in this order

☐	LC 112	Path Sum	Easy
☐	LC 257	Binary Tree Paths	Easy
☐	LC 1022	Sum of Root To Leaf Binary Numbers	Easy
☐	LC 129	Sum Root to Leaf Numbers	Medium
☐	LC 113	Path Sum II	Medium
☐	LC 437	Path Sum III	Medium

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

8.6 Binary search tree operations

`O(h)`

Spot it when the problem looks like this

- *Binary search tree*: every key in the left subtree is smaller than the node and every key in the right subtree is larger.
- Search, insert, delete, *k-th smallest*, or *an iterator over the BST in sorted order*.
- *Validate*: decide whether any binary tree follows the rule, with values from -2^{31} to $2^{31} - 1$.

How it works

The BST rule turns every operation into a walk down one path. At each node the key is either here, in the left subtree if smaller, or in the right if larger. Search and insert cost O(h): O(log n) on a balanced tree, O(n) on a chain.

Delete has three cases. A node with no left child is replaced by its right subtree, and the mirror case likewise. A node with two children takes the value of its successor, the smallest key on its right (one step right, then left to the end), and the successor is deleted from the right subtree. It has no left child, so that second delete is an easy case.

To validate, comparing a node with its children is not enough: the rule is about whole subtrees. Pass down the open range `(lo, hi)` that the ancestors allow; going left sets `hi = t.val`, going right sets

`lo = t.val`. Inorder visits a BST in increasing order, so the k-th pop of the iterative inorder is the k-th smallest.

Worked example **delete(root, 3) on [5, 3, 6, 2, 4, null, 7]; then validate another tree**

```
delete(5, 3): 3 < 5, go left
  delete(3, 3): found, 2 children: successor 4, copy it here
    delete(4, 4): found, no left child: return its right child, null
result (5 (4 (2 # #) #) (6 # (7 # #))) = [5, 4, 6, 2, null, null, 7]
---- isValidBST([5, 4, 6, null, null, 3, 7]) ----
5 in (-inf, +inf)? true
4 in (-inf, 5)? true
6 in (5, +inf)? true
3 in (5, 6)? false
```

Search and insert

memorise

```
// Search and insert follow one root-to-leaf path: O(h).
static TreeNode search(TreeNode t, int key) {
    while (t != null && t.val != key) t = key < t.val ? t.left : t.right;
    return t;
}

static TreeNode insert(TreeNode t, int key) {
    if (t == null) return new TreeNode(key);
    if (key < t.val) t.left = insert(t.left, key);
    else if (key > t.val) t.right = insert(t.right, key);
    return t;                                   // key already there: no change
}
```

Delete with the successor

memorise

```
// Delete: find the key, then lift a child up or copy in the successor.
static TreeNode delete(TreeNode t, int key) {
    if (t == null) return null;                 // key not in the tree
    if (key < t.val) t.left = delete(t.left, key);
    else if (key > t.val) t.right = delete(t.right, key);
    else {
        if (t.left == null) return t.right;     // 0 or 1 child: lift it up
        if (t.right == null) return t.left;
        TreeNode s = t.right;                   // 2 children: the successor
        while (s.left != null) s = s.left;      // is leftmost on the right
        t.val = s.val;                          // copy it here, then
        t.right = delete(t.right, s.val);       // delete it from the right
    }
    return t;
}
```

Validate with long bounds; k-th smallest

memorise

```java
// Validate: every node must lie strictly inside the range its ancestors
// allow. long bounds, so a node equal to Integer.MIN_VALUE still fits.
static boolean isValidBST(TreeNode root) {
    return valid(root, Long.MIN_VALUE, Long.MAX_VALUE);
}

static boolean valid(TreeNode t, long lo, long hi) {
    if (t == null) return true;
    if (t.val <= lo || t.val >= hi) return false;
    return valid(t.left, lo, t.val) && valid(t.right, t.val, hi);
}

// Kth smallest: inorder visits values in increasing order; stop at pop k.
static int kthSmallest(TreeNode root, int k) {
    Deque<TreeNode> st = new ArrayDeque<>();
    TreeNode cur = root;
    while (true) {
        while (cur != null) { st.push(cur); cur = cur.left; }
        cur = st.pop();
        if (--k == 0) return cur.val;
        cur = cur.right;
    }
}
```

Cost Time `O(h)` Space `O(h)`

Search, insert and delete walk one root-to-leaf path. h is about log n for random inserts and n for sorted ones; validate is O(n), k-th smallest O(h + k).

Variants same idea, small change

- Range sum of a BST: skip the left subtree when `t.val < low` and the right one when `t.val > high`.
- BST iterator (LC 173): keep the inorder stack as a field. The constructor pushes the left spine; `next()` pops t, pushes the left spine of `t.right` and returns `t.val`. O(h) memory and O(1) amortised per call.
- Inorder successor of a node: the leftmost node of its right subtree if it has one, else the last ancestor at which the search went left.
- No tree given, but you need floor, ceiling or ordered keys? Java's `TreeMap` and `TreeSet` are balanced BSTs (13.5 **Ordered maps (TreeMap, TreeSet)** (p. 375)).

Traps bugs people actually write

- Comparing each node only with its children: [5, 4, 6, null, null, 3, 7] passes that check, but 3 is in the right subtree of 5 (the trace).
- `int` bounds that start at `Integer.MIN_VALUE` reject the valid tree [-2147483648], because `t.val <= lo` is true. Use `long` bounds, or `Integer` bounds with `null` for none.
- After copying the successor, `t.right = delete(t.right, key)` looks for the old key, which is not on the right: deleting 3 from [5, 3, 6, 2, 4, null, 7] leaves two 4s.
- Sorted inserts build a chain, so h = n and the recursive insert and delete go n calls deep. With n near 10^4 or more, walk down with a loop as `search` does.

Practise on LeetCode, in this order

☐	`LC 700`	Search in a Binary Search Tree	Easy
☐	`LC 701`	Insert into a Binary Search Tree	Medium
☐	`LC 98`	Validate Binary Search Tree	Medium
☐	`LC 230`	Kth Smallest Element in a BST	Medium
☐	`LC 450`	Delete Node in a BST	Medium
☐	`LC 173`	Binary Search Tree Iterator	Medium

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

8.7 Lowest common ancestor `O(n)`

Spot it when the problem looks like this

- *Lowest common ancestor of p and q*, where a node counts as a descendant of itself.
- A path between two nodes: it climbs from one to their LCA and goes down to the other, so lengths and directions come from it.
- In a BST, the values alone say which side p and q are on.

How it works

Each call answers *is p or q in this subtree?* with a node: t itself if t is p or q, null for an empty subtree, else a combination of its children. If both sides return non-null, p and q are on different sides, so t is the lowest node above both. If one side found something, pass it up: p, q, or an LCA found lower down.

Returning at p without looking below it is safe. If q is under p, p is the answer; if not, another branch finds q and the two meet higher up. Each node is visited at most once: O(n).

In a BST, both values below `t.val` put the LCA on the left, both above put it on the right, and anything else means t separates them: an O(h) walk.

The path from s to d climbs to the LCA, then goes down. Written as L and R moves, the two root-to-node paths share a prefix that leads to the LCA.

Worked example lowestCommonAncestor(p = 6, q = 2), root = [3, 5, 1, 6, 2, 0, 8, null, null, 7, 4]

```
node  left   right  returns          (in finishing order)
6     -      -      6 (it is p, stop)
2     -      -      2 (it is q, stop; 7 and 4 are never visited)
5     6      2      5 (both sides: the LCA)
0     null   null   null
8     null   null   null
1     null   null   null
3     5      null   5
```

LCA in a binary tree

memorise

```
// LCA: return p or q if found in t's subtree, else null. Both sides: t.
static TreeNode lowestCommonAncestor(TreeNode t, TreeNode p, TreeNode q) {
    if (t == null || t == p || t == q) return t;
    TreeNode l = lowestCommonAncestor(t.left, p, q);
    TreeNode r = lowestCommonAncestor(t.right, p, q);
    if (l != null && r != null) return t;      // p and q on different sides
    return l != null ? l : r;                  // pass up what was found
}
```

Step-by-step directions (LC 2096)

example

```
// Step-by-step directions: root-to-node paths as strings of L and R.
// Their common prefix leads to the LCA; climb from s to it, then go down.
static String getDirections(TreeNode root, int s, int d) {
    StringBuilder a = new StringBuilder(), b = new StringBuilder();
    find(root, s, a);
    find(root, d, b);
    int i = 0;
    while (i < a.length() && i < b.length() && a.charAt(i) == b.charAt(i)) i++;
    for (int j = i; j < a.length(); j++) a.setCharAt(j, 'U');    // climb
    return a.substring(i) + b.substring(i);                      // descend
}

static boolean find(TreeNode t, int target, StringBuilder path) {
    if (t == null) return false;
    if (t.val == target) return true;
    path.append('L');
    if (find(t.left, target, path)) return true;
    path.setCharAt(path.length() - 1, 'R');
    if (find(t.right, target, path)) return true;
    path.deleteCharAt(path.length() - 1);      // not below t: undo
    return false;
}
```

LCA in a BST

memorise

```java
// LCA in a BST: walk down while p and q are on the same side of t.
static TreeNode lcaBst(TreeNode t, TreeNode p, TreeNode q) {
    while (true) {                          // p and q are in the tree
        if (p.val < t.val && q.val < t.val) t = t.left;
        else if (p.val > t.val && q.val > t.val) t = t.right;
        else return t;                      // they split here, or t is one
    }
}
```

Cost Time `O(n)` Space `O(h)`

The binary tree version visits each node at most once; the BST walk is O(h); directions makes two root-to-node searches.

Variants same idea, small change

- LCA of the deepest leaves (LC 1123): each subtree returns `(depth, node)`. Equal depths on both sides make t the answer; otherwise pass up the deeper side.
- Distance between p and q: `depth(p) + depth(q) - 2 * depth(lca)`.
- With parent pointers: walk p and q up together; when one passes the root, restart it at the other node. They meet at the LCA, as two linked lists meet at their intersection.
- Many LCA queries on a large general tree: **8.10 Binary lifting (k-th ancestor, LCA)** (p. 210) answers each in O(log n), and **8.13 Euler tour (subtree = range)** (p. 218) gives the ancestor test in O(1).

Traps bugs people actually write

- The template assumes p and q are both in the tree. If q is missing it returns p as if p were the LCA; when that can happen, count how many of the two you actually found.
- In the BST walk, `||` instead of `&&`: for p = 2 and q = 8 in [6, 2, 8, 0, 4, 7, 9, null, null, 3, 5], `p.val < t.val || q.val < t.val` goes left at 6 and returns 2 instead of 6.
- Directions built as `path + "L"` copy the string at every level, $O(n^2)$ on a chain. LC 2096 allows 10^5 nodes, so a chain also means a recursion 10^5 deep; if `find` overflows, fill a parent map with a loop and climb up from s and d.

Practise on LeetCode, in this order

- [ ] LC 235 Lowest Common Ancestor of a Binary Search Tree — Medium
- [ ] LC 236 Lowest Common Ancestor of a Binary Tree — Medium
- [ ] LC 1123 Lowest Common Ancestor of Deepest Leaves — Medium
- [ ] LC 2096 Step-By-Step Directions From a Binary Tree Node to Another — Medium

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

8.8 Build and serialize a tree

O(n)

Spot it when the problem looks like this

- *Construct the binary tree from preorder and inorder* (or inorder and postorder); *values are unique*.
- *Serialize and deserialize*: you design the format, and decoding must give back the same tree.
- n up to 3000, so even an $O(n^2)$ inorder search for each root passes.

How it works

Preorder lists a root before its whole subtree; inorder lists the left subtree, the root, then the right subtree. So the next preorder value is the root, and its position `mid` in inorder splits the range into the left subtree `[lo, mid - 1]` and the right subtree `[mid + 1, hi]`. Build the left side first: preorder lists all of it before the right, so one counter `next` always points at the root of the range being built. A `HashMap` from value to inorder index makes each step O(1), so the build is O(n).

One traversal alone does not fix the shape: preorder `1, 2` fits 2 as either child. A marker for every null child removes the doubt. Reading the tokens back in order rebuilds the tree: each token is a null, or a node followed by its subtrees.

Worked example **buildTree(pre = [3, 9, 20, 15, 7], in = [9, 3, 15, 20, 7])**

```
in range   next  root  mid  left range     right range
[0, 4]     0     3     1    [0, 0] [9]     [2, 4] [15,20,7]
[0, 0]     1     9     0    -              -
[2, 4]     2     20    3    [2, 2] [15]    [4, 4] [7]
[2, 2]     3     15    2    -              -
[4, 4]     4     7     4    -              -
tree [3, 9, 20, null, null, 15, 7]; serialize: 3,9,#,#,20,15,#,#,7,#,#,
```

Preorder + inorder with an index map

memorise

```java
// Build from preorder + inorder (distinct values). pre[next] is the root of
// the current inorder range; its position there splits left from right.
static int next;
static Map<Integer, Integer> pos;

static TreeNode buildTree(int[] pre, int[] in) {
    next = 0;
    pos = new HashMap<>();
    for (int i = 0; i < in.length; i++) pos.put(in[i], i);  // O(1) lookups
    return build(pre, 0, in.length - 1);
}

static TreeNode build(int[] pre, int lo, int hi) {     // inorder [lo, hi]
    if (lo > hi) return null;
    TreeNode t = new TreeNode(pre[next++]);
    int mid = pos.get(t.val);
    t.left = build(pre, lo, mid - 1);          // left first: preorder is
    t.right = build(pre, mid + 1, hi);         // root, left, right
    return t;
}
```

Serialize: preorder with # for null

memorise

```java
// Serialize: preorder, with "#" for every null child.
static String serialize(TreeNode root) {
    StringBuilder sb = new StringBuilder();
    write(root, sb);
    return sb.toString();
}

static void write(TreeNode t, StringBuilder sb) {
    if (t == null) { sb.append("#,"); return; }
    sb.append(t.val).append(',');
    write(t.left, sb);
    write(t.right, sb);
}
```

Deserialize in the same order

memorise

```java
// Deserialize: read the tokens in the same order write() produced them.
static TreeNode deserialize(String data) {
    return read(new ArrayDeque<>(Arrays.asList(data.split(","))));
}

static TreeNode read(Deque<String> tokens) {
    String s = tokens.poll();
    if (s.equals("#")) return null;
    TreeNode t = new TreeNode(Integer.parseInt(s));
    t.left = read(tokens);                    // same order as write()
    t.right = read(tokens);
    return t;
}
```

Cost Time `O(n)` Space `O(n)`

Each value is placed once with an O(1) map lookup; the map and the token queue hold n entries, the recursion one path.

Variants same idea, small change

- Inorder + postorder (LC 106): take roots from the end of postorder and build the **right** subtree first.
- Preorder + postorder (LC 889): the value after the root in preorder is the left child, and its position in postorder gives the size of the left subtree. The tree may not be unique.
- BST from preorder alone (LC 1008): pass an upper bound down and take the next value while it is below the bound; O(n), no inorder needed. The same idea lets a BST serialize without null markers (LC 449).
- LeetCode's level-order strings such as `[1,2,null,3]`: BFS, write both children of each polled node, and read them back two at a time (8.2 **Level order (BFS by levels)** (p. 189)).

Traps bugs people actually write

- Building `t.right` before `t.left` takes the wrong values as roots: on the LC 105 sample it runs past the end of preorder with `ArrayIndexOutOfBoundsException`.
- `next` and `pos` are static. Without the reset at the top of `buildTree`, a second test case starts at the old index.
- Without null markers the string is ambiguous: `1,2` decodes the same whether 2 was a left or a right child, so the round trip can fail.
- Repeated values break the index map: a value's inorder position is no longer unique, so the build can split at the wrong place.

Practise on LeetCode, in this order

☐	`LC 1008`	Construct Binary Search Tree from Preorder Traversal	Medium
☐	`LC 105`	Construct Binary Tree from Preorder and Inorder Traversal	Medium
☐	`LC 106`	Construct Binary Tree from Inorder and Postorder Traversal	Medium
☐	`LC 889`	Construct Binary Tree from Preorder and Postorder Traversal	Medium
☐	`LC 449`	Serialize and Deserialize BST	Medium
☐	`LC 297`	Serialize and Deserialize Binary Tree	Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

8.9 General tree from an edge list `O(n)`

Spot it when the problem looks like this

- *n nodes labelled 0 to n − 1 and n − 1 edges, rooted at node 0*, with each edge in no particular direction.
- n up to 10^5: a recursive DFS on a path of that length can overflow Java's stack.
- The answer adds up over subtrees: apples below a node, people in a subtree, labels below, sizes.

How it works

Store the tree as adjacency lists. Each edge goes into both lists, because the input does not say which end is the parent. A DFS with an explicit stack from the root then fixes the direction: when v is popped, every neighbour except `parent[v]` is a child, so it gets `parent = v` and `depth[v] + 1`. A tree has no cycles, so skipping the parent is enough to never meet a node twice.

`order[]` lists nodes as they are popped. Every node is pushed by its parent, so a parent always comes before its children, and read backwards the children always come first. That is all a bottom-up pass needs. Subtree sizes take one loop that adds `size[v]` into `size[parent[v]]`, and the same loop shape combines anything else: apples below, people crossing an edge, label counts.

Each node is pushed once and each edge is read from both ends, so the walk is O(n). The only stack is a `Deque` on the heap, which holds 10^5 nodes without trouble.

Worked example walk(g, 0), edges = [[0,1], [0,2], [1,4], [1,5], [2,3], [2,6]]; stack bottom to top

```
pop  parent  depth   push   stack after
0    -1      0       1 2    [1, 2]
2    0       1       3 6    [1, 3, 6]
6    2       2       -      [1, 3]
3    2       2       -      [1]
1    0       1       4 5    [4, 5]
5    1       2       -      [4]
4    1       2       -      []
order [0, 2, 6, 3, 1, 5, 4]; backwards: 4 5 1 3 6 2 0
size  [7, 3, 3, 1, 1, 1, 1]  (by node 0..6)
```

Adjacency lists from an edge list

memorise

```
// A general tree given as n and an edge list. Each edge goes into both
// lists, because the input does not say which end is the parent.
static List<Integer>[] adjacency(int n, int[][] edges) {
    List<Integer>[] g = new List[n];
    for (int v = 0; v < n; v++) g[v] = new ArrayList<>();
    for (int[] e : edges) { g[e[0]].add(e[1]); g[e[1]].add(e[0]); }
    return g;
}
```

Iterative walk: parent, depth, order, size

memorise

```
// One iterative DFS fills parent[], depth[] and order[] (each node after
// its parent). order[] read backwards puts children first: sizes add up.
static int[] parent, depth, order, size;

static void walk(List<Integer>[] g, int root) {
    int n = g.length, k = 0;
    parent = new int[n]; depth = new int[n]; order = new int[n];
    Deque<Integer> st = new ArrayDeque<>();
    st.push(root);
    parent[root] = -1;
    while (!st.isEmpty()) {
        int v = st.pop();
        order[k++] = v;
        for (int u : g[v]) {
            if (u == parent[v]) continue;      // the edge back up
            parent[u] = v;
            depth[u] = depth[v] + 1;
            st.push(u);
        }
    }
    size = new int[n];
    for (int i = n - 1; i >= 0; i--) {         // children before parents
        int v = order[i];
        size[v] += 1;
        if (parent[v] >= 0) size[parent[v]] += size[v];
    }
}
```

Minimum time to collect all apples

example

```
// Apples: edge (v, parent) is walked twice iff v's subtree has an apple.
static int minTime(int n, int[][] edges, List<Boolean> hasApple) {
    walk(adjacency(n, edges), 0);
    boolean[] need = new boolean[n];
    int time = 0;
    for (int i = n - 1; i > 0; i--) {          // i > 0: order[0] is the root
        int v = order[i];
        if (hasApple.get(v)) need[v] = true;
        if (need[v]) {
            time += 2;
            need[parent[v]] = true;
        }
    }
    return time;
}
```

Cost Time `O(n)` Space `O(n)`

Every node is pushed and popped once and every edge is read from both ends; parent, depth, order and size are four int arrays.

Variants same idea, small change

- Minimum fuel cost (LC 2477): the `size[v]` people of v's subtree all cross the edge to `parent[v]`, in `(size[v] + seats - 1) / seats` cars. The answer is that sum over every v except the root.
- Same label in the subtree (LC 1519): an `int[26]` count per node. Walking `order` backwards, add each node's counts into its parent's; the answer for v is its count of its own label.
- Reorder routes to city 0: walk from 0; an original road `[a, b]` with `parent[b] == a` points away from 0 and must be reversed. K-divisible components (LC 2872): count the nodes whose subtree sum is divisible by k.
- A parent array instead of edges (LC 2049): build child lists from `parents[]`. Removing v leaves parts of size `size[child]` and `n - size[v]`.

Traps bugs people actually write

- Assuming `edges[i][0]` is the parent: LC 1443 has edges `[[0, 2], [0, 3], [1, 2]]`, where 1 hangs below 2. With an apple only at 1 that version answers 0 instead of 4.
- Leaving `parent[root]` at 0 instead of −1: the size loop adds the root into itself, and `size[0]` comes out 14 for the 7-node trace.
- A recursive DFS overflowed Java's default 1 MB thread stack in testing on a path of 10^4 nodes. The iterative walk handles 10^5.
- Fuel needs ceiling division, `(size[v] + seats - 1) / seats`, and a `long` total: one seat per car on a 10^5-node path costs 4,999,950,000 litres.

Practise on LeetCode, in this order

- [ ] LC 1466 Reorder Routes to Make All Paths Lead to the City Zero — Medium
- [ ] LC 1443 Minimum Time to Collect All Apples in a Tree — Medium
- [ ] LC 1519 Number of Nodes in the Sub-Tree With the Same Label — Medium
- [ ] LC 2049 Count Nodes With the Highest Score — Medium
- [ ] LC 2477 Minimum Fuel Cost to Report to the Capital — Medium
- [ ] LC 2872 Maximum Number of K-Divisible Components — Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

8.10 Binary lifting (k-th ancestor, LCA)

`O(log n) each`

Spot it when the problem looks like this

- Up to $5 \cdot 10^4$ or 10^5 queries on a tree of up to 10^5 nodes, so climbing one parent at a time (O(n) per query) is too slow.
- *k-th ancestor*, *LCA of many pairs*, or questions about the path between two nodes (its length, the weights on it).
- The tree does not change between queries.

How it works

Store `up[j][v]`, the ancestor 2^j levels above v, or –1 if there is none. Row 0 is the parent array. A jump of 2^j is two jumps of 2^(j–1), so `up[j][v] = up[j-1][up[j-1][v]]`, and each row is built from the finished row below it. With 2^LOG ≥ n the table has n·LOG entries: O(n log n) time and memory.

To go k levels up, write k in binary and make one jump per set bit; 13 = 8 + 4 + 1 is three jumps. That is at most LOG jumps.

For the LCA of a and b, first lift the deeper one to the other's depth. If they meet, that node is the answer. Otherwise try the jumps from the largest down, and take a jump only if a and b land on different nodes. The LCA and everything above it are shared, so a jump that keeps them apart stays below the LCA. After the last jump they are two different children of the LCA, which is `up[0][a]`.

Worked example edges [[0,1], [1,2], [2,3], [3,4], [4,5], [1,6], [6,7], [3,8]], root 0, LOG = 4

```
v            0  1  2  3  4  5  6  7  8
depth        0  1  2  3  4  5  2  3  4
up[0]  +1   -1  0  1  2  3  4  1  6  3
up[1]  +2   -1 -1  0  1  2  3  0  1  2
up[2]  +4   -1 -1 -1 -1  0  1 -1 -1  0
up[3]  +8   -1 -1 -1 -1 -1 -1 -1 -1 -1
kthAncestor(5, 5), 5 = 101b: up[0][5] = 4, then up[2][4] = 0
lca(5, 7): lift 5 by 2 to 3, depth 3 like 7
  j = 3, 2: both land on -1, stay;  j = 1: both land on 1, stay
  j = 0: up[0][3] = 2, up[0][7] = 6 differ, jump; answer up[0][2] = 1
```

The up table and the k-th ancestor

memorise

```
// up[j][v] = the 2^j-th ancestor of v (-1 if none) = up[j-1][up[j-1][v]].
static int LOG;
static int[][] up;

static void buildLifting(int[] parent) {       // parent[root] = -1
    int n = parent.length;
    LOG = 1;
    while ((1 << LOG) < n) LOG++;              // 2^LOG >= n > any depth
    up = new int[LOG][n];
    up[0] = parent.clone();
    for (int j = 1; j < LOG; j++)              // row j needs row j-1 only
        for (int v = 0; v < n; v++) {
            int mid = up[j - 1][v];
            up[j][v] = mid < 0 ? -1 : up[j - 1][mid];
        }
}

// k-th ancestor: one jump per set bit of k, so at most LOG jumps.
static int kthAncestor(int v, int k) {
    if (k >= up[0].length) return -1;          // more than any depth
    for (int j = 0; k > 0 && v >= 0; j++, k >>= 1)
        if ((k & 1) == 1) v = up[j][v];
    return v;
}
```

LCA by lifting

memorise

```
// LCA (depth[] from walk(), 8.9): same depth, then jump while apart.
static int lca(int a, int b) {
    if (depth[a] < depth[b]) { int t = a; a = b; b = t; }
    a = kthAncestor(a, depth[a] - depth[b]);
    if (a == b) return a;                      // b was an ancestor of a
    for (int j = LOG - 1; j >= 0; j--)
        if (up[j][a] != up[j][b]) {            // both still below the LCA
            a = up[j][a];
            b = up[j][b];
        }
    return up[0][a];
}
```

Edge weight equilibrium queries

example

```java
// Edge weight equilibrium queries (weights 1..26): make every edge on the
// path a-b equal with the fewest changes = path length - most common weight.
// cnt[v][w] counts weight w on the root-to-v path; the path a-b has
// cnt[a][w] + cnt[b][w] - 2 * cnt[lca][w] edges of weight w.
static int[] minOperationsQueries(int n, int[][] edges, int[][] queries) {
    walk(adjacency(n, edges), 0);              // adjacency ignores e[2]
    buildLifting(parent);
    int[] w = new int[n];                      // weight of edge v-parent[v]
    for (int[] e : edges) w[parent[e[0]] == e[1] ? e[0] : e[1]] = e[2];
    int[][] cnt = new int[n][];
    cnt[0] = new int[27];
    for (int i = 1; i < n; i++) {              // parents before children
        int v = order[i];
        cnt[v] = cnt[parent[v]].clone();
        cnt[v][w[v]]++;
    }
    int[] ans = new int[queries.length];
    for (int i = 0; i < queries.length; i++) {
        int a = queries[i][0], b = queries[i][1], c = lca(a, b), most = 0;
        for (int x = 1; x <= 26; x++)
            most = Math.max(most, cnt[a][x] + cnt[b][x] - 2 * cnt[c][x]);
        ans[i] = depth[a] + depth[b] - 2 * depth[c] - most;
    }
    return ans;
}
```

Cost Time `O(log n) each` Space `O(n log n)`

Building the table is O(n log n) time and memory; a k-th ancestor query makes at most LOG jumps and an LCA query at most 2·LOG.

Variants same idea, small change

- LC 1483 wraps the same code in a class: build `up` in the constructor from `parent`, and answer `getKthAncestor` with the bit loop.
- Path minimum, maximum or sum: store with each jump the value over the edges it covers, `mn[j][v] = min(mn[j-1][v], mn[j-1][up[j-1][v]])`, and combine them while lifting.
- Counts on a path need no lifting beyond the LCA: keep counts on each root path, as in the equilibrium code; the path a-b has `cnt[a] + cnt[b] - 2 * cnt[lca]`.
- One successor per node, cycles allowed (the ball passing game, LC 2836): the same table jumps k steps in O(log k). There k reaches 10^{10}, so LOG must cover k, not n.

Traps bugs people actually write

- `LOG = (int) (Math.log(n) / Math.log(2))` is 2 for n = 5, so a jump of 4 has no row: on the path 0-1-2-3-4, `kthAncestor(4, 4)` throws `ArrayIndexOutOfBoundsException`. Grow LOG until `1 << LOG >= n`.
- Filling the table with v in the outer loop reads entries of nodes not done yet: for parent `[-1, 2, 3, 0, 0]` the 4th ancestor of node 1 comes out 0 instead of −1. Loop over j outside.
- Without `if (a == b) return a` after the first lift, the loop never moves when b is an ancestor of a, and the answer is b's parent: `lca(5, 3)` in the trace tree gives 2 instead of 3.

Practise on LeetCode, in this order

- [ ] `LC 1483` Kth Ancestor of a Tree Node — Hard
- [ ] `LC 2846` Minimum Edge Weight Equilibrium Queries in a Tree — Hard
- [ ] `LC 2836` Maximize Value of Function in a Ball Passing Game — Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

8.11 Tree DP (take or skip a node) `O(n)`

Spot it when the problem looks like this

- *No two directly linked nodes* may both be chosen: houses, halved prices, an independent set.
- Every node must be covered by a chosen node at distance at most 1: cameras, guards.
- Taking every other level fails on small examples; what a node may do depends on what its children did.

How it works

Solve each subtree once for every state its root can be in, and return all the answers. For house robber the states are *robbed* and *not robbed*. If t is robbed its children are not: `take = t.val + l[1] + r[1]`. If not, each child is free: `skip = max(l) + max(r)`. The rest of the tree meets the subtree only through t's state, so these two numbers are all the parent needs, at O(1) per node.

Cameras need three states: t has a camera, t is covered by a child's camera, or t is not covered yet and its parent must cover it. A null child counts as covered but cannot hold a camera, so its camera state is `INF`. On a general tree, fill the states children-first over the `order` of 8.9 **General tree from an edge list** (p. 207), read backwards.

Worked example rob(root), root = [3, 4, 5, 1, 3, null, 1]; pairs are [take, skip]

node	left	right	take	skip	returns
1 under 4	[0, 0]	[0, 0]	1 + 0 + 0 = 1	0 + 0 = 0	[1, 0]
3 under 4	[0, 0]	[0, 0]	3 + 0 + 0 = 3	0 + 0 = 0	[3, 0]
4 under 3	[1, 0]	[3, 0]	4 + 0 + 0 = 4	1 + 3 = 4	[4, 4]
1 under 5	[0, 0]	[0, 0]	1 + 0 + 0 = 1	0 + 0 = 0	[1, 0]
5 under 3	[0, 0]	[1, 0]	5 + 0 + 0 = 5	0 + 1 = 1	[5, 1]
3 (root)	[4, 4]	[5, 1]	3 + 4 + 1 = 8	4 + 5 = 9	[8, 9]

answer max(8, 9) = 9: rob 4 and 5

House robber III: {take, skip}

memorise

```java
// House robber III: each node returns two answers for its subtree,
// {best if t is robbed, best if t is not robbed}.
static int rob(TreeNode root) {
    int[] r = robPair(root);
    return Math.max(r[0], r[1]);
}

static int[] robPair(TreeNode t) {
    if (t == null) return new int[] {0, 0};
    int[] l = robPair(t.left), r = robPair(t.right);
    int take = t.val + l[1] + r[1];            // children must be skipped
    int skip = Math.max(l[0], l[1]) + Math.max(r[0], r[1]);   // free choice
    return new int[] {take, skip};
}
```

Binary tree cameras: three states

variant

```java
// Binary tree cameras. Three states for t, each with everything below t
// covered: {camera at t, covered by a child's camera, not covered yet}.
static final int INF = 1 << 20;

static int minCameraCover(TreeNode root) {
    int[] s = cam(root);
    return Math.min(s[0], s[1]);               // the root has no parent
}

static int[] cam(TreeNode t) {
    if (t == null) return new int[] {INF, 0, 0};    // null: covered, no camera
    int[] l = cam(t.left), r = cam(t.right);
    int here = 1 + Math.min(l[0], Math.min(l[1], l[2]))   // t covers both
                 + Math.min(r[0], Math.min(r[1], r[2]));
    int byChild = Math.min(l[0] + Math.min(r[0], r[1]),  // one child has one
                           r[0] + Math.min(l[0], l[1]));
    int open = l[1] + r[1];                    // children covered, no camera
    return new int[] {here, byChild, open};
}
```

Max weight independent set, general tree

example

```java
// Maximum weight independent set on a general tree: the same take/skip
// pair, filled children-first by walking order[] (from walk(), 8.9) back.
static long maxIndependentSet(int n, int[][] edges, int[] w) {
    walk(adjacency(n, edges), 0);
    long[] take = new long[n], skip = new long[n];
    for (int i = n - 1; i >= 0; i--) {
        int v = order[i], p = parent[v];
        take[v] += w[v];                        // children already added
        if (p >= 0) {
            take[p] += skip[v];
            skip[p] += Math.max(take[v], skip[v]);
        }
    }
    return Math.max(take[0], skip[0]);
}
```

Cost Time `O(n)` Space `O(h)`

O(1) work per state per node. The binary versions recurse one path deep; the general version keeps two long arrays, O(n).

Variants same idea, small change

- Cameras greedily: put a camera at a node only when a child is uncovered, and return one of three codes (needs cover, covered, has camera). Same answer, one int per node.
- Minimize the total price of the trips (LC 2646): count how many trips pass each node, then run take/skip where *take* halves a node's price and its neighbours must stay full price.
- Maximum score after operations (LC 2925): `keep[v]`, the least value left in v's subtree so every root-to-leaf path keeps a node, is `val[v]` at a leaf, else `min(val[v], sum of keep[children])`. The answer is total − keep[root].
- More states when the rule needs them: the distance to the nearest chosen node, a count of chosen nodes, or a parity.

Traps bugs people actually write

- Taking every other level: on [4, 1, null, 2, null, 3] the levels give max(4 + 2, 1 + 3) = 6, but robbing 4 and 3, which are not linked, gives 7.
- `rob(t) = max(t.val + rob(grandchildren), rob(children))` without a memo solves each node again from its parent and its grandparent; on a chain the calls grow like Fibonacci numbers.
- `Integer.MAX_VALUE` as INF overflows when 1 is added: [0, null, 0] answers −2147483648 instead of 1. Use an INF with headroom, such as `1 << 20`.
- At the root, taking the min of all three camera states counts *not covered*: [0] returns 0. Nobody is above the root, so only the first two states count.

Practise on LeetCode, in this order

- [ ] LC 337 House Robber III — Medium
- [ ] LC 2925 Maximum Score After Applying Operations on a Tree — Medium
- [ ] LC 968 Binary Tree Cameras — Hard
- [ ] LC 2646 Minimize the Total Price of the Trips — Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

8.12 Rerooting (every node as root)

O(n)

Spot it when the problem looks like this

- *For each node i, return …*: an answer array of length n with n up to $3 \cdot 10^4$ or 10^5, so a separate DFS from every node, $O(n^2)$, is too slow.
- *Which nodes could be the root?* or *the best root.*
- Moving the root across one edge changes the answer in a way you can compute in O(1).

How it works

Solve for one root, node 0, with the children-first pass of **8.9 General tree from an edge list** (p. 207). Then move the root along the edges, parents first, and compute `ans[v]` from `ans[parent[v]]`. When the root moves from p to its child v, only the edge (p, v) turns around. Every other edge keeps its direction relative to the root, so the change depends only on the two sides of that edge, `size[v]` nodes and `n - size[v]` nodes, and costs O(1).

For the sum of distances, `ans[0]` is the sum of all depths. Moving to v brings v's `size[v]` nodes one step closer and the other `n - size[v]` one step further: `ans[v] = ans[p] - size[v] + (n - size[v])`. For edge reversals, an edge that points down (p → v) was fine for root p and must flip for root v: +1. One that points up: −1.

Worked example **sumOfDistancesInTree(6, [[0,1], [0,2], [2,3], [2,4], [2,5]]); order [0, 2, 5, 4, 3, 1]**

```
v   parent  depth  size  ans[v] = ans[parent] - size + (n - size)
0   -1      0      6     sum of depths = 8
2   0       1      4     8 - 4 + 2 = 6
5   2       2      1     6 - 1 + 5 = 10
4   2       2      1     6 - 1 + 5 = 10
3   2       2      1     6 - 1 + 5 = 10
1   0       1      1     8 - 1 + 5 = 12
answer [8, 12, 6, 10, 10, 10]
```

Sum of distances in tree

memorise

```
// Pass 1, walk() from 8.9 at root 0: the sum of depths is ans[0].
// Pass 2, parents first: root p -> child v, ans[v] from ans[p] in O(1).
static int[] sumOfDistancesInTree(int n, int[][] edges) {
    walk(adjacency(n, edges), 0);
    int[] ans = new int[n];
    for (int v = 0; v < n; v++) ans[0] += depth[v];
    for (int i = 1; i < n; i++) {              // order[0] is the root
        int v = order[i];
        ans[v] = ans[parent[v]] - size[v] + (n - size[v]);
    }
    return ans;
}
```

Minimum edge reversals for every root

variant

```
// Min edge reversals to reach every node from v. For root 0, flip every
// edge that points up; moving the root across (p, v) changes only that one.
static int[] minEdgeReversals(int n, int[][] edges) {
    walk(adjacency(n, edges), 0);
    boolean[] down = new boolean[n];           // edge parent[v] -> v exists
    for (int[] e : edges) if (parent[e[1]] == e[0]) down[e[1]] = true;
    int[] ans = new int[n];
    for (int v = 1; v < n; v++) if (!down[v]) ans[0]++;
    for (int i = 1; i < n; i++) {
        int v = order[i];                      // root moves parent[v] -> v
        ans[v] = ans[parent[v]] + (down[v] ? 1 : -1);
    }
    return ans;
}
```

Count the possible roots (guesses)

example

```
// Count possible roots: guess [u, v] claims parent(v) == u. Moving the
// root from p to v makes [p, v] false and [v, p] true; nothing else changes.
static int rootCount(int[][] edges, int[][] guesses, int k) {
    int n = edges.length + 1;
    walk(adjacency(n, edges), 0);
    Set<Long> g = new HashSet<>();
    for (int[] q : guesses) g.add((long) q[0] * n + q[1]);
    int[] right = new int[n];
    for (int v = 1; v < n; v++)
        if (g.contains((long) parent[v] * n + v)) right[0]++;
    int roots = right[0] >= k ? 1 : 0;
    for (int i = 1; i < n; i++) {
        int v = order[i], p = parent[v];
        right[v] = right[p];
        if (g.contains((long) p * n + v)) right[v]--;   // p is not v's parent
        if (g.contains((long) v * n + p)) right[v]++;   // v is now p's parent
        if (right[v] >= k) roots++;
    }
    return roots;
}
```

Cost Time `O(n)` Space `O(n)`
One walk for root 0 and one O(1) update per edge; the guess set adds O(g) for g guesses.

Variants same idea, small change

- When the value for a root is a maximum (height from every root for minimum height trees, max price sum), removing v from p's answer needs the best child other than v: keep the top two child values per node and use the second when the best came through v.
- Count possible roots (LC 2581): when the root moves from p to v, guess `[p, v]` stops being true and `[v, p]` becomes true, as in `rootCount`.
- Answers for every node that depend on all other nodes: first work out what crossing one edge changes, before writing n DFS runs.

Traps bugs people actually write

- Updating in label order, `for (v = 1; v < n; v++)`, instead of over `order[]` reads `ans[parent]` before it is set: on edges [[0, 2], [2, 1]] node 1 gets 1 instead of 3.
- A BFS from every node is $O(n^2)$: about $9 \cdot 10^8$ steps at $n = 3 \cdot 10^4$, far over the time limit.
- `q[0] * n + q[1]` in `int` overflows at $n = 10^5$, since $10^{10} > 2^{31}$. Cast to `long` before multiplying, as `rootCount` does.
- The sum of distances fits `int` for LC 834 (a $3 \cdot 10^4$-node path gives 449,985,000), but a 10^5-node path reaches about $5 \cdot 10^9$: use `long` for larger n.

Practise on LeetCode, in this order

☐	`LC 310`	Minimum Height Trees	Medium
☐	`LC 834`	Sum of Distances in Tree	Hard
☐	`LC 2858`	Minimum Edge Reversals So Every Node Is Reachable	Hard
☐	`LC 2581`	Count Number of Possible Root Nodes	Hard
☐	`LC 2538`	Difference Between Maximum and Minimum Price Sum	Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

8.13 Euler tour (subtree = range) `O(n)`

Spot it when the problem looks like this

- Queries or updates on *the subtree of v* (a sum, XOR or count), mixed with point updates.
- Many *is u an ancestor of v* tests, such as trying every pair of removed edges with n ≤ 1000.
- An edge weight changes, and with it the root distance of every node below it: a range update over a subtree.

How it works

Number the nodes in DFS preorder: `tin[v]` is v's position. A DFS finishes v's whole subtree before it moves on, so the subtree fills the positions `tin[v]` to `tout[v] = tin[v] + size[v] - 1`. The stack walk of 8.9 **General tree from an edge list** (p. 207) does the same: when v is popped its children go on top, and nothing below them comes off until all of v's descendants are done. So its `order[]` is the tour, with no recursion.

Now u is an ancestor of v exactly when `tin[v]` is inside u's block, an O(1) test, and a subtree question is a range question over positions. Without updates the children-first pass already gives subtree sums; with point updates, a Fenwick tree (13.2 **Fenwick tree (binary indexed)** (p. 366)) answers each one in O(log n).

Worked example **tour(g, 0), edges [[0,1], [0,2], [1,4], [1,5], [2,3], [2,6]]; order [0, 2, 6, 3, 1, 5, 4]**

```
v   val  tin   tout  size order[tin..tout]  subtreeSum
0   5    0     6     7    [0,2,6,3,1,5,4]   28
1   1    4     6     3    [1,5,4]           10
2   4    1     3     3    [2,6,3]           13
3   2    3     3     1    [3]               2
4   3    6     6     1    [4]               3
5   6    5     5     1    [5]               6
6   7    2     2     1    [6]               7
isAncestor(2, 6) = true, isAncestor(1, 6) = false
add(6, 10): subtreeSum(2) = 23, subtreeSum(1) = 10, subtreeSum(0) = 38
```

tin, tout and the ancestor test memorise

```
// Euler tour: tin[v] = v's position in order[] from walk() (8.9), a DFS
// preorder. v's subtree is the block tin[v] .. tout[v] = tin[v]+size[v]-1.
static int[] tin, tout;

static void tour(List<Integer>[] g, int root) {
    walk(g, root);
    int n = g.length;
    tin = new int[n]; tout = new int[n];
    for (int i = 0; i < n; i++) tin[order[i]] = i;
    for (int v = 0; v < n; v++) tout[v] = tin[v] + size[v] - 1;
}

// u is an ancestor of v (or v itself) iff v's position is in u's block.
static boolean isAncestor(int u, int v) {
    return tin[u] <= tin[v] && tin[v] <= tout[u];
}
```

Minimum score after removals

example

```
// Minimum score after removals (n <= 1000). Cutting edge (v, parent[v])
// splits off v's subtree, so a pair of cuts is a pair of non-root nodes.
// If one is an ancestor of the other, the inner subtree leaves the outer.
static int minimumScore(int[] nums, int[][] edges) {
    int n = nums.length;
    tour(adjacency(n, edges), 0);
    int[] x = new int[n];                       // XOR of each subtree
    for (int i = n - 1; i >= 0; i--) {
        int v = order[i];
        x[v] ^= nums[v];
        if (parent[v] >= 0) x[parent[v]] ^= x[v];
    }
    int best = Integer.MAX_VALUE;
    for (int a = 1; a < n; a++)
        for (int b = a + 1; b < n; b++) {
            int u = a, v = b, p, q, r;          // p, q, r: XOR of each part
            if (isAncestor(v, u)) { u = b; v = a; }
            if (isAncestor(u, v)) {             // v's subtree inside u's
                p = x[v]; q = x[u] ^ x[v]; r = x[0] ^ x[u];
            } else {                            // two separate subtrees
                p = x[u]; q = x[v]; r = x[0] ^ x[u] ^ x[v];
            }
            int hi = Math.max(p, Math.max(q, r));
            int lo = Math.min(p, Math.min(q, r));
            best = Math.min(best, hi - lo);
        }
    return best;
}
```

Subtree sums with point updates

variant

```
// A Fenwick tree over tour positions: node v's value sits at tin[v].
static long[] fw;                               // fw = new long[n + 1]

static void add(int v, long delta) {           // value of node v += delta
    for (int i = tin[v] + 1; i < fw.length; i += i & -i) fw[i] += delta;
}

static long prefix(int i) {                     // sum of positions [0, i)
    long s = 0;
    for (; i > 0; i -= i & -i) s += fw[i];
    return s;
}

static long subtreeSum(int v) { return prefix(tout[v] + 1) - prefix(tin[v]); }
```

Cost Time `O(n)` Space `O(n)`

The tour is one O(n) walk and each ancestor test is O(1); a Fenwick update or subtree sum is O(log n); the removals example tries all $O(n^2)$ pairs.

Variants same idea, small change

- Range update, point query (LC 3515): changing edge (p, v) from w to w' adds w' – w to the root distance of every node in v's subtree. Add at `tin[v]`, subtract at `tout[v] + 1` in a Fenwick over differences; a node's distance is the prefix sum up to `tin[x]`.
- Postorder numbering works too: v's subtree is the block of length `size[v]` that ends at v. LC 3327 builds its DFS string in postorder once and checks each subtree's block for a palindrome (**11.4 Rolling hash (Rabin–Karp)** (p. 329) or **11.6 Manacher's algorithm** (p. 335)).
- Paths are not ranges in this numbering. For path questions use **8.10 Binary lifting (k-th ancestor, LCA)** (p. 210).
- The recursive tour sets `tin` on entry and `tout` on exit with one counter. Each subtree is again one block, but it recurses n deep on a path.

Traps bugs people actually write

- `tout = tin + size` is one too far: node 2's block becomes positions 1 to 4 and takes in node 1 at position 4, so `isAncestor(2, 1)` says true.
- A BFS order is not a tour: in [0, 1, 2, 4, 5, 3, 6] the subtree {1, 4, 5} sits at positions 1, 3 and 4, not in one block.
- A Fenwick tree at position 0 loops forever, since `i += i & -i` never moves from 0. Shift by one, as `add` does with `tin[v] + 1`.
- Removals without the ancestor test treat nested cuts as two separate subtrees: LC 2322 sample 1 gives 1 instead of 9.

Practise on LeetCode, in this order

- [ ] `LC 2322` Minimum Score After Removals on a Tree — Hard
- [ ] `LC 3515` Shortest Path in a Weighted Tree — Hard
- [ ] `LC 3327` Check if DFS Strings Are Palindromes — Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

Part 9

Graphs

A graph is a set of nodes and edges. Store it as adjacency lists, where `g[u]` lists the neighbours of u, so one full traversal costs O(V + E) for V nodes and E edges (n and m in some windows). The first half of this part works on unweighted graphs: DFS for components, BFS for fewest steps, two-colouring, cycle checks, topological order and Union-Find; the second half puts weights on the edges for shortest paths and minimum spanning trees, and ends with bridges, strongly connected components and Euler paths.

9.1	**Graph DFS and connected components** Which nodes or cells can reach each other: count the groups, measure them, or copy the graph.	O(V + E)	p. 223
9.2	**BFS shortest path (unweighted)** Fewest moves or edges from a start to a target when every step costs the same.	O(V + E)	p. 226
9.3	**Multi-source BFS** Distance from every cell to the nearest of many sources, or the time for something to spread from all of them.	O(mn)	p. 229
9.4	**BFS over states** Fewest moves when the position alone is not enough: the node also carries walls broken, nodes visited, or a whole board.	O(S·b)	p. 232
9.5	**Bipartite check (two colours)** Split the nodes into two groups so that every edge goes across; an odd cycle makes it impossible.	O(V + E)	p. 235
9.6	**Directed cycles with three colours** Does a directed graph have a cycle, and which nodes can reach one? Prerequisites, deadlocks, safe states.	O(V + E)	p. 238
9.7	**Topological sort (Kahn's algorithm)** Order tasks so that every dependency comes first, or prove that no such order exists.	O(V + E)	p. 241
9.8	**DP on a DAG in topological order** Longest path, number of paths or earliest finish time in a directed graph without cycles.	O(V + E)	p. 244
9.9	**Union-Find (disjoint set union)** Merge groups as connections arrive, and ask whether two items share a group or how many groups are left.	O(α(n)) per op	p. 247
9.10	**Dijkstra's shortest paths** Cheapest path from one source when every edge weight is ≥ 0.	O(m log n)	p. 250
9.11	**0-1 BFS (deque)** Shortest path where every move costs 0 or 1.	O(V + E)	p. 254
9.12	**Bellman–Ford and at most k edges** Negative edge weights, a limit on the number of edges, or a negative cycle to find.	O(n · m)	p. 257
9.13	**Floyd–Warshall (all pairs)**	O(n³)	p. 260

Shortest distance between every pair of nodes when n is a few hundred at most.

9.14 **Kruskal's MST (sort + DSU)** O(m log m) p. 263
Cheapest set of edges that connects every node, from an edge list.

9.15 **Prim's MST (dense or heap)** O(n²) p. 265
MST of a dense or complete graph (every pair of points), or a tree grown from one node.

9.16 **Bridges and articulation points** O(n + m) p. 268
An edge or a vertex whose removal disconnects an undirected graph.

9.17 **Strongly connected components** O(n + m) p. 271
Groups of nodes in a directed graph that can all reach each other.

9.18 **Euler path (Hierholzer)** O(n + m) p. 274
One walk that uses every edge exactly once.

9.1 Graph DFS and connected components

O(V + E)

Spot it when the problem looks like this

- *Number of connected components*, *provinces*, *islands*, *groups*: count the pieces of a graph or a grid.
- The input is an edge list `edges[i] = [u, v]`, an n × n matrix `isConnected`, or a grid of `'1'` and `'0'` cells joined up, down, left and right.
- Only reachability matters, not the length of a path (fewest steps is BFS, **9.2 BFS shortest path (unweighted)** (p. 226)).
- n and the edge count reach 10^5 (or a `300` × `300` grid), so aim for O(V + E).

How it works

Build the lists once: for each edge `[u, v]`, add v to `g[u]` and u to `g[v]`. `dfs(u)` marks u, then recurses into every unmarked neighbour. When it returns, it has marked exactly the nodes reachable from u: on a path from u, the first unmarked node would come right after a marked node whose whole list was scanned.

To count components, start a DFS from every node that is still unmarked. Each start finds one new component, so the number of starts is the answer. A node is marked once and never unmarked, and each list is scanned once, so the whole loop costs O(V + E) however many DFS calls it makes.

A grid is a graph whose nodes are cells and whose edges go to the four neighbours in `DIRS`. Overwriting a land cell with water is the mark, so no `seen` array is needed, and the DFS that sinks an island can also count its cells.

Worked example countComponents(6, [[0,1], [1,2], [3,4], [0,2]])

```
g = [[1, 2], [0, 2], [1, 0], [4], [3], []]
v  seen[v]  count  dfs(v) enters
0  false    1      0 1 2
1  true     1      -
2  true     1      -
3  false    2      3 4
4  true     2      -
5  false    3      5        (no edges: a component of its own)
```

Adjacency lists and connected components

memorise

```
// Adjacency lists for an undirected graph on nodes 0..n-1.
static List<Integer>[] build(int n, int[][] edges) {
    List<Integer>[] g = new List[n];
    for (int v = 0; v < n; v++) g[v] = new ArrayList<>();
    for (int[] e : edges) {
        g[e[0]].add(e[1]);
        g[e[1]].add(e[0]);                // leave out for a directed graph
    }
    return g;
}

// Connected components: one DFS from every node not seen yet.
static int countComponents(int n, int[][] edges) {
    List<Integer>[] g = build(n, edges);
    boolean[] seen = new boolean[n];
    int count = 0;
    for (int v = 0; v < n; v++)
        if (!seen[v]) { count++; dfs(g, v, seen); }
    return count;
}

static void dfs(List<Integer>[] g, int u, boolean[] seen) {
    seen[u] = true;                       // mark on entry, never unmark
    for (int v : g[u])
        if (!seen[v]) dfs(g, v, seen);
}
```

Iterative DFS with an explicit stack

variant

```
// The same reach with an explicit stack: no recursion depth limit.
static void dfsIter(List<Integer>[] g, int src, boolean[] seen) {
    Deque<Integer> stack = new ArrayDeque<>();
    seen[src] = true;
    stack.push(src);
    while (!stack.isEmpty()) {
        int u = stack.pop();
        for (int v : g[u])
            if (!seen[v]) { seen[v] = true; stack.push(v); }  // mark on push
    }
}
```

Marking on push puts each node on the stack once. It reaches the same set as `dfs`, in a different order.

Number of islands; max area of island

memorise

```java
static final int[][] DIRS = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};

// Number of islands: each DFS sinks one whole island, '1' -> '0'.
static int numIslands(char[][] grid) {
    int count = 0;
    for (int r = 0; r < grid.length; r++)
        for (int c = 0; c < grid[0].length; c++)
            if (grid[r][c] == '1') { count++; sink(grid, r, c); }
    return count;
}

static void sink(char[][] g, int r, int c) {
    if (r < 0 || c < 0 || r >= g.length || c >= g[0].length || g[r][c] != '1')
        return;                          // off the grid, water or done
    g[r][c] = '0';                       // mark by overwriting
    for (int[] d : DIRS) sink(g, r + d[0], c + d[1]);
}

// Max area of island: the same DFS returns how many cells it sank.
// Call it from every 1 cell and keep the maximum.
static int area(int[][] g, int r, int c) {
    if (r < 0 || c < 0 || r >= g.length || c >= g[0].length || g[r][c] != 1)
        return 0;
    g[r][c] = 0;
    int size = 1;
    for (int[] d : DIRS) size += area(g, r + d[0], c + d[1]);
    return size;
}
```

Cost Time `O(V + E)` Space `O(V)`

Each node is marked once and each list is scanned once (an undirected edge from both ends). A grid costs O(mn). The recursion can be V deep.

Variants same idea, small change

- Number of provinces gives an n × n matrix instead of edges: in the DFS, loop `v` over `0..n-1` and follow `isConnected[u][v] == 1`. That is $O(n^2)$, the size of the input.
- Clone graph: DFS with a `HashMap<Node, Node>` from each old node to its copy. Put the copy in the map *before* visiting the neighbours, so a cycle finds the copy instead of recursing forever.
- Flood from the border (surrounded regions, number of enclaves): start a DFS from every border cell; whatever it cannot reach is enclosed. Pacific Atlantic floods from each ocean and keeps the cells both reach.
- Edges arrive over time and you only need *same component?* or the count: Union-Find (9.9 **Union-Find (disjoint set union)** (p. 247)) needs no lists.

Traps bugs people actually write

- An undirected edge needs `g[u].add(v)` **and** `g[v].add(u)`. With only the first, n = 2 and edges `[[1, 0]]` gives 2 components: the DFS from 0 cannot see 1.
- Mark the cell before recursing. A `sink` that tests for `'1'` but never writes `'0'` bounces between two land cells until StackOverflowError.
- The recursion is as deep as the DFS path. On Java's default 1 MB stack, `dfs` on a path of 10^4 nodes throws StackOverflowError (measured), and a snake-shaped island in a 300 × 300 grid has about $4.5 \cdot 10^4$ cells. Use `dfsIter` when the depth can pass a few thousand.

Practise on LeetCode, in this order

- [ ] `LC 1971` Find if Path Exists in Graph — Easy
- [ ] `LC 547` Number of Provinces — Medium
- [ ] `LC 200` Number of Islands — Medium
- [ ] `LC 695` Max Area of Island — Medium
- [ ] `LC 133` Clone Graph — Medium
- [ ] `LC 130` Surrounded Regions — Medium

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

9.2 BFS shortest path (unweighted) `O(V + E)`

Spot it when the problem looks like this

- *Minimum number of steps*, *shortest path*, *fewest moves*, *nearest exit*, where every move costs 1.
- A graph or grid of up to about 10^5 nodes; the answer is a distance, and sometimes the path itself.
- If moves cost different amounts, this is the wrong tool: see **9.10 Dijkstra's shortest paths** (p. 250), or **9.11 0-1 BFS (deque)** (p. 254) when costs are 0 and 1.

How it works

BFS takes nodes from a FIFO queue, so it finishes every node at distance d before it pops any node at distance d + 1. The queue only ever holds part of level d followed by part of level d + 1. So the first time BFS reaches a node, it reaches it by a shortest path: a shorter one would have reached it from an earlier level.

That is why `dist[v]` is written when v is **pushed**, not when it is popped. `dist[v] != -1` then doubles as the visited test, every node enters the queue once, and the cost is O(V + E). Record `parent[v] = u` at the same moment. Walking `parent` back from the target gives one shortest path in reverse.

In a grid, the nodes are cells and the edges are the 4 or 8 moves. Either store a `dist` grid, or process the queue one level at a time (`size = q.size()`) and count the levels. Stop as soon as the target is reached.

Worked example **bfs(g, 0) with edges 0-1, 0-2, 1-2, 1-3, 2-4, 3-5, 4-5**

```
pop u  dist[u]  push v (dist, parent)  already set  queue after
0      0        1 (1, 0), 2 (1, 0)                 [1, 2]
1      1        3 (2, 1)               0 2         [2, 3]
2      1        4 (2, 2)               0 1         [3, 4]
3      2        5 (3, 3)               1           [4, 5]
4      2                               2 5         [5]
5      3                               3 4         []
dist = [0, 1, 1, 2, 2, 3], parent = [-1, 0, 0, 1, 2, 3]
path(5) walks 5, 3, 1, 0, adding at the front: [0, 1, 3, 5]
Popping 4 did not push 5 again: dist[5] was already set by 3.
```

Distances, parents and the path

memorise

```java
// dist[v] = fewest edges from src (-1: unreachable); parent[v] = node before v.
static int[] bfs(List<Integer>[] g, int src, int[] parent) {
    int[] dist = new int[g.length];
    Arrays.fill(dist, -1);
    Arrays.fill(parent, -1);
    Deque<Integer> q = new ArrayDeque<>();
    dist[src] = 0;
    q.add(src);
    while (!q.isEmpty()) {
        int u = q.poll();
        for (int v : g[u])
            if (dist[v] == -1) {              // set on push: pushed once
                dist[v] = dist[u] + 1;
                parent[v] = u;
                q.add(v);
            }
    }
    return dist;
}

// Path src -> dst: walk parent[] back from dst, adding at the front.
static List<Integer> path(int[] parent, int[] dist, int dst) {
    LinkedList<Integer> p = new LinkedList<>();
    if (dist[dst] == -1) return p;            // unreachable: empty path
    for (int v = dst; v != -1; v = parent[v]) p.addFirst(v);
    return p;
}
```

Shortest path in binary matrix (8 directions)

variant

```
// Shortest clear path (0,0) -> (n-1,n-1) in 8 directions, counted in cells.
static int shortestPathBinaryMatrix(int[][] grid) {
    int n = grid.length;
    if (grid[0][0] == 1 || grid[n - 1][n - 1] == 1) return -1;
    int[][] dist = new int[n][n];              // 0 = not reached
    Deque<int[]> q = new ArrayDeque<>();
    dist[0][0] = 1;
    q.add(new int[] {0, 0});
    while (!q.isEmpty()) {
        int[] cur = q.poll();
        for (int dr = -1; dr <= 1; dr++)
            for (int dc = -1; dc <= 1; dc++) {    // dr = dc = 0: reached
                int r = cur[0] + dr, c = cur[1] + dc;
                if (r < 0 || c < 0 || r >= n || c >= n) continue;
                if (grid[r][c] == 1 || dist[r][c] != 0) continue;
                dist[r][c] = dist[cur[0]][cur[1]] + 1;
                q.add(new int[] {r, c});
            }
    }
    return dist[n - 1][n - 1] == 0 ? -1 : dist[n - 1][n - 1];
}
```

Nearest exit, one level per step

variant

```
static final int[][] DIRS = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
// Nearest exit from the entrance: BFS level by level, one level per step.
static int nearestExit(char[][] maze, int[] entrance) {
    int m = maze.length, n = maze[0].length;
    Deque<int[]> q = new ArrayDeque<>();
    q.add(entrance);
    maze[entrance[0]][entrance[1]] = '+';     // mark as a wall when pushed
    for (int steps = 1; !q.isEmpty(); steps++) {
        for (int size = q.size(); size > 0; size--) {
            int[] cur = q.poll();
            for (int[] d : DIRS) {
                int r = cur[0] + d[0], c = cur[1] + d[1];
                if (r < 0 || c < 0 || r >= m || c >= n) continue;
                if (maze[r][c] == '+') continue;
                if (r == 0 || c == 0 || r == m - 1 || c == n - 1) return steps;
                maze[r][c] = '+';
                q.add(new int[] {r, c});
            }
        }
    }
    return -1;
}
```

Cost Time `O(V + E)` Space `O(V)`

Every node is pushed and popped at most once, and each adjacency list is scanned once. A grid costs O(mn).

Variants same idea, small change

- Many starts at once (every rotten orange, every 0): push them all at distance 0 (9.3 **Multi-source BFS** (p. 229)).
- The position alone does not decide the future (keys held, walls broken, a lock code): BFS over states (9.4 **BFS over states** (p. 232)).
- Snakes and ladders: squares are nodes and a die roll gives up to 6 edges. Follow the snake or ladder first, then test `dist` of the square you land on.
- Jump game IV: index i has edges to i ± 1 and to every index with the same value. Clear a value's list after using it once, or a long run of equal values costs $O(n^2)$.

Traps bugs people actually write

- Marking a node visited when it is popped, while writing `dist` when it is pushed, lets it be pushed twice, and the second push overwrites its distance. On the triangle 0-1, 0-2, 1-2, `dist[2]` from 0 comes out 2 instead of 1.
- A DFS (a stack) finds *a* path, not the shortest: on the same triangle it can reach 2 through 1, at depth 2.
- Binary matrix counts cells, not edges, so `dist[0][0] = 1`. Test `grid[0][0]` before you start: without that test `[[1, 0], [0, 0]]` returns 2 instead of –1.
- Nearest exit: test for an exit when you push a neighbour, not when you pop. Testing on pop counts the entrance itself, so LC 1926 sample 3 (`[".+"]`, entrance `[0, 0]`) returns 0 instead of –1.

Practise on LeetCode, in this order

☐	LC 1306	Jump Game III	Medium
☐	LC 1926	Nearest Exit from Entrance in Maze	Medium
☐	LC 1091	Shortest Path in Binary Matrix	Medium
☐	LC 909	Snakes and Ladders	Medium
☐	LC 1345	Jump Game IV	Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

9.3 Multi-source BFS

O(mn)

Spot it when the problem looks like this

- *Distance to the nearest 0*, *nearest land*, *closest gate*, asked for every cell at once.
- Something spreads from several places at the same time: rotting oranges, fire, rising water; *minimum minutes until every cell…*
- m·n is up to 10^4 or 10^5, so one BFS per source, or per cell, is too slow.

How it works

Push every source into the queue at distance 0, then run one ordinary BFS (9.2 BFS shortest path (unweighted) (p. 226)). Think of a new node S with an edge to each source. A BFS from S reaches all the sources after one step and then does exactly what this BFS does, so `dist[cell]` here is the BFS distance from S minus 1: the distance to the *nearest* source. Each cell is still pushed once, so the cost is O(mn) however many sources there are.

When something spreads over time, one BFS level is one minute. Rotting oranges counts the fresh oranges first and stops as soon as the count is 0. Oranges still fresh at the end are cut off, and the answer is –1. The `fresh > 0` test in the loop matters: without it, the last level, which rots nothing, would count as one more minute.

01 matrix and *as far from land as possible* are both one call to `nearest`, with the zeros or the land cells as the sources.

Worked example **orangesRotting, rotten oranges at two opposite corners**

```
minute 0     minute 1     minute 2     minute 3
2 1 1 1 1    2 2 1 1 1    2 2 2 1 2    2 2 2 2 2
1 1 0 1 1    2 1 0 1 2    2 2 0 2 2    2 2 0 2 2
1 1 1 1 2    1 1 1 2 2    2 1 2 2 2    2 2 2 2 2
fresh 12     fresh 8      fresh 2      fresh 0
queue 2      queue 4      queue 6      queue 2
answer 3. With only one of the two corners rotten it takes 6 minutes.
```

Rotting oranges (one level = one minute) memorise

```
static final int[][] DIRS = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};

// Rotting oranges: every rotten orange is in the queue at minute 0.
static int orangesRotting(int[][] grid) {
    int m = grid.length, n = grid[0].length, fresh = 0, minutes = 0;
    Deque<int[]> q = new ArrayDeque<>();
    for (int r = 0; r < m; r++)
        for (int c = 0; c < n; c++) {
            if (grid[r][c] == 2) q.add(new int[] {r, c});
            if (grid[r][c] == 1) fresh++;
        }
    while (fresh > 0 && !q.isEmpty()) {
        minutes++;                              // one level = one minute
        for (int size = q.size(); size > 0; size--) {
            int[] cur = q.poll();
            for (int[] d : DIRS) {
                int r = cur[0] + d[0], c = cur[1] + d[1];
                if (r < 0 || c < 0 || r >= m || c >= n) continue;
                if (grid[r][c] != 1) continue;
                grid[r][c] = 2;                 // rots now: marked when pushed
                fresh--;
                q.add(new int[] {r, c});
            }
        }
    }
    return fresh == 0 ? minutes : -1;
}
```

Distance to the nearest source; 01 matrix; far from land

memorise

```java
// dist[r][c] = steps from (r, c) to the nearest cell whose value is src.
// All src cells start at 0; any other cell gets its dist once, when pushed.
static int[][] nearest(int[][] grid, int src) {
    int m = grid.length, n = grid[0].length;
    int[][] dist = new int[m][n];
    Deque<int[]> q = new ArrayDeque<>();
    for (int r = 0; r < m; r++)
        for (int c = 0; c < n; c++) {
            dist[r][c] = grid[r][c] == src ? 0 : -1;
            if (dist[r][c] == 0) q.add(new int[] {r, c});
        }
    while (!q.isEmpty()) {
        int[] cur = q.poll();
        for (int[] d : DIRS) {
            int r = cur[0] + d[0], c = cur[1] + d[1];
            if (r < 0 || c < 0 || r >= m || c >= n || dist[r][c] != -1) continue;
            dist[r][c] = dist[cur[0]][cur[1]] + 1;
            q.add(new int[] {r, c});
        }
    }
    return dist;
}

// 01 matrix: distance to the nearest 0.
static int[][] updateMatrix(int[][] mat) { return nearest(mat, 0); }

// As far from land as possible: the largest distance to the nearest 1.
static int maxDistance(int[][] grid) {
    int best = -1;
    for (int[] row : nearest(grid, 1))
        for (int d : row) best = Math.max(best, d);
    return best <= 0 ? -1 : best;            // 0: no water, -1: no land
}
```

Cost Time `O(mn)` Space `O(mn)`

Each cell is pushed at most once, whatever the number of sources. One BFS per source would cost O(k·mn).

Variants same idea, small change

- Map of highest peak: the water cells are the sources, and each land cell's height is its distance to water.
- Shortest bridge: mark one island with a DFS (9.1 **Graph DFS and connected components** (p. 223)) and push all its cells at distance 0. When the BFS first reaches a cell of the other island at distance d, the bridge needs d – 1 cells.
- Walls stop the spread: skip them in the neighbour test. Cells they cut off keep –1.
- Find the safest path in a grid: a multi-source BFS from every thief gives each cell's safety; then find the path whose smallest safety is largest, by 3.3 **Binary search on the answer** (p. 74) or a max-heap in the style of 9.10 **Dijkstra's shortest paths** (p. 250).

Traps bugs people actually write

- `while (!q.isEmpty())` with `minutes++` per level also counts the last level, which rots nothing: LC 994 sample 1 returns 5 instead of 4, and `[[0, 2]]` returns 1 instead of 0.
- Rotting an orange when it is popped instead of when it is pushed lets two neighbours push it, so `fresh--` runs twice for it and sample 1 returns –1 instead of 4.
- A separate BFS from every cell to its nearest source is $O((mn)^2)$: 10^8 steps on a 100 × 100 grid.
- As far from land returns –1 both when there is no land and when there is no water. `best <= 0` covers both cases.

Practise on LeetCode, in this order

☐	LC 994	Rotting Oranges	Medium
☐	LC 542	01 Matrix	Medium
☐	LC 1765	Map of Highest Peak	Medium
☐	LC 1162	As Far from Land as Possible	Medium
☐	LC 934	Shortest Bridge	Medium

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

9.4 BFS over states

`O(S·b)`

Spot it when the problem looks like this

- *Minimum number of moves* where each move changes a whole configuration: a lock, a word, a puzzle board.
- A walk with a resource or a memory: *you may eliminate at most k obstacles*, keys and doors, *visit every node*.
- The states can be counted: 10^4 lock codes, 6! = 720 boards, 40·40·(k + 1) cells with breaks left, $n \cdot 2^n$ with $n \le 12$.

How it works

Plain BFS (9.2 BFS shortest path (unweighted) (p. 226)) is correct on any graph whose edges all cost 1, and nothing says a node must be a cell. Make the node everything that decides the moves still possible: the four wheels of the lock, the cell plus the eliminations left, the current node plus the set already visited. Two paths that reach the same state have the same futures, so the first arrival is the shortest and the state is marked for good.

Choose the state with care. Leave out a part, and different situations merge: with only the cell in `seen`, an early arrival with no eliminations left blocks a later arrival that still has some. The cost is O(S·b) for S states with b moves each, so count S first: 10^4 for the lock, 1600·(k + 1) for a 40 × 40 grid, $12 \cdot 2^{12} = 49152$ for 12 nodes.

Generate neighbours on the fly. Keep `seen` in a `HashSet<String>`, an array indexed by the parts of the state, or an `int` bitmask.

Worked example **openLock, deadends [0201, 0101, 0102, 1212, 2002], target 0202**

```
moves  queue size  front of the queue
0          1       0000
1          8       1000 9000 0100 0900 0010 0090 ...
2         31       2000 1100 1900 1010 1090 1001 ...
3         86       3000 2100 2900 2010 2090 2001 ...
4        190       4000 3100 3900 3010 3090 3001 ...
5        356       5000 4100 4900 4010 4090 4001 ...
6        576       ... 0202 is in this level: answer 6
one shortest path: 0000 1000 1100 1200 1201 1202 0202
seen holds 1511 of the 10000 codes (dead ends included) when 0202 is popped
```

Open the lock

memorise

```java
// Open the lock: a state is the 4-digit string; a move turns one wheel by 1.
static int openLock(String[] deadends, String target) {
    Set<String> seen = new HashSet<>(Arrays.asList(deadends));
    if (!seen.add("0000")) return -1;            // the start is a dead end
    Deque<String> q = new ArrayDeque<>();
    q.add("0000");
    for (int moves = 0; !q.isEmpty(); moves++) {
        for (int size = q.size(); size > 0; size--) {
            String s = q.poll();
            if (s.equals(target)) return moves;
            char[] w = s.toCharArray();
            for (int i = 0; i < 4; i++) {
                char keep = w[i];
                for (int turn = 1; turn <= 9; turn += 8) {  // +1, and +9 = -1
                    w[i] = (char) ('0' + (keep - '0' + turn) % 10);
                    String next = new String(w);
                    if (seen.add(next)) q.add(next);   // dead ends start seen
                }
                w[i] = keep;
            }
        }
    }
    return -1;
}
```

Grid with obstacles elimination

variant

```
static final int[][] DIRS = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};

// Grid with obstacles elimination: state = (row, col, eliminations left).
static int shortestPath(int[][] grid, int k) {
    int m = grid.length, n = grid[0].length;
    boolean[][][] seen = new boolean[m][n][k + 1];
    Deque<int[]> q = new ArrayDeque<>();
    q.add(new int[] {0, 0, k, 0});                    // row, col, left, steps
    seen[0][0][k] = true;
    while (!q.isEmpty()) {
        int[] s = q.poll();
        if (s[0] == m - 1 && s[1] == n - 1) return s[3];
        for (int[] d : DIRS) {
            int r = s[0] + d[0], c = s[1] + d[1];
            if (r < 0 || c < 0 || r >= m || c >= n) continue;
            int left = s[2] - grid[r][c];             // a wall costs one
            if (left < 0 || seen[r][c][left]) continue;
            seen[r][c][left] = true;
            q.add(new int[] {r, c, left, s[3] + 1});
        }
    }
    return -1;
}
```

The step count travels inside the state here, instead of a level loop.

Shortest path visiting all nodes

variant

```
// Shortest path visiting all nodes: state = (node, bitmask of visited).
static int shortestPathLength(int[][] graph) {
    int n = graph.length, full = (1 << n) - 1;
    boolean[][] seen = new boolean[n][1 << n];
    Deque<int[]> q = new ArrayDeque<>();
    for (int v = 0; v < n; v++) {                     // any start: all at step 0
        q.add(new int[] {v, 1 << v});
        seen[v][1 << v] = true;
    }
    for (int steps = 0; !q.isEmpty(); steps++) {
        for (int size = q.size(); size > 0; size--) {
            int[] s = q.poll();
            if (s[1] == full) return steps;
            for (int v : graph[s[0]]) {
                int mask = s[1] | 1 << v;
                if (seen[v][mask]) continue;
                seen[v][mask] = true;
                q.add(new int[] {v, mask});
            }
        }
    }
    return -1;
}
```

Cost Time `O(S·b)` Space `O(S)`

Each state is pushed once: $10^4 \times 8$ moves for the lock, mn(k + 1) × 4 for the grid.

Variants same idea, small change

- Word ladder: a state is a word. Its neighbours change one position to each of `a..z` and must be in the word set: 26·L tries per word instead of comparing every pair of words.
- Sliding puzzle: the state is the board written as a string of 6 digits, and each move swaps the `0` with one of its neighbours.
- Shortest path with alternating colours: the state is (node, colour of the last edge), so `seen` is a `boolean[n][2]`.
- Obstacles elimination: when `k >= m + n - 3`, you can break every cell of an L-shaped path between the corners, so return `m + n - 2` without a BFS.

Traps bugs people actually write

- Turning a wheel down with `(c - '0' - 1) % 10` gives –1 for 0, because Java's `%` keeps the sign, and `'0'` becomes `'/'`. Add 9 instead, as the template does.
- In the obstacle grid, marking only the cell is wrong: on `[[0,0],[1,1],[1,1],[1,0]]` with k = 2, a 2D `seen` returns –1; the answer is 4.
- Visiting all nodes may start anywhere, so every `(v, 1 << v)` starts at step 0. Starting only from node 0 gives 5 on LC 847 sample 1 (`[[1,2,3],[0],[0],[0]]`); the answer is 4.

Practise on LeetCode, in this order

☐	LC 1129	Shortest Path with Alternating Colors	Medium
☐	LC 752	Open the Lock	Medium
☐	LC 127	Word Ladder	Hard
☐	LC 773	Sliding Puzzle	Hard
☐	LC 1293	Shortest Path in a Grid with Obstacles Elimination	Hard
☐	LC 847	Shortest Path Visiting All Nodes	Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

9.5 Bipartite check (two colours) `O(V + E)`

Spot it when the problem looks like this

- *Is the graph bipartite? Split into two groups* so that no disliking pair is together.
- An edge means *these two must differ*: two teams, two colours, even and odd positions.
- *The graph may not be connected*: every component needs its own start.

How it works

Colour a start node 1. Its neighbours must be −1, their neighbours 1, and so on, so every colour in the component is forced: it is the parity of the node's BFS distance from the start. If an edge joins two nodes of the same colour, no split exists. The two BFS tree paths from those nodes up to their common ancestor have lengths of the same parity, so with the edge they close a cycle of odd length, and an odd cycle cannot alternate two colours. If no edge clashes, the colouring itself is the split.

The first colour is a free choice, but only inside its own component. So loop over all nodes and start a new BFS from each one that is still uncoloured. DFS colouring works the same way. Both look at each node once and each edge from both ends: O(V + E).

Worked example **isBipartite([[1,2,3], [0,2], [0,1,3], [0,2]]), LC 785 sample 1**

```
start 0: color[0] = 1
pop u  color[u]  v  color[v]  action
0        1       1    0       color[1] = -1, push
0        1       2    0       color[2] = -1, push
0        1       3    0       color[3] = -1, push
1       -1       0    1       differs: ok
1       -1       2   -1       same colour: odd cycle 0-1-2, return false
```

BFS colouring over every component

memorise

```java
// graph[u] lists u's neighbours. color: 0 = none yet, 1 or -1 = a side.
static boolean isBipartite(int[][] graph) {
    int n = graph.length;
    int[] color = new int[n];
    Deque<Integer> q = new ArrayDeque<>();
    for (int s = 0; s < n; s++) {
        if (color[s] != 0) continue;          // its component is done
        color[s] = 1;                         // new component: any side
        q.add(s);
        while (!q.isEmpty()) {
            int u = q.poll();
            for (int v : graph[u]) {
                if (color[v] == 0) { color[v] = -color[u]; q.add(v); }
                else if (color[v] == color[u]) return false;   // odd cycle
            }
        }
    }
    return true;
}
```

Possible bipartition (DFS colouring)

variant

```java
// Possible bipartition: people 1..n, and each disliking pair must split.
static boolean possibleBipartition(int n, int[][] dislikes) {
    List<Integer>[] g = new List[n + 1];
    for (int v = 0; v <= n; v++) g[v] = new ArrayList<>();
    for (int[] d : dislikes) { g[d[0]].add(d[1]); g[d[1]].add(d[0]); }
    int[] color = new int[n + 1];
    for (int v = 1; v <= n; v++)
        if (color[v] == 0 && !paint(g, v, 1, color)) return false;
    return true;
}

// DFS colouring: give u colour c, its neighbours -c. False on a clash.
static boolean paint(List<Integer>[] g, int u, int c, int[] color) {
    color[u] = c;
    for (int v : g[u]) {
        if (color[v] == c) return false;
        if (color[v] == 0 && !paint(g, v, -c, color)) return false;
    }
    return true;
}
```

Cost Time `O(V + E)` Space `O(V)`

Each node is coloured once and each edge is checked from both ends.

Variants same idea, small change

- Need the two groups? After a successful check they are the nodes with colour 1 and the nodes with colour –1.
- Maximize target nodes after connecting trees II: in a tree, the nodes at even distance from i are exactly those with i's colour, so count each colour class once instead of running a BFS per node.
- Divide nodes into the maximum number of groups: if the graph is not bipartite the answer is –1; otherwise, for each component, take the most BFS levels from any of its nodes, and add these up.

Traps bugs people actually write

- Starting the BFS only from node 0 misses the other components: `[[], [2,3], [1,3], [1,2]]` (node 0 alone, then a triangle) returns true; the answer is false.
- Use three values (0 = none, 1, –1). With a `boolean[]` colour, an uncoloured node looks the same as a node coloured false.
- Possible bipartition numbers people 1..n. Arrays of size n throw `ArrayIndexOutOfBoundsException` on person n; allocate n + 1 and loop from 1.
- `paint` recurses as deep as the component. That is fine for n ≤ 2000 in LC 886; for 10^5 nodes use the BFS version.

Practise on LeetCode, in this order

- [ ] LC 785 Is Graph Bipartite? Medium
- [ ] LC 886 Possible Bipartition Medium
- [ ] LC 3373 Maximize the Number of Target Nodes After Connecting Trees II Hard
- [ ] LC 2493 Divide Nodes Into the Maximum Number of Groups Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

9.6 Directed cycles with three colours

O(V + E)

Spot it when the problem looks like this

- Directed edges, a before b: *can you finish all courses?*, *is there a circular dependency?*
- *Safe* or *eventually terminal* nodes: every path from them ends, so none of them reaches a loop.
- n up to 10^4 with a few times that many edges: one O(V + E) pass.

How it works

Run a DFS with three states: 0 = not visited, 1 = on the current DFS path (entered, not finished), 2 = finished. There is a cycle exactly when the DFS follows an edge into a state-1 node. That node is an ancestor of the current one, so the path down from it plus this edge is a cycle. Conversely, take the first node c of a cycle that the DFS enters. The rest of the cycle is still unvisited, so the DFS reaches c's predecessor before it finishes c, and sees the edge into c while c is still 1.

An edge into a state-2 node is not a cycle: that node finished earlier without finding one. That is why two states are not enough. Every node is entered once and finished once: O(V + E).

In an undirected graph each edge shows up from both ends, so the node you came from always looks visited. There the test is *visited and not the parent*.

Worked example **hasCycle(g, 0), edges 0→1, 0→2, 0→3, 1→2, 3→4, 4→3**

```
step              state[0..4]
enter 0           1 0 0 0 0
enter 1           1 1 0 0 0
enter 2           1 1 1 0 0
finish 2          1 1 2 0 0
finish 1          1 2 2 0 0
0 -> 2: state 2   1 2 2 0 0   finished before: skip, not a cycle
enter 3           1 2 2 1 0
enter 4           1 2 2 1 1
4 -> 3: state 1   1 2 2 1 1   back edge: cycle 3 -> 4 -> 3
```

Course schedule with three states

memorise

```java
// Course schedule: {a, b} means take b before a, so the edge is b -> a.
// state: 0 = unvisited, 1 = on the current DFS path, 2 = finished.
static boolean canFinish(int n, int[][] prerequisites) {
    List<Integer>[] g = new List[n];
    for (int v = 0; v < n; v++) g[v] = new ArrayList<>();
    for (int[] p : prerequisites) g[p[1]].add(p[0]);
    int[] state = new int[n];
    for (int v = 0; v < n; v++)
        if (state[v] == 0 && hasCycle(g, v, state)) return false;
    return true;
}

static boolean hasCycle(List<Integer>[] g, int u, int[] state) {
    state[u] = 1;                              // u is on the path
    for (int v : g[u]) {
        if (state[v] == 1) return true;        // back edge: a cycle
        if (state[v] == 0 && hasCycle(g, v, state)) return true;
        // state[v] == 2: explored before, no cycle that way
    }
    state[u] = 2;                              // finished
    return false;
}
```

Eventual safe states

variant

```java
// Eventual safe states: a node is safe when no path from it reaches a cycle.
static List<Integer> eventualSafeNodes(int[][] graph) {
    int[] state = new int[graph.length];
    List<Integer> safe = new ArrayList<>();
    for (int v = 0; v < graph.length; v++)
        if (!reachesCycle(graph, v, state)) safe.add(v);
    return safe;
}

static boolean reachesCycle(int[][] g, int u, int[] state) {
    if (state[u] != 0) return state[u] == 1;  // 1: on a path into a cycle
    state[u] = 1;
    for (int v : g[u])
        if (reachesCycle(g, v, state)) return true;   // u stays 1: unsafe
    state[u] = 2;                              // 2: safe for good
    return false;
}
```

Returning early leaves every node that leads into a cycle at state 1, so a later visit reads it as unsafe.

Contrast: the parent check in an undirected graph

variant

```
// Undirected graph: a seen neighbour other than the parent closes a cycle.
static boolean cycleUndirected(List<Integer>[] g, int u, int parent,
                               boolean[] seen) {
    seen[u] = true;
    for (int v : g[u]) {
        if (!seen[v]) {
            if (cycleUndirected(g, v, u, seen)) return true;
        } else if (v != parent) return true;
    }
    return false;
}
```

Call it with parent `-1` for each start node.

Cost Time `O(V + E)` Space `O(V)`

Each node goes 0 → 1 → 2 once and each edge is followed once. The recursion can be V deep.

Variants same idea, small change

- Need an order, not only yes or no? The reverse of the finish order is a topological order. Or use Kahn's algorithm (**9.7 Topological sort (Kahn's algorithm)** (p. 241)), where fewer than n popped nodes means a cycle.
- Eventual safe states without recursion: reverse every edge and run Kahn's algorithm from the nodes with no outgoing edges; exactly the popped nodes are safe.
- Detect cycles in 2D grid: the undirected parent check, over cells with the same letter.
- At most one outgoing edge per node (longest cycle in a graph): stamp each node with the step at which the current walk reached it; meeting a node stamped in this walk closes a cycle of `step - stamp[v]` nodes.

Traps bugs people actually write

- One `visited` boolean in a directed graph reports cycles that are not there: on 0→1, 0→2, 1→2, node 2 is already visited when the DFS comes back to 0.
- Resetting the state to 0 on exit, instead of 2, keeps the answer right but explores every path again: 20 diamonds in a row (61 nodes) take $4 \cdot 10^6$ calls.
- The recursion is as deep as the longest path. `hasCycle` on a path of 10^4 nodes throws StackOverflowError on Java's default stack (measured); for larger inputs use Kahn's algorithm (**9.7 Topological sort (Kahn's algorithm)** (p. 241)).

Practise on LeetCode, in this order

- [ ] LC 207 Course Schedule — Medium
- [ ] LC 802 Find Eventual Safe States — Medium
- [ ] LC 1559 Detect Cycles in 2D Grid — Medium

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

9.7 Topological sort (Kahn's algorithm)

O(V + E)

Spot it when the problem looks like this

- *Return an order* in which to take the courses, build the projects or apply the steps; *any valid order* is accepted.
- Pairs `[a, b]` meaning *b must come before a*, or a list of what each item depends on.
- *If it is impossible, return an empty array*: a cycle must be detected along the way.

How it works

A node with indegree 0 has no prerequisite left, so it can go next. Kahn's algorithm queues every such node. It pops one, appends it to the order, and removes its outgoing edges by decrementing each target's indegree. A target that reaches 0 has lost its last prerequisite, so it joins the queue. An edge u → v is removed only when u is popped, so v reaches 0, and is placed, after u.

On a cycle no node ever reaches indegree 0, because each one waits for the node before it. Those nodes, and everything that depends on them, are never popped. So `order.size() < n` is the cycle test, with no separate DFS. Each node is pushed once and each edge decrements once: O(V + E).

The queue can hold several nodes at once, and any of them may go next, so there are often many valid orders. A `PriorityQueue` gives the smallest one.

Worked example edges 0→1, 0→2, 1→3, 2→3 (LC 210 sample 2), then with 3→1 added

```
indeg = [0, 1, 1, 2], queue = [0]
pop u  order          indeg updates      queue after
0      [0]            1: 1->0, 2: 1->0   [1, 2]
1      [0, 1]         3: 2->1            [2]
2      [0, 1, 2]      3: 1->0            [3]
3      [0, 1, 2, 3]                      []
---- add 3 -> 1: indeg = [0, 2, 1, 2] ----
0      [0]            1: 2->1, 2: 1->0   [2]
2      [0, 2]         3: 2->1            []
2 of 4 popped: 1 and 3 wait for each other, so there is a cycle
```

Kahn's algorithm; course schedule II

memorise

```java
// Kahn's algorithm, edges u -> v in g[u]. An order shorter than n = a cycle.
static List<Integer> topoOrder(List<Integer>[] g) {
    int n = g.length;
    int[] indeg = new int[n];
    for (List<Integer> out : g) for (int v : out) indeg[v]++;
    Deque<Integer> q = new ArrayDeque<>();
    for (int v = 0; v < n; v++) if (indeg[v] == 0) q.add(v);   // 1. sources
    List<Integer> order = new ArrayList<>();
    while (!q.isEmpty()) {
        int u = q.poll();
        order.add(u);                    // 2. u's prerequisites are all out
        for (int v : g[u])
            if (--indeg[v] == 0) q.add(v);    // 3. v's last one is out
    }
    return order;
}

static List<Integer>[] graph(int n) {
    List<Integer>[] g = new List[n];
    for (int v = 0; v < n; v++) g[v] = new ArrayList<>();
    return g;
}

// Course schedule II: {a, b} means take b before a, so the edge is b -> a.
static int[] findOrder(int n, int[][] prerequisites) {
    List<Integer>[] g = graph(n);
    for (int[] p : prerequisites) g[p[1]].add(p[0]);
    List<Integer> order = topoOrder(g);
    if (order.size() < n) return new int[0];      // a cycle: no order
    return order.stream().mapToInt(Integer::intValue).toArray();
}
```

Build a matrix with conditions

variant

```java
// Build a matrix with conditions: one order for rows, one for columns.
static int[][] buildMatrix(int k, int[][] rowConds, int[][] colConds) {
    int[] row = position(k, rowConds), col = position(k, colConds);
    if (row == null || col == null) return new int[0][0];
    int[][] m = new int[k][k];
    for (int v = 0; v < k; v++) m[row[v]][col[v]] = v + 1;
    return m;
}

// pos[v] = place of value v + 1 in a topological order; null on a cycle.
static int[] position(int k, int[][] conds) {
    List<Integer>[] g = graph(k);
    for (int[] e : conds) g[e[0] - 1].add(e[1] - 1);  // values are 1..k
    List<Integer> order = topoOrder(g);
    if (order.size() < k) return null;
    int[] pos = new int[k];
    for (int i = 0; i < k; i++) pos[order.get(i)] = i;
    return pos;
}
```

Sort items by groups (two levels of order)

example

```java
// Sort items by groups: order the groups, order the items, then output each
// group's items in item order. An item with group -1 gets a group of its own.
static int[] sortItems(int n, int m, int[] group, List<List<Integer>> before) {
    for (int i = 0; i < n; i++) if (group[i] == -1) group[i] = m++;
    List<Integer>[] items = graph(n), groups = graph(m);
    for (int i = 0; i < n; i++)
        for (int b : before.get(i)) {
            items[b].add(i);
            if (group[b] != group[i]) groups[group[b]].add(group[i]);
        }
    List<Integer> itemOrder = topoOrder(items), groupOrder = topoOrder(groups);
    if (itemOrder.size() < n || groupOrder.size() < m) return new int[0];
    List<Integer>[] members = graph(m);
    for (int i : itemOrder) members[group[i]].add(i);
    int[] out = new int[n];
    int k = 0;
    for (int gr : groupOrder) for (int i : members[gr]) out[k++] = i;
    return out;
}
```

Items inside a group keep their order from `itemOrder`, so the dependencies between them hold too.

Cost Time `O(V + E)` Space `O(V + E)`

Every node is pushed and popped once, and every edge lowers one indegree once.

Variants same idea, small change

- Smallest order in lexicographic order: replace the queue with a `PriorityQueue<Integer>`; the cost becomes O(V log V + E).
- All ancestors of a node in a DAG: in topological order, a node's ancestors are its parents plus the union of their ancestor sets. With `boolean[n]` sets that is O(n·E); for n ≤ 100, the closure in 9.13 **Floyd–Warshall (all pairs)** (p. 260) is shorter.
- Find all possible recipes: recipes and ingredients are the nodes, and the supplies start in the queue.
- Minimum height trees: peel the leaves (degree 1) layer by layer, Kahn-style on an undirected tree; the last one or two nodes left are the centres.

Traps bugs people actually write

- For `[a, b]` the edge is `b -> a`. Writing `g[a].add(b)` returns the order backwards: `[1, 0]` for LC 210 sample 1, whose only answer is `[0, 1]`.
- Returning `order` without checking its size hands back a partial order when there is a cycle: the second run in the trace would return `[0, 2]` instead of `[]`.
- Sort items by groups: give every item with group –1 a group of its own. If they all share one, groups `[-1, 0, -1]` with 1 after 0 and 2 after 1 come out impossible, although `[0, 1, 2]` works.

Practise on LeetCode, in this order

- [ ] `LC 210` Course Schedule II — Medium
- [ ] `LC 2115` Find All Possible Recipes from Given Supplies — Medium
- [ ] `LC 2192` All Ancestors of a Node in a Directed Acyclic Graph — Medium
- [ ] `LC 2392` Build a Matrix With Conditions — Hard
- [ ] `LC 1203` Sort Items by Groups Respecting Dependencies — Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

9.8 DP on a DAG in topological order

`O(V + E)`

Spot it when the problem looks like this

- *Minimum time to finish all courses* when courses can run in parallel; the *longest* chain of dependent tasks.
- *Largest value along any path*, *number of paths*, *longest increasing path*: the best or the total over all paths.
- Edges go only one way (prerequisites, strictly increasing values), so there is no cycle, or a cycle must be reported.

How it works

On a general graph, the longest simple path is NP-hard. On a DAG it is a DP: the answer for v depends only on its predecessors, and a topological order puts every predecessor before v. So run Kahn's algorithm (**9.7 Topological sort (Kahn's algorithm)** (p. 241)) and, when you pop u, push its value along each edge, as in `best[v] = max(best[v], best[u] + w)`. By the time v is popped, all its incoming edges have been relaxed, so its value is final. Each edge is relaxed once: O(V + E).

Parallel courses III keeps `start[v]`, the latest finish among v's prerequisites; v then finishes at `start[v] + time[v]`. Counting paths uses a sum instead of max. The largest colour value keeps 26 counts per node and relaxes all of them, and the same loop reports a cycle: fewer than n popped.

When the DAG is implicit, as with strictly increasing steps in a matrix, a memoised DFS is shorter: each cell is solved once, after the cells it can step to.

Worked example **minimumTime(5, [[1,5], [2,5], [3,5], [3,4], [4,5]], time = [1, 2, 3, 4, 5])**

```
pop  start  time  finish  relax start[v]          best
1    0      1     1       5: 0 -> 1               1
2    0      2     2       5: 1 -> 2               2
3    0      3     3       5: 2 -> 3, 4: 0 -> 3    3
4    3      4     7       5: 3 -> 7               7
5    7      5     12                              12
Course 5 starts when its slowest prerequisite (4, after 3) ends.
```

Parallel courses III (earliest finish) memorise

```java
// Parallel courses III: {a, b} (1-based) means a before b; course v takes
// time[v - 1]. Earliest finish of v = time + latest finish of a prerequisite.
static int minimumTime(int n, int[][] relations, int[] time) {
    List<Integer>[] g = new List[n];
    for (int v = 0; v < n; v++) g[v] = new ArrayList<>();
    int[] indeg = new int[n], start = new int[n];  // start = all prereqs done
    for (int[] e : relations) { g[e[0] - 1].add(e[1] - 1); indeg[e[1] - 1]++; }
    Deque<Integer> q = new ArrayDeque<>();
    for (int v = 0; v < n; v++) if (indeg[v] == 0) q.add(v);
    int best = 0;
    while (!q.isEmpty()) {
        int u = q.poll();                          // start[u] is final now
        int finish = start[u] + time[u];
        best = Math.max(best, finish);
        for (int v : g[u]) {
            start[v] = Math.max(start[v], finish);   // relax u -> v
            if (--indeg[v] == 0) q.add(v);
        }
    }
    return best;
}
```

Largest colour value (26 DPs at once)

variant

```java
// Largest colour value: cnt[v][c] = most nodes of colour c on a path ending
// at v. Push the counts along each edge in Kahn order; a cycle gives -1.
static int largestPathValue(String colors, int[][] edges) {
    int n = colors.length();
    List<Integer>[] g = new List[n];
    for (int v = 0; v < n; v++) g[v] = new ArrayList<>();
    int[] indeg = new int[n];
    for (int[] e : edges) { g[e[0]].add(e[1]); indeg[e[1]]++; }
    int[][] cnt = new int[n][26];
    Deque<Integer> q = new ArrayDeque<>();
    for (int v = 0; v < n; v++) if (indeg[v] == 0) q.add(v);
    int done = 0, best = 0;
    while (!q.isEmpty()) {
        int u = q.poll();
        done++;
        best = Math.max(best, ++cnt[u][colors.charAt(u) - 'a']);  // add u
        for (int v : g[u]) {
            for (int c = 0; c < 26; c++)
                cnt[v][c] = Math.max(cnt[v][c], cnt[u][c]);
            if (--indeg[v] == 0) q.add(v);
        }
    }
    return done < n ? -1 : best;
}
```

Longest increasing path (memoised DFS)

variant

```java
static final int[][] DIRS = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};

// Longest increasing path in a matrix: "step to a bigger neighbour" is a DAG,
// so a memoised DFS computes each cell's answer once.
static int longestIncreasingPath(int[][] m) {
    int[][] memo = new int[m.length][m[0].length];    // 0 = not computed
    int best = 0;
    for (int r = 0; r < m.length; r++)
        for (int c = 0; c < m[0].length; c++)
            best = Math.max(best, longest(m, r, c, memo));
    return best;
}

static int longest(int[][] m, int r, int c, int[][] memo) {
    if (memo[r][c] != 0) return memo[r][c];
    int best = 1;                                     // the cell alone
    for (int[] d : DIRS) {
        int nr = r + d[0], nc = c + d[1];
        if (nr < 0 || nc < 0 || nr >= m.length || nc >= m[0].length) continue;
        if (m[nr][nc] > m[r][c])                      // an edge of the DAG
            best = Math.max(best, 1 + longest(m, nr, nc, memo));
    }
    return memo[r][c] = best;
}
```

Cost Time `O(V + E)` Space `O(V + E)`
One relaxation per edge (26 in the colour version); the matrix version solves each cell once.

Variants same idea, small change

- Count paths from s to t: `ways[s] = 1`, then `ways[v] += ways[u]` along each edge in topological order, mod $10^9 + 7$ (number of increasing paths in a grid).
- Loud and rich: the answer for x is the quietest person among x and everyone richer than x, a min over the *richer than* DAG; memoised DFS or Kahn.
- Shortest path in a weighted DAG: the same relaxation with min. It works with negative weights, which 9.10 **Dijkstra's shortest paths** (p. 250) does not.

Traps bugs people actually write

- `start[v] = finish` keeps the prerequisite popped last, not the slowest: relations `[[1,3], [2,3]]` with time `[10, 2, 1]` give 10, not 11. Take the max.
- The colour version must return −1 on a cycle. Without the `done < n` test, `"a"` with the self-loop `[[0, 0]]` returns 0.
- The matrix DFS needs strict `>`. With `>=`, two equal neighbours form a cycle and call each other forever, because the memo is written only on return: `[[1, 1]]` throws StackOverflowError.
- A 200 × 200 matrix can hold an increasing path of 20101 cells, and the memoised DFS then overflows Java's default stack (measured). Without recursion: Kahn's algorithm on cells, starting from those with no smaller neighbour; the number of levels is the answer.

Practise on LeetCode, in this order

☐	`LC 851`	Loud and Rich	Medium
☐	`LC 329`	Longest Increasing Path in a Matrix	Hard
☐	`LC 2050`	Parallel Courses III	Hard
☐	`LC 2328`	Number of Increasing Paths in a Grid	Hard
☐	`LC 1857`	Largest Color Value in a Directed Graph	Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

9.9 Union-Find (disjoint set union)

`O(α(n)) per op`

Spot it when the problem looks like this

- Connections arrive one at a time: *are a and b connected now?*, *how many groups remain?*
- Items joined by something shared: the same email, row or column, or an `a==b` equation.
- Up to 10^5 items and unions, so a fresh DFS for every query (O(V + E) each) is too slow.

How it works

Each set is a tree stored in `parent[]`, and its root names the set. `find(x)` walks up to the root; `union(a, b)` hangs one root under the other. Two items are in the same set exactly when their roots are equal.

Two rules keep the trees flat. **Union by size** hangs the smaller tree under the bigger one, so a node gets one level deeper only when the size of its tree at least doubles: no node is deeper than $\log_2 n$. **Path compression** points every node that `find` passes straight at the root, so the next `find` from any of them takes one step. Together they make each operation $O(\alpha(n))$ amortised, where α, the inverse Ackermann function, is at most 4 for any n you will meet.

`union` returns false when both items already have the same root. That is the cycle test in redundant connection. `count` starts at n and drops by one per successful union, so it is always the number of components.

Worked example **DSU(5): five unions, then find(1)**

```
op           roots  result  parent           count
start                       [0, 1, 2, 3, 4]  5
union(0, 1)  0, 1   joined  [0, 0, 2, 3, 4]  4
union(2, 3)  2, 3   joined  [0, 0, 2, 2, 4]  3
union(3, 4)  2, 4   joined  [0, 0, 2, 2, 2]  2   size 2 vs 1: 4 under 2
union(1, 4)  0, 2   joined  [2, 0, 2, 2, 2]  1   size 2 vs 3: 0 under 2
union(0, 3)  2, 2   false   [2, 0, 2, 2, 2]  1   same root: extra edge
find(1)      1 -> 0 -> 2    [2, 2, 2, 2, 2]      compressed: 1 points at 2
```

DSU with union by size and path compression memorise

```
static class DSU {
    int[] parent, size;
    int count;                                  // number of sets
    DSU(int n) {
        parent = new int[n];
        size = new int[n];
        count = n;
        for (int i = 0; i < n; i++) { parent[i] = i; size[i] = 1; }
    }

    // Root of x's set. Path compression: point every node passed at the root.
    int find(int x) {
        return parent[x] == x ? x : (parent[x] = find(parent[x]));
    }

    // Join the sets of a and b; false if they were already one set.
    boolean union(int a, int b) {
        int ra = find(a), rb = find(b);
        if (ra == rb) return false;
        if (size[ra] < size[rb]) { int t = ra; ra = rb; rb = t; }
        parent[rb] = ra;                        // smaller tree under the bigger
        size[ra] += size[rb];
        count--;
        return true;
    }
}
```

Redundant connection; equality equations

example

```java
// Redundant connection: the first edge whose two ends are already joined.
static int[] findRedundantConnection(int[][] edges) {
    DSU d = new DSU(edges.length + 1);        // nodes 1..n and n = edges
    for (int[] e : edges)
        if (!d.union(e[0], e[1])) return e;
    return new int[0];
}

// Equality equations: join every "a==b" first, then test every "a!=b".
static boolean equationsPossible(String[] equations) {
    DSU d = new DSU(26);
    for (String e : equations)
        if (e.charAt(1) == '=') d.union(e.charAt(0) - 'a', e.charAt(3) - 'a');
    for (String e : equations) {
        int a = e.charAt(0) - 'a', b = e.charAt(3) - 'a';
        if (e.charAt(1) == '!' && d.find(a) == d.find(b)) return false;
    }
    return true;
}
```

Accounts merge

example

```java
// Accounts merge: join two accounts when they share an email.
static List<List<String>> accountsMerge(List<List<String>> acc) {
    DSU d = new DSU(acc.size());
    Map<String, Integer> firstOwner = new HashMap<>();   // email -> account
    for (int i = 0; i < acc.size(); i++)
        for (String email : acc.get(i).subList(1, acc.get(i).size())) {
            Integer j = firstOwner.putIfAbsent(email, i);
            if (j != null) d.union(i, j);
        }
    Map<Integer, TreeSet<String>> byRoot = new HashMap<>();
    for (Map.Entry<String, Integer> e : firstOwner.entrySet())
        byRoot.computeIfAbsent(d.find(e.getValue()), k -> new TreeSet<>())
              .add(e.getKey());
    List<List<String>> out = new ArrayList<>();
    for (Map.Entry<Integer, TreeSet<String>> e : byRoot.entrySet()) {
        List<String> row = new ArrayList<>();
        row.add(acc.get(e.getKey()).get(0));         // the name
        row.addAll(e.getValue());                    // sorted emails
        out.add(row);
    }
    return out;
}
```

Cost Time `O(α(n)) per op` Space `O(n)`

With both rules, m operations cost O(m·α(n)), which is O(m) for every practical n. Union by size alone gives O(log n) per find.

Variants same idea, small change

- Number of provinces, or operations to make a network connected: union every edge and read `count`. The network needs `count - 1` moved cables, provided it has at least n – 1 edges.
- Most stones removed: union each stone's row with its column (shift the column ids past the row ids). The answer is the number of stones minus the number of sets that hold a stone.
- Smallest string with swaps: union the swap pairs; each set gets its letters in sorted order.
- Sorted edges plus a DSU give a minimum spanning tree: **9.14 Kruskal's MST (sort + DSU)** (p. 263).

Traps bugs people actually write

- `union` must link roots. `parent[a] = b` moves one node and cuts it from its set: after `union(0, 1)` and `union(0, 2)` written that way, node 1 is a root on its own again.
- Compare `find(a) == find(b)`, not `parent[a] == parent[b]`. After `union(0,1)`, `union(2,3)`, `union(0,2)`, `parent` is `[0, 0, 0, 2]`: 1 and 3 are in one set, but their parents differ.
- Without union by size, the unions `(0,1), (1,2), …` build a chain, and the recursive `find(0)` on a chain of 10^5 nodes throws StackOverflowError (measured). By size, depth ≤ $\log_2 n$.
- Redundant connection numbers nodes 1..n: use `new DSU(n + 1)`, or node n is out of bounds.

Practise on LeetCode, in this order

- [ ] `LC 684` Redundant Connection — Medium
- [ ] `LC 990` Satisfiability of Equality Equations — Medium
- [ ] `LC 1319` Number of Operations to Make Network Connected — Medium
- [ ] `LC 721` Accounts Merge — Medium
- [ ] `LC 1202` Smallest String With Swaps — Medium
- [ ] `LC 947` Most Stones Removed with Same Row or Column — Medium

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

9.10 Dijkstra's shortest paths

`O(m log n)`

Spot it when the problem looks like this

- *Minimum time*, *cheapest price*, *shortest distance* from one node, and the edges have different weights. If every weight is 1, use **9.2 BFS shortest path (unweighted)** (p. 226).
- Weights are non-negative; n up to 10^5 and m up to $2 \cdot 10^5$, so O(m log n) is the target.
- The cost of a path is not a plain sum but never gets better as the path grows: the largest step (*minimum effort*), a product of probabilities ≤ 1.
- *Number of shortest paths*, or the shortest path with a tie-break: the same run with one more array.

How it works

Keep `dist[v]`, the best distance found so far, and a min-heap of `{dist, node}`. Pop the smallest entry `(d, u)`. Every entry left in the heap is ≥ d and no edge is negative, so no other path can reach `u` for less than d: `dist[u]` is final. Then relax each edge `u → v`: if `d + w < dist[v]`, store it and push `{d + w, v}`.

Java's `PriorityQueue` cannot lower a key, so a node can sit in the heap several times. Only the entry with `d == dist[u]` is current; skip the others with `if (d > dist[u]) continue`. Each edge pushes at most one entry, so the heap holds at most m entries and the run costs O(m log m) = O(m log n).

The proof only needs that extending a path never makes it cheaper. So it also works when a path costs its largest step, as in *minimum effort*.

Worked example **src = 0; edges 0→1 (4), 0→2 (1), 2→1 (2), 1→3 (1), 2→3 (5)**

```
pop       action                              dist
(0, 0)    relax 1: inf -> 4, 2: inf -> 1      [0, 4, 1, inf]
(1, 2)    relax 1: 4 -> 3, 3: inf -> 6        [0, 3, 1, 6]
(3, 1)    relax 3: 6 -> 4                     [0, 3, 1, 4]
(4, 1)    stale: 4 > dist[1] = 3
(4, 3)    settled, no out-edges               [0, 3, 1, 4]
(6, 3)    stale: 6 > dist[3] = 4
```

Dijkstra and network delay time

memorise

```java
// Dijkstra from src. g[u] holds edges {v, w} with w >= 0.
// dist[v] = Long.MAX_VALUE means v cannot be reached.
static long[] dijkstra(List<int[]>[] g, int src) {
    long[] dist = new long[g.length];
    Arrays.fill(dist, Long.MAX_VALUE);
    dist[src] = 0;
    PriorityQueue<long[]> pq =
        new PriorityQueue<>((a, b) -> Long.compare(a[0], b[0]));
    pq.add(new long[] {0, src});                    // {distance, node}
    while (!pq.isEmpty()) {
        long[] top = pq.poll();
        long d = top[0];
        int u = (int) top[1];
        if (d > dist[u]) continue;                  // stale: u was settled cheaper
        for (int[] e : g[u]) {                      // d == dist[u] is final here
            int v = e[0];
            long nd = d + e[1];
            if (nd < dist[v]) {                     // relax u -> v
                dist[v] = nd;
                pq.add(new long[] {nd, v});
            }
        }
    }
    return dist;
}

// Network delay time (LC 743): nodes 1..n, times[i] = {u, v, w}.
static int networkDelayTime(int[][] times, int n, int k) {
    List<int[]>[] g = new List[n + 1];
    for (int i = 0; i <= n; i++) g[i] = new ArrayList<>();
    for (int[] t : times) g[t[0]].add(new int[] {t[1], t[2]});
    long[] dist = dijkstra(g, k);
    long worst = 0;
    for (int v = 1; v <= n; v++) worst = Math.max(worst, dist[v]);
    return worst == Long.MAX_VALUE ? -1 : (int) worst;
}
```

Number of shortest paths

variant

```java
// Number of ways to arrive at destination (LC 1976): count the shortest
// paths 0 -> n-1, mod 1e9+7. Roads are undirected, times are >= 1.
static int countPaths(int n, int[][] roads) {
    List<int[]>[] g = new List[n];
    for (int i = 0; i < n; i++) g[i] = new ArrayList<>();
    for (int[] r : roads) {
        g[r[0]].add(new int[] {r[1], r[2]});
        g[r[1]].add(new int[] {r[0], r[2]});
    }
    long[] dist = new long[n], ways = new long[n];
    Arrays.fill(dist, Long.MAX_VALUE);
    dist[0] = 0;
    ways[0] = 1;
    PriorityQueue<long[]> pq =
        new PriorityQueue<>((a, b) -> Long.compare(a[0], b[0]));
    pq.add(new long[] {0, 0});
    while (!pq.isEmpty()) {
        long[] top = pq.poll();
        long d = top[0];
        int u = (int) top[1];
        if (d > dist[u]) continue;                // ways[u] is complete here
        for (int[] e : g[u]) {
            int v = e[0];
            long nd = d + e[1];
            if (nd < dist[v]) {                   // shorter: a new count
                dist[v] = nd;
                ways[v] = ways[u];
                pq.add(new long[] {nd, v});
            } else if (nd == dist[v]) {           // as short: add u's ways
                ways[v] = (ways[v] + ways[u]) % 1_000_000_007;
            }
        }
    }
    return (int) ways[n - 1];
}
```

When u pops for real, every road into u on a shortest path starts at a node with a smaller distance (times are ≥ 1), so `ways[u]` is complete.

Cost Time `O(m log n)` Space `O(n + m)`

Each edge is relaxed once, from its settled start, and pushes at most one heap entry; a push or pop costs O(log m) = O(log n).

Variants same idea, small change

- Path with maximum probability: multiply instead of add and pop the largest first. Probabilities are ≤ 1, so a path only gets less likely as it grows and the proof still holds.
- Path with minimum effort (LC 1631): the nodes are cells `r * n + c`, the new cost is `Math.max(d, Math.abs(h[nr][nc] - h[r][c]))` instead of `d + w`, and you can return when the corner pops for real.
- The path must pass a meeting point (LC 2203): run Dijkstra from both sources and from the target on the reversed graph, then take the minimum of `d1[x] + d2[x] + d3[x]`.
- Weights only 0 and 1: a deque replaces the heap (9.11 **0-1 BFS (deque)** (p. 254)). Negative weights: 9.12 **Bellman–Ford and at most k edges** (p. 257). A state such as (node, stops used) gets its own `dist` entry, as in 9.4 **BFS over states** (p. 232).

Traps bugs people actually write

- `(a, b) -> (int) (a[0] - b[0])` on `long` distances breaks once two distances differ by 2^{31} or more: the cast drops the high bits and the heap pops in the wrong order. Use `Long.compare`.
- Marking `v` done when it is first pushed, as BFS does, keeps the first distance found. In the example node 1 is pushed with 4 and only later improves to 3.
- Leaving out `if (d > dist[u]) continue` keeps the answer right but scans u's edges again for every stale entry, up to O(n · m) work on a dense graph.
- A negative edge breaks the proof. With 0→1 (2), 0→2 (3), 2→1 (−2), node 1 pops at 2 but its distance is 1, so an early return at the target is wrong. Use 9.12 **Bellman–Ford and at most k edges** (p. 257).

Practise on LeetCode, in this order

☐	LC 743	Network Delay Time	Medium
☐	LC 1514	Path with Maximum Probability	Medium
☐	LC 1631	Path With Minimum Effort	Medium
☐	LC 1976	Number of Ways to Arrive at Destination	Medium
☐	LC 2203	Minimum Weighted Subgraph With the Required Paths	Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

9.11 0-1 BFS (deque)

`O(V + E)`

Spot it when the problem looks like this

- Each move is free or costs 1: follow the sign for free or change it, walk on empty cells for free or remove an obstacle.
- *Minimum number of changes*, *removals* or *flips* from start to target, on up to 10^5 cells.
- Plain BFS is wrong (the steps differ in cost), and a heap's log factor is not needed.

How it works

Dijkstra pops nodes in order of distance. With weights 0 and 1 a deque keeps that order without a heap. Relaxing an edge out of `u` (distance d) gives d or d + 1. Put a weight-0 neighbour at the **front**, since it ties with `u`, and a weight-1 neighbour at the **back**.

The invariant: from front to back the deque holds some entries of distance d and then some of distance d + 1, never a third value. So the front is always a smallest entry, exactly what a heap would pop, and when a cell leaves the front its distance is final. A cell is pushed at most twice (first at d + 1 from one neighbour, then at d from another), so the run is O(V + E); for an m × n grid that is O(mn).

Push only when `dist` improves, and read `dist[cell]` when you pop: an old copy of a cell then does no harm.

Worked example **LC 1368, grid = [[4,3,1],[1,3,4],[4,2,1]]; arrows 1 R, 2 L, 3 D, 4 U**

```
pop   d  improved neighbours                deque after
(0,0) 0  R (0,1)=1 back, D (1,0)=1 back     [(0,1) (1,0)]
(0,1) 1  R (0,2)=2 back, D (1,1)=1 front    [(1,1) (1,0) (0,2)]
(1,1) 1  R (1,2)=2 back, D (2,1)=1 front    [(2,1) (1,0) (0,2) (1,2)]
(2,1) 1  R (2,2)=2 back, L (2,0)=1 front    [(2,0) (1,0) (0,2) (1,2) (2,2)]
(2,0) 1  -                                  [(1,0) (0,2) (1,2) (2,2)]
(1,0) 1  -                                  [(0,2) (1,2) (2,2)]
(0,2) 2  -                                  [(1,2) (2,2)]
(1,2) 2  -                                  [(2,2)]
(2,2) 2  -                                  []
answer: dist of (2,2) = 2
```

Minimum cost to make a valid path

memorise

```java
// Minimum cost to make at least one valid path (LC 1368). Following the
// cell's arrow costs 0; any other move costs 1 (change the arrow once).
static int minCost(int[][] grid) {
    int m = grid.length, n = grid[0].length;
    int[] dr = {0, 0, 0, 1, -1}, dc = {0, 1, -1, 0, 0};  // arrows 1..4: R L D U
    int[] dist = new int[m * n];                          // cell id = r * n + c
    Arrays.fill(dist, Integer.MAX_VALUE);
    Deque<Integer> dq = new ArrayDeque<>();
    dist[0] = 0;
    dq.addFirst(0);
    while (!dq.isEmpty()) {
        int cell = dq.pollFirst(), r = cell / n, c = cell % n;
        for (int k = 1; k <= 4; k++) {
            int nr = r + dr[k], nc = c + dc[k];
            if (nr < 0 || nr >= m || nc < 0 || nc >= n) continue;
            int w = grid[r][c] == k ? 0 : 1;
            int next = nr * n + nc;
            if (dist[cell] + w < dist[next]) {
                dist[next] = dist[cell] + w;
                if (w == 0) dq.addFirst(next);          // same distance: front
                else dq.addLast(next);                  // one more: back
            }
        }
    }
    return dist[m * n - 1];
}
```

Minimum obstacle removal

variant

```java
// Minimum obstacle removal to reach corner (LC 2290): stepping onto a
// cell costs grid value (0 = empty, 1 = obstacle to remove).
static int minimumObstacles(int[][] grid) {
    int m = grid.length, n = grid[0].length;
    int[][] dist = new int[m][n];
    for (int[] row : dist) Arrays.fill(row, Integer.MAX_VALUE);
    int[] dr = {1, -1, 0, 0}, dc = {0, 0, 1, -1};
    Deque<int[]> dq = new ArrayDeque<>();
    dist[0][0] = 0;
    dq.add(new int[] {0, 0});
    while (!dq.isEmpty()) {
        int[] cur = dq.pollFirst();
        int r = cur[0], c = cur[1];
        for (int k = 0; k < 4; k++) {
            int nr = r + dr[k], nc = c + dc[k];
            if (nr < 0 || nr >= m || nc < 0 || nc >= n) continue;
            int nd = dist[r][c] + grid[nr][nc];
            if (nd >= dist[nr][nc]) continue;
            dist[nr][nc] = nd;
            if (grid[nr][nc] == 0) dq.addFirst(new int[] {nr, nc});
            else dq.addLast(new int[] {nr, nc});
        }
    }
    return dist[m - 1][n - 1];
}
```

Here the cost belongs to the cell you step onto, so the start cell is free.

Cost Time `O(V + E)` Space `O(V)`

Each cell is pushed at most twice and looks at 4 neighbours per pop; on an m × n grid V and E are both O(mn).

Variants same idea, small change

- Find a safe walk (LC 3286): unsafe cells cost 1, the start cell included. The walk works when the 0-1 distance is less than `health`.
- Minimum sideway jumps: a state is (point, lane). Moving forward costs 0 and changing lane costs 1.
- Weights 0..k for a small k: keep k + 1 buckets by distance (Dial's algorithm), or use 9.10 **Dijkstra's shortest paths** (p. 250).
- All moves cost 1 but there are free moves (portals, teleports): free moves go to the front of the same deque.

Traps bugs people actually write

- Marking a cell done when it is first pushed, as in plain BFS, keeps a cost that a later 0-move could lower: LC 1368 sample 2 returns 1 instead of 0.
- The cost depends on the arrow of the cell you leave, `grid[r][c] == k`. Testing `grid[nr][nc] == k` returns 2 on sample 2 (answer 0).
- `dr` and `dc` must follow the problem's numbering: 1 right, 2 left, 3 down, 4 up. The usual right, down, left, up order returns 2 on sample 1 (answer 3).

Practise on LeetCode, in this order

- [ ] LC 3286 Find a Safe Walk Through a Grid — Medium
- [ ] LC 1824 Minimum Sideway Jumps — Medium
- [ ] LC 2290 Minimum Obstacle Removal to Reach Corner — Hard
- [ ] LC 1368 Minimum Cost to Make at Least One Valid Path in a Grid — Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

9.12 Bellman–Ford and at most k edges

`O(n · m)`

Spot it when the problem looks like this

- *At most k stops* (k + 1 flights) on the cheapest route. Dijkstra settles a node once and ignores how many edges it used.
- Some edge weights are negative, or you must report a negative cycle (an arbitrage loop).
- n·m is small, about 10^6 ($n \le 100$, $m \le 10^4$), because the cost is O(n·m).

How it works

Without a negative cycle a shortest path is simple, so it has at most n – 1 edges. Bellman–Ford relaxes every edge in rounds. After round i, `dist[v]` is at most the cost of the best path with ≤ i edges: that path is a best path with i – 1 edges plus one more edge, and round i relaxes that edge. So n – 1 rounds are enough, and a round that changes nothing means you can stop.

If round n still lowers a distance, a path with n edges beats all shorter ones. It must repeat a vertex, so there is a negative cycle.

In place, one round can chain edges: relax `a → b`, then `b → c` in the same pass. That is fine for plain shortest paths but breaks an edge limit. For *at most k + 1 edges*, read from `prev`, a copy of last round, so round i only extends paths with fewer than i edges. Cost: O(n·m), or O(k·m) with the limit.

Worked example **LC 787 sample 1: src 0, dst 3, k = 1 stop (at most 2 flights)**

```
round  edge       prev[u] + w   dist after
  1    0->1 100   0 + 100       [0, 100, inf, inf]
  1    1->2 100   inf (skip)    [0, 100, inf, inf]
  1    2->0 100   inf (skip)    [0, 100, inf, inf]
  1    1->3 600   inf (skip)    [0, 100, inf, inf]
  1    2->3 200   inf (skip)    [0, 100, inf, inf]
  2    0->1 100   0 + 100       [0, 100, inf, inf]
  2    1->2 100   100 + 100     [0, 100, 200, inf]
  2    2->0 100   inf (skip)    [0, 100, 200, inf]
  2    1->3 600   100 + 600     [0, 100, 200, 700]
  2    2->3 200   inf (skip)    [0, 100, 200, 700]
answer 700. In place, round 1 alone gives [0, 100, 200, 400]: 2 stops.
```

Bellman–Ford with the negative-cycle round

memorise

```java
static final long INF = Long.MAX_VALUE / 4;

// Shortest paths from src with any edge weights; edges[i] = {u, v, w}.
// Returns null if a negative cycle can be reached from src.
static long[] bellmanFord(int n, int[][] edges, int src) {
    long[] dist = new long[n];
    Arrays.fill(dist, INF);
    dist[src] = 0;
    for (int round = 1; round < n; round++) {      // n - 1 rounds
        boolean changed = false;
        for (int[] e : edges) {
            if (dist[e[0]] == INF) continue;          // u not reached yet
            if (dist[e[0]] + e[2] < dist[e[1]]) {
                dist[e[1]] = dist[e[0]] + e[2];
                changed = true;
            }
        }
        if (!changed) return dist;                    // stable: no cycle either
    }
    for (int[] e : edges)                             // round n: still improves?
        if (dist[e[0]] != INF && dist[e[0]] + e[2] < dist[e[1]]) return null;
    return dist;
}
```

Cheapest flights within k stops

memorise

```
// Cheapest flights within k stops (LC 787): at most k + 1 edges.
// Each round reads last round's copy, so after round i every path
// in dist uses at most i edges.
static int findCheapestPrice(int n, int[][] flights, int src, int dst, int k) {
    long[] dist = new long[n];
    Arrays.fill(dist, INF);
    dist[src] = 0;
    for (int round = 1; round <= k + 1; round++) {
        long[] prev = dist.clone();                    // paths with < round edges
        for (int[] f : flights)
            if (prev[f[0]] != INF && prev[f[0]] + f[2] < dist[f[1]])
                dist[f[1]] = prev[f[0]] + f[2];
    }
    return dist[dst] == INF ? -1 : (int) dist[dst];
}
```

Cost Time `O(n · m)` Space `O(n)`

Up to n – 1 rounds over all m edges (k + 1 rounds with the stop limit, plus an O(n) copy per round).

Variants same idea, small change

- Any negative cycle in the graph, not only one reachable from src: start with every `dist` at 0, as if a new source had a 0-edge to each node.
- Count walks with exactly k edges (knight dialer): the same rounds from a copy, with `+=` instead of `min`.
- Cheapest trip within a time limit (LC 1928): the round is the time t, and `dp[t][v]` relaxes each edge from the earlier round `t - w`.
- The queue version (SPFA) relaxes only the edges out of nodes that changed. It is often faster but has the same O(n·m) worst case.

Traps bugs people actually write

- Relaxing in place in the k-stop version lets one round chain several flights. On sample 1, round 1 already finds 0→1→2→3 = 400, which has 2 stops. Read from `prev`.
- Without `if (dist[u] == INF) continue`, an unreached u with a –5 edge sets `dist[v] = INF - 5`. That is no longer `INF`, so v looks reachable.
- `INF = Long.MAX_VALUE` wraps to a negative number on `INF + w` for w > 0. Use `Long.MAX_VALUE / 4`, or skip unreached nodes before adding.
- Stopping after n – 1 rounds without the extra round returns garbage when a negative cycle is reachable: the distances are wherever the loop stopped.

Practise on LeetCode, in this order

- [ ] LC 787 Cheapest Flights Within K Stops — Medium
- [ ] LC 935 Knight Dialer — Medium
- [ ] LC 1928 Minimum Cost to Reach Destination in Time — Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

9.13 Floyd–Warshall (all pairs) `O(n³)`

Spot it when the problem looks like this

- Many questions *from any u to any v*, and n ≤ 400 or so, so $n^3 \le 6.4 \cdot 10^7$ steps.
- A small fixed set of nodes with conversion costs between them: 26 letters, 10 digits, a few cities.
- *Is a a prerequisite of b*, *can u reach v*, asked for many pairs: reachability for all pairs.

How it works

After round k, `d[i][j]` is the shortest path from i to j whose middle nodes all come from `0..k`. A best such path either avoids k, so round k – 1 already has it, or passes k once and costs `d[i][k] + d[k][j]` with round k – 1's values. Taking the smaller one for every pair finishes round k. After round n – 1 any node may be in the middle.

That is why k must be the **outer** loop: round k needs round k – 1 finished for all pairs. One array is enough, because round k does not change `d[i][k]` or `d[k][j]` (a detour through k to reach k gains nothing). Three loops give $O(n^3)$ time and $O(n^2)$ space.

The same loops work with other operations. With booleans, `reach[i][j] |= reach[i][k] && reach[k][j]` gives reachability for all pairs. On the 26 letters of LC 2976 Floyd costs 26^3 steps, however long the strings are.

Worked example **LC 1334 sample 1: edges 0-1 (3), 1-2 (1), 1-3 (4), 2-3 (1); threshold 4**

```
k  improved (i < j; d[j][i] changes the same way)
0  -
1  d[0][2] inf -> 4, d[0][3] inf -> 7
2  d[0][3] 7 -> 5, d[1][3] 4 -> 2
3  -
final d       cities within 4
0: 0 3 4 5    2
1: 3 0 1 2    3
2: 4 1 0 1    3
3: 5 2 1 0    2     0 and 3 tie at 2; the larger index wins: 3
```

Floyd–Warshall and the threshold city

memorise

```
static final long INF = Long.MAX_VALUE / 4;       // INF + INF does not overflow

// All pairs, in place: d[i][i] = 0, d[i][j] = edge weight or INF (w >= 0).
// After round k, d[i][j] is the best path whose middle nodes are all <= k.
static void floyd(long[][] d) {
    int n = d.length;
    for (int k = 0; k < n; k++)                    // k must be the outer loop
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                d[i][j] = Math.min(d[i][j], d[i][k] + d[k][j]);
}

// LC 1334: the city with the fewest cities within the threshold.
// Ties go to the largest index, hence <= below. Edges are undirected.
static int findTheCity(int n, int[][] edges, int threshold) {
    long[][] d = new long[n][n];
    for (long[] row : d) Arrays.fill(row, INF);
    for (int i = 0; i < n; i++) d[i][i] = 0;
    for (int[] e : edges) {
        d[e[0]][e[1]] = Math.min(d[e[0]][e[1]], e[2]);
        d[e[1]][e[0]] = Math.min(d[e[1]][e[0]], e[2]);
    }
    floyd(d);
    int best = -1, bestCount = Integer.MAX_VALUE;
    for (int i = 0; i < n; i++) {
        int count = 0;
        for (int j = 0; j < n; j++) if (j != i && d[i][j] <= threshold) count++;
        if (count <= bestCount) { bestCount = count; best = i; }
    }
    return best;
}
```

Course schedule IV (reachability)

variant

```
// Course schedule IV (LC 1462): is a a prerequisite of b, directly or not?
// Same three loops with OR instead of min: the transitive closure.
static List<Boolean> checkIfPrerequisite(int n, int[][] pre, int[][] queries) {
    boolean[][] reach = new boolean[n][n];
    for (int[] p : pre) reach[p[0]][p[1]] = true;
    for (int k = 0; k < n; k++)
        for (int i = 0; i < n; i++)
            if (reach[i][k])
                for (int j = 0; j < n; j++)
                    if (reach[k][j]) reach[i][j] = true;
    List<Boolean> ans = new ArrayList<>();
    for (int[] q : queries) ans.add(reach[q[0]][q[1]]);
    return ans;
}
```

Minimum cost to convert string I

example

```
// Minimum cost to convert string I (LC 2976): Floyd on the 26 letters,
// then pay d[s][t] for every position.
static long minimumCost(String source, String target, char[] original,
                        char[] changed, int[] cost) {
    long[][] d = new long[26][26];
    for (long[] row : d) Arrays.fill(row, INF);
    for (int i = 0; i < 26; i++) d[i][i] = 0;
    for (int i = 0; i < cost.length; i++) {
        int a = original[i] - 'a', b = changed[i] - 'a';
        d[a][b] = Math.min(d[a][b], cost[i]);       // keep the cheapest rule
    }
    floyd(d);
    long total = 0;
    for (int i = 0; i < source.length(); i++) {
        long c = d[source.charAt(i) - 'a'][target.charAt(i) - 'a'];
        if (c >= INF) return -1;
        total += c;
    }
    return total;
}
```

Cost Time $O(n^3)$ Space $O(n^2)$

n rounds over n^2 pairs. In LC 2976 n is 26, so Floyd is 26^3 steps and the string costs one pass.

Variants same idea, small change

- Negative edges: Floyd still works if you skip terms where `d[i][k]` or `d[k][j]` is INF. A negative `d[i][i]` at the end means i is on a negative cycle.
- Count houses at a certain distance (n ≤ 100): Floyd on the line plus the one extra edge, then count pairs per distance.
- Closing branches (n ≤ 10): for each of the 2^{10} subsets of kept nodes, run Floyd on that subset and check its largest distance.
- Distances from one source only: **9.10 Dijkstra's shortest paths** (p. 250) costs O(m log n) instead of $O(n^3)$.

Traps bugs people actually write

- k as the innermost loop is wrong. With edges 0→1 (1), 1→3 (2), 3→2 (3), `d[0][2]` is finished before `d[0][3]` exists and stays INF; the answer is 6.
- `INF = Integer.MAX_VALUE` makes `d[i][k] + d[k][j]` overflow to a negative number, so unreachable pairs look cheap. Use `Long.MAX_VALUE / 4`.
- The same pair can appear twice with different costs. `d[a][b] = cost` keeps the last one; keep the minimum.
- LC 1334 breaks ties toward the larger index. `count < bestCount` returns 0 on sample 1; the answer is 3.

Practise on LeetCode, in this order

- [ ] LC 1334 Find the City With the Smallest Number of Neighbors at a Threshold Distance — Medium
- [ ] LC 1462 Course Schedule IV — Medium
- [ ] LC 2976 Minimum Cost to Convert String I — Medium
- [ ] LC 3015 Count the Number of Houses at a Certain Distance I — Medium
- [ ] LC 2959 Number of Possible Sets of Closing Branches — Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

9.14 Kruskal's MST (sort + DSU)

`O(m log m)`

Spot it when the problem looks like this

- *Connect all* cities, points or computers *at minimum total cost*, and the input is a list of weighted edges.
- *Which edges are in every or in some minimum spanning tree*, or the smallest possible largest edge on a path (a bottleneck).
- m up to about 10^5, so sorting the edges is the main cost.

How it works

Sort the edges by weight and walk through them. Take an edge if its ends are in different components, which a **9.9 Union-Find (disjoint set union)** (p. 247) tells you in almost O(1); otherwise it would close a cycle, so skip it. Stop after n – 1 edges: the tree is complete.

Why the cheapest edge is safe (the cut property): split the nodes into two sides. Some MST edge crosses the split. If the cheapest crossing edge e is not in that MST, add e, which closes a cycle, and remove another crossing edge of that cycle. The new tree is no heavier. When Kruskal takes an edge, it is the cheapest edge leaving the component of one of its ends, so it is the cheapest edge across that split.

Sorting costs O(m log m) and the DSU work is almost linear. LC 1489 runs Kruskal twice per edge, once skipping it and once forcing it in: O(m^2) after one sort.

Worked example LC 1489 sample 1: n = 5, edges sorted by weight

```
edge  u-v  w  find(u) find(v)  action  total used
e0    0-1  1     0       1     take      1   1
e1    1-2  1     0       2     take      2   2
e2    2-3  2     0       3     take      4   3
e3    0-3  2     0       0     skip      4   3
e4    0-4  3     0       4     take      7   4
e5    3-4  3  not looked at: used = n - 1
e6    1-4  6  not looked at: used = n - 1
MST 7. Skip e0 or e1: 8, so both are critical.
Force e2, e3, e4 or e5: still 7, so pseudo-critical. Force e6: 10.
```

Kruskal

memorise

```java
// Kruskal: MST weight of nodes 0..n-1, edges[i] = {u, v, w}; -1 if disconnected.
static long kruskal(int n, int[][] edges) {
    Arrays.sort(edges, (a, b) -> Integer.compare(a[2], b[2]));   // in place
    DSU dsu = new DSU(n);
    long total = 0;
    int used = 0;
    for (int[] e : edges) {
        if (used == n - 1) break;                    // the tree is complete
        if (dsu.union(e[0], e[1])) {                 // joins two parts: take it
            total += e[2];
            used++;
        }
    }
    return used == n - 1 ? total : -1;
}
```

`DSU` is the class from **9.9 Union-Find (disjoint set union)** (p. 247); `union` returns false if the edge would close a cycle.

Critical and pseudo-critical edges

variant

```java
// LC 1489. Critical: without edge i the MST is heavier or impossible.
// Pseudo-critical: not critical, but some MST uses it (force it in first).
static List<List<Integer>> findCriticalAndPseudoCriticalEdges(
        int n, int[][] edges) {
    Integer[] order = new Integer[edges.length];
    for (int i = 0; i < order.length; i++) order[i] = i;
    Arrays.sort(order, (a, b) -> Integer.compare(edges[a][2], edges[b][2]));
    long best = mst(n, edges, order, -1, -1);
    List<Integer> critical = new ArrayList<>(), pseudo = new ArrayList<>();
    for (int i = 0; i < edges.length; i++) {
        if (mst(n, edges, order, i, -1) > best) critical.add(i);
        else if (mst(n, edges, order, -1, i) == best) pseudo.add(i);
    }
    return Arrays.asList(critical, pseudo);
}

// Kruskal that skips edge `skip` and takes edge `force` first (-1: none).
static long mst(int n, int[][] edges, Integer[] order, int skip, int force) {
    DSU dsu = new DSU(n);
    long total = 0;
    int used = 0;
    if (force >= 0 && dsu.union(edges[force][0], edges[force][1])) {
        total += edges[force][2];
        used++;
    }
    for (int i : order)
        if (i != skip && dsu.union(edges[i][0], edges[i][1])) {
            total += edges[i][2];
            used++;
        }
    return used == n - 1 ? total : Long.MAX_VALUE;  // MAX: no spanning tree
}
```

Cost Time `O(m log m)` Space `O(n + m)`

The sort dominates; each `find` is almost O(1) with path compression and union by size. LC 1489 adds 2m more passes.

Variants same idea, small change

- Min cost to connect all points: build all n(n – 1)/2 edges with Manhattan weights and run Kruskal. For n = 1000 that is $5 \cdot 10^5$ edges; **9.15 Prim's MST (dense or heap)** (p. 265) needs no edge list.
- Queries *is there a path using only edges < limit* (LC 1697): sort the queries by limit too, add the edges below each limit, then test `find(u) == find(v)`.
- Two kinds of users (LC 1579): add the shared edges to both DSUs first, then each type's own edges. Every edge that joins nothing can be removed.
- Maximum spanning tree: sort by weight, largest first. The rest is the same.

Traps bugs people actually write

- Linking the raw nodes, `parent[u] = v`, instead of the roots `find(u)` and `find(v)`, cuts u away from its old component, and later cycle tests are wrong.
- A disconnected graph gives a forest. Check `used == n - 1` before you return the total.
- In LC 1489, *pseudo-critical* is not *skipping it keeps the weight*: that holds for e6 in the sample, and no MST uses e6. Force the edge in and compare.

Practise on LeetCode, in this order

☐	LC 1584	Min Cost to Connect All Points	Medium
☐	LC 1579	Remove Max Number of Edges to Keep Graph Fully Traversable	Hard
☐	LC 1697	Checking Existence of Edge Length Limited Paths	Hard
☐	LC 1489	Find Critical and Pseudo-Critical Edges in Minimum Spanning Tree	Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

9.15 Prim's MST (dense or heap) `O(n²)`

Spot it when the problem looks like this

- *Connect all points* where every pair is an edge: n ≤ 1000, so an edge list holds $5 \cdot 10^5$ entries but n^2 is only 10^6.
- The graph is given as an adjacency list and you can grow the tree from one node.
- *Lowest possible highest cell (or edge) on the way* to a target: grow the tree from the start, always taking the lowest step.

How it works

Prim grows one tree from node 0. `best[v]` is the cheapest edge from the tree to v. Each step adds the outside node with the smallest `best`, then lowers `best` for the rest. Every step is safe by the cut property from **9.14 Kruskal's MST (sort + DSU)** (p. 263), with the tree on one side of the split.

The array version scans all n nodes to pick one and all n again to update: $O(n^2)$ in total, and it never stores an edge. For a complete graph, with $m = n(n - 1)/2$, that beats the $O(m \log m)$ of sorting or of a heap.

On a sparse graph (m close to n) the heap wins. Push `{w, v}` for each edge out of the new node, and skip a popped node that is already in the tree: $O(m \log m)$. It is Dijkstra's loop with the key `w` instead of `d + w`.

Choose by m: near n^2, the array; near n, the heap or **9.14 Kruskal's MST (sort + DSU)** (p. 263).

Worked example **LC 1584 sample 1: points (0,0), (2,2), (3,10), (5,2), (7,0)**

```
step  take  cost  total  best[0..4] after (- = in tree)
  0     0     0     0   [-, 4, 13, 7, 7]
  1     1     4     4   [-, -, 9, 3, 7]
  2     3     3     7   [-, -, 9, -, 4]
  3     4     4    11   [-, -, 9, -, -]
  4     2     9    20   [-, -, -, -, -]
```

O(n²) Prim for all pairs of points

memorise

```java
// Min cost to connect all points (LC 1584), Manhattan distance, O(n^2).
// best[v] = cheapest edge from the tree to v. No edge list is built.
static int minCostConnectPoints(int[][] pts) {
    int n = pts.length, total = 0;
    int[] best = new int[n];
    boolean[] inTree = new boolean[n];
    Arrays.fill(best, Integer.MAX_VALUE);
    best[0] = 0;                                    // start the tree at point 0
    for (int step = 0; step < n; step++) {
        int u = -1;
        for (int v = 0; v < n; v++)                 // 1. closest point outside
            if (!inTree[v] && (u == -1 || best[v] < best[u])) u = v;
        inTree[u] = true;                           // 2. add it with its edge
        total += best[u];
        for (int v = 0; v < n; v++) {               // 3. u may be closer to v
            if (inTree[v]) continue;
            int d = Math.abs(pts[u][0] - pts[v][0])
                  + Math.abs(pts[u][1] - pts[v][1]);
            best[v] = Math.min(best[v], d);
        }
    }
    return total;
}
```

Prim with a heap (sparse graphs) variant

```java
// Prim with a heap for sparse graphs: g[u] holds {v, w}. -1 if disconnected.
// Like Dijkstra, but the key is the edge weight, not the path length.
static long primHeap(List<int[]>[] g) {
    int n = g.length, taken = 0;
    boolean[] inTree = new boolean[n];
    PriorityQueue<int[]> pq =
        new PriorityQueue<>((a, b) -> Integer.compare(a[0], b[0]));
    pq.add(new int[] {0, 0});                       // {edge weight, node}
    long total = 0;
    while (!pq.isEmpty() && taken < n) {
        int[] top = pq.poll();
        int u = top[1];
        if (inTree[u]) continue;                    // stale: u joined already
        inTree[u] = true;
        total += top[0];
        taken++;
        for (int[] e : g[u])
            if (!inTree[e[0]]) pq.add(new int[] {e[1], e[0]});
    }
    return taken == n ? total : -1;
}
```

Cost Time $O(n^2)$ Space $O(n)$

The array version runs n rounds of two O(n) scans. The heap version is O(m log m) time with an O(m) heap.

Variants same idea, small change

- Swim in rising water: grow from (0, 0) with a heap keyed by cell height. The answer is the highest cell taken by the time the corner joins.
- Points from grid queries (LC 2503): sort the queries; for each one keep popping cells lower than it from the same growing heap. The number taken so far is its answer.
- Minimize the maximum edge weight (LC 3419): reverse the edges and grow from node 0 with a heap keyed by weight. The answer is the largest edge taken, or –1 if a node never joins.
- Practise the array version on min cost to connect all points from **9.14 Kruskal's MST (sort + DSU)** (p. 263), and compare the run times.

Traps bugs people actually write

- Dijkstra's key `dist[u] + w` in place of `w` builds a shortest-path tree, not an MST: on sample 1 its edges add up to 31, not 20.
- In the heap version a node can be popped again after it joined. Without `if (inTree[u]) continue` its edge is counted twice.
- In a disconnected graph the heap empties with `taken < n`. Return –1, not the weight of one component.

Practise on LeetCode, in this order

- [ ] LC 3419 Minimize the Maximum Edge Weight of Graph — Medium
- [ ] LC 778 Swim in Rising Water — Hard
- [ ] LC 2503 Maximum Number of Points From Grid Queries — Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

9.16 Bridges and articulation points

`O(n + m)`

Spot it when the problem looks like this

- *Critical connections*: which single cable or road, if cut, splits the network.
- *Remove one cell, node or router* so that the rest falls apart: articulation (cut) vertices.
- n and m up to 10^5, so removing each edge in turn and running a DFS, O(m·(n + m)), is too slow.

How it works

Run one DFS and stamp each node with its entry time `tin[u]`. `low[u]` is the smallest `tin` that the subtree of u can reach with tree edges down and then one back edge up. Compute it on the way back up: start at `tin[u]`, take `tin[v]` for each back edge, and `low[v]` for each child v.

A tree edge u–v is a bridge exactly when `low[v] > tin[u]`. Any other way out of v's subtree is a back edge, and none of them reaches u or higher, so removing u–v cuts the subtree off. A non-root u is a cut vertex when some child has `low[v] >= tin[u]`: that subtree can climb back to u but not above it, so it hangs on u alone. The root is a cut vertex when it has two or more DFS children, because an undirected DFS leaves no edges between sibling subtrees.

One DFS looks at every edge twice: O(n + m).

Worked example LC 1192 sample 1: edges 0-1, 1-2, 2-0, 1-3; DFS from 0

```
enter 0: tin = low = 0
  enter 1: tin = low = 1
    enter 2: tin = low = 2
      back edge 2-0: low[2] = 0
  back at 1 from 2: low[1] = 0; low[2] = 0 <= tin[1] = 1
    enter 3: tin = low = 3
  back at 1 from 3: low[1] = 0; low[3] = 3 > tin[1] = 1: bridge 1-3
back at 0 from 1: low[0] = 0; low[1] = 0 <= tin[0] = 0
  back edge 0-2: low[0] = 0
```

Bridges (critical connections)

memorise

```java
// Critical connections (LC 1192) = bridges of an undirected graph.
// tin[u]: DFS entry time. low[u]: smallest tin that u's subtree reaches
// with tree edges down and one back edge up.
static int timer;
static int[] tin, low;
static List<int[]>[] g;                              // g[u] holds {v, edge id}
static List<List<Integer>> bridges;

static List<List<Integer>> criticalConnections(int n, List<List<Integer>> conn) {
    g = new List[n];
    for (int i = 0; i < n; i++) g[i] = new ArrayList<>();
    for (int id = 0; id < conn.size(); id++) {
        int a = conn.get(id).get(0), b = conn.get(id).get(1);
        g[a].add(new int[] {b, id});
        g[b].add(new int[] {a, id});
    }
    tin = new int[n];
    low = new int[n];
    Arrays.fill(tin, -1);                            // -1 = not visited
    timer = 0;
    bridges = new ArrayList<>();
    for (int s = 0; s < n; s++) if (tin[s] == -1) dfs(s, -1);
    return bridges;
}

static void dfs(int u, int parentEdge) {
    tin[u] = low[u] = timer++;
    for (int[] e : g[u]) {
        int v = e[0];
        if (e[1] == parentEdge) continue;            // not the edge we came by
        if (tin[v] == -1) {                          // tree edge
            dfs(v, e[1]);
            low[u] = Math.min(low[u], low[v]);
            if (low[v] > tin[u]) bridges.add(Arrays.asList(u, v));
        } else {                                     // back edge
            low[u] = Math.min(low[u], tin[v]);
        }
    }
}
```

Articulation points

variant

```
// Articulation points: removing u disconnects its component.
// Uses tin, low and timer from above; adj[u] lists the neighbours of u.
static boolean[] isCut;

static boolean[] articulationPoints(List<Integer>[] adj) {
    int n = adj.length;
    tin = new int[n];
    low = new int[n];
    isCut = new boolean[n];
    Arrays.fill(tin, -1);
    timer = 0;
    for (int s = 0; s < n; s++) if (tin[s] == -1) dfsCut(adj, s, -1);
    return isCut;
}

static void dfsCut(List<Integer>[] adj, int u, int parent) {
    tin[u] = low[u] = timer++;
    int children = 0;
    for (int v : adj[u]) {
        if (v == parent) continue;
        if (tin[v] == -1) {
            dfsCut(adj, v, u);
            low[u] = Math.min(low[u], low[v]);
            children++;
            if (parent != -1 && low[v] >= tin[u]) isCut[u] = true;   // >=, not >
        } else {
            low[u] = Math.min(low[u], tin[v]);
        }
    }
    if (parent == -1 && children >= 2) isCut[u] = true;   // the root rule
}
```

Cost Time `O(n + m)` Space `O(n + m)`

Each vertex is entered once and each edge is looked at from both ends; the recursion can go n levels deep.

Variants same idea, small change

- Minimum days to disconnect island (LC 1568): the answer is 0, 1 or 2. It is 0 if there is not exactly one island, 1 if the island is one cell or has a cut cell, else 2: the top-left land cell has at most two land neighbours, and removing them cuts it off (with only two cells, remove both).
- 2-edge-connected components: delete the bridges, and each piece that stays connected is one component. Contracted, they form a tree whose edges are the bridges.
- In a directed graph the question *which nodes reach each other both ways* is 9.17 **Strongly connected components** (p. 271).
- LeetCode has only two free problems for low-link, both listed below and both Hard.

Traps bugs people actually write

- Skipping the parent by vertex (`v == parent`) instead of by edge id breaks on parallel edges: two edges 0-1 are not a bridge, but the vertex test reports 0-1.
- `low[v] >= tin[u]` is the cut-vertex test. Used for bridges it also reports 0-1 on the sample, which lies on the cycle 0-1-2.
- The cut-vertex test fails at the root: `low[v] >= tin[root]` holds for every child. Count the root's DFS children instead.
- dfs recurses once per node on the current path; a 10^5-node path overflowed Java's default 1 MB stack in the tests. Run it in `new Thread(null, task, "dfs", 1 << 26)`, or go iterative: keep `{u, next edge index}` frames on a stack and fold a frame's `low` into its parent's when you pop it.

Practise on LeetCode, in this order

- [ ] `LC 1192` Critical Connections in a Network — Hard
- [ ] `LC 1568` Minimum Number of Days to Disconnect Island — Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

9.17 Strongly connected components

`O(n + m)`

Spot it when the problem looks like this

- A directed graph where cycles matter: *reach each other*, *the same group both ways*, *treat every cycle as one node*.
- Every node has at most one out-edge (`edges[i]`, `favorite[i]`): a functional graph, where each SCC is one cycle or one node.
- LeetCode has few free SCC problems. The three below are functional graphs, where a walk finds each cycle without the full algorithm.

How it works

Contract each SCC to one node and you get the condensation, which has no cycles: a cycle of SCCs would merge them into one. Kosaraju uses this DAG in two passes.

Pass 1 runs DFS on g and lists the vertices by finish time. If an edge goes from SCC A to SCC B, the latest finish in A is later than every finish in B. So the vertex that finishes last lies in a source SCC.

Pass 2 takes vertices from the latest finish down and floods the **reversed** graph from each one that has no label yet. Reversing keeps every SCC but flips the DAG, so from a source SCC the flood reaches only its own SCC; nodes labelled earlier are skipped. Each flood is one SCC, and the ids come out in topological order. Both passes use an explicit stack, so a 10^5-node path cannot overflow it. Time O(n + m).

Worked example edges 0→1, 1→0, 1→2, 3→4, 4→5, 5→3, 5→0

```
pass 1, DFS on g: finish order 2, 1, 0, 5, 4, 3
pass 2, latest first, flood the reversed graph:
  3      reaches 3, 5, 4   -> comp 0
  4, 5   already labelled
  0      reaches 0, 1      -> comp 1   (its rg edge to 5 is skipped)
  1      already labelled
  2      reaches 2         -> comp 2
comp = [1, 1, 2, 0, 0, 0]; edges 5->0 and 1->2 go from a lower id to a higher
```

Kosaraju (iterative)

memorise

```java
// Kosaraju, iterative. comp[v] = id of v's strongly connected component.
// Ids come out in topological order: every edge u -> v has comp[u] <= comp[v].
static int[] scc(List<Integer>[] g) {
    int n = g.length;
    List<Integer>[] rg = new List[n];                // the reversed graph
    for (int v = 0; v < n; v++) rg[v] = new ArrayList<>();
    for (int u = 0; u < n; u++) for (int v : g[u]) rg[v].add(u);
    // 1. DFS on g; order[] lists vertices by finish time
    int[] order = new int[n], next = new int[n];     // next[u] = next edge to try
    boolean[] seen = new boolean[n];
    int done = 0;
    for (int s = 0; s < n; s++) {
        if (seen[s]) continue;
        Deque<Integer> st = new ArrayDeque<>();
        st.push(s);
        seen[s] = true;
        while (!st.isEmpty()) {
            int u = st.peek();
            if (next[u] < g[u].size()) {
                int v = g[u].get(next[u]++);
                if (!seen[v]) { seen[v] = true; st.push(v); }
            } else {
                order[done++] = st.pop();            // all of u's edges are done
            }
        }
    }
    // 2. Latest finisher first, flood rg; one flood = one component
    int[] comp = new int[n];
    Arrays.fill(comp, -1);
    int c = 0;
    for (int i = n - 1; i >= 0; i--) {
        if (comp[order[i]] != -1) continue;
        Deque<Integer> st = new ArrayDeque<>();
        st.push(order[i]);
        comp[order[i]] = c;
        while (!st.isEmpty())
            for (int v : rg[st.pop()])
                if (comp[v] == -1) { comp[v] = c; st.push(v); }
        c++;
    }
    return comp;
}
```

Longest cycle in a functional graph

example

```java
// Longest cycle (LC 2360). Each node has at most one out-edge, so every
// SCC with 2+ nodes is one cycle, found by walking. stamp[v] = global
// step when v was first visited (0 = never).
static int longestCycle(int[] edges) {
    int n = edges.length, best = -1, clock = 1;
    int[] stamp = new int[n];
    for (int s = 0; s < n; s++) {
        int start = clock, u = s;                    // this walk: stamps >= start
        while (u != -1 && stamp[u] == 0) {
            stamp[u] = clock++;
            u = edges[u];
        }
        if (u != -1 && stamp[u] >= start)            // ran into its own path
            best = Math.max(best, clock - stamp[u]);
    }
    return best;
}
```

Cost Time `O(n + m)` Space `O(n + m)`

Two passes over every vertex and edge, plus a copy of the graph reversed. The functional-graph walk is O(n).

Variants same idea, small change

- Condensation DP: an edge `comp[u] -> comp[v]` for each `comp[u] != comp[v]`. Ids are topological, so a loop from the last id down to 0 sees every successor first. Count visited nodes (LC 2876): an SCC's answer is its size plus the answer of the SCC it points to.
- Longest cycle with the full template: the answer is the size of the largest SCC with at least 2 nodes, or –1.
- Maximum employees (LC 2127): a cycle of length ≥ 3 sits alone. Each 2-cycle seats both plus the longest chain into each end, and all of those groups fit at one table.
- 2-SAT: one node for x and one for ¬x, an edge for each implication. The formula has a solution unless some x and ¬x share an SCC.

Traps bugs people actually write

- Pass 2 must flood the reversed graph: flooding g from 3 in the example reaches all six nodes.
- Pass 2 must start from the latest finish: flooding the reversed graph from `order[0]` = 2 also reaches all six nodes.
- In longest cycle, `stamp[u] != 0` alone does not mean a cycle; u may belong to an earlier walk. Without `stamp[u] >= start`, sample 2 `[2, -1, 3, 1]` returns 3 instead of –1.

Practise on LeetCode, in this order

- [ ] LC 2360 Longest Cycle in a Graph — Hard
- [ ] LC 2876 Count Visited Nodes in a Directed Graph — Hard
- [ ] LC 2127 Maximum Employees to Be Invited to a Meeting — Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

9.18 Euler path (Hierholzer) O(n + m)

Spot it when the problem looks like this

- *Use all the tickets (pairs, dominoes) exactly once* and return the order.
- *Shortest string that contains every n-digit code* (a de Bruijn sequence).
- Every **node** once is a Hamiltonian path instead: see 10.12 **Bitmask DP over subsets** (p. 309).

How it works

A directed graph has an Euler path when every vertex has out = in, except at most one start with out − in = 1 and one end with in − out = 1, and all edges are connected. If every vertex is balanced, the path is a circuit and can start at any vertex with an edge.

Hierholzer walks from the start along unused edges until it is stuck. Stuck at u means u has no unused edge left, so u can be the last vertex of what remains: pop it and put it at the **front** of the answer, which grows from its end. The walk then goes on from the vertex below it on the stack, which splices in the cycles the first walk missed. Each edge is pushed once and popped once: O(n + m).

For the smallest lexical order, hand out each vertex's edges smallest first.

Worked example **LC 332, tickets JFK→KUL, JFK→NRT, NRT→JFK: KUL is smallest but a dead end**

top	action	stack (bottom..top)	route
JFK	fly JFK->KUL	JFK KUL	
KUL	stuck: pop KUL	JFK	KUL
JFK	fly JFK->NRT	JFK NRT	KUL
NRT	fly NRT->JFK	JFK NRT JFK	KUL
JFK	stuck: pop JFK	JFK NRT	JFK KUL
NRT	stuck: pop NRT	JFK	NRT JFK KUL
JFK	stuck: pop JFK		JFK NRT JFK KUL

Reconstruct itinerary (lexical order) memorise

```
// Reconstruct itinerary (LC 332): each airport hands out tickets smallest first.
static List<String> findItinerary(List<List<String>> tickets) {
    Map<String, PriorityQueue<String>> g = new HashMap<>();
    for (List<String> t : tickets)
        g.computeIfAbsent(t.get(0), k -> new PriorityQueue<>()).add(t.get(1));
    LinkedList<String> route = new LinkedList<>();
    Deque<String> st = new ArrayDeque<>();
    st.push("JFK");
    while (!st.isEmpty()) {
        PriorityQueue<String> out = g.get(st.peek());
        if (out != null && !out.isEmpty()) st.push(out.poll());
        else route.addFirst(st.pop());              // stuck: fix it at the front
    }
    return route;
}
```

Any directed graph, with the start-vertex rule

memorise

```
// Hierholzer: a path in directed graph g that uses every edge once, or null.
// Start at the vertex with out - in = 1 if there is one (else any with edges).
static List<Integer> eulerPath(List<Integer>[] g) {
    int n = g.length, edges = 0, start = -1, plus = 0, minus = 0;
    int[] in = new int[n];
    for (int u = 0; u < n; u++) for (int v : g[u]) { in[v]++; edges++; }
    for (int u = 0; u < n; u++) {
        int d = g[u].size() - in[u];
        if (d == 1) { plus++; start = u; }
        else if (d == -1) minus++;
        else if (d != 0) return null;
    }
    if (plus != minus || plus > 1) return null;     // degrees rule it out
    for (int u = 0; u < n && start == -1; u++) if (!g[u].isEmpty()) start = u;
    if (start == -1) return new ArrayList<>();      // no edges at all
    int[] next = new int[n];                        // next unused edge of u
    Deque<Integer> st = new ArrayDeque<>();
    List<Integer> path = new ArrayList<>();
    st.push(start);
    while (!st.isEmpty()) {
        int u = st.peek();
        if (next[u] < g[u].size()) st.push(g[u].get(next[u]++));   // walk on
        else path.add(st.pop());                    // stuck: u goes last
    }
    Collections.reverse(path);
    return path.size() == edges + 1 ? path : null;  // edges left: not connected
}
```

Cracking the safe (de Bruijn)

example

```
// Cracking the safe (LC 753): node = last n-1 digits as a number; typing d
// moves u to (u * k + d) % size. Each of the k^n passwords is one edge.
static String crackSafe(int n, int k) {
    int size = 1;
    for (int i = 1; i < n; i++) size *= k;          // k^(n-1) nodes
    int[] nextDigit = new int[size];
    Deque<int[]> st = new ArrayDeque<>();           // {node, digit typed}
    StringBuilder digits = new StringBuilder();
    st.push(new int[] {0, -1});
    while (!st.isEmpty()) {
        int u = st.peek()[0];
        if (nextDigit[u] < k) {
            int d = nextDigit[u]++;
            st.push(new int[] {(u * k + d) % size, d});
        } else {
            int d = st.pop()[1];
            if (d >= 0) digits.append((char) ('0' + d));
        }
    }
    for (int i = 1; i < n; i++) digits.append('0'); // start node 00..0
    return digits.reverse().toString();
}
```

Cost Time `O(n + m)` Space `O(n + m)`

Every edge is pushed and popped once. The `PriorityQueue` per airport adds O(m log m) for the lexical order.

Variants same idea, small change

- Valid arrangement of pairs (LC 2097): values go up to 10^9, so compress them to small indexes (**3.11 Coordinate compression** (p. 96)), call `eulerPath`, and read the pairs off consecutive path values.
- Undirected graph: all degrees even (circuit) or exactly two odd (start at one). Each edge sits in two adjacency lists, so mark it used by its id.
- Word or domino chains: each word is an edge from its first letter to its last, so there are only 26 nodes and the same code runs.

Traps bugs people actually write

- Adding a vertex to the answer when you first reach it, instead of when it is stuck, breaks at dead ends: the example gives JFK KUL NRT JFK, and KUL→NRT is not a ticket.
- *Always take the smallest ticket* without the stack gets stuck at KUL with two tickets unused.
- The start must be the vertex with out − in = 1. Starting at `pairs[0][0]` in LC 2097 sample 1 means starting at 5, and the walk uses only the pair [5, 1].
- Taking tickets with `list.remove(0)` on an `ArrayList` costs O(deg) each time. A `PriorityQueue`, or a sorted list and an index, keeps it fast.

Practise on LeetCode, in this order

☐	LC 332	Reconstruct Itinerary	Hard
☐	LC 2097	Valid Arrangement of Pairs	Hard
☐	LC 753	Cracking the Safe	Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

Part 10

Dynamic programming

Dynamic programming answers a question from the answers to smaller versions of it, and solves each of them once. Every pattern here is the same recipe with a different state: a position, a cell, a capacity, two prefixes, a range, a mask or a digit. Reach for it when you need a count, a minimum, a maximum or a yes/no over many choices, and no greedy rule is safe.

10.1	**The DP recipe (memo, table, rolling)** Best total or number of ways when each choice limits the next ones.	O(n)	p. 278
10.2	**Linear DP over a sequence** An answer for each prefix, built from the answers a few positions back.	O(n)	p. 280
10.3	**Grid DP (one row or in place)** Right and down moves only: count paths, cheapest path, largest square.	O(m·n)	p. 283
10.4	**0/1 knapsack (reverse loop)** Pick a subset, each item at most once, under a capacity or to an exact sum.	O(n·W)	p. 286
10.5	**Unbounded knapsack (forward loop)** Coins you can use any number of times: fewest coins or number of ways.	O(k·A)	p. 289
10.6	**Longest increasing subsequence** Longest increasing chain, not necessarily adjacent, or pairs that nest.	O(n log n)	p. 292
10.7	**Longest common subsequence** Two strings: the longest subsequence they share, and what follows from it.	O(m·n)	p. 294
10.8	**Edit distance and alignment** Turn one string into another with the fewest edits, or count or test the ways two strings line up.	O(m·n)	p. 297
10.9	**Palindrome DP on s[i..j]** Longest palindromic subsequence, number of palindromic substrings, or fewest cuts into palindromes.	$O(n^2)$	p. 300
10.10	**Interval DP (last split point)** Best way to burst, cut, merge or print a whole range, when each step's cost depends on its neighbours.	$O(n^3)$	p. 303
10.11	**Stock DP (hold and free states)** Buy and sell under rules (any number of trades, a fee, a cooldown, at most k trades): the best profit per state per day.	O(n·k)	p. 306
10.12	**Bitmask DP over subsets** Order or group at most about 20 items, when the state is the set of items used so far.	$O(2^n \cdot n)$	p. 309
10.13	**Digit DP (count numbers ≤ N)** Count the integers in [L, R] whose digits obey a rule, with R up to 10^{18} or a string of 100 digits.	O(L·S·10)	p. 313
10.14	**Partition into k contiguous groups** Cut an array or string into k contiguous pieces to minimise or maximise a total over the pieces.	$O(k \cdot n^2)$	p. 316

10.1 The DP recipe (memo, table, rolling)

O(n)

Spot it when the problem looks like this

- *Number of ways*, *minimum cost*, *maximum amount* or *is it possible*, over a sequence of choices.
- A choice now limits later choices (rob house i and you cannot rob house i + 1), and no fixed rule picks the right one.
- A plain recursion is easy to write but calls itself with the same arguments again and again.
- n up to 10^4 or 10^5: the 2^n choice sequences are far too many, but n states with O(1) work each fit.

How it works

Write five things down before any code. **State**: what `f(i)` means in words (most money from the first i houses). **Transition**: how `f(i)` follows from smaller states (skip house i–1 or rob it: `max(f(i-1), f(i-2) + a[i-1])`). **Base case**: `f(0) = 0`. **Order**: smaller i first. **Answer**: `f(n)`.

The plain recursion is correct but slow: `f(i-2)` is called from `f(i)` and again from `f(i-1)`, so for n = 40 it makes 535,828,591 calls. These are *overlapping subproblems*. A memo solves each state once: 81 calls. The table fills the same values in a loop with no recursion, and since `dp[i]` reads only the last two entries, two variables can replace the array.

Use DP when the best answer for i is built from best answers of smaller states (*optimal substructure*) and those states repeat (*overlapping subproblems*). Greedy needs more: a proof that one choice is always safe (6.5 **Jump game (farthest reach)** (p. 152)). On `[3, 4, 3]`, robbing the richest house first gets 4; the answer is 6.

Worked example rob(a), a = [2, 7, 9, 3, 1]: dp[i] = max(skip, rob)

```
i  a[i-1]  skip: dp[i-1]  rob: dp[i-2] + a[i-1]  dp[i]
0      -              -                      -      0
1      2              0                0 + 2 = 2      2
2      7              2                0 + 7 = 7      7
3      9              7               2 + 9 = 11     11
4      3             11               7 + 3 = 10     11
5      1             11              11 + 1 = 12     12
n = 40: plain recursion makes 535,828,591 calls; with the memo, 81.
```

Top-down: recursion + memo memorise

```java
// House robber, top-down. State: f(i) = most money from the first i houses.
// Transition: skip house i-1 -> f(i-1), or rob it -> f(i-2) + a[i-1].
static int robMemo(int[] a) {
    Integer[] memo = new Integer[a.length + 1];    // null = not solved yet
    return f(a, a.length, memo);                   // answer: f(n)
}

static int f(int[] a, int i, Integer[] memo) {
    if (i <= 0) return 0;                          // base: no houses, no money
    if (memo[i] != null) return memo[i];
    int best = Math.max(f(a, i - 1, memo), f(a, i - 2, memo) + a[i - 1]);
    return memo[i] = best;
}
```

Bottom-up table, then two variables

memorise

```java
// Bottom-up: the same recurrence, filled so dp[i-1] and dp[i-2] are ready.
static int robTable(int[] a) {
    int n = a.length;
    int[] dp = new int[n + 1];                    // dp[i] = f(i); dp[0] = 0
    if (n > 0) dp[1] = a[0];
    for (int i = 2; i <= n; i++)
        dp[i] = Math.max(dp[i - 1], dp[i - 2] + a[i - 1]);
    return dp[n];
}

// Rolling: dp[i] reads only the two values before it, so keep two numbers.
static int rob(int[] a) {
    int prev2 = 0, prev1 = 0;                     // dp[i-2], dp[i-1]
    for (int x : a) {
        int cur = Math.max(prev1, prev2 + x);
        prev2 = prev1;
        prev1 = cur;
    }
    return prev1;
}
```

Climbing stairs and min cost climbing stairs

example

```java
// Climbing stairs: ways(i) = ways(i-1) + ways(i-2); ways(0) = ways(1) = 1.
static int climbStairs(int n) {
    int a = 1, b = 1;                             // ways(i-2), ways(i-1)
    for (int i = 2; i <= n; i++) {
        int c = a + b;
        a = b;
        b = c;
    }
    return b;
}

// Min cost climbing stairs: pay cost[i] to step up 1 or 2 from step i.
// reach(i) = min(reach(i-1) + cost[i-1], reach(i-2) + cost[i-2]).
static int minCostClimbingStairs(int[] cost) {
    int a = 0, b = 0;                             // reach(0) = reach(1) = 0
    for (int i = 2; i <= cost.length; i++) {
        int c = Math.min(b + cost[i - 1], a + cost[i - 2]);
        a = b;
        b = c;
    }
    return b;                                     // reach(n): the top
}
```

Cost Time `O(n)` Space `O(1)`

Each of the n + 1 states is solved once in O(1). The memo and the table take O(n) memory; the rolling version keeps two numbers.

Variants same idea, small change

- Memo as `long[]` filled with –1: use it when answers need `long`. It only works if –1 is never a real answer.
- N-th Tribonacci: the rolling version with three variables instead of two.
- House robber on a binary tree: the state is a node plus *robbed or not* (8.11 **Tree DP (take or skip a node)** (p. 213)).
- Two or more values per position (holding a stock or not, in cooldown): a state machine, see 10.11 **Stock DP (hold and free states)** (p. 306).

Traps bugs people actually write

- An `int[]` memo with `0` for *not solved* cannot tell it from the answer `0`. On `30` houses of value `0` it makes 4,356,617 calls, as many as no memo. Use `Integer[]` and `null`, or fill with –1.
- `prev1 = cur; prev2 = prev1;` updates in the wrong order and sets both to `cur`: `[2, 7, 9, 3, 1]` gives 22 instead of 12. Move `prev2` first.
- The memo version recurses n deep. On a default Java stack, n = `5000` already threw `StackOverflowError` in a test run. When n is large, use the table.
- Climbing stairs grows like Fibonacci: n = 45 gives 1,836,311,903, the last value that fits in `int`, and n = 46 wraps to –1,323,752,223. Use `long` or the modulus the problem gives.

Practise on LeetCode, in this order

☐	`LC 70`	Climbing Stairs	Easy
☐	`LC 1137`	N-th Tribonacci Number	Easy
☐	`LC 746`	Min Cost Climbing Stairs	Easy
☐	`LC 198`	House Robber	Medium

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

10.2 Linear DP over a sequence `O(n)`

Spot it when the problem looks like this

- A string or array where the answer for the first i elements depends on how the last piece ends: one digit or two, which word, rob or skip.
- *Number of ways to decode*, *can s be segmented into words*, houses *arranged in a circle*, *deleting x also deletes x – 1 and x + 1*.
- n up to 10^5, and each position looks back at only a few earlier positions (or at most L, the longest word).

How it works

Let `dp[i]` be the answer for the first i elements (`dp[0]` is the empty prefix) and ask how the prefix can end. In decode ways the last letter is one digit (`1`–`9`) or two (`10`–`26`), so `dp[i] = dp[i-1] + dp[i-2]`,

each term only when its digits form a letter. A `0` cannot stand alone: it must pair with a 1 or 2 before it, and after any other digit the count drops to 0.

Word break asks the same question: the last word is `s[j..i)` for some j with `ok[j]`. Only j with `i - j ≤ L`, the longest word length, can work, so the inner loop is short.

Two problems reduce to house robber. In a circle, houses 0 and n−1 are neighbours, so every answer skips at least one: take the better of two line runs. In delete and earn, taking v forbids every v−1 and v+1 but no other copy of v, so take all copies: total the points per value and rob the values.

Worked example **numDecodings("11206")**

```
i  s[i-1]  one digit         two digits          dp[i]
0       -                                          1
1       1  1 ok: +dp[0] = 1  -                      1
2       1  1 ok: +dp[1] = 1  11 ok: +dp[0] = 1     2
3       2  2 ok: +dp[2] = 2  12 ok: +dp[1] = 1     3
4       0  0: no             20 ok: +dp[2] = 2     2
5       6  6 ok: +dp[4] = 2  06: no                2
"12305": dp = [1, 1, 2, 3, 0, 0]. "30" is no letter, so nothing ends at the 0.
```

Decode ways (zeros included)

memorise

```java
// Decode ways ('1' -> A ... '26' -> Z).
// dp[i] = number of ways to decode the first i characters.
static int numDecodings(String s) {
    int n = s.length();
    int[] dp = new int[n + 1];
    dp[0] = 1;                                      // empty prefix: one way
    for (int i = 1; i <= n; i++) {
        char c = s.charAt(i - 1);
        if (c != '0') dp[i] += dp[i - 1];           // last letter is one digit
        if (i >= 2) {
            int two = (s.charAt(i - 2) - '0') * 10 + (c - '0');
            if (two >= 10 && two <= 26) dp[i] += dp[i - 2];   // two digits
        }
    }
    return dp[n];
}
```

Word break (set and longest word)

memorise

```java
// Word break: ok[i] = the first i characters split into dictionary words.
static boolean wordBreak(String s, List<String> words) {
    Set<String> dict = new HashSet<>(words);
    int maxLen = 0;
    for (String w : words) maxLen = Math.max(maxLen, w.length());
    int n = s.length();
    boolean[] ok = new boolean[n + 1];
    ok[0] = true;
    for (int i = 1; i <= n; i++)                    // last word is s[j..i)
        for (int j = i - 1; j >= Math.max(0, i - maxLen) && !ok[i]; j--)
            if (ok[j] && dict.contains(s.substring(j, i))) ok[i] = true;
    return ok[n];
}
```

House robber II and delete and earn

variant

```java
// House robber II: houses in a circle, so house 0 and house n-1 touch.
// An answer skips house 0 or skips house n-1: run the line version twice.
static int robCircle(int[] a) {
    int n = a.length;
    if (n == 1) return a[0];
    return Math.max(robLine(a, 0, n - 2), robLine(a, 1, n - 1));
}

static int robLine(int[] a, int lo, int hi) {     // house robber on a[lo..hi]
    int prev2 = 0, prev1 = 0;
    for (int i = lo; i <= hi; i++) {
        int cur = Math.max(prev1, prev2 + a[i]);
        prev2 = prev1;
        prev1 = cur;
    }
    return prev1;
}

// Delete and earn: taking value v deletes every v-1 and v+1. Total the
// points per value; then values are houses in a row.
static int deleteAndEarn(int[] nums) {
    int max = 0;
    for (int x : nums) max = Math.max(max, x);
    int[] points = new int[max + 1];
    for (int x : nums) points[x] += x;
    return robLine(points, 0, max);
}
```

Cost Time `O(n)` Space `O(n)`

Decode ways and both robbers are one pass. Word break tries at most L ends per position, each a substring and hash of length ≤ L, so $O(n \cdot L^2)$. Delete and earn adds O(max value) for the points array.

Variants same idea, small change

- Maximum subarray sum with one deletion: two states per i. `keep` is Kadane (1.7 **Kadane's maximum subarray** (p. 34)); `del = max(del + a[i], keep)`, with `keep` from before `a[i]`: the deletion was earlier, or it is `a[i]`.
- Solving questions with brainpower: fill from the back, `dp[i] = max(dp[i+1], points[i] + dp[i+1+skip[i]])`, past the end counting as 0. Use `long`: the total can reach 10^{10}.
- Decode ways II (`*` is any of 1–9): the same two terms, each multiplied by how many digits or pairs the stars allow, mod $10^9 + 7$.
- Word break II (list every sentence) is a search (7.7 **Cut a prefix, recurse on the rest** (p. 178)); `ok[]` from this table prunes the dead branches.

Traps bugs people actually write

- Testing the pair with only `two <= 26` lets "06" count as a letter: `"106"` gives 2 instead of 1. The pair must be in 10–26.
- House robber II without the `n == 1` case: both runs skip the only house, so `[5]` returns 0.
- Greedy word break fails both ways: on "goalspecial" with `[go, goal, goals, special]`, the shortest first match (go) and the longest (goals) both get stuck; goal + special works.
- Robbing the sorted distinct values as a line makes 2 and 4 neighbours: `[2, 4]` gives 4, but they do not conflict and the answer is 6. Index `points` by value so the gaps stay.

Practise on LeetCode, in this order

☐	LC 213	House Robber II	Medium
☐	LC 740	Delete and Earn	Medium
☐	LC 2140	Solving Questions With Brainpower	Medium
☐	LC 983	Minimum Cost For Tickets	Medium
☐	LC 91	Decode Ways	Medium
☐	LC 139	Word Break	Medium

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

10.3 Grid DP (one row or in place)

`O(m·n)`

Spot it when the problem looks like this

- You may move only **right or down** (or down a triangle), so no path can come back to a cell.
- *Number of unique paths*, *minimum path sum*, *largest square of 1s*, *minimum initial health*.
- m and n up to 100 or 200, so O(m·n) cells with O(1) work each is the target.

How it works

A cell (r, c) is entered from (r–1, c) above or from (r, c–1) on the left, so filling row by row, left to right, has both ready. Paths: `ways = up + left`, and a rock is 0. Cheapest path: `g[r][c] += min(up, left)`, in place. One row is enough: before `ways[c]` is updated it still holds the row above, while `ways[c-1]` already holds the current row.

Maximal square: a square of side k ends at (r, c) exactly when squares of side k – 1 end above, to the left and on the diagonal. So `side = 1 + min(up, left, diag)`.

Dungeon game runs backwards. Forward, a cell would need two numbers, the health needed so far and the health left, and the way in that needs less may have less left, so neither number picks the better way in. Backward, one number describes the rest of the path:
`need = max(1, min(need below, need right) - d[r][c])`.

Worked example LC samples for minPathSum (in place) and calculateMinimumHP

```
minPathSum                    calculateMinimumHP
grid       cost to reach      d             need on entering
 1  3  1    1  4  5           -2  -3   3     7  5  2
 1  5  1    2  7  6           -5 -10   1     6 11  5
 4  2  1    6  8  7           10  30  -5     1  1  6
top-left first: answer 7      bottom-right first: answer 7
```

Unique paths II (one row), min path sum (in place)

memorise

```java
// Unique paths II (moves: right or down; 1 = rock). ways[c] is one row:
// before the update it holds the row above, after it the current row.
static int uniquePathsWithObstacles(int[][] g) {
    int n = g[0].length;
    int[] ways = new int[n];
    ways[0] = 1;                                    // one way to stand on (0, 0)
    for (int[] row : g)
        for (int c = 0; c < n; c++) {
            if (row[c] == 1) ways[c] = 0;           // nobody passes a rock
            else if (c > 0) ways[c] += ways[c - 1];    // from above + from left
        }
    return ways[n - 1];
}

// Minimum path sum, in place: g[r][c] becomes the cheapest cost to reach it.
static int minPathSum(int[][] g) {
    int m = g.length, n = g[0].length;
    for (int r = 0; r < m; r++)
        for (int c = 0; c < n; c++) {
            if (r == 0 && c == 0) continue;
            int up = r > 0 ? g[r - 1][c] : Integer.MAX_VALUE;
            int left = c > 0 ? g[r][c - 1] : Integer.MAX_VALUE;
            g[r][c] += Math.min(up, left);
        }
    return g[m - 1][n - 1];
}
```

Maximal square

variant

```java
// Maximal square: side[r][c] = largest all-'1' square whose bottom-right
// corner is cell (r-1, c-1). Row 0 and column 0 are padding.
static int maximalSquare(char[][] g) {
    int m = g.length, n = g[0].length, best = 0;
    int[][] side = new int[m + 1][n + 1];
    for (int r = 1; r <= m; r++)
        for (int c = 1; c <= n; c++)
            if (g[r - 1][c - 1] == '1') {
                side[r][c] = 1 + Math.min(side[r - 1][c - 1],
                                 Math.min(side[r - 1][c], side[r][c - 1]));
                best = Math.max(best, side[r][c]);
            }
    return best * best;
}
```

Dungeon game, filled backwards

variant

```java
// Dungeon game: need[r][c] = least health on entering (r, c) that still
// reaches the end alive. It depends on the cells after, so fill backwards.
static int calculateMinimumHP(int[][] d) {
    int m = d.length, n = d[0].length;
    int[][] need = new int[m + 1][n + 1];          // row m, column n: walls
    for (int[] row : need) Arrays.fill(row, Integer.MAX_VALUE);
    need[m][n - 1] = need[m - 1][n] = 1;           // leave the end with 1
    for (int r = m - 1; r >= 0; r--)
        for (int c = n - 1; c >= 0; c--) {
            int next = Math.min(need[r + 1][c], need[r][c + 1]);
            need[r][c] = Math.max(1, next - d[r][c]);
        }
    return need[0][0];
}
```

Cost Time `O(m·n)` Space `O(n)`

Each cell is filled once from at most three neighbours. Paths keeps one row, min path sum reuses the grid, and the other two keep a padded (m + 1) × (n + 1) table.

Variants same idea, small change

- Triangle: fill from the bottom row up, `t[r][c] += min(t[r+1][c], t[r+1][c+1])`; the top holds the answer and there are no edge cases.
- Minimum falling path sum: each cell takes the min of the three cells above it (up-left, up, up-right).
- Count square submatrices of 1s: the same `side` table; each corner adds `side[r][c]` squares, so sum them.
- Moves in all four directions allow cycles, so no fill order exists. Use a shortest-path search: **9.2 BFS shortest path (unweighted)** (p. 226) or **9.10 Dijkstra's shortest paths** (p. 250).

Traps bugs people actually write

- Padding with 0 instead of infinity for min path sum lets every border cell pretend a free cell sits outside: the LC sample returns 3 instead of 7.
- Filling the first row and column with 1 for unique paths ignores rocks there: `[[0, 1, 0]]` gives 1 instead of 0. Everything after a rock in that row or column is 0.
- Dungeon game forward, keeping the way in that needed less health so far: on `[[0, -1, 2], [0, 0, -2]]` it answers 3, but the path along the top row needs only 2.
- Maximal square asks for the area: return `best * best`, not the side.

Practise on LeetCode, in this order

- [ ] LC 62 Unique Paths — Medium
- [ ] LC 64 Minimum Path Sum — Medium
- [ ] LC 63 Unique Paths II — Medium
- [ ] LC 120 Triangle — Medium
- [ ] LC 221 Maximal Square — Medium
- [ ] LC 174 Dungeon Game — Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

10.4 0/1 knapsack (reverse loop)

`O(n·W)`

Spot it when the problem looks like this

- Each item is used **at most once**: a subset with total weight ≤ W, or a sum of exactly S.
- *Partition into two subsets with equal sum*, *put + or – before each number*, *smash stones together*, *at most m zeros and n ones*.
- The capacity or the sum is small (up to about 10^4), while 2^n subsets is far too many.

How it works

Let `best[i][c]` be the most value from the first i items within weight c. Item i is skipped or taken once: `max(best[i-1][c], best[i-1][c-w] + v)`. Row i reads only row i – 1, so one array can hold both.

The loop over c must go **down**. Then `best[c - w]` is not updated for this item yet and still means *without item i*. Going up, `best[c - w]` may already contain item i, which is then taken twice: that is 10.5 **Unbounded knapsack (forward loop)** (p. 289).

The same loop answers yes/no (`can[s - x]` sets `can[s]`) and counts (`ways[s] += ways[s - x]`). Target sum is a count: the plus set P and the minus set N have `P - N = target` and `P + N = total`, so count subsets with sum `(total + target) / 2`. Two capacities give a 2D array with both loops reversed. A yes/no row is a row of bits: shifting it left by x and OR-ing adds x to every reachable sum, 64 sums per `long`.

Worked example knapsack(w = [1, 3, 4, 5], v = [1, 4, 5, 7], cap = 7): best[c] after each item

```
c                0  1  2  3  4  5  6  7
w=1 v=1          0  1  1  1  1  1  1  1
w=3 v=4          0  1  1  4  5  5  5  5
w=4 v=5          0  1  1  4  5  6  6  9
w=5 v=7          0  1  1  4  5  7  8  9   best[7] = 9: w=3 and w=4
forward, w=1     0  1  2  3  4  5  6  7   item 0 taken up to 7 times
```

Knapsack and equal partition

memorise

```java
// 0/1 knapsack: each item at most once. best[c] = most value in weight <= c.
static int knapsack(int[] w, int[] v, int cap) {
    int[] best = new int[cap + 1];
    for (int i = 0; i < w.length; i++)
        // reverse: best[c - w[i]] is not updated yet, so it means "without i"
        for (int c = cap; c >= w[i]; c--)
            best[c] = Math.max(best[c], best[c - w[i]] + v[i]);
    return best[cap];
}

// Partition equal subset sum: can some subset reach half the total?
static boolean canPartition(int[] nums) {
    int total = 0;
    for (int x : nums) total += x;
    if (total % 2 == 1) return false;
    int half = total / 2;
    boolean[] can = new boolean[half + 1];         // can[s]: a subset sums to s
    can[0] = true;
    for (int x : nums)
        for (int s = half; s >= x; s--)
            if (can[s - x]) can[s] = true;
    return can[half];
}
```

Equal partition with a bitset

variant

```java
// The same with bits: bit s of the long words = "sum s is reachable".
// java.util.BitSet has no shift, so do bits |= bits << x by hand.
static boolean canPartitionBits(int[] nums) {
    int total = 0;
    for (int x : nums) total += x;
    if (total % 2 == 1) return false;
    int half = total / 2;
    long[] bits = new long[half / 64 + 1];
    bits[0] = 1;                                   // sum 0
    for (int x : nums) {
        int ws = x / 64, bs = x % 64;              // whole words, then bits
        for (int i = bits.length - 1; i >= ws; i--) {     // high words first
            bits[i] |= bits[i - ws] << bs;
            if (bs > 0 && i > ws) bits[i] |= bits[i - ws - 1] >>> (64 - bs);
        }
    }
    return (bits[half / 64] >>> (half % 64) & 1) == 1;
}
```

Target sum and ones and zeroes

variant

```java
// Target sum: the '+' numbers P and the '-' numbers N give P - N = target and
// P + N = total, so P = (total + target) / 2. Count subsets with sum P.
static int findTargetSumWays(int[] nums, int target) {
    int total = 0;
    for (int x : nums) total += x;
    if (Math.abs(target) > total || (total + target) % 2 != 0) return 0;
    int p = (total + target) / 2;
    int[] ways = new int[p + 1];
    ways[0] = 1;
    for (int x : nums)                              // x = 0 doubles every count
        for (int s = p; s >= x; s--) ways[s] += ways[s - x];
    return ways[p];
}

// Ones and zeroes: two capacities (m zeros, n ones), both loops reversed.
static int findMaxForm(String[] strs, int m, int n) {
    int[][] best = new int[m + 1][n + 1];
    for (String s : strs) {
        int zeros = 0, ones = 0;
        for (char ch : s.toCharArray()) if (ch == '0') zeros++; else ones++;
        for (int i = m; i >= zeros; i--)
            for (int j = n; j >= ones; j--)
                best[i][j] = Math.max(best[i][j], best[i - zeros][j - ones] + 1);
    }
    return best[m][n];
}
```

Cost Time `O(n·W)` Space `O(W)`

n items times W + 1 capacities, where W is the capacity or half the sum. The bitset does the same work in about W/64 word operations per item.

Variants same idea, small change

- Last stone weight II: the smashes split the stones into two piles and leave their difference. Find the largest reachable s ≤ total/2 and return `total - 2·s`.
- Target sum with an offset: `dp[s + total]` over every s in –total..total, each number sending counts to `s + x` and `s - x`. Same answer, twice the width.
- Profitable schemes: two dimensions again (members used, profit capped at `minProfit`), counts mod $10^9 + 7$.
- n ≤ 40 with huge weights: no table fits, so split the items in half and combine (**7.8 Meet in the middle** (p. 181)).

Traps bugs people actually write

- A forward loop, `for (int s = x; s <= half; s++)`, reuses a number: `[1, 2, 5]` reports true (1 + 1 + 1 + 1 = 4) although no subset sums to 4.
- Target sum without the `Math.abs(target) > total` check: `[1, 1]` with target −4 gives `p = -1`, so `new int[0]` and then `ways[0] = 1` throws.
- Skipping zeros loses answers: each 0 can take + or −, so `[0, 0, 1]` with target 1 has 4 ways. With x = 0 the loop `s >= x` runs down to s = 0 and doubles every count, which is right.

Practise on LeetCode, in this order

- [ ] LC 416 Partition Equal Subset Sum — Medium
- [ ] LC 494 Target Sum — Medium
- [ ] LC 1049 Last Stone Weight II — Medium
- [ ] LC 474 Ones and Zeroes — Medium
- [ ] LC 879 Profitable Schemes — Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

10.5 Unbounded knapsack (forward loop)

`O(k·A)`

Spot it when the problem looks like this

- *An infinite number of each kind of coin*, *you may reuse* an element, *any number of times*.
- *Fewest coins*, *number of combinations*, or a *number of combinations* where different orders count as different (LC 377).
- The amount is small (up to about 10^4), so an array indexed by amount fits.

How it works

`dp[a]` describes the amount a. When every coin can be used again, the last coin of a solution for a can be any coin c, and the rest is a solution for a − c that may use c too. So the inner loop goes **up**: `dp[a - c]` is already updated for coin c, which is the reuse. This is the one line that differs from **10.4 0/1 knapsack (reverse loop)** (p. 286).

For fewest coins, unreachable amounts hold `INF = amount + 1`: more than any real answer (at most `amount` coins of value 1), and safe to add 1 to.

For counting, the loop order decides what is counted. **Coins outside, amounts inside counts combinations**: coin c is added only after the earlier coins are done, so every multiset is built in one fixed coin order and counted once. **Amounts outside, coins inside counts sequences**: for each amount every coin may come last, so (1, 2) and (2, 1) both count.

Worked example nums = [1, 2, 3], amount 4: the loop order changes the count

```
a                    0  1  2  3  4
coins outer: start   1  0  0  0  0
  after coin 1       1  1  1  1  1
  after coin 2       1  1  2  2  3
  after coin 3       1  1  2  3  4   combinations: 1111 112 13 22
amount outer         1  1  2  4  7   sequences: 1111 112 121 13 211 22 31
```

Coin change (fewest coins)

memorise

```java
// Coin change: fewest coins that make amount; every coin can be used again.
static int coinChange(int[] coins, int amount) {
    final int INF = amount + 1;                    // more coins than any answer
    int[] dp = new int[amount + 1];                // dp[a] = fewest coins for a
    Arrays.fill(dp, INF);
    dp[0] = 0;
    for (int coin : coins)
        for (int a = coin; a <= amount; a++)       // forward: reuse allowed
            dp[a] = Math.min(dp[a], dp[a - coin] + 1);
    return dp[amount] == INF ? -1 : dp[amount];
}
```

Combinations vs sequences

memorise

```java
// Coin change II: number of combinations ({1, 2} and {2, 1} are the same).
// Coins outer: each combination is built in coin order, so it counts once.
static int change(int amount, int[] coins) {
    int[] ways = new int[amount + 1];
    ways[0] = 1;
    for (int coin : coins)
        for (int a = coin; a <= amount; a++) ways[a] += ways[a - coin];
    return ways[amount];
}

// Combination sum IV: number of sequences ((1, 2) and (2, 1) differ).
// Amount outer: for each total, try every number as the last one.
static int combinationSum4(int[] nums, int target) {
    int[] ways = new int[target + 1];
    ways[0] = 1;
    for (int a = 1; a <= target; a++)
        for (int x : nums)
            if (x <= a) ways[a] += ways[a - x];
    return ways[target];
}
```

Loop order rule: coins outside → combinations (LC 518); amount outside → sequences (LC 377).

Perfect squares

example

```java
// Perfect squares: coin change with coins 1, 4, 9, ... up to n.
static int numSquares(int n) {
    int[] dp = new int[n + 1];
    for (int a = 0; a <= n; a++) dp[a] = a;        // a ones: always possible
    for (int s = 2; s * s <= n; s++)
        for (int a = s * s; a <= n; a++)
            dp[a] = Math.min(dp[a], dp[a - s * s] + 1);
    return dp[n];
}
```

Cost Time `O(k·A)` Space `O(A)`

For k coins and amount A, each (coin, amount) pair is one O(1) step. Perfect squares has √n coins, so O(n·√n).

Variants same idea, small change

- Rod cutting or any reusable piece with a price: the same forward loop with `max` instead of `min`.
- Perfect squares by BFS: amounts are nodes and each square is an edge; the level where BFS first reaches 0 is the answer (**9.2 BFS shortest path (unweighted)** (p. 226)).
- Largest number from digit costs (LC 1449): first the most digits for each cost (a max unbounded knapsack), then build from the left, taking the largest digit that keeps the count at that maximum.
- List the combinations instead of counting them: that is backtracking (**7.4 Combinations from a start index** (p. 169)).

Traps bugs people actually write

- `Arrays.fill(dp, Integer.MAX_VALUE)` and then `dp[a - coin] + 1` overflows: coins `[2]`, amount 3 gives `dp[3] = -2147483648`, which beats every real answer. Use `amount + 1`.
- Swapping the loops in coin change II counts orders: coins `[1, 2, 5]`, amount 5 gives 9 instead of 4.
- The reverse loop copied from **10.4 0/1 knapsack (reverse loop)** (p. 286) uses each coin once: coins `[1, 2, 5]`, amount 11 gives –1 instead of 3.
- LC 377 promises only that the final answer fits in an `int`; a smaller amount can have far more sequences (about $3 \cdot 10^{29}$ on one 100-number input). Java's `int` wraps mod 2^{32}, so the final sum still comes out right, but never compare or `min` such values.

Practise on LeetCode, in this order

- [ ] LC 322 Coin Change — Medium
- [ ] LC 279 Perfect Squares — Medium
- [ ] LC 518 Coin Change II — Medium
- [ ] LC 377 Combination Sum IV — Medium
- [ ] LC 1449 Form Largest Integer With Digits That Add up to Target — Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

10.6 Longest increasing subsequence

`O(n log n)`

Spot it when the problem looks like this

- *Subsequence* (any elements, order kept) that is *strictly increasing* or *non-decreasing*.
- n up to 2500 allows the $O(n^2)$ dp; n up to 10^5 needs O(n log n).
- Pairs that must nest in both dimensions (envelopes, boxes): sort by one, LIS on the other.
- *Number of* longest increasing subsequences, or the *fewest removals* to sort (n – LIS).

How it works

$O(n^2)$: `len[i]`, the LIS that ends at i, is 1 + the largest `len[j]` with `j < i` and `a[j] < a[i]`.

O(n log n) keeps `tails[k]`, the smallest last value of any increasing subsequence of length k + 1. A smaller last value is never worse: whatever extends a larger one also extends it. `tails` is strictly increasing, so binary search for the first `tails[k] >= x` (3.1 **Lower bound and upper bound** (p. 68)). If none exists, x makes the longest one longer; otherwise x becomes the smaller end for length k + 1. The answer is `len`, the number of entries. For non-decreasing, an equal value may extend, so search for the first `tails[k] > x` (upper bound).

Envelopes: sort by width ascending and, for equal widths, height **descending**. Equal widths cannot nest, and with heights descending no two of them form an increasing pair. A strict LIS on the heights then uses strictly larger widths too.

Worked example **lis(a, true), a = [10, 9, 2, 5, 3, 7, 101, 18]**

```
x    pos  action   tails
10     0  extend   [10]
9      0  replace  [9]
2      0  replace  [2]
5      1  extend   [2, 5]
3      1  replace  [2, 3]
7      2  extend   [2, 3, 7]
101    3  extend   [2, 3, 7, 101]
18     3  replace  [2, 3, 7, 18]     length 4
```

LIS in O(n log n), strict or non-decreasing

memorise

```java
// tails[k] = smallest last value of a valid subsequence of length k + 1.
// strict (a[i] < a[j]): x replaces the first tails[k] >= x (lower bound);
// non-decreasing (a[i] <= a[j]): the first tails[k] > x (upper bound).
static int lis(int[] a, boolean strict) {
    int[] tails = new int[a.length];
    int len = 0;
    for (int x : a) {
        int lo = 0, hi = len;
        while (lo < hi) {
            int mid = (lo + hi) >>> 1;
            if (tails[mid] < x || (!strict && tails[mid] == x)) lo = mid + 1;
            else hi = mid;
        }
        tails[lo] = x;                              // smaller end for length lo+1
        if (lo == len) len++;                       // or a new, longer one
    }
    return len;
}
```

O(n²) dp and the number of LIS

variant

```java
// O(n^2): len[i] = longest strictly increasing subsequence ending at i,
// cnt[i] = how many subsequences of that length end at i.
static int findNumberOfLIS(int[] a) {
    int n = a.length, best = 0, total = 0;
    int[] len = new int[n], cnt = new int[n];
    for (int i = 0; i < n; i++) {
        len[i] = cnt[i] = 1;                        // a[i] alone
        for (int j = 0; j < i; j++) {
            if (a[j] >= a[i]) continue;
            if (len[j] + 1 > len[i]) { len[i] = len[j] + 1; cnt[i] = cnt[j]; }
            else if (len[j] + 1 == len[i]) cnt[i] += cnt[j];
        }
        if (len[i] > best) { best = len[i]; total = cnt[i]; }
        else if (len[i] == best) total += cnt[i];
    }
    return total;
}
```

Russian doll envelopes

variant

```java
// Russian doll envelopes: sort width ascending and, for equal widths, height
// descending; then the answer is the strict LIS of the heights.
static int maxEnvelopes(int[][] env) {
    Arrays.sort(env, (p, q) -> p[0] != q[0] ? Integer.compare(p[0], q[0])
                                            : Integer.compare(q[1], p[1]));
    int[] h = new int[env.length];
    for (int i = 0; i < env.length; i++) h[i] = env[i][1];
    return lis(h, true);
}
```

Cost Time `O(n log n)` Space `O(n)`

One binary search over at most n tails per element. Counting the LIS needs the $O(n^2)$ dp.

Variants same idea, small change

- Longest string chain, largest divisible subset: sort, then the $O(n^2)$ dp with another `j → i` test.
- Fewest removals to make a mountain: the LIS ending at each i from the left and from the right; the answer is the smallest `n - (left[i] + right[i] - 1)` over peaks where both are > 1.
- Print one LIS in O(n log n): store indices in `tails`, and when i is placed at k, set `parent[i]` to the index at `k - 1`.
- Fewest removals to leave a non-decreasing array: n – the non-decreasing LIS.

Traps bugs people actually write

- The lower bound for non-decreasing: `[7, 7, 7]` gives 1, and the answer is 3. Equal values need the upper bound.
- Equal widths sorted by height ascending: `[[1, 1], [1, 2]]` gives 2, but they cannot nest; the answer is 1.
- Number of LIS: returning `cnt[n-1]` instead of summing `cnt[i]` over every i with the best length. `[1, 2, 4, 3]` gives 1 instead of 2.
- Printing `tails` as the LIS: `[2, 5, 3, 7, 4, 1]` leaves `[1, 3, 4]`, but 1 comes last in the array.

Practise on LeetCode, in this order

☐	LC 300	Longest Increasing Subsequence	Medium
☐	LC 368	Largest Divisible Subset	Medium
☐	LC 1048	Longest String Chain	Medium
☐	LC 673	Number of Longest Increasing Subsequence	Medium
☐	LC 354	Russian Doll Envelopes	Hard
☐	LC 1671	Minimum Number of Removals to Make Mountain Array	Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

10.7 Longest common subsequence `O(m·n)`

Spot it when the problem looks like this

- Two sequences and the word *subsequence*: characters in order, not necessarily adjacent.
- *Fewest deletions to make two strings equal, shortest string that has both as subsequences, lines between equal numbers that do not cross.*
- Lengths up to 1000 each, so an (m + 1) × (n + 1) table of about 10^6 cells fits.

How it works

`dp[i][j]` is the LCS length of the prefixes `a[0..i)` and `b[0..j)`; row 0 and column 0 are the empty prefixes. Look at the last characters. If `a[i-1] == b[j-1]`, some LCS pairs them, so `dp[i][j] = dp[i-1][j-1] + 1`. If not, no common subsequence can pair them, so one of them can be dropped: `max(dp[i-1][j], dp[i][j-1])`.

To get one LCS, walk back from (m, n). A match is part of it; otherwise step to the neighbour with the larger value, which is where the length came from.

Deleting until the strings are equal leaves a common subsequence, so the fewest deletions is `m + n - 2·LCS`. A common supersequence writes every character once and the shared ones only once, so the shortest has `m + n - LCS` characters, and the same walk builds it. Uncrossed lines are an LCS of two arrays. Row i reads only row i – 1, so one row plus the saved diagonal gives O(n) space.

Worked example a = "abcbdab" (rows), b = "bdcaba" (columns); * = match on the walk back

dp	""	b	d	c	a	b	a
""	0	0	0	0	0	0	0
a	0	0	0	0	1	1	1
b	0	1*	1.	1	1	2	2
c	0	1	1	2*	2.	2	2
b	0	1	1	2	2	3*	3
d	0	1	2	2	2	3.	3
a	0	1	2	2	3	3	4*
b	0	1	2	2	3	4	4.

. = step without a match. LCS = "bcba"; deletions 13 - 8 = 5; SCS 13 - 4 = 9

The LCS table

memorise

```
// dp[i][j] = LCS length of a[0..i) and b[0..j). Row 0 and column 0 stand
// for an empty prefix, so they stay 0 and need no special case.
static int[][] lcsTable(String a, String b) {
    int m = a.length(), n = b.length();
    int[][] dp = new int[m + 1][n + 1];
    for (int i = 0; i < m; i++)
        for (int j = 0; j < n; j++)
            dp[i + 1][j + 1] = a.charAt(i) == b.charAt(j)
                    ? dp[i][j] + 1                              // use both
                    : Math.max(dp[i][j + 1], dp[i + 1][j]);     // drop one
    return dp;
}
```

Walk back: one LCS, shortest supersequence

memorise

```
// One LCS: walk back from (m, n). Equal chars are in it; otherwise step
// to the neighbour that kept the same length.
static String oneLcs(String a, String b) {
    int[][] dp = lcsTable(a, b);
    StringBuilder sb = new StringBuilder();
    int i = a.length(), j = b.length();
    while (i > 0 && j > 0) {
        if (a.charAt(i - 1) == b.charAt(j - 1)) {
            sb.append(a.charAt(--i));              // in the LCS
            j--;
        } else if (dp[i - 1][j] >= dp[i][j - 1]) i--;
        else j--;
    }
    return sb.reverse().toString();
}

// Shortest common supersequence: the same walk back, but every char is
// written; a shared (LCS) char is written once.
static String shortestCommonSupersequence(String a, String b) {
    int[][] dp = lcsTable(a, b);
    StringBuilder sb = new StringBuilder();
    int i = a.length(), j = b.length();
    while (i > 0 && j > 0) {
        if (a.charAt(i - 1) == b.charAt(j - 1)) {
            sb.append(a.charAt(--i));              // shared: written once
            j--;
        } else if (dp[i - 1][j] >= dp[i][j - 1]) sb.append(a.charAt(--i));
        else sb.append(b.charAt(--j));
    }
    while (i > 0) sb.append(a.charAt(--i));        // one side is left over
    while (j > 0) sb.append(b.charAt(--j));
    return sb.reverse().toString();
}
```

One row: uncrossed lines

variant

```
// Uncrossed lines = LCS of two int arrays, with one row of the table.
// diag saves dp[i][j], the old row[j] that the step before overwrote.
static int maxUncrossedLines(int[] a, int[] b) {
    int[] row = new int[b.length + 1];             // row i of the table
    for (int x : a) {
        int diag = 0;                              // dp[i][0]
        for (int j = 0; j < b.length; j++) {
            int up = row[j + 1];                   // dp[i][j + 1]
            row[j + 1] = x == b[j] ? diag + 1 : Math.max(up, row[j]);
            diag = up;
        }
    }
    return row[b.length];
}
```

Cost Time `O(m·n)` Space `O(m·n)`

One O(1) step per cell; the walk back adds O(m + n). The one-row version needs O(n) memory but cannot rebuild the string.

Variants same idea, small change

- Edit distance also allows a replace: the mismatch case becomes `1 + min` of three neighbours (10.8 **Edit distance and alignment** (p. 297)).
- Longest common substring (maximum length of repeated subarray): on a mismatch the cell is `0`, not a max, and the answer is the largest cell.
- Longest palindromic subsequence is the LCS of s and its reverse, though the interval form is more usual (10.9 **Palindrome DP on s[i..j]** (p. 300)).
- Minimum ASCII delete sum: the same table, but it stores the largest ASCII total kept instead of a length.

Traps bugs people actually write

- Checking the neighbours before the match in the walk back: for `"a"` and `"a"`, `dp[0][1] >= dp[1][0]` moves up and the LCS comes out empty.
- The one-row version with `row[j] + 1` on a match reads the new row, not the diagonal: `[1]` and `[1, 1]` gives 2 instead of 1.
- The supersequence walk without the two leftover loops drops the rest of one string: `"abac"`, `"cab"` gives "abac", which does not contain "cab".

Practise on LeetCode, in this order

☐	`LC 1143`	Longest Common Subsequence	Medium
☐	`LC 583`	Delete Operation for Two Strings	Medium
☐	`LC 1035`	Uncrossed Lines	Medium
☐	`LC 718`	Maximum Length of Repeated Subarray	Medium
☐	`LC 1092`	Shortest Common Supersequence	Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

10.8 Edit distance and alignment `O(m·n)`

Spot it when the problem looks like this

- *Minimum operations (insert, delete, replace) to convert word1 into word2.*
- Two strings of length up to 500 or 1000, so a table of about 10^6 cells fits.
- *Count the subsequences of s equal to t, is s3 an interleaving of s1 and s2.*
- Matching with `?` and `*` (wildcards) or `.` and `*` (regular expressions).

How it works

`dp[i][j]` is the fewest edits that turn the first i chars of a into the first j chars of b. Look at what happens to the last chars. If `a[i-1] == b[j-1]`, keep both for free: `dp[i-1][j-1]`. Otherwise the last edit is a replace (`dp[i-1][j-1] + 1`), a delete of `a[i-1]` (`dp[i-1][j] + 1`) or an insert of `b[j-1]` (`dp[i][j-1] + 1`). Every edit script ends in one of these cases, so the minimum of the three is optimal. The base row and column are the empty string: `dp[i][0] = i` deletes and `dp[0][j] = j` inserts.

Each cell reads three neighbours (up, left, up-left), so filling row by row is O(1) per cell and O(m·n) in total. Distinct subsequences, interleaving and wildcard matching use the same table with a different cell rule: add counts instead of taking a minimum, or OR two booleans.

Worked example **minDistance("horse", "ros") = 3**

```
row i = first i chars of "horse", column j = first j chars of "ros"
       ""  r  o  s
  ""    0  1  2  3
  h     1  1  2  3
  o     2  2  1  2
  r     3  2  2  2
  s     4  3  3  2
  e     5  4  4  3
dp[2][2]: o == o, so dp[1][1] = 1
dp[5][3]: e != s, 1 + min(replace 3, delete 2, insert 4) = 3
one script: replace h -> r (rorse), delete r (rose), delete e (ros)
```

Edit distance memorise

```java
// dp[i][j] = fewest edits that turn a[0..i) into b[0..j).
static int minDistance(String a, String b) {
    int m = a.length(), n = b.length();
    int[][] dp = new int[m + 1][n + 1];
    for (int i = 0; i <= m; i++) dp[i][0] = i;       // delete all i chars
    for (int j = 0; j <= n; j++) dp[0][j] = j;       // insert all j chars
    for (int i = 1; i <= m; i++)
        for (int j = 1; j <= n; j++) {
            if (a.charAt(i - 1) == b.charAt(j - 1)) {
                dp[i][j] = dp[i - 1][j - 1];         // last chars match: free
            } else {
                dp[i][j] = 1 + Math.min(dp[i - 1][j - 1],   // replace
                               Math.min(dp[i - 1][j],       // delete a[i-1]
                                        dp[i][j - 1]));     // insert b[j-1]
            }
        }
    return dp[m][n];
}
```

Distinct subsequences; interleaving string

variant

```java
// Distinct subsequences: ways[i][j] = ways to pick t[0..j) out of s[0..i).
static int numDistinct(String s, String t) {
    int m = s.length(), n = t.length();
    int[][] ways = new int[m + 1][n + 1];
    for (int i = 0; i <= m; i++) ways[i][0] = 1;     // empty t: one way
    for (int i = 1; i <= m; i++)
        for (int j = 1; j <= n; j++) {
            ways[i][j] = ways[i - 1][j];             // do not use s[i-1]
            if (s.charAt(i - 1) == t.charAt(j - 1))
                ways[i][j] += ways[i - 1][j - 1];    // use it as t[j-1]
        }
    return ways[m][n];
}

// Interleaving string: ok[i][j] = s3[0..i+j) interleaves s1[0..i), s2[0..j).
static boolean isInterleave(String s1, String s2, String s3) {
    int m = s1.length(), n = s2.length();
    if (m + n != s3.length()) return false;
    boolean[][] ok = new boolean[m + 1][n + 1];
    ok[0][0] = true;
    for (int i = 0; i <= m; i++)
        for (int j = 0; j <= n; j++) {
            if (i == 0 && j == 0) continue;
            char c = s3.charAt(i + j - 1);           // the char placed last
            ok[i][j] = (i > 0 && ok[i - 1][j] && s1.charAt(i - 1) == c)
                    || (j > 0 && ok[i][j - 1] && s2.charAt(j - 1) == c);
        }
    return ok[m][n];
}
```

Cells you never need can pass 2^{31} even when the answer fits. `int` wraps, and with only additions the final value is still right.

Wildcard matching

variant

```java
// Wildcard matching: '?' is any one char, '*' is any run, also empty.
static boolean isMatch(String s, String p) {
    int m = s.length(), n = p.length();
    boolean[][] dp = new boolean[m + 1][n + 1];      // s[0..i) matches p[0..j)
    dp[0][0] = true;
    for (int j = 1; j <= n; j++)                     // only stars match ""
        dp[0][j] = dp[0][j - 1] && p.charAt(j - 1) == '*';
    for (int i = 1; i <= m; i++)
        for (int j = 1; j <= n; j++) {
            char c = p.charAt(j - 1);
            if (c == '*') dp[i][j] = dp[i][j - 1]    // '*' matches nothing
                                   || dp[i - 1][j];  // '*' also takes s[i-1]
            else dp[i][j] = dp[i - 1][j - 1]
                          && (c == '?' || c == s.charAt(i - 1));
        }
    return dp[m][n];
}
```

Cost Time `O(m·n)` Space `O(m·n)`

One O(1) step per cell of the (m + 1) × (n + 1) table. Each row reads only the row above, so two rows bring the space down to O(n).

Variants same idea, small change

- One row (O(n) space): keep the old up-left value in a variable. For distinct subsequences, loop j from n down to 1 so `ways[j-1]` is still the previous row's.
- Delete only, or delete with a cost per char (LC 583, LC 712): the same table with no replace; it is **10.7 Longest common subsequence** (p. 294) in disguise.
- Regular expression matching: `x*` is zero or more x, so a star looks two columns back: `dp[i][j] = dp[i][j-2] || (p[j-2] matches s[i-1] && dp[i-1][j])`.

Traps bugs people actually write

- No base row or column: with `dp[i][0]` left at 0, deleting a whole prefix is free, and `minDistance("horse", "ros")` returns 2 instead of 3.
- Wildcard base row `dp[0][j] = p.charAt(j - 1) == '*'` forgets that every earlier char must be a star too: `isMatch("", "a*")` returns true.
- Interleaving without the `m + n == s3.length()` check: the table looks only at the first m + n chars, so `("a", "b", "abc")` returns true.
- The table has m + 1 rows, so row i compares `a.charAt(i - 1)`. Writing `a.charAt(i)` is off by one and throws at i = m.

Practise on LeetCode, in this order

☐	LC 72	Edit Distance	Medium
☐	LC 97	Interleaving String	Medium
☐	LC 115	Distinct Subsequences	Hard
☐	LC 44	Wildcard Matching	Hard
☐	LC 10	Regular Expression Matching	Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

10.9 Palindrome DP on s[i..j]

$O(n^2)$

Spot it when the problem looks like this

- *Longest palindromic subsequence*: chars may be skipped, so **11.1 Expand around the centre** (p. 320) does not apply.
- *Fewest cuts* into palindromes, or *fewest insertions* to make s one; n up to 1000 or 2000.
- The answer for `s[i..j]` depends on `s[i]`, `s[j]` and the ranges one char shorter.

How it works

`dp[i][j]` is the answer for `s[i..j]`. For the longest palindromic subsequence, if `s[i] == s[j]` the two ends go on the outside: `2 + dp[i+1][j-1]`. Taking both is never worse, because a palindrome that skips an end is at most two chars longer than the best one inside `s[i+1..j-1]`, and wrapping that one adds two. If the ends differ, one of them is unused: `max(dp[i+1][j], dp[i][j-1])`.

Every cell reads ranges that are one or two chars shorter, so fill i from n – 1 down to 0 (or by increasing length). That is $n^2/2$ cells at O(1) each.

The boolean table `isPal[i][j]` uses the same order, and *count palindromic substrings* counts its true cells. For the fewest cuts, `cuts[j]` tries every last piece `s[i..j]` that is a palindrome: `cuts[i-1] + 1`, or 0 when i = 0. The best split of `s[0..i-1]` does not depend on the last piece. Both are $O(n^2)$.

Worked example **longestPalindromeSubseq("bbbab") = 4**

```
row i, column j; "." is below the diagonal and reads as 0 (empty range)
         0b 1b 2b 3a 4b
  0 b     1  2  3  3  4
  1 b     .  1  2  2  3
  2 b     .  .  1  1  3
  3 a     .  .  .  1  1
  4 b     .  .  .  .  1
dp[1][4]: s[1] = s[4] = b, so 2 + dp[2][3] = 2 + 1 = 3
dp[0][3]: b != a, so max(dp[1][3], dp[0][2]) = max(2, 3) = 3
minCut("aabba"): cuts = [0, 0, 1, 1, 1]; cuts[4] = cuts[0] + 1 via "abba"
```

Longest palindromic subsequence

memorise

```java
// Longest palindromic subsequence. dp[i][j] = answer for s[i..j].
// i goes down, so row i + 1 (shorter ranges) is ready when row i reads it.
static int longestPalindromeSubseq(String s) {
    int n = s.length();
    int[][] dp = new int[n][n];
    for (int i = n - 1; i >= 0; i--) {
        dp[i][i] = 1;
        for (int j = i + 1; j < n; j++) {
            if (s.charAt(i) == s.charAt(j)) dp[i][j] = 2 + dp[i + 1][j - 1];
            else dp[i][j] = Math.max(dp[i + 1][j], dp[i][j - 1]);
        }
    }
    return dp[0][n - 1];
}
```

Count palindromic substrings

variant

```java
// isPal[i][j]: s[i..j] is a palindrome. Count the true cells (LC 647).
static int countSubstrings(String s) {
    int n = s.length(), count = 0;
    boolean[][] isPal = new boolean[n][n];
    for (int i = n - 1; i >= 0; i--)
        for (int j = i; j < n; j++) {
            isPal[i][j] = s.charAt(i) == s.charAt(j)
                          && (j - i < 2 || isPal[i + 1][j - 1]);
            if (isPal[i][j]) count++;
        }
    return count;
}
```

Palindrome partitioning II (fewest cuts)

variant

```java
// Palindrome partitioning II. cuts[j] = fewest cuts for s[0..j].
// Column j of isPal only reads column j - 1, so both fill left to right.
static int minCut(String s) {
    int n = s.length();
    boolean[][] isPal = new boolean[n][n];
    int[] cuts = new int[n];
    for (int j = 0; j < n; j++) {
        cuts[j] = j;                                    // one char per piece
        for (int i = 0; i <= j; i++) {
            isPal[i][j] = s.charAt(i) == s.charAt(j)
                          && (j - i < 2 || isPal[i + 1][j - 1]);
            if (isPal[i][j])                            // last piece is s[i..j]
                cuts[j] = Math.min(cuts[j], i == 0 ? 0 : cuts[i - 1] + 1);
        }
    }
    return cuts[n - 1];
}
```

Cost Time $O(n^2)$ Space $O(n^2)$

$n^2/2$ cells at O(1) each; the cut loop tries every last piece, also $O(n^2)$. LPS can keep two rows for O(n) space.

Variants same idea, small change

- Minimum insertions (or deletions) to make s a palindrome: n – LPS. Every char outside the longest palindromic subsequence needs a partner.
- Palindrome partitioning IV (exactly three palindromes): fill `isPal`, then try every pair of cut points in $O(n^2)$.
- Longest palindromic *substring*: the boolean table works in $O(n^2)$ space, 11.1 **Expand around the centre** (p. 320) in O(1) space, 11.6 **Manacher's algorithm** (p. 335) in O(n) time.
- Every partition, not the fewest cuts: 7.7 **Cut a prefix, recurse on the rest** (p. 178).

Traps bugs people actually write

- Filling i from 0 upwards: row i reads row i + 1 before it is filled, and `"bbbab"` gives 2 instead of 4.
- `j == i` instead of `j - i < 2`: for j = i + 1 it reads `isPal[i+1][i]`, which is never set, so `"aaa"` counts 4 instead of 6.
- The subsequence recurrence on a substring question: `"abcda"` has a palindromic subsequence of length 3 (aca), but no palindromic substring longer than 1.
- Min cut: when the whole prefix `s[0..j]` is a palindrome the answer is 0 cuts. Without the `i == 0 ? 0` case you read `cuts[-1]`.

Practise on LeetCode, in this order

- [ ] `LC 647` Palindromic Substrings — Medium
- [ ] `LC 516` Longest Palindromic Subsequence — Medium
- [ ] `LC 1312` Minimum Insertion Steps to Make a String Palindrome — Hard
- [ ] `LC 1745` Palindrome Partitioning IV — Hard
- [ ] `LC 132` Palindrome Partitioning II — Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

10.10 Interval DP (last split point)

`O(n³)`

Spot it when the problem looks like this

- Removing an element makes its neighbours adjacent, so the cost of a step depends on the order of the steps.
- *Minimum cost to cut*, *triangulate*, *merge* or *print*, with n ≤ 100 to 500, so $O(n^3)$ fits.
- Greedy orders fail on small cases, but once one step is fixed, the parts to its left and right no longer affect each other.

How it works

`dp[l][r]` is the best result for the range between l and r. Choose the step that splits the range into two independent halves. For burst balloons that is the balloon k burst **last** in `(l, r)`. Until then it stands between the two sides, so they never touch, and when it bursts its neighbours are exactly l and r: `dp[l][k] + a[l]·a[k]·a[r] + dp[k][r]`. Taking k as the *first* balloon fails, because its neighbours then depend on the order and the two sides join after it bursts.

Every choice reads strictly shorter ranges, so loop over the length first (or l from right to left). n^2 ranges × n split points is $O(n^3)$; only triples l < k < r count, about $4.5 \cdot 10^6$ for n = 300.

Cutting a stick is the mirror image: the first cut in a piece costs the piece length and splits it in two. Padding with 1s (balloons) or with 0 and n (stick) gives the two ends a value.

Worked example maxCoins([3, 1, 5, 8]) = 167

```
pad = [1, 3, 1, 5, 8, 1]; dp[l][r] for r - l >= 2
l\r     2    3    4    5
0       3   30  159  167
1           15  135  159
2                40   48
3                     40
dp[0][5] = best k of dp[0][k] + 1*a[k]*1 + dp[k][5]
k = 1 (3):    0 + 3 + 159 = 162
k = 2 (1):    3 + 1 +  48 =  52
k = 3 (5):   30 + 5 +  40 =  75
k = 4 (8):  159 + 8 +   0 = 167   <- 8 bursts last
```

Burst balloons (k bursts last)

memorise

```java
// Burst balloons. Pad with a 1 at both ends. dp[l][r] = most coins from
// bursting every balloon strictly between l and r. k is the LAST one to
// burst, so at that moment its neighbours are l and r.
static int maxCoins(int[] nums) {
    int n = nums.length + 2;
    int[] a = new int[n];
    a[0] = a[n - 1] = 1;
    for (int i = 0; i < nums.length; i++) a[i + 1] = nums[i];
    int[][] dp = new int[n][n];
    for (int len = 2; len < n; len++)                // len = r - l
        for (int l = 0; l + len < n; l++) {
            int r = l + len;
            for (int k = l + 1; k < r; k++)
                dp[l][r] = Math.max(dp[l][r],
                                    dp[l][k] + a[l] * a[k] * a[r] + dp[k][r]);
        }
    return dp[0][n - 1];
}
```

Minimum cost to cut a stick

variant

```java
// Minimum cost to cut a stick; a cut costs the length of the piece it cuts.
// p = 0, the sorted cuts, n. dp[i][j] = cheapest way to make every cut
// strictly between p[i] and p[j]; the first one costs p[j] - p[i].
static int minCost(int n, int[] cuts) {
    int m = cuts.length + 2;
    int[] p = new int[m];
    for (int i = 0; i < cuts.length; i++) p[i + 1] = cuts[i];
    p[m - 1] = n;
    Arrays.sort(p);
    int[][] dp = new int[m][m];
    for (int len = 2; len < m; len++)
        for (int i = 0; i + len < m; i++) {
            int j = i + len;
            dp[i][j] = Integer.MAX_VALUE;
            for (int k = i + 1; k < j; k++)          // k = the first cut
                dp[i][j] = Math.min(dp[i][j], dp[i][k] + dp[k][j]);
            dp[i][j] += p[j] - p[i];
        }
    return dp[0][m - 1];
}
```

Strange printer

variant

```java
// Strange printer: dp[l][r] = fewest turns to print s[l..r]. Either s[r]
// gets its own turn, or the turn that prints some s[k] == s[r] runs on to r.
static int strangePrinter(String s) {
    int n = s.length();
    int[][] dp = new int[n][n];                      // dp[r][r-1] = 0: empty
    for (int l = n - 1; l >= 0; l--) {
        dp[l][l] = 1;
        for (int r = l + 1; r < n; r++) {
            dp[l][r] = dp[l][r - 1] + 1;
            for (int k = l; k < r; k++)
                if (s.charAt(k) == s.charAt(r))
                    dp[l][r] = Math.min(dp[l][r], dp[l][k] + dp[k + 1][r - 1]);
        }
    }
    return dp[0][n - 1];
}
```

Cost Time $O(n^3)$ Space $O(n^2)$

About $n^2/2$ ranges, each trying up to n split points. The strange printer has the same bound.

Variants same idea, small change

- Minimum score triangulation: the balloon loop with no padding. Triangle (i, k, j) stands on edge i–j: `dp[i][j] = min(dp[i][k] + v[i]·v[k]·v[j] + dp[k][j])`.
- Minimum cost tree from leaf values: `dp[l][r] = min(dp[l][k] + dp[k+1][r] + max(l..k)·max(k+1..r))`. A monotonic stack also solves it in O(n) (4.3 **Monotonic stack: next greater** (p. 105)).
- Strange printer: the turn that prints `s[r]` either prints it alone (+1), or is the turn that printed an earlier `s[k] == s[r]`, stretched to r; `s[k+1..r-1]` is printed on top of it.
- Merge stones k piles at a time: add the number of piles the range has become to the state, `dp[l][r][p]`.

Traps bugs people actually write

- Coins written as `a[k-1] * a[k] * a[k+1]` treat k as the first balloon. Its real neighbours depend on the order, and `[3, 1, 5, 8]` gives 98 instead of 167.
- `for l up, for r up` reads `dp[k][r]` with k > l before it is filled: `[3, 1, 5, 8]` gives 63. Loop by length, or l from n – 1 down.
- Cut stick without `Arrays.sort(p)`: piece lengths go negative, and LC sample 2 (n = 9, cuts `[5, 6, 1, 4, 2]`) gives 2 instead of 22.
- Strange printer: counting runs of equal chars is not the answer. `"aba"` takes 2 turns (print aaa, then b on top), not 3.

Practise on LeetCode, in this order

☐	LC 1130	Minimum Cost Tree From Leaf Values	Medium
☐	LC 1039	Minimum Score Triangulation of Polygon	Medium
☐	LC 1547	Minimum Cost to Cut a Stick	Hard
☐	LC 312	Burst Balloons	Hard
☐	LC 664	Strange Printer	Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

10.11 Stock DP (hold and free states) O(n·k)

Spot it when the problem looks like this

- *Best time to buy and sell stock* with *you must sell before you buy again*, plus a twist: a cooldown day, a fee per trade, at most k transactions.
- At the end of each day you are in one of a few states (holding, not holding, cooling down), and the rules say which state can follow which.
- n up to $5 \cdot 10^4$ (or 1000 with k ≤ 100), so O(n) or O(n·k) fits; trying every set of buy and sell days does not.

How it works

For each day keep the best profit so far in each state: `hold` while you own a share, `free` while you do not. Each day you either rest or switch: `free = max(free, hold + p - fee)` sells, `hold = max(hold, free - p)` buys. What you can still earn depends only on the state, not on how you reached it, so the best profit per state is all you need. That is O(1) per day.

A cooldown adds a state `cool`: sold today. It turns into `free` the next day, and you may buy only from `free`, so a buy never comes the day after a sale.

For at most k trades, keep `hold[t]` and `free[t]` for t buys so far. Buy number t starts from `free[t-1]`: O(n·k). A trade takes two days, so when k ≥ n/2 the limit never binds and the answer is the Stock II sum of every rise `p[i] - p[i-1] > 0`.

Worked example **maxProfitFee([1, 3, 2, 8, 4, 9], fee = 2) = 8**

```
day  price  sell: hold+p-2   buy: free-p   hold  free
0      1          -              -1         -1     0
1      3    -1+3-2 =  0      0-3 = -3       -1     0
2      2    -1+2-2 = -1      0-2 = -2       -1     0
3      8    -1+8-2 =  5      0-8 = -8       -1     5
4      4    -1+4-2 =  1      5-4 =  1        1     5
5      9     1+9-2 =  8      5-9 = -4        1     8
sell and buy use yesterday's hold and free; answer = free = 8
(buy at 1, sell at 8, buy at 4, sell at 9: 5 + 3)
```

Any number of trades, with a fee

memorise

```
// Any number of trades, a fee per sale (fee = 0 is Stock II).
// hold = best profit so far while owning a share, free = while not.
static int maxProfitFee(int[] prices, int fee) {
    int hold = -prices[0], free = 0;                 // day 0: buy, or not
    for (int i = 1; i < prices.length; i++) {
        int p = prices[i];
        int newFree = Math.max(free, hold + p - fee);   // rest, or sell today
        int newHold = Math.max(hold, free - p);         // rest, or buy today
        free = newFree;
        hold = newHold;
    }
    return free;                          // still holding at the end never helps
}
```

With fee = 0 this is Stock II.

With cooldown (three states)

variant

```java
// Cooldown: the day after a sale you may not buy.
// cool = sold today; tomorrow that profit becomes free, but not today.
static int maxProfitCooldown(int[] prices) {
    int hold = -prices[0], free = 0, cool = 0;
    for (int i = 1; i < prices.length; i++) {
        int p = prices[i];
        int newHold = Math.max(hold, free - p);      // buy from free, never cool
        int newCool = hold + p;                      // sell today
        int newFree = Math.max(free, cool);          // yesterday's sale clears
        hold = newHold;
        cool = newCool;
        free = newFree;
    }
    return Math.max(free, cool);
}
```

At most k trades (Stock III and IV)

variant

```java
// At most k trades (Stock III is k = 2). hold[t] / free[t]: best profit
// after t buys. Buy number t starts from free[t-1].
static int maxProfitK(int k, int[] prices) {
    int n = prices.length;
    if (k >= n / 2) {                    // a trade needs 2 days: no real limit
        int sum = 0;
        for (int i = 1; i < n; i++)
            sum += Math.max(0, prices[i] - prices[i - 1]);  // each rise
        return sum;
    }
    int[] hold = new int[k + 1], free = new int[k + 1];
    Arrays.fill(hold, Integer.MIN_VALUE / 2);       // no share yet
    for (int p : prices)
        for (int t = k; t >= 1; t--) {    // t down: free[t-1] is yesterday's
            free[t] = Math.max(free[t], hold[t] + p);
            hold[t] = Math.max(hold[t], free[t - 1] - p);
        }
    return free[k];
}
```

Cost Time `O(n·k)` Space `O(k)`

Fee and cooldown are O(n) time and O(1) space; k trades keep 2(k + 1) numbers and do k steps per day.

Variants same idea, small change

- Stock III (at most two trades) is `maxProfitK(2, prices)`; written out it is four variables, `buy1, sell1, buy2, sell2`.
- Maximum alternating subsequence sum: the same two states; `even` adds the next number, `odd` subtracts it.
- One trade only (LC 121): `hold = max(hold, -p)` buys from 0 instead of from `free`; 1.7 **Kadane's maximum subarray** (p. 34) shows the same loop as a max-subarray.
- House robber has the same shape: the states are *robbed this house* and *did not*, and robbed cannot follow robbed (10.2 **Linear DP over a sequence** (p. 280)).

Traps bugs people actually write

- `hold = 0` on day 0 means you own a share that cost nothing: `[5, 4, 3]` returns 4 instead of 0. Start with `hold = -prices[0]`.
- Cooldown updated in place, `cool = hold + p; free = max(free, cool); hold = max(hold, free - p);`, moves a sale into `free` on the same day, so the next day may buy: `[1, 2, 3, 0, 2]` gives 4 instead of 3. Compute all new values from yesterday's.
- Charging the fee on the buy and on the sell doubles it: the fee sample `[1, 3, 2, 8, 4, 9]` gives 4 instead of 8.

Practise on LeetCode, in this order

☐	LC 122	Best Time to Buy and Sell Stock II	Medium
☐	LC 1911	Maximum Alternating Subsequence Sum	Medium
☐	LC 714	Best Time to Buy and Sell Stock with Transaction Fee	Medium
☐	LC 309	Best Time to Buy and Sell Stock with Cooldown	Medium
☐	LC 123	Best Time to Buy and Sell Stock III	Hard
☐	LC 188	Best Time to Buy and Sell Stock IV	Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

10.12 Bitmask DP over subsets

$O(2^n \cdot n)$

Spot it when the problem looks like this

- n ≤ 12, 16 or 20: far too many orders to try ($20! \approx 2 \cdot 10^{18}$), but $2^{20} \approx 10^6$ subsets fit.
- *Visit every* city, word or node once in the best order: `dp[mask][last]`, the travelling-salesman shape.
- *Assign* n people to n items, *partition into k equal-sum subsets*, *fewest sessions or bins of capacity T*.

How it works

`mask` is a set of items: bit i is set when item i is used. `dp[mask]` keeps the best partial solution that used exactly those items. Two partial solutions with the same mask have the same future, so only the best one matters. When the order matters, the future also depends on the last item, and the state becomes `dp[mask][last]`.

Each step adds one item, `mask | 1 << i`, which is a bigger number than `mask`. So a plain loop over masks in increasing order has every predecessor ready. The cost is 2^n masks × n moves, or $2^n \times n^2$ with `last`.

For packing, store a pair per mask: sessions opened and time used in the last one. Keep the smallest pair, fewer sessions first. A state with fewer sessions can always open a new empty one, so it is never worse than a state with more.

Worked example **minSessions([1, 2, 3], T = 3) = 2; bit i is tasks[i], named by length**

```
mask  tasks done  (sessions, fill)  best way in
000   -           (1, 0)            start: one empty session
001   1           (1, 1)            000 + task 1
010   2           (1, 2)            000 + task 2
011   1, 2        (1, 3)            001 + 2 or 010 + 1
100   3           (1, 3)            000 + task 3
101   1, 3        (2, 1)            100 + 1 = (2, 1) beats 001 + 3 = (2, 3)
110   2, 3        (2, 2)            100 + 2 = (2, 2) beats 010 + 3 = (2, 3)
111   1, 2, 3     (2, 3)            every way in gives (2, 3): 2 sessions
```

Work sessions; k equal-sum subsets

memorise

```java
// Fewest work sessions of length T that finish every task (n <= 16).
// For the tasks in mask: cnt = sessions opened, fill = time used in the
// last one. Keep the smallest (cnt, fill) pair: fewer sessions first.
static int minSessions(int[] tasks, int T) {
    int n = tasks.length, full = (1 << n) - 1;
    int[] cnt = new int[1 << n], fill = new int[1 << n];
    Arrays.fill(cnt, Integer.MAX_VALUE);
    cnt[0] = 1;                                     // one empty session open
    for (int mask = 0; mask < full; mask++)         // masks only grow
        for (int i = 0; i < n; i++) {
            if ((mask >> i & 1) == 1) continue;     // task i is done
            int c = cnt[mask], f = fill[mask] + tasks[i];
            if (f > T) { c++; f = tasks[i]; }       // open a new session
            int nm = mask | 1 << i;
            if (c < cnt[nm] || (c == cnt[nm] && f < fill[nm])) {
                cnt[nm] = c;
                fill[nm] = f;
            }
        }
    return cnt[full];
}

// k equal-sum subsets: k sessions of length sum / k must hold everything.
static boolean canPartitionKSubsets(int[] nums, int k) {
    int sum = 0, max = 0;
    for (int x : nums) { sum += x; max = Math.max(max, x); }
    if (sum % k != 0 || max > sum / k) return false;
    return minSessions(nums, sum / k) == k;
}
```

Shortest superstring, dp[mask][last]

variant

```java
// Shortest superstring (TSP shape). dp[mask][last] = most chars saved by
// overlaps, over every order of the words in mask that ends with last.
static String shortestSuperstring(String[] w) {
    int n = w.length, full = (1 << n) - 1;
    int[][] ov = new int[n][n];                 // end of w[i] = start of w[j]
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            for (int k = Math.min(w[i].length(), w[j].length()); k > 0; k--)
                if (i != j && w[i].endsWith(w[j].substring(0, k))) {
                    ov[i][j] = k;
                    break;
                }
    int[][] dp = new int[1 << n][n], par = new int[1 << n][n];
    for (int[] row : dp) Arrays.fill(row, -1);
    for (int i = 0; i < n; i++) dp[1 << i][i] = 0;
    for (int mask = 1; mask <= full; mask++)
        for (int last = 0; last < n; last++) {
            if (dp[mask][last] < 0) continue;        // last is not in mask
            for (int j = 0; j < n; j++) {
                int nm = mask | 1 << j, v = dp[mask][last] + ov[last][j];
                if (nm != mask && v > dp[nm][j]) {   // j is new: append it
                    dp[nm][j] = v;
                    par[nm][j] = last;
                }
            }
        }
    int last = 0;
    for (int i = 1; i < n; i++) if (dp[full][i] > dp[full][last]) last = i;
    int[] order = new int[n];                    // walk the parents back
    for (int pos = n - 1, mask = full; pos >= 0; pos--) {
        order[pos] = last;
        int prev = par[mask][last];
        mask ^= 1 << last;
        last = prev;
    }
    StringBuilder sb = new StringBuilder(w[order[0]]);
    for (int t = 1; t < n; t++)
        sb.append(w[order[t]].substring(ov[order[t - 1]][order[t]]));
    return sb.toString();
}
```

Cost Time $O(2^n \cdot n)$ Space $O(2^n)$

2^n masks with n moves each. With `last` in the state it is $O(2^n \cdot n^2)$ time and $O(2^n \cdot n)$ memory.

Variants same idea, small change

- Number of ways to wear different hats: the mask goes over the 10 *people*, and the loop over the 40 hats: `dp[h][mask]` = ways using hats 1..h. Always put the small set in the mask.
- Minimum XOR sum of two arrays: the next item of nums1 is `i = Integer.bitCount(mask)`; try every free j of nums2. O($2^n \cdot n$), no `last` needed.
- Shortest path visiting all nodes: the same `(mask, node)` state, searched with BFS because every edge costs 1 (**9.4 BFS over states** (p. 232)).
- All submasks of a mask: `for (int s = mask; s > 0; s = (s - 1) & mask)`. Over all masks that is 3^n steps (**12.1 Bit tricks: lowest bit, submasks** (p. 339)).

Traps bugs people actually write

- `(mask & (1 << i)) == 1` is true only for i = 0. Test `(mask >> i & 1) == 1` or `(mask & (1 << i)) != 0`.
- Keeping only the session count and the first fill seen: tasks `[2, 2, 4, 5, 1]` with T = 7 gives 3 instead of 2. Break ties by the smaller fill.
- `int[1 << n][n]` for n = 20 is $2 \cdot 10^7$ ints, 80 MB per table, and the superstring code has two. Check n before you add `last`.

Practise on LeetCode, in this order

☐	LC 698	Partition to K Equal Sum Subsets	Medium
☐	LC 1986	Minimum Number of Work Sessions to Finish the Tasks	Medium
☐	LC 1879	Minimum XOR Sum of Two Arrays	Hard
☐	LC 1434	Number of Ways to Wear Different Hats to Each Other	Hard
☐	LC 943	Find the Shortest Superstring	Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

10.13 Digit DP (count numbers ≤ N)

`O(L·S·10)`

Spot it when the problem looks like this

- *How many integers in [1, n]* with n up to 10^9 or 10^{18}, or given as a string: a loop over the numbers is too slow.
- The rule is about digits: how many 1s, no repeated digit, only digits from a set, a digit sum in a range.
- A range `[lo, hi]`: answer it as `f(hi) - f(lo - 1)`, where f counts from 0 up to a bound.

How it works

Build the number from the left, one digit per position, and count the ways to finish. `tight` says the digits so far equal the prefix of n, so this digit may go only up to `d[pos]`; a smaller digit frees every later

position to 0–9. `started` says a non-zero digit has been placed, so leading zeros are not counted as digits. `state` is what the rule must remember about the prefix: the set of used digits, the digit sum, the number of 1s.

The number of ways to finish depends only on `(pos, tight, started, state)`, not on the digits that led there, so memoise it. With 10 positions, 2 × 2 flags and 2^{10} masks there are about $4 \cdot 10^4$ states, each trying 10 digits.

For a range, count x in [0, bound] and return `f(hi) - f(lo - 1)`. With string inputs, subtract 1 by turning the trailing 0s into 9s and lowering the digit before them.

Worked example **n = 235: numbers in [1, 235] whose digits are all different**

pos	prefix	digit	next call	count	numbers
0	-	0	go(1, free, not started)	90	1..99
0	-	1	go(1, free, {1})	72	1__
0	-	2	go(1, tight, {2})	20	2__ up to 235
1	2	0, 1	go(2, free, {0,2}), {1,2}	8 + 8	20_, 21_
1	2	2	repeated digit, skipped		
1	2	3	go(2, tight, {2,3})	4	23_ up to 235
2	23	0,1,4	go(3, free, ...)	1 each	230, 231, 234
2	23	2, 3	repeated digit, skipped		
2	23	5	go(3, tight, {2,3,5})	1	235

distinct = 90 + 72 + 20 = 182; with a repeated digit: 235 - 182 = 53

Numbers with repeated digits

memorise

```java
// Numbers with repeated digits (LC 1012) = n - (numbers in [1, n] whose
// digits are all different). go() places the digits left to right.
static char[] d;                                    // the digits of n
static Integer[][][][] memo;                        // [pos][tight][started][mask]

static int numDupDigitsAtMostN(int n) {
    d = String.valueOf(n).toCharArray();
    memo = new Integer[d.length][2][2][1 << 10];
    return n - go(0, 1, 0, 0);
}

// tight = 1: the prefix equals n's prefix, so this digit is at most d[pos].
// started = 0: only leading zeros so far; they are not digits of the number.
static int go(int pos, int tight, int started, int mask) {
    if (pos == d.length) return started;            // built one number (not 0)
    Integer m = memo[pos][tight][started][mask];
    if (m != null) return m;
    int lim = tight == 1 ? d[pos] - '0' : 9, res = 0;
    for (int dig = 0; dig <= lim; dig++) {
        int nt = tight == 1 && dig == lim ? 1 : 0;
        if (started == 0 && dig == 0)
            res += go(pos + 1, nt, 0, mask);        // still a leading zero
        else if ((mask >> dig & 1) == 0)
            res += go(pos + 1, nt, 1, mask | 1 << dig); // dig is new: use it
    }
    return memo[pos][tight][started][mask] = res;
}
```

Digit sum in a range: f(hi) − f(lo − 1)

variant

```java
// Count of integers (LC 2719): x in [num1, num2], strings up to 10^22,
// digit sum in [minSum, maxSum], mod 1e9+7.
static final int MOD = 1_000_000_007;
static String bound;                                // count x in [0, bound]
static int minSum, maxSum;
static long[][][] ways;                             // [pos][tight][sum]; -1 = new

static int count(String num1, String num2, int min_sum, int max_sum) {
    minSum = min_sum;
    maxSum = max_sum;
    long r = upTo(num2) - upTo(minusOne(num1));     // f(hi) - f(lo - 1)
    return (int) ((r % MOD + MOD) % MOD);
}

static long upTo(String n) {
    bound = n;
    ways = new long[n.length()][2][maxSum + 1];
    for (long[][] a : ways) for (long[] b : a) Arrays.fill(b, -1);
    return walk(0, 1, 0);
}

// Leading zeros add 0 to the sum, so this rule needs no 'started' flag.
static long walk(int pos, int tight, int sum) {
    if (sum > maxSum) return 0;                     // the sum only grows
    if (pos == bound.length()) return sum >= minSum ? 1 : 0;
    if (ways[pos][tight][sum] >= 0) return ways[pos][tight][sum];
    int lim = tight == 1 ? bound.charAt(pos) - '0' : 9;
    long res = 0;
    for (int dig = 0; dig <= lim; dig++)
        res += walk(pos + 1, tight == 1 && dig == lim ? 1 : 0, sum + dig);
    return ways[pos][tight][sum] = res % MOD;
}

static String minusOne(String x) {                  // "1000" -> "0999"
    char[] c = x.toCharArray();
    int i = c.length - 1;
    while (c[i] == '0') c[i--] = '9';
    c[i]--;
    return new String(c);
}
```

Cost Time `O(L·S·10)` Space `O(L·S)`

L digits, S values of the state (2^{10} masks, or sums up to 400), times 2 × 2 flags; each state tries 10 digits.

Variants same idea, small change

- Number of digit one: no `started`; the state is the count of 1s so far, and `pos == len` returns that count instead of 1, so the memo sums the 1s of every number.
- Numbers at most n from a digit set: loop only over the allowed digits. `started` lets shorter numbers count; no mask.
- Count numbers with unique digits (all $x < 10^n$): `go` with the bound $10^n - 1$, plus 1 for the number 0.
- Divisible by k: the state is the remainder so far, `(rem * 10 + dig) % k`; for a digit sum divisible by k, `(rem + dig) % k`.

Traps bugs people actually write

- No `started` flag: 7 becomes 007 with two zeros, so it counts as repeated. For n = 100 you get 72 numbers with distinct digits instead of 90.
- Memo without `tight`: a tight call reuses the count that a free call stored for the same position and mask. Under n = 321, `23_` (free) and `32_` (tight) share the mask {2, 3}, and the answer comes out 63 instead of 69.
- `f(hi) - f(lo - 1)` under a modulus can be negative; return `(r % MOD + MOD) % MOD`.

Practise on LeetCode, in this order

☐	LC 357	Count Numbers with Unique Digits	Medium
☐	LC 902	Numbers At Most N Given Digit Set	Hard
☐	LC 233	Number of Digit One	Hard
☐	LC 1012	Numbers With Repeated Digits	Hard
☐	LC 2719	Count of Integers	Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

10.14 Partition into k contiguous groups

$O(k \cdot n^2)$

Spot it when the problem looks like this

- *Split nums into k non-empty subarrays, schedule the jobs over d days, in order, at least one per day.*
- The cost of a piece depends only on what is inside it: its sum, its maximum, its average, the changes that make it a palindrome.
- n up to a few hundred and k ≤ 50, so $O(k \cdot n^2)$ fits.

How it works

`dp[g][i]` is the best cost for cutting the first i elements into exactly g groups. Look at the last group, `a[j..i-1]`. Before it come g – 1 groups of the first j elements, and how those are cut does not change

the last group's cost. So `dp[g][i]` is the best over j of `dp[g-1][j]` combined with the cost of `a[j..i-1]`. Combine with `+` for summed costs (difficulty, averages, palindrome changes) and with `max` when the largest group counts.

Start with `dp[0][0] = 0` and everything else at infinity; `j ≥ g - 1` keeps every earlier group non-empty. Walk j down from i – 1, so the last group grows by one element per step and its sum or maximum updates in O(1). That is k·n states × n cuts = $O(k \cdot n^2)$.

For *split array largest sum*, binary search on the answer is O(n log S) (3.3 Binary search on the answer (p. 74)); the DP is the version that works for any cost.

Worked example **splitArray([7, 2, 5, 10, 8], k = 2) = 18**

```
dp[g][i] = smallest largest sum for the first i elements in g groups
i          1    2    3    4    5
g = 1      7    9   14   24   32
g = 2      -    7    7   14   18
dp[2][5]: last group a[j..4], before it dp[1][j]
j = 4    max(24, 8)  = 24
j = 3    max(14, 18) = 18   <- best: [7, 2, 5] [10, 8]
j = 2    max(9, 23)  = 23
j = 1    max(7, 25)  = 25
```

Split array largest sum (DP) memorise

```
// Split array largest sum: cut a into k contiguous groups and minimise the
// largest group sum. dp[g][i] = best for the first i elements in g groups;
// the last group is a[j..i-1].
static int splitArray(int[] a, int k) {
    int n = a.length, INF = Integer.MAX_VALUE;
    int[][] dp = new int[k + 1][n + 1];
    for (int[] row : dp) Arrays.fill(row, INF);
    dp[0][0] = 0;
    for (int g = 1; g <= k; g++)
        for (int i = g; i <= n; i++) {
            int last = 0;                              // sum of a[j..i-1]
            for (int j = i - 1; j >= g - 1; j--) {   // g-1 groups need j >= g-1
                last += a[j];
                dp[g][i] = Math.min(dp[g][i], Math.max(dp[g - 1][j], last));
            }
        }
    return dp[k][n];
}
```

Minimum difficulty of a job schedule

variant

```java
// Minimum difficulty of a job schedule: d days, jobs in order, at least one
// a day; a day costs its hardest job. Same table; groups add up.
static int minDifficulty(int[] jobs, int d) {
    int n = jobs.length, INF = Integer.MAX_VALUE / 2;   // INF + cost: no overflow
    if (n < d) return -1;
    int[][] dp = new int[d + 1][n + 1];
    for (int[] row : dp) Arrays.fill(row, INF);
    dp[0][0] = 0;
    for (int g = 1; g <= d; g++)
        for (int i = g; i <= n; i++) {
            int hardest = 0;                          // max of jobs[j..i-1]
            for (int j = i - 1; j >= g - 1; j--) {
                hardest = Math.max(hardest, jobs[j]);
                dp[g][i] = Math.min(dp[g][i], dp[g - 1][j] + hardest);
            }
        }
    return dp[d][n];
}
```

Palindrome partitioning III

variant

```java
// Palindrome partitioning III: change the fewest chars so that s splits
// into k palindromes. cost[j][i] = changes that make s[j..i] a palindrome.
static int palindromePartition(String s, int k) {
    int n = s.length(), INF = Integer.MAX_VALUE / 2;
    int[][] cost = new int[n][n];
    for (int j = n - 1; j >= 0; j--)
        for (int i = j + 1; i < n; i++)
            cost[j][i] = cost[j + 1][i - 1]
                       + (s.charAt(j) == s.charAt(i) ? 0 : 1);
    int[][] dp = new int[k + 1][n + 1];
    for (int[] row : dp) Arrays.fill(row, INF);
    dp[0][0] = 0;
    for (int g = 1; g <= k; g++)
        for (int i = g; i <= n; i++)
            for (int j = g - 1; j < i; j++)           // last piece s[j..i-1]
                dp[g][i] = Math.min(dp[g][i], dp[g - 1][j] + cost[j][i - 1]);
    return dp[k][n];
}
```

Cost Time $O(k \cdot n^2)$ Space $O(k \cdot n)$

k·n cells, each trying up to n cuts. Palindrome partitioning III adds an $O(n^2)$ table of piece costs.

Variants same idea, small change

- Largest sum of averages: combine with `+`, cost = the group's average from prefix sums. More groups never lower the score, so *at most k* is the same as exactly k.
- Any number of groups, each with a cost: drop g and use `dp[i] = min over j of dp[j] + cost(j, i-1)` (**10.2 Linear DP over a sequence** (p. 280)).
- Allocate mailboxes: sort the houses; a group's cost is the distance of its houses to their median, precomputed for every range.
- Split array largest sum with n = 10^5: the DP is too slow; binary search on the answer with a greedy check (**3.3 Binary search on the answer** (p. 74)).

Traps bugs people actually write

- `INF = Integer.MAX_VALUE` with `dp[g-1][j] + hardest` overflows to a large negative number that wins the min: the job sample `[6, 5, 4, 3, 2, 1]`, d = 2 returns –2147483646. Use `MAX_VALUE / 2` (with `max` instead of `+`, MAX_VALUE is safe).
- No `if (n < d) return -1`: with fewer jobs than days the answer cell stays at INF, and you return 1073741823.
- j going up from g – 1 cannot keep a running sum, because the last group `a[j..i-1]` shrinks as j grows. Go down from i – 1, or use prefix sums; summing each group in a loop adds a factor of n.

Practise on LeetCode, in this order

- [ ] LC 813 Largest Sum of Averages — Medium
- [ ] LC 1335 Minimum Difficulty of a Job Schedule — Hard
- [ ] LC 410 Split Array Largest Sum — Hard
- [ ] LC 1278 Palindrome Partitioning III — Hard
- [ ] LC 1478 Allocate Mailboxes — Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

Part 11

Strings

String questions have structure that general array tools miss: a palindrome mirrors around its centre, a failed match still tells you what the text was, and a hash turns a substring into a number. Each pattern here replaces a compare-every-substring loop, $O(n^2)$ or worse, with a linear pass, an O(1) comparison or a walk down a tree of prefixes. Reach for this part when the question is about substrings, prefixes, palindromes or many words at once.

11.1	**Expand around the centre** Longest palindromic substring, or how many substrings are palindromes, for n up to a few thousand.	$O(n^2)$	p. 320
11.2	**KMP prefix function** Find a pattern in a text in O(n + m), or use a string's borders (a prefix that is also a suffix).	O(n + m)	p. 323
11.3	**Z-function** For every position i, how far the text from i matches the start of the string.	O(n)	p. 326
11.4	**Rolling hash (Rabin–Karp)** Compare any two substrings in O(1) by comparing numbers; find repeats with a set of hashes.	O(n log n)	p. 329
11.5	**Trie (prefix tree)** Many words and questions about prefixes, wildcards, or finding all the words in a grid.	O(L)	p. 332
11.6	**Manacher's algorithm** The longest palindrome at every centre in O(n); longest or count of palindromic substrings for n up to 10^5 or 10^6.	O(n)	p. 335

11.1 Expand around the centre

$O(n^2)$

Spot it when the problem looks like this

- *Longest palindromic substring*, *count palindromic substrings*: the piece is contiguous, so every palindrome has a centre.
- n ≤ 1000 to 3000, so $O(n^2)$ is fine, but checking each of the $n^2/2$ substrings from scratch ($O(n^3)$) is not.
- You want the palindromes at many centres with O(1) extra memory, not an n × n table.

How it works

A palindrome mirrors around its centre. If `s[l..r]` is a palindrome and `s[l-1] == s[r+1]`, then `s[l-1..r+1]` is one too; if they differ, no longer palindrome has this centre. So from each centre, grow outward one pair at a time until the chars differ or an end is reached. Every palindrome is found from its own centre, so nothing is missed.

Odd lengths centre on a char and even lengths on the gap between two chars: 2n – 1 centres. Number them `c = 0..2n-2` and start at `l = c / 2`, `r = l + c % 2`. An even c starts both ends on one char, an odd c on two neighbours, so one loop covers both kinds.

A centre grows at most n/2 steps, so the worst case (all one letter) is $O(n^2)$; text with few palindromes stops almost at once. Each successful step is one more palindrome, which is all the count needs.

Worked example **s = "babad": 9 centres, the longest and the count**

```
c  centre    start l, r  longest here  palindromes  best
0  b         0, 0        "b"           +1           "b"
1  gap b|a   0, 1        -             +0           "b"
2  a         1, 1        "bab"         +2           "bab"
3  gap a|b   1, 2        -             +0           "bab"
4  b         2, 2        "aba"         +2           "bab"
5  gap b|a   2, 3        -             +0           "bab"
6  a         3, 3        "a"           +1           "bab"
7  gap a|d   3, 4        -             +0           "bab"
8  d         4, 4        "d"           +1           "bab"
7 palindromes. "aba" ties "bab" but is not longer, so "bab" stays.
```

Longest palindromic substring

memorise

```java
// Longest palindromic substring. Centre c runs over 0..2n-2: an even c is
// the char c/2 (odd lengths), an odd c is the gap after it (even lengths).
static String longestPalindrome(String s) {
    int n = s.length(), bestL = 0, bestLen = 0;
    for (int c = 0; c < 2 * n - 1; c++) {
        int l = c / 2, r = l + c % 2;
        while (l >= 0 && r < n && s.charAt(l) == s.charAt(r)) {
            l--;
            r++;
        }
        // the loop stopped one step too far: s[l+1..r-1] is the palindrome
        if (r - l - 1 > bestLen) {
            bestL = l + 1;
            bestLen = r - l - 1;
        }
    }
    return s.substring(bestL, bestL + bestLen);
}
```

Count palindromic substrings

variant

```java
// Palindromic substrings: every step of the inner loop finds one more
// palindrome with this centre (the next one is two chars longer).
static int countSubstrings(String s) {
    int n = s.length(), count = 0;
    for (int c = 0; c < 2 * n - 1; c++) {
        int l = c / 2, r = l + c % 2;
        while (l >= 0 && r < n && s.charAt(l) == s.charAt(r)) {
            count++;
            l--;
            r++;
        }
    }
    return count;
}
```

Cost Time `O(n²)` Space `O(1)`

2n – 1 centres, each growing at most n/2 steps. Only a few indexes are kept; the answer string is cut once at the end.

Variants same idea, small change

- Longest palindromic *subsequence* (chars may be skipped): centres do not apply; use 10.9 **Palindrome DP on s[i..j]** (p. 300).
- n up to 10^5, or the longest palindrome at every centre is needed: 11.6 **Manacher's algorithm** (p. 335) does the same growth in O(n) in total.
- Palindrome table for a partition search: run the loop once and set `isPal[l][r] = true` at every step. That fills all the true cells in $O(n^2)$ without a DP order (7.7 **Cut a prefix, recurse on the rest** (p. 178)).
- Non-overlapping palindromes of length ≥ k (LC 2472): from each centre keep only the first palindrome of length ≥ k (k or k + 1 chars), then choose the most disjoint ones by earliest right end (6.3 **Interval scheduling (sort by end)** (p. 147)).

Traps bugs people actually write

- Odd centres only (`l = r = c` for `c < n`) misses every even palindrome: `"cbbd"` returns `"c"` instead of `"bb"`.
- Testing `s.charAt(l) == s.charAt(r)` before `l >= 0 && r < n` throws `StringIndexOutOfBoundsException` as soon as a palindrome reaches an end of s, for example on `"aba"`.
- After the loop, l and r are one step past the palindrome: it is `s[l+1..r-1]`, of length `r - l - 1`. Using `r - l + 1` reports two chars too many.

Practise on LeetCode, in this order

- [ ] LC 647 Palindromic Substrings — Medium
- [ ] LC 5 Longest Palindromic Substring — Medium
- [ ] LC 2472 Maximum Number of Non-overlapping Palindrome Substrings — Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

11.2 KMP prefix function

`O(n + m)`

Spot it when the problem looks like this

- *Find the first occurrence*, *all occurrences*, *is s a rotation of t*, with lengths up to 10^5 or more.
- A *prefix that is also a suffix*: longest happy prefix, the period of a string, *s is a block repeated k times*.
- Add the fewest chars in front to make s a palindrome: that is the longest palindromic prefix.

How it works

A **border** of a string is a proper prefix that is also a suffix, and `pi[i]` is the longest border of `s[0..i]`. A border of `s[0..i]` is a border of `s[0..i-1]` followed by `s[i]`. The borders of `s[0..i-1]` are `pi[i-1]`, then `pi[pi[i-1]-1]`, and so on down to 0, because a border of a border is a border. So try them longest first: the first length `k` with `s[k] == s[i]` gives `pi[i] = k + 1`.

The search keeps `k`, the number of chars of p that match the text ending at `i`. On a mismatch, `p[0..k)` is text you have already read, so the next candidate is its longest border, and `i` never moves back.

Each step raises `k` by at most 1, and each pass of the `while` lowers it by at least 1 without going below 0. So the `while` runs at most n times in total: O(n) for `pi` and O(n + m) for the search.

Worked example prefixFunction("abacabab")

```
i  s[i]  borders tried (k)                pi[i]
0  a     -                                0
1  b     0: s[0] = a, no                  0
2  a     0: s[0] = a, match               1
3  c     1: s[1] = b, no -> pi[0] = 0, no 0
4  a     0: s[0] = a, match               1
5  b     1: s[1] = b, match               2
6  a     2: s[2] = a, match               3
7  b     3: s[3] = c, no -> pi[2] = 1:
           s[1] = b, match                2
pi = [0, 0, 1, 0, 1, 2, 3, 2]. At i = 7, "aba" + b is not a border,
but the border of "aba", "a", grows to "ab".
```

Prefix function

memorise

```java
// pi[i] = length of the longest proper prefix of s[0..i] that is also a
// suffix of s[0..i] (its longest "border").
static int[] prefixFunction(String s) {
    int n = s.length();
    int[] pi = new int[n];
    for (int i = 1; i < n; i++) {
        int k = pi[i - 1];                      // longest border of s[0..i-1]
        while (k > 0 && s.charAt(i) != s.charAt(k))
            k = pi[k - 1];                      // next shorter border
        if (s.charAt(i) == s.charAt(k)) k++;    // extend it by s[i]
        pi[i] = k;
    }
    return pi;
}
```

Search without moving back

memorise

```java
// First index of p in t, or -1. i never moves back; on a mismatch only k
// (the number of chars of p matched so far) falls back through pi.
static int strStr(String t, String p) {
    int m = p.length();
    if (m == 0) return 0;
    int[] pi = prefixFunction(p);
    for (int i = 0, k = 0; i < t.length(); i++) {
        while (k > 0 && t.charAt(i) != p.charAt(k)) k = pi[k - 1];
        if (t.charAt(i) == p.charAt(k)) k++;
        if (k == m) return i - m + 1;    // all matches: record, k = pi[k - 1]
    }
    return -1;
}
```

Period, happy prefix, shortest palindrome

example

```java
// LC 459: the shortest period of s is n - pi[n-1]. s is a block repeated
// at least twice exactly when that period is shorter than n and divides n.
static boolean repeatedSubstringPattern(String s) {
    int n = s.length(), period = n - prefixFunction(s)[n - 1];
    return period < n && n % period == 0;
}

// LC 1392: the longest happy prefix is the longest border of s.
static String longestPrefix(String s) {
    return s.substring(0, prefixFunction(s)[s.length() - 1]);
}

// LC 214: keep the longest palindromic prefix of s, then put the reverse
// of the rest in front. That prefix is the longest border of
// s + '#' + reverse(s); the '#' stops a border from passing n.
static String shortestPalindrome(String s) {
    String rev = new StringBuilder(s).reverse().toString();
    int[] pi = prefixFunction(s + "#" + rev);
    int keep = pi[pi.length - 1];                // s[0..keep) is a palindrome
    return rev.substring(0, s.length() - keep) + s;
}
```

Cost Time `O(n + m)` Space `O(m)`

k rises at most once per char and every fallback lowers it, so all the fallbacks together are at most n. pi holds one int per char of p.

Variants same idea, small change

- All occurrences: on `k == m`, record `i - m + 1` and set `k = pi[k - 1]`; overlapping matches are found too.
- Rotate string (LC 796): goal is a rotation of s when the lengths are equal and goal occurs in `s + s`.
- Remove all occurrences of a substring (LC 1910): keep the kept chars on a stack with the KMP state after each one. When the state reaches m, pop m chars and continue from the state below them: O(n), no rescans.
- The same answers come from the Z-array (11.3 **Z-function** (p. 326)). Many patterns at once: put them in a 11.5 **Trie (prefix tree)** (p. 332).

Traps bugs people actually write

- `if` instead of `while` for the fallback leaves k on a border that was never checked: `"aaab"` gets `pi = [0, 1, 2, 1]`, which claims that "a" is a suffix of "aaab".
- Falling back with `k = pi[k]` instead of `pi[k - 1]` can loop forever: on `"aab"` at i = 2, k = 1 and `pi[1] = 1`.
- Shortest palindrome without the `#`: for `"aa"`, `"aaaa"` has a border of 3, longer than s, and `rev.substring(0, 2 - 3)` throws.

Practise on LeetCode, in this order

- [ ] LC 28 Find the Index of the First Occurrence in a String — Easy
- [ ] LC 459 Repeated Substring Pattern — Easy
- [ ] LC 1910 Remove All Occurrences of a Substring — Medium
- [ ] LC 1392 Longest Happy Prefix — Hard
- [ ] LC 214 Shortest Palindrome — Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

11.3 Z-function

O(n)

Spot it when the problem looks like this

- Compare every suffix with the whole string: the *score* of each suffix, *is word[i..] a prefix of word*.
- Pattern search where you want an answer per position (every match, matches with one change), n up to 10^5 or 10^6.
- A prefix of the string that comes back later, asked for every position at once.

How it works

`z[i]` is the length of the longest common prefix of `s` and `s[i..]`. Keep the box `[l, r)`, the match found so far that reaches furthest right, so `s[l..r) == s[0..r-l)`. For `i < r`, `s[i..r)` is a copy of `s[i-l..r-l)`, whose match is already known, so `z[i] ≥ min(r - i, z[i - l])`. If `z[i - l] < r - i`, that match stopped inside the box at a mismatch that is copied too: the value is exact and the first compare fails. Otherwise compare the chars past `r`.

Every successful compare moves the right end of the box. `r` never moves left and stops at n, so there are at most n successful compares, plus at most one failed compare per `i`: O(n).

To search, build `p + '$' + t`. A z-value of `m` at position `m + 1 + j` means p occurs at `t[j]`.

Worked example zFunction("aabcaabxaaz"); z[0] = n = 11

i	s[i]	box [l, r)	start from the box	compares	z[i]	box after
1	a	-	0	2	1	[1, 2)
2	b	-	0	1	0	
3	c	-	0	1	0	[3, 3)
4	a	-	0	4	3	[4, 7)
5	a	[4, 7)	min(7-5, z[1]) = 1	1	1	
6	b	[4, 7)	min(7-6, z[2]) = 0	1	0	
7	x	-	0	1	0	
8	a	-	0	3	2	[8, 10)
9	a	[8, 10)	min(10-9, z[1]) = 1	1	1	
10	z	-	0	1	0	

z = [11, 1, 0, 0, 3, 1, 0, 0, 2, 1, 0], 16 compares for n = 11

Z-array with the [l, r) box

memorise

```java
// z[i] = length of the longest common prefix of s and s[i..]; z[0] = n.
// [l, r) is the match s[l..r) == s[0..r-l) that reaches furthest right.
static int[] zFunction(String s) {
    int n = s.length();
    int[] z = new int[n];
    if (n > 0) z[0] = n;
    for (int i = 1, l = 0, r = 0; i < n; i++) {
        if (i < r) z[i] = Math.min(r - i, z[i - l]);   // copy from the box
        while (i + z[i] < n && s.charAt(z[i]) == s.charAt(i + z[i]))
            z[i]++;                                    // compare past r
        if (i + z[i] > r) {                            // box moves right
            l = i;
            r = i + z[i];
        }
    }
    return z;
}
```

All matches via p + '$' + t

memorise

```java
// Every start of p in t. '$' is in neither string, so no z-value of
// p + '$' + t can pass m, and z[i] == m marks a match.
static List<Integer> occurrences(String t, String p) {
    int m = p.length();
    int[] z = zFunction(p + "$" + t);
    List<Integer> out = new ArrayList<>();
    for (int i = m + 1; i < z.length; i++)
        if (z[i] == m) out.add(i - m - 1);
    return out;
}
```

Sum of scores; time to revert a word

example

```java
// LC 2223: the score of the suffix s[i..] is z[i]. The sum reaches
// n(n+1)/2, about 5e9 for n = 1e5, so add in a long.
static long sumScores(String s) {
    long sum = 0;
    for (int v : zFunction(s)) sum += v;
    return sum;
}

// LC 3029 / 3031: after t seconds the word starts with word[t*k..] and
// the rest is free. It is back once that suffix is a prefix of word
// (z[t*k] covers it), or once nothing of the old word is left.
static int minimumTimeToInitialState(String word, int k) {
    int n = word.length();
    int[] z = zFunction(word);
    int t = 1;
    while (t * k < n && z[t * k] < n - t * k) t++;
    return t;
}
```

Cost Time `O(n)` Space `O(n)`

r only moves right, so successful compares total at most n, and each i adds at most one failed compare. Search on p + '$' + t is O(n + m).

Variants same idea, small change

- One change allowed (LC 3303): z of `p + '$' + s` gives the prefix match at each start, and z of the two reversed strings gives the suffix match at each end. Start i works when the two add up to at least m − 1.
- Longest happy prefix: the first i with `i + z[i] == n`; the answer is `s[i..]`. Same answer as **11.2 KMP prefix function** (p. 323).
- Beautiful indices (LC 3008): find all starts of a and of b with Z, then for each start of a look for a start of b within k with two pointers or **3.1 Lower bound and upper bound** (p. 68).
- Prefixes of another string t at every position of s: z of `t + '$' + s`.

Traps bugs people actually write

- No separator: in `p + t` a z-value can run past m into t, and `z == m` misses that spot. p = "aa", t = "aaa" gives z = `[5, 4, 3, 2, 1]` on "aaaaa", so the match at 0 (z = 3) is lost.
- `z[i] = z[i - l]` without the `min(r - i, ·)` claims chars past r that were never compared: on "aaa" it gives `z[2] = 2`, which runs off the end of the string.
- Leaving `z[0] = 0` drops the whole string's own score: LC 2223 on "babab" sums to 4 instead of 9. The sum reaches $5 \cdot 10^9$ for n = 10^5, so add in a `long`.

Practise on LeetCode, in this order

☐	LC 3029	Minimum Time to Revert Word to Initial State I	Medium
☐	LC 2223	Sum of Scores of Built Strings	Hard
☐	LC 3031	Minimum Time to Revert Word to Initial State II	Hard
☐	LC 3008	Find Beautiful Indices in the Given Array II	Hard
☐	LC 3303	Find the Occurrence of First Almost Equal Substring	Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

11.4 Rolling hash (Rabin–Karp)

`O(n log n)`

Spot it when the problem looks like this

- *Longest substring that occurs twice*, *distinct substrings of the form a + a*, n up to $3 \cdot 10^4$ or 10^5.
- Many equality tests between pieces of one string, where each char-by-char compare would cost O(n).
- "If length L works, every shorter length works": binary search the length, and test one length with a hash set.

How it works

Read a string as a number in base B: `h[i+1] = h[i]*B + s[i]` mod M, so `h[i]` is the hash of the first i chars. A substring is a difference: `hash(s[l..r)) = h[r] - h[l]*B^(r-l)`, since shifting the prefix `s[0..l)` up by r – l digits lines it up with `h[r]`. With the prefix hashes and powers stored, each substring hash is O(1), and equal substrings always get equal hashes.

The converse can fail: two different strings can share a hash (a **collision**). With one prime near 10^9 and 10^5 windows, about $n^2/2M \approx 5$ pairs collide. With two primes a false match needs both residues to collide, about $5 \cdot 10^{-9}$. Java 8 has no 128-bit multiply for mod $2^{61} - 1$, so the template uses two primes below 2^{30}: every product stays below 2^{60} in a `long`. Where a wrong answer is not acceptable, confirm each hit with `regionMatches`.

Worked example s = "banana", B = 131, M1 = $10^9 + 7$ (the M2 half works the same way)

```
i       0   1    2      3        4          5         6
h1[i]   0   98   12935  1694595  221992042  80957409  605420606
get(1, 4) = h1[4] - h1[1]*B^3 = 221992042 - 98*2248091 = 1679124  "ana"
get(3, 6) = h1[6] - h1[3]*B^3  (mod M1)                = 1679124  "ana"
longestDupSubstring: binary search on L, answer in [lo, hi]
lo  hi  mid  windows of length mid   first repeat
0   5   3    ban ana nan ana         at 3, same key as at 1: lo = 3
3   5   4    bana anan nana          none: hi = 3
lo = hi = 3: answer s.substring(3, 6) = "ana"
```

Prefix hashes, two primes, O(1) substring

memorise

```
// Two polynomial hashes, mod two primes below 2^30, so every product of
// two residues is below 2^60 and fits in a long. Chars are >= 1.
static final long M1 = 1_000_000_007L, M2 = 1_000_000_009L, B = 131;

static class Hasher {
    final long[] h1, h2, p1, p2;     // prefix hashes and powers of B

    Hasher(String s) {
        int n = s.length();
        h1 = new long[n + 1]; h2 = new long[n + 1];
        p1 = new long[n + 1]; p2 = new long[n + 1];
        p1[0] = p2[0] = 1;
        for (int i = 0; i < n; i++) {      // h[i+1] = h[i] * B + s[i]
            h1[i + 1] = (h1[i] * B + s.charAt(i)) % M1;
            h2[i + 1] = (h2[i] * B + s.charAt(i)) % M2;
            p1[i + 1] = p1[i] * B % M1;
            p2[i + 1] = p2[i] * B % M2;
        }
    }

    // Hash of s[l..r) in O(1): h[r] - h[l] * B^(r-l), both mods in one key.
    long get(int l, int r) {
        long a = ((h1[r] - h1[l] * p1[r - l]) % M1 + M1) % M1;
        long b = ((h2[r] - h2[l] * p2[r - l]) % M2 + M2) % M2;
        return a * M2 + b;               // < 2^63, and a different (a, b)
    }                                    // always gives a different key
}
```

Binary search on length + hash set

memorise

```
// LC 1044: if some substring of length L occurs twice, so does its first
// L - 1 chars. "A repeat of length L exists" is monotone: binary search L.
static String longestDupSubstring(String s) {
    Hasher h = new Hasher(s);
    int lo = 0, hi = s.length() - 1, start = 0;    // answer length in [lo, hi]
    while (lo < hi) {
        int mid = (lo + hi + 1) / 2;
        int at = repeatAt(h, s.length(), mid);
        if (at >= 0) { lo = mid; start = at; }
        else hi = mid - 1;
    }
    return s.substring(start, start + lo);
}

// Start of the second copy of some length-len substring, or -1.
static int repeatAt(Hasher h, int n, int len) {
    Set<Long> seen = new HashSet<>();
    for (int i = 0; i + len <= n; i++)
        if (!seen.add(h.get(i, i + len))) return i;
    return -1;
}
```

Distinct echo substrings

example

```
// LC 1316: count distinct a + a. For each half length, compare the two
// halves in O(1) and collect the hashes of the first half.
static int distinctEchoSubstrings(String text) {
    Hasher h = new Hasher(text);
    int n = text.length(), count = 0;
    for (int len = 1; 2 * len <= n; len++) {
        Set<Long> halves = new HashSet<>();
        for (int i = 0; i + 2 * len <= n; i++)
            if (h.get(i, i + len) == h.get(i + len, i + 2 * len))
                halves.add(h.get(i, i + len));
        count += halves.size();
    }
    return count;
}
```

Cost Time `O(n log n)` Space `O(n)`

Prefix hashes take O(n) and each get is O(1). Longest duplicate runs O(log n) rounds of an O(n) set pass; echo substrings tries all $O(n^2)$ pairs.

Variants same idea, small change

- Rabin–Karp search: hash p once and compare it with `get(i, i + m)` for every i; confirm a hit with `t.regionMatches(i, p, 0, m)`. O(n + m) expected.
- Longest common piece of two arrays (LC 718): binary search L, put the window hashes of one array in a set and look up the other's. For many paths (LC 1923), intersect the sets of all paths.
- Is `s[l..r)` a palindrome? Hash s and its reverse and compare the two directions. 11.6 **Manacher's algorithm** (p. 335) answers the same with no collisions.
- Repeated DNA sequences (LC 187): 4 letters and windows of 10, so keep the window as a 20-bit number, `x = ((x << 2) | code[c]) & 0xFFFFF`. The code is exact, so a set of ints finds the repeats with no collisions.

Traps bugs people actually write

- One prime is not enough: in a test on 10^5 random 10-letter windows, mod $10^9 + 7$ alone gave 6 colliding pairs, and both primes together gave none.
- In Java `%` keeps the sign, so `(h[r] - h[l] * p[r - l]) % M` can be negative and equal substrings compare unequal. Add M and take `% M` again.
- Mapping 'a' to 0 makes "a", "aa" and "aaa" all hash to 0. A set that mixes lengths then sees false repeats. Use the char itself or `c - 'a' + 1`.
- With a prime near 2^{61}, `h[l] * p[r - l]` overflows a `long` without a split multiply; with primes below 2^{30} it cannot.

Practise on LeetCode, in this order

- [ ] LC 187 Repeated DNA Sequences — Medium
- [ ] LC 718 Maximum Length of Repeated Subarray — Medium
- [ ] LC 1316 Distinct Echo Substrings — Hard
- [ ] LC 1044 Longest Duplicate Substring — Hard
- [ ] LC 1923 Longest Common Subpath — Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

11.5 Trie (prefix tree)

O(L)

Spot it when the problem looks like this

- *Starts with*, *shortest root*, *autocomplete*, *search with '.'*: queries about prefixes of the words in one dictionary.
- Find every dictionary word on a board or in a text at once, instead of one search per word.
- Lowercase letters only and a total word length up to about 10^5, so 26 links per node fit in memory.

How it works

A trie stores words letter by letter on the edges of a tree. Every word that starts with p passes through one node, the one reached by spelling p from the root. So `insert`, `search` and `startsWith` each follow at most L links, O(L) for a word of length L, however many words are stored. `end` marks the nodes where a word stops; without it, "app" is found as soon as "apple" is in.

A wildcard `.` matches any letter, so at a dot the search tries all 26 children. Fixed letters still narrow the search to one child.

For word search II, run the board backtracking of **7.6 Grid DFS with mark and restore** (p. 175) and carry the trie node of the path so far. A path that leaves the trie cannot start any word, so it stops at once. A word is reported at its end node and then cleared. A node with nothing left below it is cut from its parent, so later paths do not enter it.

Worked example **insert car, cat, cart, dog; * marks end = true**

```
(root)
  c
    a
      r *     car
        t *   cart
      t *     cat
  d
    o
      g *     dog
search("ca") = false: the node exists, but end is false
startsWith("ca") = true    matches(".a.") = true (car)    matches("d.") = false
```

Trie with Node[26] and an end flag

memorise

```
static class Node {
    Node[] next = new Node[26];      // child for each letter, or null
    boolean end;                     // some word ends at this node
}

static class Trie {                  // LC 208
    final Node root = new Node();

    void insert(String w) {
        Node cur = root;
        for (char c : w.toCharArray()) {
            if (cur.next[c - 'a'] == null) cur.next[c - 'a'] = new Node();
            cur = cur.next[c - 'a'];
        }
        cur.end = true;
    }

    // The node reached by spelling p from the root, or null.
    Node walk(String p) {
        Node cur = root;
        for (int i = 0; i < p.length() && cur != null; i++)
            cur = cur.next[p.charAt(i) - 'a'];
        return cur;
    }

    boolean search(String w) { Node x = walk(w); return x != null && x.end; }
    boolean startsWith(String p) { return walk(p) != null; }
}
```

Search with '.' wildcards

variant

```
// LC 211: a letter follows one child; '.' tries all 26 children.
// Call matches(trie.root, word, 0).
static boolean matches(Node x, String w, int i) {
    if (x == null) return false;
    if (i == w.length()) return x.end;
    char c = w.charAt(i);
    if (c != '.') return matches(x.next[c - 'a'], w, i + 1);
    for (Node child : x.next)
        if (matches(child, w, i + 1)) return true;
    return false;
}
```

Word search II (trie + backtracking)

example

```java
// LC 212: one backtracking walk of the board moves through the trie at
// the same time. An end node stores its word; clear it once found.
static class WNode { WNode[] next = new WNode[26]; String word; }

static List<String> findWords(char[][] board, String[] words) {
    WNode root = new WNode();
    for (String w : words) {
        WNode cur = root;
        for (char c : w.toCharArray()) {
            if (cur.next[c - 'a'] == null) cur.next[c - 'a'] = new WNode();
            cur = cur.next[c - 'a'];
        }
        cur.word = w;
    }
    List<String> found = new ArrayList<>();
    for (int r = 0; r < board.length; r++)
        for (int c = 0; c < board[0].length; c++) dfs(board, r, c, root, found);
    return found;
}

static void dfs(char[][] b, int r, int c, WNode parent, List<String> found) {
    if (r < 0 || c < 0 || r >= b.length || c >= b[0].length) return;
    char ch = b[r][c];
    if (ch == '#' || parent.next[ch - 'a'] == null) return;  // used, or no word
    WNode x = parent.next[ch - 'a'];
    if (x.word != null) { found.add(x.word); x.word = null; }  // report once
    b[r][c] = '#';                                  // mark the cell used
    dfs(b, r + 1, c, x, found); dfs(b, r - 1, c, x, found);
    dfs(b, r, c + 1, x, found); dfs(b, r, c - 1, x, found);
    b[r][c] = ch;                                   // undo
    if (x.word == null && isLeaf(x)) parent.next[ch - 'a'] = null;  // prune
}

static boolean isLeaf(WNode x) {
    for (WNode y : x.next) if (y != null) return false;
    return true;
}
```

Cost Time `O(L)` Space `O(26 · N)`

A word of length L follows L links. N is the total length of the words, and each of the at most N nodes has 26 links. Word search II is bounded by the backtracking, at most 4·3^(L−1) paths per start cell.

Variants same idea, small change

- Replace words (LC 648): insert the roots, then walk each word down the trie and stop at the first node with `end`. That prefix is the shortest root.
- Sum of prefix scores (LC 2416): keep a `cnt` in each node and add 1 along the path on every insert. A word's score is the sum of `cnt` along its path.
- Large or unknown alphabet: use a `HashMap<Character, Node>` for the children instead of an array.
- Maximum XOR of two numbers: a trie over the bits, with 2 children per node (12.3 **Binary trie for maximum XOR** (p. 345)).

Traps bugs people actually write

- `search` written as `walk(w) != null` is true for every prefix: after `insert("apple")`, `search("app")` returns true.
- Word search II without `x.word = null` reports a word once for each path that spells it: board `["aa"]` with words `["a"]` returns `[a, a]`.
- Without the prune line, used-up branches are walked again and again: a 12 × 12 board of a's with the words a to a^{10} makes 7,534,800 calls instead of 184.

Practise on LeetCode, in this order

☐	LC 208	Implement Trie (Prefix Tree)	Medium
☐	LC 648	Replace Words	Medium
☐	LC 211	Design Add and Search Words Data Structure	Medium
☐	LC 2416	Sum of Prefix Scores of Strings	Hard
☐	LC 212	Word Search II	Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

11.6 Manacher's algorithm

O(n)

Spot it when the problem looks like this

- Palindromic substrings with n up to 10^5 or more, where 11.1 **Expand around the centre** (p. 320) is $O(n^2)$ on inputs such as `aaaa…a`.
- You need the palindrome at **every** centre (the longest one inside each prefix, or ending at each i), not only the overall best.
- Many questions of the form *is s[l..r] a palindrome?* after one O(n) pass.

How it works

Put `#` between the chars and at both ends. Then t has 2n + 1 chars, and every palindrome of s, odd or even, is an odd palindrome of t with the same centre. `p[i]` is its radius in t, which equals its length in s.

Keep `c` and `r = c + p[c]` for the palindrome that reaches furthest right. For `i < r`, the mirror `j = 2c - i` lies in the same palindrome, so up to r the text around i mirrors the text around j, and `p[i] ≥ min(r - i, p[j])`. If `p[j] < r - i`, the palindrome at i stops where the one at j does, and the first compare fails. Only a palindrome that reaches r compares past r, and each success moves r right. r never moves left, so the compares total O(n), as with the box in 11.3 Z-function (p. 326).

The `#` never matters: a compare is always between two positions of the same parity, so `#` meets `#` and letters meet letters.

Worked example s = "babcbabc", t has 17 chars

```
i    0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16
t    #  b  #  a  #  b  #  c  #  b  #  a  #  b  #  c  #
p    0  1  0  3  0  1  0  7  0  1  0  5  0  1  0  1  0
i = 7:  no box covers it; 7 compares succeed, box (c, r) = (7, 14)
i = 9:  mirror 5, p[5] = 1 < 14 - 9: copy 1, the next compare fails
i = 11: mirror 3, p[3] = 3 = 14 - 11: start at 3, 2 more succeed -> 5
i = 15: box (11, 16), mirror 7: min(16 - 15, p[7] = 7) = 1, t ends there
longest: p[7] = 7, start (7 - 7) / 2 = 0 -> "babcbab"
count: sum of (p[i] + 1) / 2 = 14
```

Radius array on #a#b#..., mirror rule memorise

```
// t = "#s0#s1#...#s(n-1)#" has 2n + 1 chars and only odd palindromes.
// p[i] = radius of the longest palindrome of t centred at i. It equals the
// length of the palindrome of s with the same centre, which starts at
// s[(i - p[i]) / 2].
static int[] radii(String s) {
    int m = 2 * s.length() + 1;
    char[] t = new char[m];
    for (int i = 0; i < m; i++) t[i] = i % 2 == 0 ? '#' : s.charAt(i / 2);
    int[] p = new int[m];
    for (int i = 0, c = 0, r = 0; i < m; i++) {     // t[2c-r..r] is a palindrome
        if (i < r) p[i] = Math.min(r - i, p[2 * c - i]);   // mirror rule
        while (i - p[i] - 1 >= 0 && i + p[i] + 1 < m
                && t[i - p[i] - 1] == t[i + p[i] + 1]) p[i]++;
        if (i + p[i] > r) {                          // reaches further right
            c = i;
            r = i + p[i];
        }
    }
    return p;
}
```

Longest and count in O(n)

memorise

```java
static String longestPalindrome(String s) {           // LC 5 in O(n)
    int[] p = radii(s);
    int best = 0;
    for (int i = 1; i < p.length; i++) if (p[i] > p[best]) best = i;
    int start = (best - p[best]) / 2;
    return s.substring(start, start + p[best]);
}

// LC 647 in O(n): centre i holds palindromes of length p[i], p[i] - 2,
// ..., down to 1 or 2, which is (p[i] + 1) / 2 of them.
static long countSubstrings(String s) {
    long count = 0;
    for (int r : radii(s)) count += (r + 1) / 2;
    return count;
}
```

Two non-overlapping palindromes (LC 1960)

example

```java
// LC 1960: two non-overlapping odd palindromes, largest product of lengths.
// left[i] = longest one inside s[0..i], right[i] = inside s[i..n-1].
static long maxProduct(String s) {
    int n = s.length();
    int[] p = radii(s), left = new int[n], right = new int[n];
    for (int j = 0; j < n; j++) {          // odd centre s[j] is t[2j + 1]
        int len = p[2 * j + 1], h = len / 2;
        left[j + h] = Math.max(left[j + h], len);     // ends at j + h
        right[j - h] = Math.max(right[j - h], len);   // starts at j - h
    }
    // Drop one char from each end: a palindrome ending at i + 1 gives a
    // palindrome two shorter ending at i (and the same for starts).
    for (int i = n - 2; i >= 0; i--) left[i] = Math.max(left[i], left[i + 1] - 2);
    for (int i = 1; i < n; i++) right[i] = Math.max(right[i], right[i - 1] - 2);
    for (int i = 1; i < n; i++) left[i] = Math.max(left[i], left[i - 1]);
    for (int i = n - 2; i >= 0; i--) right[i] = Math.max(right[i], right[i + 1]);
    long best = 0;
    for (int i = 0; i + 1 < n; i++)                  // split after i
        best = Math.max(best, (long) left[i] * right[i + 1]);
    return best;
}
```

Cost Time `O(n)` Space `O(n)`

r only moves right, so successful compares total at most 2n + 1, and each i adds at most one failed compare. t and p have 2n + 1 slots.

Variants same idea, small change

- Retry LC 5 and LC 647 (listed under **11.1 Expand around the centre** (p. 320)) with this template; LC 647 sums `(p[i] + 1) / 2`.
- Is `s[l..r]` a palindrome? Its centre in t is `l + r + 1`, so the answer is `p[l + r + 1] >= r - l + 1`. With that O(1) test, palindrome partitioning IV tries every pair of cuts in $O(n^2)$.
- Longest palindromic prefix (shortest palindrome, LC 214): the largest `p[i]` with `i - p[i] == 0`. Same answer as **11.2 KMP prefix function** (p. 323) on `s + '#' + reverse(s)`.
- DFS strings of a tree (LC 3327): build one postorder string, run Manacher's once, and test each subtree's block with the O(1) check (**8.13 Euler tour (subtree = range)** (p. 218)).

Traps bugs people actually write

- The mirror without the `min`: `p[i] = p[2*c - i]` claims chars past r that were never compared. On `"baba"` it sets `p[7] = 3` instead of 1, and the count says 7 instead of 6.
- The count reaches $n(n+1)/2 = 5 \cdot 10^9$ for 10^5 equal letters. Keep it in a `long`; LC 647 returns `int` only because $n \le 1000$ there.
- LC 1960 on 10^5 a's: the best product is $49999 \cdot 50001 = 2{,}499{,}999{,}999$, more than `int` holds. Multiply as `long`.

Practise on LeetCode, in this order

- [ ] `LC 1745` Palindrome Partitioning IV — Hard
- [ ] `LC 3327` Check if DFS Strings Are Palindromes — Hard
- [ ] `LC 1960` Maximum Product of the Length of Two Palindromic Substrings — Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

Part 12

Math and bits

These patterns work on the numbers themselves instead of on a data structure. Bit tricks and XOR turn sets and parity into O(1) operations, a binary trie finds the best XOR partner one bit at a time, and gcd, fast powers, sieves, binomials and matrix powers answer counting and divisibility questions that a loop up to n cannot. Reach for this part when the statement says *bitwise*, *XOR*, *divides*, *prime* or *modulo* $10^9 + 7$, or when n is up to 10^{18}.

12.1	**Bit tricks: lowest bit, submasks** Test, set or count bits of an int in O(1), or visit every subset of a small set.	O(1) per op	p. 339
12.2	**XOR tricks (pairs cancel)** Every value appears twice except one, or the XOR of a range, in O(n) time and O(1) space.	O(n)	p. 342
12.3	**Binary trie for maximum XOR** Maximum XOR of a value with any element of a set, for many values or queries.	O(n·B)	p. 345
12.4	**GCD, LCM and extended Euclid** Largest common divisor, a common period, or which amounts a·x + b·y can reach.	O(log n)	p. 348
12.5	**Fast power and modular inverse** a^n modulo $10^9 + 7$ with n up to 10^{18}, or a division under a prime modulus.	O(log n)	p. 350
12.6	**Prime sieve and factorisation** All primes up to n, or the prime factors of many numbers up to about 10^7.	O(n log log n)	p. 353
12.7	**nCr mod p (factorial tables)** Count ways to choose positions, modulo a prime, with many C(n, r) up to $n \approx 10^6$.	O(n) + O(1)	p. 356
12.8	**Matrix power for recurrences** The n-th term of a linear recurrence, or strings of length n, with n up to 10^{18}.	O(k³ log n)	p. 359

12.1 Bit tricks: lowest bit, submasks

O(1) per op

Spot it when the problem looks like this

- The statement says *bits*, *binary representation*, *bitwise AND/OR* or *power of two*, or packs up to 20 or 32 flags into one int.
- Count or change 1 bits without building a string, in O(1) or O(number of 1 bits).
- A DP or search over subsets needs every subset of a given set: for n ≤ 15 that is $3^n \approx 1.4 \cdot 10^7$ steps in total.

How it works

Subtracting 1 turns the lowest 1 bit of x into 0 and every 0 below it into 1; the higher bits stay. So `x & (x - 1)` clears only the lowest 1. In two's complement `-x` is `~x + 1`. The +1 carries through the trailing 1s of `~x` and stops at the lowest 1 of x, so that bit is the only 1 that `-x` and x share, and `x & -x` keeps it alone.

`i >> 1` is i without its last bit, so `bits[i] = bits[i >> 1] + (i & 1)`.

For submasks, `s - 1` clears the lowest 1 of s and sets all bits below it; `& mask` drops the bits outside mask. That is the largest submask below s, so the loop visits each of the 2^k submasks once, for k set bits in mask. Over all masks of n bits it is 3^n steps: each bit is out of the mask, in the mask only, or in both.

Worked example **submasks(13), mask = 1101: s = (s - 1) & mask**

```
s     s - 1   (s - 1) & mask   s in decimal
1101  1100    1100             13
1100  1011    1001             12
1001  1000    1000              9
1000  0111    0101              8   bit 1 is not in mask
0101  0100    0100              5
0100  0011    0001              4
0001  0000    0000              1
0000  -       -                 0   added, then break
```

Test, set, clear, lowest bit, popcount memorise

```
// Bit i counts from 0 at the right (the 1s place).
static boolean has(int x, int i)  { return ((x >> i) & 1) == 1; }
static int set(int x, int i)      { return x | (1 << i); }
static int clear(int x, int i)    { return x & ~(1 << i); }
static int toggle(int x, int i)   { return x ^ (1 << i); }
static int dropLowest(int x)      { return x & (x - 1); }   // 101100 -> 101000
static int lowestBit(int x)       { return x & -x; }        // 101100 -> 000100

// Number of 1 bits: each round removes the lowest one. Same as Integer.bitCount.
static int popcount(int x) {
    int c = 0;
    for (; x != 0; x &= x - 1) c++;
    return c;
}

static boolean isPowerOfTwo(int n) { return n > 0 && (n & (n - 1)) == 0; }

// bits[i] = number of 1 bits in i; i >> 1 is i without its last bit.
static int[] countBits(int n) {
    int[] bits = new int[n + 1];
    for (int i = 1; i <= n; i++) bits[i] = bits[i >> 1] + (i & 1);
    return bits;
}
```

Every submask of a mask

memorise

```java
// Every submask of mask, largest first, 0 last.
// (s - 1) & mask is the largest submask below s.
static List<Integer> submasks(int mask) {
    List<Integer> out = new ArrayList<>();
    for (int s = mask; ; s = (s - 1) & mask) {
        out.add(s);
        if (s == 0) break;          // 0 is a submask too; stop after it
    }
    return out;
}
```

The usual loop `for (int s = mask; s > 0; s = (s - 1) & mask)` skips the empty set.

Reverse bits; minimum flips for a | b == c

example

```java
// Reverse bits: move n's lowest bit onto r, 32 times.
static int reverseBits(int n) {
    int r = 0;
    for (int i = 0; i < 32; i++) {
        r = (r << 1) | (n & 1);
        n >>>= 1;                   // >>> shifts in 0s, also when n < 0
    }
    return r;
}

// Minimum flips so that (a | b) == c, one bit position at a time.
static int minFlips(int a, int b, int c) {
    int flips = 0;
    for (int i = 0; i < 31; i++) {
        int x = (a >> i) & 1, y = (b >> i) & 1, z = (c >> i) & 1;
        if (z == 1) flips += (x | y) == 0 ? 1 : 0;   // turn one of them on
        else flips += x + y;                          // turn both off
    }
    return flips;
}
```

Cost Time `O(1) per op` Space `O(1)`

popcount runs once per 1 bit, countBits is O(n), and the submask loop is $O(2^k)$ for k set bits in mask.

Variants same idea, small change

- Visit only the set bits: `for (int m = x; m != 0; m &= m - 1)` and take `Integer.numberOfTrailingZeros(m)` as the index.
- Masks wider than 31 bits: `long` with `1L << i`, `Long.bitCount`, `Long.numberOfTrailingZeros`.
- Minimum flips in one line: `Integer.bitCount((a | b) ^ c) + Integer.bitCount(a & b & ~c)`. The second term pays the extra flip where c is 0 and both a and b are 1.
- Gray code (LC 89): `i ^ (i >> 1)` for i = 0..2^n – 1 lists every n-bit mask, and neighbours differ in one bit.

Traps bugs people actually write

- `1 << i` is an `int`: `1 << 31` is negative and `1 << 32` is 1, because Java uses the shift count mod 32. In a `long` mask, `mask | (1 << 40)` sets bit 8. Write `1L << i`.
- `while (n != 0) { c += n & 1; n >>= 1; }` never ends for a negative n: `-1 >> 1` is –1. Use `>>>`, or `n &= n - 1`.
- `+` binds tighter than shifts: `l + (r - l) >> 1` is `r >> 1`, so l = 10, r = 20 gives 10, not 15. Bracket the shift.
- `(n & (n - 1)) == 0` alone calls 0 and `Integer.MIN_VALUE` powers of two. Test `n > 0` first.

Practise on LeetCode, in this order

☐	LC 231	Power of Two	Easy
☐	LC 191	Number of 1 Bits	Easy
☐	LC 338	Counting Bits	Easy
☐	LC 190	Reverse Bits	Easy
☐	LC 1318	Minimum Flips to Make a OR b Equal to c	Medium
☐	LC 1178	Number of Valid Words for Each Puzzle	Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

12.2 XOR tricks (pairs cancel) O(n)

Spot it when the problem looks like this

- *Every element appears twice (or three times) except one*, with *linear runtime* and *constant extra space*.
- One value of 0..n is missing, or one letter was added to a shuffled copy of a string.
- The XOR of a subarray, or an array given as XORs of neighbours: `encoded[i] = arr[i] ^ arr[i + 1]`.

How it works

XOR adds each bit mod 2 with no carry. So it is associative and commutative, `a ^ 0 == a` and `a ^ a == 0`. The XOR of a list, in any order, is the XOR of the values that appear an odd number of times. That gives *single number* in one pass, and *missing number*: XOR every index 0..n with every value, and each present number meets itself.

With two lonely values x and y the total is `x ^ y`, which is not 0. Its lowest 1 bit, `xy & -xy`, is a bit where x and y differ. Split the values by that bit: both copies of a pair land in one group, and x and y land in different groups. XOR one group to get x.

Three copies leave one copy behind, so for single number II count bit i over all values instead: the count is 3k plus the lonely value's bit. Prefix XOR works like a prefix sum, because XOR undoes itself.

Worked example singleNumberIII([1, 2, 1, 3, 2, 5])

```
xy = 1 ^ 2 ^ 1 ^ 3 ^ 2 ^ 5 = 6 = 110; bit = xy & -xy = 010
v  binary  v & bit  x (group with bit)  rest (not computed)
1  001     0        0                   1
2  010     set      2                   1
1  001     0        2                   0
3  011     set      1                   0
2  010     set      3                   0
5  101     0        3                   5
x = 3, y = xy ^ x = 6 ^ 3 = 5. Each pair met itself inside one group.
```

Single number, missing number, two singles

memorise

```java
// Every value appears twice except one: the pairs cancel (a ^ a == 0).
static int singleNumber(int[] a) {
    int x = 0;
    for (int v : a) x ^= v;
    return x;
}

// a holds n distinct values from 0..n: XOR every index and every value.
static int missingNumber(int[] a) {
    int x = a.length;
    for (int i = 0; i < a.length; i++) x ^= i ^ a[i];
    return x;
}

// Two values x != y appear once, all others twice.
static int[] singleNumberIII(int[] a) {
    int xy = 0;
    for (int v : a) xy ^= v;                // = x ^ y, not 0
    int bit = xy & -xy;                     // a bit where x and y differ
    int x = 0;
    for (int v : a) if ((v & bit) != 0) x ^= v;   // only x's group
    return new int[] {x, xy ^ x};
}
```

Single number II (bit counts mod 3)

variant

```java
// Every value appears three times except one: count each bit mod 3.
static int singleNumberII(int[] a) {
    int ans = 0;
    for (int i = 0; i < 32; i++) {
        int ones = 0;
        for (int v : a) ones += (v >> i) & 1;
        if (ones % 3 != 0) ans |= 1 << i;    // i = 31 sets the sign bit
    }
    return ans;
}
```

Prefix XOR queries; decode XORed array

variant

```java
// p[i] = a[0] ^ ... ^ a[i-1]; the XOR of a[l..r] is p[r + 1] ^ p[l].
static int[] xorQueries(int[] a, int[][] queries) {
    int[] p = new int[a.length + 1];
    for (int i = 0; i < a.length; i++) p[i + 1] = p[i] ^ a[i];
    int[] ans = new int[queries.length];
    for (int q = 0; q < queries.length; q++)
        ans[q] = p[queries[q][1] + 1] ^ p[queries[q][0]];
    return ans;
}

// encoded[i] = arr[i] ^ arr[i + 1], so arr[i + 1] = encoded[i] ^ arr[i].
static int[] decode(int[] encoded, int first) {
    int[] arr = new int[encoded.length + 1];
    arr[0] = first;
    for (int i = 0; i < encoded.length; i++) arr[i + 1] = encoded[i] ^ arr[i];
    return arr;
}
```

Cost Time `O(n)` Space `O(1)`

One or two passes; single number II makes 32 passes. Prefix XOR keeps an O(n) array and answers each query in O(1).

Variants same idea, small change

- Every value appears k times except one: the same bit count with `% k`.
- Find the difference (LC 389): XOR every char of s and t; the added letter is what is left.
- Count triplets with equal XOR (LC 1442): `a[i..j-1]` and `a[j..k]` have equal XOR exactly when `p[i] == p[k + 1]`, and then every j in `i+1..k` works, so add `k - i` per equal pair.
- Decode XORed permutation (LC 1734): the XOR of 1..n is known, and n is odd, so `encoded[1] ^ encoded[3] ^ ...` is the XOR of `arr[1..n-1]`; the two together give `arr[0]`.

Traps bugs people actually write

- `if ((v & bit) == 1)` is true only when bit is 1. On `[1, 2, 1, 3, 2, 5]` the bit is 2, no value matches, and you return `[0, 6]`.
- XOR-ing everything for single number II: `[2, 2, 3, 2]` gives `2 ^ 2 ^ 3 ^ 2 = 1`, not 3.
- Looping `i < 31` in the bit count drops the sign bit: −4 comes back as 2147483644.
- `p[r] ^ p[l]` with p of length n + 1 leaves out `a[r]`. The range `[l, r]` is `p[r + 1] ^ p[l]`.

Practise on LeetCode, in this order

- [ ] LC 136 Single Number — Easy
- [ ] LC 268 Missing Number — Easy
- [ ] LC 1720 Decode XORed Array — Easy
- [ ] LC 1310 XOR Queries of a Subarray — Medium
- [ ] LC 260 Single Number III — Medium
- [ ] LC 137 Single Number II — Medium

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

12.3 Binary trie for maximum XOR

O(n·B)

Spot it when the problem looks like this

- *Maximum XOR of two numbers* in an array of up to $2 \cdot 10^5$ values: checking all pairs ($O(n^2)$) is too slow.
- Each query allows only some elements: values ≤ m, a sliding window, the nodes on a tree path.
- Values are below 2^{31}, so each one is a path of 31 bits.

How it works

Bit b is worth 2^b, more than all lower bits together ($2^b - 1$). So the best partner for x is decided from the top bit down. Among the values that match the choices so far, if one has the opposite of x's bit b, the answer has a 1 there, and every value without it loses, whatever its lower bits are. A binary trie stores each value as a path of bits 30..0. The values that match the choices so far are the subtree under the current node, so each greedy choice is one step to a child: 31 steps per insert or query.

For a pair, insert each value and then query it. The later value of the best pair finds the earlier one, so nothing is missed.

When a query may only use values ≤ m, sort the values and the queries by m. Before each query, insert the values ≤ its m. The trie then holds exactly the allowed values, and each value goes in once.

Worked example **findMaximumXOR([3, 10, 5, 25, 2, 8]): the query for v = 25**

```
x = 25 = 11001; trie holds 3, 10, 5, 25 (00011 01010 00101 11001)
bits 30..5 are 0 in every value: want 1, no child, go to 0, best stays 0
b  x's bit  want  child?  values left on the path  best
4     1      0    yes     3, 10, 5                 10000
3     1      0    yes     3, 5                     11000
2     0      1    yes     5                        11100
1     0      1    no      5                        11100
0     1      0    no      5                        11100 = 28 = 25 ^ 5
```

Binary trie and the max XOR pair

memorise

```
// Binary trie over bits 30..0 (values 0 <= v < 2^31), for max XOR queries.
static class XorTrie {
    static final int HIGH = 30;
    final int[][] child;          // child[bit][node]; 0 means none
    int nodes = 1;                // node 0 is the root

    XorTrie(int maxValues) {
        child = new int[2][maxValues * (HIGH + 1) + 1];
    }

    void insert(int v) {
        int cur = 0;
        for (int b = HIGH; b >= 0; b--) {
            int bit = (v >> b) & 1;
            if (child[bit][cur] == 0) child[bit][cur] = nodes++;
            cur = child[bit][cur];
        }
    }

    // max of x ^ v over the inserted v (the trie must not be empty)
    int maxXor(int x) {
        int cur = 0, best = 0;
        for (int b = HIGH; b >= 0; b--) {
            int want = ((x >> b) & 1) ^ 1;          // the opposite bit
            if (child[want][cur] != 0) {
                best |= 1 << b;                     // bit b of x ^ v is 1
                cur = child[want][cur];
            } else {
                cur = child[want ^ 1][cur];
            }
        }
        return best;
    }
}

static int findMaximumXOR(int[] a) {
    XorTrie t = new XorTrie(a.length);
    int best = 0;
    for (int v : a) {
        t.insert(v);                                // insert first: never empty
        best = Math.max(best, t.maxXor(v));
    }
    return best;
}
```

Max XOR with a limit, queries offline

variant

```
// queries[i] = {x, m}: max x ^ v over nums v <= m, or -1 if no v <= m.
// Offline: answer queries by increasing m, inserting nums as they qualify.
static int[] maximizeXor(int[] nums, int[][] queries) {
    Arrays.sort(nums);
    Integer[] order = new Integer[queries.length];
    for (int i = 0; i < order.length; i++) order[i] = i;
    Arrays.sort(order, (p, q) -> Integer.compare(queries[p][1], queries[q][1]));
    XorTrie t = new XorTrie(nums.length);
    int[] ans = new int[queries.length];
    int j = 0;                                          // nums[0..j) are inserted
    for (int qi : order) {
        while (j < nums.length && nums[j] <= queries[qi][1]) t.insert(nums[j++]);
        ans[qi] = j == 0 ? -1 : t.maxXor(queries[qi][0]);
    }
    return ans;
}
```

Cost Time `O(n·B)` Space `O(n·B)`

B = 31 bits: every insert and query walks B levels, and the trie has at most n·B + 1 nodes. The offline version adds O(n log n + q log q) to sort.

Variants same idea, small change

- Deletions: keep a count per node, add 1 along the path on insert and subtract 1 on delete, and treat a child with count 0 as missing. Maximum strong pair XOR II takes the values in increasing order as y, inserts y, deletes the smallest x while `2 * x < y`, then queries y.
- Count pairs with XOR < K (LC 1803 takes `f(high + 1) - f(low)`): keep a count per node and walk K's bits. Where K has a 1, add the count of the child that makes this XOR bit 0 and move to the other child; where K has a 0, move to the child that makes it 0.
- Without a trie (LC 421): decide the answer from the top bit, keep the prefixes `v >> b` in a hash set, and test whether `candidate ^ p` is in the set for some prefix p.
- Maximum genetic difference (LC 1938): DFS the tree, insert a node on entry and delete it on exit, so the trie holds exactly the root path.

Traps bugs people actually write

- Walking the bits from 0 upwards makes the greedy pick low bits first: with values {1, 2} and x = 0 it returns 1; the answer is 2.
- Querying an empty trie returns 0 with this code. LC 1707 wants −1 when no value is ≤ m: test `j == 0`.
- `new int[cap][2]` reads the same but makes one small array per node: for n = $2 \cdot 10^5$ it uses 174 MB, against 50 MB for `new int[2][cap]` (measured).
- Too few levels: `HIGH = 29` is enough for LC 1707 (values ≤ 10^9), but on LC 421 the pair `[0, 2^30]` then gives 0.

Practise on LeetCode, in this order

- [ ] LC 2932 Maximum Strong Pair XOR I — Easy
- [ ] LC 421 Maximum XOR of Two Numbers in an Array — Medium
- [ ] LC 1707 Maximum XOR With an Element From Array — Hard
- [ ] LC 2935 Maximum Strong Pair XOR II — Hard
- [ ] LC 1803 Count Pairs With XOR in a Range — Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

12.4 GCD, LCM and extended Euclid

`O(log n)`

Spot it when the problem looks like this

- *Greatest common divisor*, *largest X that divides every*, *split into groups of equal size*, or a string that *divides* two strings.
- Can a target be built from steps or jugs of sizes a and b? Can some subset combine to 1?
- Least common multiple, a simplified fraction, or two cycles that line up every lcm(a, b) steps.

How it works

A number d divides both a and b exactly when it divides both b and `a % b`, because `a % b` is a − q·b. So (a, b) can become (b, `a % b`) without changing the gcd, and when b reaches 0 the answer is a. Every two steps the larger number at least halves: if b ≤ a/2 then `a % b` < b ≤ a/2, and otherwise `a % b` = a − b < a/2. That is O(log min(a, b)) steps. Consecutive Fibonacci numbers are the worst case: gcd(F_{45}, F_{44}) takes 43 steps.

Extended Euclid carries coefficients back up the same calls. Write q = a/b, rounded down. If b·x1 + (a % b)·y1 = g, put in a % b = a − q·b and regroup: a·y1 + b·(x1 − q·y1) = g. This proves Bézout: a·x + b·y = c has integer solutions exactly when gcd(a, b) divides c.

For the lcm, divide first: `a / gcd(a, b) * b` never exceeds the result.

Worked example **extGcd(252, 105) = {21, -2, 5}**

```
call              a/b  a%b  returns {g, x, y}   a*x + b*y
extGcd(252, 105)   2    42  {21, -2, 5}         252*(-2) + 105*5 = 21
extGcd(105, 42)    2    21  {21, 1, -2}         105*1 + 42*(-2)  = 21
extGcd(42, 21)     2     0  {21, 0, 1}          42*0 + 21*1      = 21
extGcd(21, 0)      -     -  {21, 1, 0}          base case
Calls go down; x, y are built on the way back up with
x = y1, y = x1 - (a/b)*y1. Row 2 from row 3: x = 1, y = 0 - 2*1 = -2.
```

gcd, lcm and extended Euclid

memorise

```java
// Euclid for a, b >= 0: gcd(a, b) = gcd(b, a % b), and gcd(a, 0) = a.
static long gcd(long a, long b) {
    while (b != 0) {
        long t = a % b;
        a = b;
        b = t;
    }
    return a;
}

static long lcm(long a, long b) { return a / gcd(a, b) * b; }   // divide first

// Extended Euclid: returns {g, x, y} with a*x + b*y == g == gcd(a, b).
static long[] extGcd(long a, long b) {
    if (b == 0) return new long[] {a, 1, 0};
    long[] r = extGcd(b, a % b);            // b*x1 + (a % b)*y1 == g
    long x = r[2], y = r[1] - (a / b) * r[2];
    return new long[] {r[0], x, y};
}
```

Arrays, strings, decks and water jugs

example

```java
// GCD of an array (LC 1979 wants gcd(min, max)); gcd(0, v) == v starts the fold.
static long gcdAll(int[] a) {
    long g = 0;
    for (int v : a) g = gcd(g, v);
    return g;
}

// GCD of strings: some string divides both iff s + t equals t + s.
static String gcdOfStrings(String s, String t) {
    if (!(s + t).equals(t + s)) return "";
    return s.substring(0, (int) gcd(s.length(), t.length()));
}

// Deck of cards: groups of equal size X >= 2 exist iff gcd of the counts >= 2.
static boolean hasGroupsSizeX(int[] deck) {
    Map<Integer, Integer> cnt = new HashMap<>();
    for (int c : deck) cnt.merge(c, 1, Integer::sum);
    long g = 0;
    for (int c : cnt.values()) g = gcd(g, c);
    return g >= 2;
}

// Water jugs (x, y >= 1): every reachable amount is a*x + b*y (Bezout),
// so z works iff it fits in both jugs and gcd(x, y) divides it.
static boolean canMeasureWater(int x, int y, int z) {
    return z <= (long) x + y && z % gcd(x, y) == 0;
}
```

`Math` has no gcd, and `BigInteger.gcd` creates objects on every call. Write the loop.

Cost Time `O(log n)` Space `O(1)`
n = min(a, b). The larger value halves every two steps; extGcd recurses to the same depth.

Variants same idea, small change

- Good array (LC 1250): some subset combines to 1 exactly when the gcd of all values is 1 (Bézout).
- Inverse mod m when m is not prime: `extGcd(a, m)` gives x with a·x ≡ 1 (mod m) if g = 1; use `((x % m) + m) % m`.
- Nth magical number: v has `v/a + v/b - v/lcm(a, b)` multiples of a or b up to it; binary search the smallest v with count ≥ n (3.3 **Binary search on the answer** (p. 74)).

Traps bugs people actually write

- `a * b / gcd(a, b)` overflows first: in `int`, 60000 · 90000 wraps to 1105032704, and the lcm comes out as 36834 instead of 180000. Write `a / gcd(a, b) * b` in `long`.
- Java's `%` keeps the sign of the dividend, so `gcd(4, -6)` returns −2. Take `Math.abs` first when values can be negative.
- Water jugs with only the gcd test say yes to x = 3, y = 5, z = 9; 9 litres do not fit. Check `z <= x + y`.
- x and y from extGcd are often negative (252·(−2) + 105·5 = 21); reduce them mod m before use.

Practise on LeetCode, in this order

☐	`LC 1979`	Find Greatest Common Divisor of Array	Easy
☐	`LC 1071`	Greatest Common Divisor of Strings	Easy
☐	`LC 914`	X of a Kind in a Deck of Cards	Easy
☐	`LC 365`	Water and Jug Problem	Medium
☐	`LC 1250`	Check If It Is a Good Array	Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

12.5 Fast power and modular inverse `O(log n)`

Spot it when the problem looks like this

- *Modulo $10^9 + 7$*, and the answer is a power: k choices at each of n positions, n up to 10^{15}.
- A division under a prime modulus (a probability, n! / k!): multiply by the inverse.
- pow(x, n) by hand with n down to -2^{31}, or an exponent too large for any type (given as digits).

How it works

Write exp in binary. For exp = 13 = 1101_2, $a^{13} = a^8 \cdot a^4 \cdot a^1$. Squaring `base` each round makes it a, a^2, a^4, a^8, ..., and the loop multiplies it into `result` when the matching bit of exp is 1. That is $\lfloor \log_2 \text{exp} \rfloor + 1$ rounds

with at most two multiplications each, so exp = 10^{18} takes 60 rounds. Reducing after every product keeps both factors below m, so a product is below $m^2 \approx 10^{18}$, which fits in a `long` (up to $9.2 \cdot 10^{18}$).

Fermat's little theorem: for a prime p that does not divide a, $a^{p-1} \equiv 1 \pmod p$. So $a \cdot a^{p-2} \equiv 1$, and a^{p-2} is the inverse of a. Dividing by a mod p means multiplying by it. That is how n! / (r!·(n – r)!) is taken mod $10^9 + 7$ in 12.7 nCr mod p (factorial tables) (p. 356).

For doubles, $x^{-n} = (1/x)^n$. Negate n in a `long`: in `int`, `-Integer.MIN_VALUE` is still negative.

Worked example modPow(3, 13, $10^9 + 7$)

```
exp (binary)  bit  result                 base after squaring
1101          1    1 * 3 = 3               3^2 = 9
110           0    3                       9^2 = 81
11            1    3 * 81 = 243            81^2 = 6561
1             1    243 * 6561 = 1594323    6561^2 = 43046721
exp = 0: stop. 13 = 8 + 4 + 1, so 3^13 = 3^8 * 3^4 * 3^1 = 1594323
```

Fast power mod m; inverse mod a prime

memorise

```
static final long MOD = 1_000_000_007L;

// base^exp mod m by squaring. m < 3e9, so every product fits in a long.
static long modPow(long base, long exp, long m) {
    long result = 1 % m;
    base %= m;
    if (base < 0) base += m;
    while (exp > 0) {
        if ((exp & 1) == 1) result = result * base % m;   // this bit is 1
        base = base * base % m;                            // a^1, a^2, a^4, ...
        exp >>= 1;
    }
    return result;
}

// Inverse of a modulo a prime p, for a % p != 0 (Fermat: a^(p-1) == 1).
static long modInverse(long a, long p) { return modPow(a, p - 2, p); }
```

pow(x, n) for doubles

variant

```
// x^n for doubles. Negate n as a long: -Integer.MIN_VALUE overflows an int.
static double myPow(double x, int n) {
    long e = n;
    if (e < 0) { x = 1 / x; e = -e; }
    double r = 1;
    while (e > 0) {
        if ((e & 1) == 1) r *= x;
        x *= x;
        e >>= 1;
    }
    return r;
}
```

Count good numbers; super pow

example

```
// Count good numbers (LC 1922): 5 even digits at the (n + 1) / 2 even
// indexes, 4 prime digits at the n / 2 odd ones.
static long countGoodNumbers(long n) {
    return modPow(5, (n + 1) / 2, MOD) * modPow(4, n / 2, MOD) % MOD;
}

// Super pow (LC 372): a^b mod 1337, b given as decimal digits.
// Reading one more digit d: a^(10k + d) = (a^k)^10 * a^d.
static int superPow(int a, int[] b) {
    long r = 1;
    for (int d : b) r = modPow(r, 10, 1337) * modPow(a, d, 1337) % 1337;
    return (int) r;
}
```

Cost Time `O(log n)` Space `O(1)`

n = exp: one squaring per bit of exp, plus one multiplication per 1 bit. An inverse mod $10^9 + 7$ is one power of about 30 rounds.

Variants same idea, small change

- Exponent given as digits (super pow): fold digit by digit, $r \to r^{10} \cdot a^d$. For a prime p that does not divide a, you may instead reduce the exponent mod p – 1.
- Inverse modulo a number that is not prime: extended Euclid (12.4 **GCD, LCM and extended Euclid** (p. 348)), and only when gcd(a, m) = 1.
- Inverses of all of 1..n in O(n): `inv[i] = (p - p / i) * inv[p % i] % p`, with `inv[1] = 1`.
- The n-th term of a linear recurrence: the same loop with matrix products (12.8 **Matrix power for recurrences** (p. 359)).

Traps bugs people actually write

- `int` products overflow before the `%`: $(10^9 + 6)^2$ mod $(10^9 + 7)$ is 1, but in `int` arithmetic it comes out as 923446813. Keep `base` and `result` in `long`.
- With `int n`, `n = -n` leaves `Integer.MIN_VALUE` negative, the loop never runs, and pow(2.0, -2^{31}) returns 1.0 instead of 0.0.
- Java's `%` can be negative: `(3 - 5) % 7` is –2. Write `((a - b) % m + m) % m`; that is also why modPow fixes a negative base.
- Fermat needs a prime modulus and `a % p != 0`. `modInverse(0, p)` returns 0 with no error, so check for 0 first.

Practise on LeetCode, in this order

☐	LC 50	Pow(x, n)	Medium
☐	LC 2550	Count Collisions of Monkeys on a Polygon	Medium
☐	LC 1922	Count Good Numbers	Medium
☐	LC 372	Super Pow	Medium
☐	LC 1969	Minimum Non-Zero Product of the Array Elements	Medium

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

12.6 Prime sieve and factorisation

`O(n log log n)`

Spot it when the problem looks like this

- *Count primes less than n*, or primes in a range, with n up to $5 \cdot 10^6$ or 10^7: a trial division per number is too slow.
- Many values up to 10^5 or 10^6 need their prime factors: *distinct prime factors*, *share a common factor greater than 1*.
- One number up to 10^{12} to test or factorise: trial division up to $\sqrt{x}$.

How it works

Every composite c has a prime factor $p \le \sqrt{c}$. So crossing out the multiples of every prime $p \le \sqrt{n}$ removes every composite up to n, and what is left is prime. When `isPrime[p]` is still true at step p, no smaller prime divides p, so p is prime. Start at $p \cdot p$: a smaller multiple $k \cdot p$ with $k < p$ has a prime factor below p and is already out. The work is $n/2 + n/3 + n/5 + \ldots$ over the primes, which is O(n log log n).

The SPF sieve keeps the first prime that reaches each cell, which is the smallest one. Then x factorises by dividing by `spf[x]` again and again. Each division at least halves x, so it takes at most $\log_2 x$ steps.

For a few numbers up to 10^{12}, use trial division. If $x = a \cdot b$ then $\min(a, b) \le \sqrt{x}$, so trying d up to $\sqrt{x}$ (10^6 steps) finds a factor or proves x prime.

Worked example **sieve(30), then distinctPrimes(84) with spfSieve(100)**

```
p  isPrime[p]  crosses out (new)                      already out
2  yes         4 6 8 10 12 14 16 18 20 22 24 26 28 30
3  yes         9 15 21 27                             12 18 24 30
4  no          -
5  yes         25                                     30
6  stop: 6*6 = 36 > 30
primes <= 30: 2 3 5 7 11 13 17 19 23 29
distinctPrimes(84): spf 2 -> 21, spf 3 -> 7, spf 7 -> 1, gives [2, 3, 7]
```

Sieve of Eratosthenes; count primes

memorise

```java
// isPrime[x] for 0 <= x <= n. Each prime p crosses out p*p, p*p + p, ...
static boolean[] sieve(int n) {
    boolean[] isPrime = new boolean[n + 1];
    if (n >= 2) Arrays.fill(isPrime, 2, n + 1, true);
    for (int p = 2; (long) p * p <= n; p++)
        if (isPrime[p])
            for (int m = p * p; m <= n; m += p) isPrime[m] = false;
    return isPrime;
}

static int countPrimes(int n) {                 // primes strictly below n
    int c = 0;
    for (boolean b : sieve(n - 1)) if (b) c++;
    return c;
}
```

Smallest prime factor sieve

memorise

```java
// spf[x] = smallest prime factor of x, for 2 <= x <= n.
static int[] spfSieve(int n) {
    int[] spf = new int[n + 1];
    for (int p = 2; p <= n; p++)
        if (spf[p] == 0)                        // no smaller prime divides p
            for (int m = p; m <= n; m += p)
                if (spf[m] == 0) spf[m] = p;
    return spf;
}

// Distinct prime factors of x, smallest first, in O(log x) divisions.
static List<Integer> distinctPrimes(int x, int[] spf) {
    List<Integer> out = new ArrayList<>();
    while (x > 1) {
        int p = spf[x];
        out.add(p);
        while (x % p == 0) x /= p;              // remove every copy of p
    }
    return out;
}
```

Trial division for one number

variant

```java
// One number: if x = a * b then min(a, b) <= sqrt(x). O(sqrt x) steps.
static List<Long> primeFactors(long x) {        // with repeats, ascending
    List<Long> out = new ArrayList<>();
    for (long d = 2; d * d <= x; d++)
        while (x % d == 0) { out.add(d); x /= d; }
    if (x > 1) out.add(x);                      // what is left is a prime
    return out;
}

static boolean isPrime(long x) {
    if (x < 2) return false;
    for (long d = 2; d * d <= x; d++) if (x % d == 0) return false;
    return true;
}
```

Largest component by common factor

example

```java
// Largest component by common factor (LC 952): join every value with each
// of its primes, so two values that share a prime end up in one set.
static int largestComponentSize(int[] nums) {
    int max = 0;
    for (int v : nums) max = Math.max(max, v);
    int[] spf = spfSieve(max);
    DSU d = new DSU(max + 1);                   // nodes: values and primes
    for (int v : nums)
        for (int p : distinctPrimes(v, spf)) d.union(v, p);
    Map<Integer, Integer> size = new HashMap<>();      // root -> values
    int best = 0;
    for (int v : nums)
        best = Math.max(best, size.merge(d.find(v), 1, Integer::sum));
    return best;
}
```

`DSU` is the union-find class from 9.9 **Union-Find (disjoint set union)** (p. 247). Its nodes go up to max(nums), so every prime factor has a slot.

Cost Time `O(n log log n)` Space `O(n)`

The inner loops add up to n/2 + n/3 + n/5 + ... ≈ n·ln ln n. Each SPF factorisation is O(log x); trial division is O(√x) per number.

Variants same idea, small change

- Closest primes in a range: sieve up to `right`, walk the primes in order, and keep the smallest gap between neighbours.
- Prime pairs with a target sum: sieve up to n, then test x and n − x for every x ≤ n/2.
- Segmented sieve for [L, R] with R up to 10^{12} and R − L ≤ 10^{6}: sieve the primes up to √R, then cross out their multiples inside the window only.
- Number of divisors from the factorisation: if p appears e times in x, a divisor takes it 0..e times, so multiply (e + 1) over the primes. Four divisors (LC 1390) is this with trial division.

Traps bugs people actually write

- Running the outer loop to n instead of √n with `m = p * p`: at p = 46349 the product overflows to a negative number, and `isPrime[m]` throws for n = 10^5. Stop at `(long) p * p <= n`.
- Without the `n >= 2` guard, `Arrays.fill(isPrime, 2, n + 1, true)` throws IllegalArgumentException for n = 0, which `countPrimes(1)` asks for.
- Adding `spf[x]` and dividing once per step lists 2 twice for 12. For distinct primes, divide out every copy of p before the next one.
- 1 is not prime and has no prime factors. In LC 952 a 1 stays a component of size 1.

Practise on LeetCode, in this order

☐	LC 204	Count Primes	Medium
☐	LC 2761	Prime Pairs With Target Sum	Medium
☐	LC 2521	Distinct Prime Factors of Product of Array	Medium
☐	LC 2523	Closest Prime Numbers in Range	Medium
☐	LC 952	Largest Component Size by Common Factor	Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

12.7 nCr mod p (factorial tables)

`O(n) + O(1)`

Spot it when the problem looks like this

- *Number of ways, modulo $10^9 + 7$*, and the count is a choice of positions: grid paths, arrangements of repeated letters, steps left or right.
- Many C(n, r) values with n up to 10^5 or 10^6: build tables once, then answer each in O(1).
- n ≤ 1000 and a modulus that is not prime, or exact values with n ≤ 66: Pascal's triangle.

How it works

C(n, r) = n! / (r!·(n − r)!). Modulo a prime p you cannot divide residues, but you can multiply by inverses: 1/r! is $(r!)^{p-2}$ (12.5 Fast power and modular inverse (p. 350)). Instead of one power per factorial, take one inverse at the top, 1/max! = $(max!)^{p-2}$, and walk down with 1/(i − 1)! = i · 1/i!. Setup is O(max + log p), and each C(n, r) is two multiplications. It needs max < p, so that no i! is 0 mod p; with p = $10^9 + 7$, any table that fits in memory is fine.

Pascal's rule C(n, r) = C(n−1, r−1) + C(n−1, r) splits on whether the last item is chosen. It uses only additions, so it works for any modulus or none, at $O(n^2)$ time and memory.

Counting problems turn into nCr once you name the choice: which m − 1 of the m + n − 2 grid moves go down, which of the k steps go right, or which positions each letter takes.

Worked example **Comb(6) with p = 13 instead of $10^9 + 7$, to keep the numbers small**

```
i  i!   fact[i]  inv[i], built from the top       fact[i]*inv[i] % 13
6  720  5        5^11 % 13 = 8 (the one inverse)  1
5  120  3        inv[6]*6 = 48 % 13 = 9           1
4  24   11       inv[5]*5 = 45 % 13 = 6           1
3  6    6        inv[4]*4 = 24 % 13 = 11          1
2  2    2        inv[3]*3 = 33 % 13 = 7           1
1  1    1        inv[2]*2 = 14 % 13 = 1           1
0  1    1        inv[1]*1 = 1                     1
C(6, 2) = fact[6]*inv[2]*inv[4] = 5*7*6 = 210 % 13 = 2, and 15 % 13 = 2
```

Factorial and inverse-factorial tables

memorise

```java
// nCr mod a prime p for 0 <= r <= n <= max, after O(max) setup (max < p).
static class Comb {
    final long[] fact, inv;                     // inv[i] = 1 / i! mod p
    Comb(int max) {
        fact = new long[max + 1];
        inv = new long[max + 1];
        fact[0] = 1;
        for (int i = 1; i <= max; i++) fact[i] = fact[i - 1] * i % MOD;
        inv[max] = modPow(fact[max], MOD - 2);  // the only Fermat inverse
        for (int i = max; i > 0; i--)
            inv[i - 1] = inv[i] * i % MOD;      // 1/(i-1)! = i * 1/i!
    }

    long nCr(int n, int r) {
        if (r < 0 || r > n) return 0;
        return fact[n] * inv[r] % MOD * inv[n - r] % MOD;
    }
}
```

`modPow(b, e)` is the template from **12.5 Fast power and modular inverse** (p. 350) with m = MOD.

Pascal's triangle; unique paths by formula

variant

```java
// Pascal: the last item is in or out. Exact in long up to n = 66.
static long[][] pascal(int n) {
    long[][] c = new long[n + 1][n + 1];
    for (int i = 0; i <= n; i++) {
        c[i][0] = 1;
        for (int j = 1; j <= i; j++) c[i][j] = c[i - 1][j - 1] + c[i - 1][j];
    }
    return c;
}

// Unique paths in an m x n grid: pick which m - 1 of the m + n - 2 moves
// go down. After step i, r = C(n - 1 + i, i), so each division is exact.
static long uniquePaths(int m, int n) {
    long r = 1;
    for (int i = 1; i <= m - 1; i++) r = r * (n - 1 + i) / i;
    return r;
}
```

Count anagrams; exactly k steps

example

```java
// Count anagrams (LC 2514): a word of length L with letter counts c1, c2, ...
// has L! / (c1! c2! ...) orders; multiply over the words.
static long countAnagrams(String s, Comb cb) {
    long ans = 1;
    for (String w : s.split(" ")) {
        int[] cnt = new int[26];
        for (char ch : w.toCharArray()) cnt[ch - 'a']++;
        ans = ans * cb.fact[w.length()] % MOD;
        for (int c : cnt) ans = ans * cb.inv[c] % MOD;
    }
    return ans;
}

// Exactly k unit steps from start to end (LC 2400): (k + d) / 2 of them go
// toward end, so choose which ones. Needs k - d >= 0 and even.
static long numberOfWays(int start, int end, int k, Comb cb) {
    int d = Math.abs(end - start);
    if (d > k || (k - d) % 2 != 0) return 0;
    return cb.nCr(k, (k + d) / 2);
}
```

Build one `Comb` as large as the longest word (10^5 for LC 2514) or as k (1000 for LC 2400).

Cost Time `O(n) + O(1)` Space `O(n)`

n = the table size. Two passes of n multiplications plus one O(log p) power; then each nCr is O(1). Pascal is $O(n^2)$ time and memory.

Variants same idea, small change

- Stars and bars: n identical items into k boxes in C(n + k – 1, k – 1) ways. Count the number of ideal arrays uses it for each prime exponent e of the last value: C(n – 1 + e, e).
- Ordered choices: P(n, r) = n! / (n – r)! is `fact[n] * inv[n - r] % MOD`.
- Catalan numbers C(2n, n) / (n + 1) count balanced bracket strings and BST shapes with n nodes.
- n ≥ p for a small prime p: Lucas's theorem multiplies $C(n_i, r_i)$ over the base-p digits of n and r.

Traps bugs people actually write

- `fact[n] / (fact[r] * fact[n - r] % MOD)` divides residues: C(20, 10) comes out as 0 instead of 184756. Multiply by `inv[r]` and `inv[n - r]`.
- `fact[n] * inv[r] * inv[n - r] % MOD` multiplies three numbers near 10^9; the second product passes $9.2 \cdot 10^{18}$. Reduce after every product.
- Pascal in `long` past n = 66: C(67, 33) wraps to –4220223336089263246. Reduce mod p, or use the tables.
- LC 2400 without the parity check: start 0, end 2, k = 3 gives C(3, 2) = 3, but 3 steps always end an odd distance away, so the answer is 0.

Practise on LeetCode, in this order

- [ ] LC 118 Pascal's Triangle — Easy
- [ ] LC 62 Unique Paths — Medium
- [ ] LC 2400 Number of Ways to Reach a Position After Exactly k Steps — Medium
- [ ] LC 2514 Count Anagrams — Hard
- [ ] LC 2338 Count the Number of Ideal Arrays — Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

12.8 Matrix power for recurrences

`O(k³ log n)`

Spot it when the problem looks like this

- *n-th term*, or *number of strings or paths of length n*, modulo $10^9 + 7$, with n up to 10^9 or 10^{18}.
- The state is small (k up to about 50) and every step applies the same rule.
- If $n \le 10^5$, the same count is a plain DP over the steps; the matrix only pays off for huge n.

How it works

A linear recurrence of order k keeps its last k values as a state, and one step is a fixed linear map: `v(i+1) = M · v(i)` for a k×k matrix M. For Fibonacci the state is `[F(i+1), F(i)]` and M = [[1, 1], [1, 0]]. So `v(n) = Mⁿ · v(0)`. Matrix products are associative, so M^n comes from the squaring loop of 12.5 **Fast power and modular inverse** (p. 350): O(log n) products of $O(k^3)$ each.

Counting strings is the same thing. Let `T[a][b] = 1` when a may follow b. An entry of a product sums over the middle symbol, so `(Tᵐ)[a][b]` counts the paths of m steps from b to a. Strings of n symbols are paths of n – 1 steps: add up all the entries of T^{n-1}.

When n is at most about 10^5, the plain DP over the steps costs O(n · edges) and is simpler. Use the matrix when n reaches 10^9 or more.

Worked example **fib(10) = matPow(M, 10)[1][0], M = [[1, 1], [1, 0]]**

```
e (bin)  bit  r after                        m after squaring
1010     0    I                              M^2  = [[2, 1], [1, 1]]
101      1    r*M^2 = [[2, 1], [1, 1]]       M^4  = [[5, 3], [3, 2]]
10       0    unchanged                      M^8  = [[34, 21], [21, 13]]
1        1    r*M^8 = [[89, 55], [55, 34]]   M^16 = [[1597, 987], [987, 610]]
e = 0: stop. 10 = 8 + 2, so r = M^10; F(10) = r[1][0] = 55
```

Matrix product and power mod p

memorise

```
static long[][] mul(long[][] a, long[][] b) {           // (n x k) * (k x m)
    int n = a.length, k = b.length, m = b[0].length;
    long[][] c = new long[n][m];
    for (int i = 0; i < n; i++)
        for (int t = 0; t < k; t++) {
            if (a[i][t] == 0) continue;                 // sparse rows are cheap
            for (int j = 0; j < m; j++)
                c[i][j] = (c[i][j] + a[i][t] * b[t][j]) % MOD;
        }
    return c;
}

// M^e by squaring: the modPow loop with matrices. O(k^3 log e).
static long[][] matPow(long[][] m, long e) {
    int k = m.length;
    long[][] r = new long[k][k];
    for (int i = 0; i < k; i++) r[i][i] = 1;            // identity
    while (e > 0) {
        if ((e & 1) == 1) r = mul(r, m);
        m = mul(m, m);
        e >>= 1;
    }
    return r;
}
```

Fibonacci and tribonacci

variant

```
// [F(n+1), F(n)] = M * [F(n), F(n-1)] with M = [[1, 1], [1, 0]],
// so M^n = [[F(n+1), F(n)], [F(n), F(n-1)]].
static long fib(long n) {
    return matPow(new long[][] {{1, 1}, {1, 0}}, n)[1][0];
}

// Tribonacci: state [T(i+2), T(i+1), T(i)]; T(0) = 0, T(1) = T(2) = 1.
static long tribonacci(long n) {
    long[][] m = {{1, 1, 1}, {1, 0, 0}, {0, 1, 0}};
    long[][] p = matPow(m, n);
    return (p[2][0] + p[2][1]) % MOD;                   // M^n * [1, 1, 0]
}
```

Count strings; knight dialer; vowels

example

```
// Strings of n symbols where symbol b may be followed by any a in next[b]:
// T[a][b] = 1, and T^(n-1)[a][b] counts the strings from b to a.
static long countStrings(int[][] next, long n) {
    int k = next.length;
    long[][] t = new long[k][k];
    for (int b = 0; b < k; b++) for (int a : next[b]) t[a][b] = 1;
    long[][] p = matPow(t, n - 1);
    long total = 0;
    for (long[] row : p) for (long x : row) total = (total + x) % MOD;
    return total;
}

// Knight dialer (LC 935): the digits a knight can jump to from each digit.
static final int[][] JUMPS = {{4, 6}, {6, 8}, {7, 9}, {4, 8}, {0, 3, 9},
                              {}, {0, 1, 7}, {2, 6}, {1, 3}, {2, 4}};
// Count vowels permutation (LC 1220): a e i o u and the vowels allowed next.
static final int[][] FOLLOW = {{1}, {0, 2}, {0, 1, 3, 4}, {2, 4}, {0}};

static long knightDialer(int n)          { return countStrings(JUMPS, n); }
static long countVowelPermutation(int n) { return countStrings(FOLLOW, n); }
```

Cost Time $O(k^3 \log n)$ Space $O(k^2)$

About $2 \cdot \log_2 n$ products of k^3 multiply-adds each. The 10×10 knight matrix with $n = 10^9$ needs about $6 \cdot 10^4$ operations.

Variants same idea, small change

- Recurrence with a constant, `f(n) = f(n-1) + f(n-2) + c`: add a 1 to the state and a matrix row that keeps it.
- Exactly n edges in a graph: the adjacency matrix to the power n. With `min` for `+`, `+` for `·`, and ∞ off the diagonal of the identity, the same loop gives the cheapest walk of exactly n edges.
- Total characters after t transformations II (LC 3337): `T[c][d] = 1` when c turns into d, with t up to 10^9. The answer adds count[c] × (sum of row c of T^t) over the 26 letters.
- Tiling or painting an n × 3 grid: write the DP over columns with a small state first; turn the transition into a matrix only if n is huge.

Traps bugs people actually write

- LC 1137 wants the exact T(37) = 2082876103, which is above $10^9 + 7$; the mod version returns 82876089. With n ≤ 37, a plain loop in `long` is the answer.
- n symbols take n − 1 transitions. `matPow(t, n)` for the knight dialer with n = 1 returns 20 instead of 10.
- One product of two entries is below $1.0 \cdot 10^{18}$, but adding 10 of them before a `%` passes $9.2 \cdot 10^{18}$. Reduce after each multiply-add.
- Reading the wrong entry: $M^n = [[F(n+1), F(n)], [F(n), F(n-1)]]$, so `[0][0]` is F(n+1), 89 for n = 10.

Practise on LeetCode, in this order

- [ ] LC 509 Fibonacci Number — Easy
- [ ] LC 1137 N-th Tribonacci Number — Easy
- [ ] LC 935 Knight Dialer — Medium
- [ ] LC 1220 Count Vowels Permutation — Hard
- [ ] LC 3337 Total Characters in String After Transformations II — Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

Part 13

Range queries and ordered maps

A range query asks about a contiguous block of an array: its minimum, its sum, the first position that reaches a bound. If the array never changes, prefix sums or a sparse table answer in O(1). If values change between queries, a Fenwick tree or a segment tree does the update and the query in O(log n) each, and lazy tags extend that to updates on whole ranges. When the keys themselves come and go, Java's TreeMap and TreeSet give the neighbours of any value in O(log n).

13.1	**Sparse table (static range min)** Many min, max, gcd, AND or OR queries on ranges of an array that never changes.	`O(1) per query`	p. **363**
13.2	**Fenwick tree (binary indexed)** Prefix sums while values change; count how many earlier values are smaller than this one.	`O(log n)`	p. **366**
13.3	**Segment tree (point update)** Point updates mixed with range queries for any associative operation, or the first index whose value reaches k.	`O(log n)`	p. **369**
13.4	**Lazy propagation (range updates)** Updates on whole ranges (add, assign, flip) mixed with range queries, each in O(log n).	`O(log n)`	p. **372**
13.5	**Ordered maps (TreeMap, TreeSet)** Keep keys sorted while they are added and removed, and find the nearest key above or below any value.	`O(log n)`	p. **375**

13.1 Sparse table (static range min)

`O(1) per query`

Spot it when the problem looks like this

- The array is fixed, and there are 10^5 or more queries for the min, max, gcd, AND or OR of `a[l..r]`.
- A two-pointer or binary-search loop needs the min or max of its current window in O(1).
- The operation has no inverse, so the prefix difference trick of 1.3 **Prefix sums** (p. 23) does not work.

How it works

`st[k][i]` is the min of the 2^k values starting at i. Row 0 is the array, and each cell of row k combines two cells of row k – 1: `st[k][i] = min(st[k-1][i], st[k-1][i + 2^(k-1)])`. There are about $\log_2 n$ rows of at most n cells, so the build is O(n log n) in time and memory.

A query `[l, r]` takes `k = floor(log2(r - l + 1))`. The block of length 2^k that starts at l and the one that ends at r cover `[l, r]` together, because 2·2^k > r – l + 1. They usually overlap, and for a min that is harmless: a value seen twice does not change a min. The same holds for every **idempotent** operation, where `op(x, x) = x`: min, max, gcd, AND, OR. So a query is two lookups, O(1).

A sum counts the overlap twice. It needs disjoint blocks, one per set bit of the length, which is O(log n), and prefix sums do sums in O(1) anyway.

Worked example **a = [5, 2, 4, 7, 1, 3, 6, 8], range min**

```
i      0  1  2  3  4  5  6  7
k = 0  5  2  4  7  1  3  6  8    the array
k = 1  2  2  4  1  1  3  6       min of a[i..i+1]
k = 2  2  1  1  1  1             min of a[i..i+3]
k = 3  1                         min of a[i..i+7]
query(2, 6): length 5, k = lg[5] = 2
  blocks a[2..5] and a[3..6]: st[2][2] = 1, st[2][3] = 1 -> min 1
  a[3..5] is in both blocks, which a min does not mind
the same two blocks for a sum: 15 + 17 = 32, but a[2..6] sums to 21
```

Sparse table for range min or max

memorise

```java
// st[k][i] = op of a[i .. i + 2^k), built from two halves of size 2^(k-1).
// op must be idempotent (op(x, x) == x): min, max, gcd, &, |.
static class SparseTable {
    final int[][] st;
    final int[] lg;                     // lg[x] = floor(log2 x)
    final boolean max;                  // false: range min, true: range max

    SparseTable(int[] a, boolean max) {
        this.max = max;
        int n = a.length;
        lg = new int[n + 1];
        for (int x = 2; x <= n; x++) lg[x] = lg[x / 2] + 1;
        st = new int[lg[n] + 1][];
        st[0] = a.clone();
        for (int k = 1; k < st.length; k++) {      // row k needs all of row k-1
            int half = 1 << (k - 1);
            st[k] = new int[n - (1 << k) + 1];
            for (int i = 0; i < st[k].length; i++)
                st[k][i] = op(st[k - 1][i], st[k - 1][i + half]);
        }
    }

    int op(int x, int y) { return max ? Math.max(x, y) : Math.min(x, y); }

    // op of a[l..r], inclusive: two blocks of length 2^k that may overlap.
    int query(int l, int r) {
        int k = lg[r - l + 1];
        return op(st[k][l], st[k][r - (1 << k) + 1]);
    }
}
```

Longest subarray with max − min ≤ limit

example

```
// LC 1438: longest subarray with max - min <= limit. Both ends only move
// right, and each check is two O(1) queries.
static int longestSubarray(int[] a, int limit) {
    SparseTable mn = new SparseTable(a, false), mx = new SparseTable(a, true);
    int best = 0;
    for (int l = 0, r = 0; r < a.length; r++) {
        while (mx.query(l, r) - mn.query(l, r) > limit) l++;
        best = Math.max(best, r - l + 1);
    }
    return best;
}
```

Cost Time `O(1) per query` Space `O(n log n)`

The build fills about $\log_2 n$ rows of up to n cells. A query is two lookups. LC 1438 adds an O(n) window walk to the O(n log n) build.

Variants same idea, small change

- OR or AND of a range (LC 2411, LC 3171): both are idempotent, so change `op`. For a fixed l the OR only grows with r, so binary search r with O(1) queries.
- Max and min of the same window (LC 1438): one table for each. LC 2398 needs only the max table, plus a prefix sum of the running costs.
- Index of the minimum, as in LCA on an Euler tour (8.7 **Lowest common ancestor** (p. 202)): store indexes in the table and compare their values in `op`.
- Values change between queries: a 13.3 **Segment tree (point update)** (p. 369) answers in O(log n) and updates in O(log n).

Traps bugs people actually write

- Filling the table with i in the outer loop and k inside reads row k − 1 cells that are not built yet: on `[5, 2, 4, 7, 1, 3, 6, 8]` every cell of row 2 comes out 0. Build row by row.
- `st[k][r - k + 1]` instead of `st[k][r - (1 << k) + 1]`: `query(0, 3)` above reads the block `a[2..5]` and returns 1 instead of 2.
- The overlapping two-block query on a sum counts the overlap twice: 32 instead of 21 in the example.

Practise on LeetCode, in this order

- [ ] LC 2411 Smallest Subarrays With Maximum Bitwise OR — Medium
- [ ] LC 1438 Longest Continuous Subarray With Absolute Diff Less Than or Equal to Limit — Medium
- [ ] LC 2398 Maximum Number of Robots Within Budget — Hard
- [ ] LC 3171 Find Subarray With Bitwise OR Closest to K — Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

13.2 Fenwick tree (binary indexed) O(log n)

Spot it when the problem looks like this

- *Update one value* and *sum a range*, mixed, with 10^5 or $3 \cdot 10^4$ operations of each.
- *How many earlier (or later) elements are smaller*: count of smaller numbers after self, inversions, the cost of inserting into a sorted list.
- Values are bounded (up to 10^5) or can be compressed, so they can serve as indexes.

How it works

Index from 1. `t[i]` holds the sum of `a(i - lowbit(i) .. i]`, where `lowbit(i) = i & -i` is the lowest set bit of i: `t[6]` holds `a[5..6]` and `t[8]` holds `a[1..8]`. The prefix `[1, i]` splits into such blocks: take `t[i]`, clear the lowest bit with `i -= i & -i`, and repeat. Each step clears one set bit, so `sum(i)` adds at most $\log_2 n + 1$ cells.

The cells whose blocks contain `a[i]` are i, then `i + lowbit(i)`, and so on. Adding the lowest bit carries into a higher bit, and that cell's block reaches back over i. So `add(i, d)` also touches O(log n) cells, and a range sum is `sum(r) - sum(l - 1)`.

For range add with point query, store the differences `d[i] = a[i] - a[i-1]`. Adding v to `a[l..r]` changes only `d[l]` and `d[r+1]`, and `a[i]` is the prefix sum of d.

Worked example a = [3, 1, 4, 1, 5, 9, 2, 6] at indexes 1..8

```
i  binary  i & -i  t[i] holds  t[i]
1  0001    1       a[1]          3
2  0010    2       a[1..2]       4
3  0011    1       a[3]          4
4  0100    4       a[1..4]       9
5  0101    1       a[5]          5
6  0110    2       a[5..6]      14
7  0111    1       a[7]          2
8  1000    8       a[1..8]      31
sum(7) = t[7] + t[6] + t[4] = 2 + 14 + 9 = 25     i: 7 -> 6 -> 4 -> 0
add(3, +2) changes t[3], t[4], t[8]                i: 3 -> 4 -> 8 -> 16 > 8
```

Fenwick tree with i & -i

memorise

```java
// Fenwick tree, 1-indexed: t[i] holds a(i - lowbit(i) .. i], where
// lowbit(i) = i & -i is the lowest set bit of i.
static class Fenwick {
    final long[] t;
    Fenwick(int n) { t = new long[n + 1]; }

    void add(int i, long delta) {              // a[i] += delta, 1 <= i <= n
        for (; i < t.length; i += i & -i) t[i] += delta;
    }

    long sum(int i) {                          // a[1] + ... + a[i]
        long s = 0;
        for (; i > 0; i -= i & -i) s += t[i];
        return s;
    }

    long sum(int l, int r) { return sum(r) - sum(l - 1); }    // a[l..r]
}
```

Range sum mutable; range add, point query

variant

```java
// LC 307: LeetCode indexes from 0, the tree from 1. update() sets a value,
// so add the difference and remember the new value.
static class NumArray {
    final int[] a;
    final Fenwick f;
    NumArray(int[] nums) {
        a = nums.clone();
        f = new Fenwick(a.length);
        for (int i = 0; i < a.length; i++) f.add(i + 1, a[i]);
    }
    void update(int i, int val) { f.add(i + 1, val - a[i]); a[i] = val; }
    int sumRange(int l, int r) { return (int) f.sum(l + 1, r + 1); }
}

// Range add, point query: keep d[i] = a[i] - a[i-1] in the tree, so a[i]
// is the prefix sum d[1] + ... + d[i]. Adding v to a[l..r] changes two d's.
static void rangeAdd(Fenwick f, int l, int r, long v) {
    f.add(l, v);
    f.add(r + 1, -v);                  // r + 1 = n + 1 is skipped by add
}
static long pointQuery(Fenwick f, int i) { return f.sum(i); }
```

Count smaller after self; sorted inserts

example

```java
// LC 315: walk from the right; the tree counts the values seen so far by
// rank. ranks() is the compression of [coord-compress], shifted to 1..m.
static List<Integer> countSmaller(int[] nums) {
    int[] rank = ranks(nums);
    Fenwick f = new Fenwick(nums.length);
    Integer[] out = new Integer[nums.length];
    for (int i = nums.length - 1; i >= 0; i--) {
        out[i] = (int) f.sum(rank[i] - 1);     // strictly smaller, to the right
        f.add(rank[i], 1);
    }
    return Arrays.asList(out);
}

// LC 1649: values are 1..1e5, so they index the tree directly.
static int createSortedArray(int[] instructions) {
    Fenwick f = new Fenwick(100_000);
    long cost = 0;
    for (int i = 0; i < instructions.length; i++) {
        int v = instructions[i];
        long less = f.sum(v - 1), greater = i - f.sum(v);  // i values inserted
        cost += Math.min(less, greater);
        f.add(v, 1);
    }
    return (int) (cost % 1_000_000_007);
}
```

`ranks(a)` gives each value its rank 1..m: the sort, unique and binary search of **3.11 Coordinate compression** (p. 96), plus 1.

Cost Time `O(log n)` Space `O(n)`

Each add or sum touches at most $\log_2 n + 1$ cells. LC 315 and LC 1649 do one add and one or two sums per element, O(n log n) with the sort for the ranks.

Variants same idea, small change

- Build in O(n): copy a into t, then for i = 1..n add `t[i]` into `t[i + (i & -i)]` when that index is ≤ n.
- k-th smallest of the counted values in O(log n): start at `pos = 0` and try each `step` from the highest power of two down. If `t[pos + step] < k`, subtract that cell from k and move `pos += step`. The answer is `pos + 1`.
- Count good triplets (LC 2179): replace each value by its position in the other array, then for each middle element multiply smaller-before by larger-after.
- Range add with range sum: two trees, or a **13.4 Lazy propagation (range updates)** (p. 372). Min or max of a range with updates: a **13.3 Segment tree (point update)** (p. 369).

Traps bugs people actually write

- Index 0: `add(0, x)` never ends, because `0 & -0` is 0 and i never moves. LC 307 indexes from 0, so add 1.
- LC 307 `update` sets a value. Adding `val` instead of `val - a[i]`: on `[1, 3, 5]`, `update(1, 2)` and then `sumRange(0, 2)` gives 11 instead of 8.
- Forgetting `a[i] = val`: a second `update(1, 2)` subtracts 1 again, and the sum becomes 7.
- Range add with `add(r, -v)` instead of `add(r + 1, -v)` takes the add back from `a[r]` itself.

Practise on LeetCode, in this order

☐	LC 1409	Queries on a Permutation With Key	Medium
☐	LC 307	Range Sum Query - Mutable	Medium
☐	LC 315	Count of Smaller Numbers After Self	Hard
☐	LC 1649	Create Sorted Array through Instructions	Hard
☐	LC 2179	Count Good Triplets in an Array	Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

13.3 Segment tree (point update)

`O(log n)`

Spot it when the problem looks like this

- Values change, and after each change you need the max, min, sum or gcd of a range; n and q up to 10^5.
- *The first position* (leftmost row, basket, seat) *with at least k*, while the values keep changing.
- A DP over values where `dp[v]` needs the best `dp` over the value range `[v - k, v - 1]`.

How it works

Node 1 covers the array. Node x with range `[lo, hi]` has children 2x for `[lo, mid]` and 2x + 1 for `[mid+1, hi]`, and stores the max and the sum of its range. There are about $\log_2 n$ levels.

`set(i, v)` changes one leaf and recomputes its ancestors. A query returns a node's value when the node is inside `[l, r]`, the identity (0 for a sum, $-\infty$ for a max) when it is outside, and asks both children otherwise. Only the two ends of `[l, r]` cut nodes, at most two per level, so a query visits O(log n) nodes.

The descent finds the smallest $i \le r$ with `a[i] ≥ k`. A node whose max is below k is skipped whole, so the search follows the path at r and one path down to the answer: O(log n). The book uses the recursive tree with 4n slots. The iterative 2n version is shorter, but unless n is a power of two its nodes do not nest into one tree, and the descent and the tags of 13.4 Lazy propagation (range updates) (p. 372) need one.

Worked example a = [5, 1, 4, 2, 6, 3]; each node shows [lo, hi] max/sum

```
x=1  [0,5] 6/21
x=2    [0,2] 5/10
x=4      [0,1] 5/6     children x=8 [0,0] 5/5, x=9 [1,1] 1/1
x=5      [2,2] 4/4
x=3    [3,5] 6/11
x=6      [3,4] 6/8     children x=12 [3,3] 2/2, x=13 [4,4] 6/6
x=7      [5,5] 3/3
sum(1, 4) = [1,1] + [2,2] + [3,4] = 1 + 4 + 8 = 13, three inside nodes
first(k = 5, r = 5): [0,5] -> [0,2] -> [0,1] -> [0,0]: index 0
first(k = 6, r = 5): [0,2] has max 5 < 6, skipped -> [3,5] -> [3,4]
                     -> [3,3] has max 2, skipped -> [4,4]: index 4
```

Max and sum, point set, range query

memorise

```java
// Recursive segment tree over a[0..n-1], all 0 at the start. Node x covers
// [lo, hi]; children 2x and 2x+1 split it at mid. Each node keeps the max
// and the sum of its range. 4n slots are always enough.
static class SegTree {
    final int n;
    final long[] mx, sm;
    SegTree(int n) { this.n = n; mx = new long[4 * n]; sm = new long[4 * n]; }

    void set(int i, long v) { set(1, 0, n - 1, i, v); }          // a[i] = v
    void set(int x, int lo, int hi, int i, long v) {
        if (lo == hi) { mx[x] = sm[x] = v; return; }
        int mid = (lo + hi) / 2;
        if (i <= mid) set(2 * x, lo, mid, i, v);
        else set(2 * x + 1, mid + 1, hi, i, v);
        mx[x] = Math.max(mx[2 * x], mx[2 * x + 1]);                // pull up
        sm[x] = sm[2 * x] + sm[2 * x + 1];
    }

    long max(int l, int r) { return max(1, 0, n - 1, l, r); }      // a[l..r]
    long max(int x, int lo, int hi, int l, int r) {
        if (r < lo || hi < l) return Long.MIN_VALUE;               // outside
        if (l <= lo && hi <= r) return mx[x];                      // inside
        int mid = (lo + hi) / 2;
        return Math.max(max(2 * x, lo, mid, l, r),
                        max(2 * x + 1, mid + 1, hi, l, r));
    }

    long sum(int l, int r) { return sum(1, 0, n - 1, l, r); }
    long sum(int x, int lo, int hi, int l, int r) {
        if (r < lo || hi < l) return 0;
        if (l <= lo && hi <= r) return sm[x];
        int mid = (lo + hi) / 2;
        return sum(2 * x, lo, mid, l, r) + sum(2 * x + 1, mid + 1, hi, l, r);
    }
}
```

Descent to the first a[i] ≥ k

memorise

```
// Smallest i <= r with a[i] >= k, or -1: call first(t, 1, 0, t.n - 1, k, r).
// A node whose max is below k cannot hold the answer, so skip it whole.
static int first(SegTree t, int x, int lo, int hi, long k, int r) {
    if (lo > r || t.mx[x] < k) return -1;
    if (lo == hi) return lo;
    int mid = (lo + hi) / 2;
    int i = first(t, 2 * x, lo, mid, k, r);              // leftmost first
    return i >= 0 ? i : first(t, 2 * x + 1, mid + 1, hi, k, r);
}
```

Longest increasing subsequence II

example

```
// LC 2407: best[v] = longest valid subsequence ending in value v. The value
// before v lies in [v - k, v - 1]: one range max, one point set per value.
static int lengthOfLIS(int[] nums, int k) {
    int maxV = 0;
    for (int v : nums) maxV = Math.max(maxV, v);
    SegTree t = new SegTree(maxV + 1);
    long best = 0;
    for (int v : nums) {                          // values are >= 1
        long len = 1 + Math.max(0, t.max(Math.max(0, v - k), v - 1));
        t.set(v, len);                            // never smaller than before
        best = Math.max(best, len);
    }
    return (int) best;
}
```

Cost Time `O(log n)` Space `O(n)`

set, max, sum and the descent each visit O(log n) nodes. LC 2407 builds the tree over the values, so it costs O(n log V) for values up to V.

Variants same idea, small change

- Range sum query mutable (LC 307): `set` and `sum` are enough; a **13.2 Fenwick tree (binary indexed)** (p. 366) tree is shorter when sums are all you need.
- Fruits into baskets III (LC 3479): the descent over the whole array finds the leftmost basket that fits, and `set(i, 0)` uses it up.
- Booking concert tickets (LC 2286): the tree holds each row's free seats. *gather* is the descent with bound maxRow, then a `set` on that row. *scatter* checks `sum(0, maxRow) ≥ k` and fills rows from a pointer to the first row that is not full. Each row fills once: O((n + q) log n) in total.
- Richer nodes (LC 3165): each node keeps the best sum for each choice of using its first and last element, four numbers that merge in O(1).

Traps bugs people actually write

- Arrays of size 2n for the recursive tree: n = 6 already uses node 13 (see the trace). Allocate 4n.
- Returning `0` for an outside node in a max query is wrong when values can be negative: on `[-3, -5]`, `max(1, 1)` comes out 0. Return `Long.MIN_VALUE`.
- LC 2407 with the range `[v - k, v]` lets a value follow an equal one: `[1, 1]` with k = 1 gives 2, but the subsequence must be strictly increasing, so the answer is 1.

Practise on LeetCode, in this order

- [ ] `LC 3479` Fruits Into Baskets III — Medium
- [ ] `LC 2407` Longest Increasing Subsequence II — Hard
- [ ] `LC 2286` Booking Concert Tickets in Groups — Hard
- [ ] `LC 3165` Maximum Sum of Subsequence With Non-adjacent Elements — Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

13.4 Lazy propagation (range updates)

`O(log n)`

Spot it when the problem looks like this

- *Add v to every element in [l, r]*, *set [l, r] to v*, *flip the bits in [l, r]*, mixed with sum or max queries.
- n and q up to 10^5, so touching every element of an update (O(n) each) is too slow.
- Squares or intervals dropped on a line, where each one reads the max over its range and then raises it.

How it works

A range update stops at the O(log n) nodes that lie fully inside `[l, r]`, the same nodes a query would use. Each gets its new value at once (`sm += v·len`) and a **tag**, `lz += v`, meaning "my children have not received this add yet". Nothing below is touched.

The invariant: `sm[x]` is right for x's range, but a child is right only once the tags of all its ancestors have been handed down. So every call that goes below a node first **pushes**: it applies the node's tag to both children and clears it. After the children change, it **pulls**: `sm[x] = sm[2x] + sm[2x+1]`. Push before going down, pull after coming back.

Each call still visits O(log n) nodes and a push is O(1), so updates and queries are both O(log n). Tags must combine into one tag: two adds are one add, two flips cancel, and an assign replaces any older tag.

Worked example n = 8, all 0; nodes are [lo, hi] with sm and lz

```
add(0, 5, +2)  [0,3] inside: sm 8, lz 2      [4,5] inside: sm 4, lz 2
               pull: [4,7] sm 4, [0,7] sm 12; the six leaves are untouched
add(2, 3, +3)  [0,3] is cut: push lz 2 to [0,1] (sm 4) and [2,3] (sm 4)
               [2,3] inside: sm 4 + 3*2 = 10, lz 2 + 3 = 5
               pull: [0,3] sm 4 + 10 = 14, [0,7] sm 14 + 4 = 18
sum(3, 3)      [2,3] is cut: push lz 5 to [2,2] and [3,3] (sm 5 each)
               [3,3] inside: the answer is 5
               without the push, [3,3] still has sm 0 and the answer is 0
```

Range add, range sum

memorise

```java
// Range add, range sum. sm[x] is the true sum of x's range; lz[x] is an
// add that x's children have not received yet.
static class LazySegTree {
    final int n;
    final long[] sm, lz;
    LazySegTree(int n) { this.n = n; sm = new long[4 * n]; lz = new long[4 * n]; }

    void apply(int x, int lo, int hi, long v) {      // add v to all of [lo, hi]
        sm[x] += v * (hi - lo + 1);
        lz[x] += v;
    }

    void push(int x, int lo, int hi) {               // hand the tag down
        if (lz[x] == 0) return;
        int mid = (lo + hi) / 2;
        apply(2 * x, lo, mid, lz[x]);
        apply(2 * x + 1, mid + 1, hi, lz[x]);
        lz[x] = 0;
    }

    void add(int l, int r, long v) { add(1, 0, n - 1, l, r, v); }
    void add(int x, int lo, int hi, int l, int r, long v) {
        if (r < lo || hi < l) return;
        if (l <= lo && hi <= r) { apply(x, lo, hi, v); return; }   // stop here
        push(x, lo, hi);                             // 1. push before going down
        int mid = (lo + hi) / 2;
        add(2 * x, lo, mid, l, r, v);
        add(2 * x + 1, mid + 1, hi, l, r, v);
        sm[x] = sm[2 * x] + sm[2 * x + 1];           // 2. pull after
    }

    long sum(int l, int r) { return sum(1, 0, n - 1, l, r); }
    long sum(int x, int lo, int hi, int l, int r) {
        if (r < lo || hi < l) return 0;
        if (l <= lo && hi <= r) return sm[x];
        push(x, lo, hi);
        int mid = (lo + hi) / 2;
        return sum(2 * x, lo, mid, l, r) + sum(2 * x + 1, mid + 1, hi, l, r);
    }
}
```

Range flip, count ones (LC 2569) variant

```java
// LC 2569: flip the bits of nums1[l..r] and count its ones. A flip turns a
// node's ones into len - ones, and two flips cancel: the tag is a boolean.
static class FlipTree {
    final int n;
    final int[] ones;
    final boolean[] flip;
    FlipTree(int n) {
        this.n = n;
        ones = new int[4 * n];
        flip = new boolean[4 * n];
    }

    void apply(int x, int lo, int hi) {          // flip all of [lo, hi]
        ones[x] = hi - lo + 1 - ones[x];
        flip[x] = !flip[x];
    }

    void flip(int l, int r) { flip(1, 0, n - 1, l, r); }
    void flip(int x, int lo, int hi, int l, int r) {
        if (r < lo || hi < l) return;
        if (l <= lo && hi <= r) { apply(x, lo, hi); return; }
        int mid = (lo + hi) / 2;
        if (flip[x]) {                           // push
            apply(2 * x, lo, mid);
            apply(2 * x + 1, mid + 1, hi);
            flip[x] = false;
        }
        flip(2 * x, lo, mid, l, r);
        flip(2 * x + 1, mid + 1, hi, l, r);
        ones[x] = ones[2 * x] + ones[2 * x + 1]; // pull
    }
}
```

For LC 2569, flip each 1 of nums1 into an empty tree, keep the sum of nums2 in a `long`, and on a type 2 query add `p * ones[1]`: the root counts every one.

Cost Time `O(log n)` Space `O(n)`

Updates and queries each visit O(log n) nodes; push and pull are O(1). LC 2569 reads the root alone for a type 2 query.

Variants same idea, small change

- Range assign: the tag is "set to v" plus a flag, since 0 is a real value. `apply` sets `sm = v·len` and replaces any older tag.
- Range add with range max or min: `apply` adds v to `mx[x]`, with no length factor.
- Falling squares (LC 699): compress the left and right edges. Each square reads the max height over its cells, adds its side, and raises the range to that height. The new height is above everything in the range, so a "raise to at least h" tag works like an assign.
- Coordinates up to 10^9 that arrive online (my calendar III, range module): create children only when a push or an update needs them. Each operation adds O(log 10^9) nodes.

Traps bugs people actually write

- No push in the query: the children still hold old values. In the example, `sum(3, 3)` returns 0 instead of 5.
- `sm[x] += v` without the length factor: adding 1 to all of `[0, 3]` raises the sum by 1, not 4.
- Testing `lz[x] == 0` for "no tag" in range assign loses every assignment of 0. Keep a separate boolean.

Practise on LeetCode, in this order

- [ ] `LC 2569` Handling Sum Queries After Update — Hard
- [ ] `LC 699` Falling Squares — Hard
- [ ] `LC 732` My Calendar III — Hard
- [ ] `LC 715` Range Module — Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

13.5 Ordered maps (TreeMap, TreeSet) `O(log n)`

Spot it when the problem looks like this

- *The closest value to x* (≤ x, ≥ x, within d) in a set that changes: inserts, deletes and queries, up to 10^5 of each.
- Intervals or bookings that arrive one at a time and must stay disjoint or merged.
- The max and min of a collection where any element can be removed or corrected, which a heap cannot do directly.

How it works

`TreeMap` and `TreeSet` are red-black trees: keys stay sorted, and `put`, `remove`, `get` and the neighbour queries `floorKey`, `ceilingKey`, `lowerKey` and `higherKey` are all O(log n). `firstKey` and `lastKey` give the min and max.

Most uses rest on one argument: only the neighbours matter. Bookings that never overlap, sorted by start, are also sorted by end. So a new booking `[s, e)` can only collide with the last one starting at or before s, or the first one starting at or after s, and two O(log n) lookups replace a scan. For *contains duplicate III*, the only value among the last k that needs checking is the smallest one ≥ x − d: if it is above x + d, no value is close enough.

Merging works the same way: an interval that overlaps or touches `[l, r]` must start at or before r + 1, so `floorEntry(r + 1)` finds them one at a time, and each stored interval is removed once.

Worked example **MyCalendar.book, then Intervals.add (touching intervals merge)**

book	floorEntry(s)	ceilingKey(s)	result	
[10,20)	null	null	true	{10=20}
[15,25)	10=20	null	false	20 > 15: it runs into 15
[20,30)	10=20	null	true	{10=20, 20=30}
[5,12)	null	10	false	10 < 12: it starts first
[5,10)	null	10	true	{5=10, 10=20, 20=30}

add	map after	count()	
[2,3]	{2=3}	2	
[7,10]	{2=3, 7=10}	6	
[5,8]	{2=3, 5=10}	8	absorbs 7=10
[11,11]	{2=3, 5=11}	9	touches 5=10
[1,1]	{1=3, 5=11}	10	touches 2=3

My calendar I; contains duplicate III

memorise

```java
// LC 729: bookings are half-open [s, e) and never overlap, so sorted by start
// they are also sorted by end. A new booking can only hit its two neighbours.
static class MyCalendar {
    final TreeMap<Integer, Integer> cal = new TreeMap<>();     // start -> end

    boolean book(int s, int e) {
        Map.Entry<Integer, Integer> prev = cal.floorEntry(s);   // start <= s
        Integer next = cal.ceilingKey(s);                       // start >= s
        if (prev != null && prev.getValue() > s) return false;  // runs into s
        if (next != null && next < e) return false;             // starts before e
        cal.put(s, e);
        return true;
    }
}

// LC 220: indexes at most k apart, values at most d apart. The set holds
// the last k values; the only candidate is the smallest value >= x - d.
static boolean containsNearbyAlmostDuplicate(int[] nums, int k, int d) {
    TreeSet<Long> window = new TreeSet<>();
    for (int i = 0; i < nums.length; i++) {
        long x = nums[i];
        Long c = window.ceiling(x - d);
        if (c != null && c <= x + d) return true;
        window.add(x);
        if (i >= k) window.remove((long) nums[i - k]);   // keep k values
    }
    return false;
}
```

The calls you need, and a multiset

usage

```
TreeMap<Integer, String> m = new TreeMap<>();
m.put(10, "a"); m.put(20, "b"); m.put(30, "c");
Integer fl = m.floorKey(25);           // 20: largest key <= 25, or null
Integer ce = m.ceilingKey(25);         // 30: smallest key >= 25
Integer lo = m.lowerKey(20);           // 10: largest key <  20
Integer hi = m.higherKey(30);          // null: no key > 30
String v = m.floorEntry(25).getValue();               // "b"
SortedMap<Integer, String> head = m.headMap(20);      // keys < 20: {10}
NavigableMap<Integer, String> tail = m.tailMap(20, true);  // >= 20: {20, 30}
Map.Entry<Integer, String> first = m.pollFirstEntry(); // removes 10=a
// head and tail are live views of m: head is now {}
// TreeSet<Integer> has the same calls without "Key": floor, ceiling,
// lower, higher, first, last, pollFirst, headSet, tailSet.

// A multiset: count per value; drop the key when its count reaches 0.
TreeMap<Integer, Integer> bag = new TreeMap<>();
for (int p : new int[] {5, 3, 5}) bag.merge(p, 1, Integer::sum);
if (bag.merge(5, -1, Integer::sum) == 0) bag.remove(5);   // one 5 left
int max = bag.lastKey(), min = bag.firstKey();            // 5 and 3
```

Disjoint intervals (LC 352, LC 2276)

example

```
// Disjoint intervals [l, r] kept merged in a TreeMap start -> end. LC 352
// calls add(v, v) and reads map.entrySet() in order; LC 2276 asks count().
static class Intervals {
    final TreeMap<Integer, Integer> map = new TreeMap<>();
    long total = 0;                          // integers covered

    void add(int l, int r) {
        // absorb every stored interval that overlaps or touches [l, r]
        Map.Entry<Integer, Integer> e = map.floorEntry(r + 1);
        while (e != null && e.getValue() >= l - 1) {
            l = Math.min(l, e.getKey());
            r = Math.max(r, e.getValue());
            total -= e.getValue() - e.getKey() + 1;
            map.remove(e.getKey());
            e = map.floorEntry(r + 1);
        }
        map.put(l, r);
        total += r - l + 1;
    }

    long count() { return total; }
}
```

Cost Time `O(log n)` Space `O(n)`

Each put, remove and neighbour query is O(log n). Every interval is merged away at most once, so n adds cost O(n log n) in total.

Variants same idea, small change

- Stock price fluctuation (LC 2034): a `HashMap` from time to price, and a `TreeMap` multiset of prices. A correction takes one copy of the old price away; the max is `lastKey()`.
- My calendar II and III: a `TreeMap` of +1 and −1 changes, swept in key order (**6.4 Sweep line (start and end events)** (p. 149)).
- Equal values in a `TreeSet`: store `(long) value * n + index`, or pairs with a comparator that breaks ties by index.
- Rank or k-th smallest is not in `TreeMap` (it keeps no subtree sizes): use a **13.2 Fenwick tree (binary indexed)** (p. 366) tree over compressed values (**3.11 Coordinate compression** (p. 96)).

Traps bugs people actually write

- `int k = map.floorKey(x);` throws `NullPointerException` when no key is ≤ x. Keep the `Integer` and test it for null.
- A `TreeSet` of prices loses duplicates: after (t1, 10), (t2, 10) and a correction of t1 to 5, the set holds only 5, and the max says 5 instead of 10. Count copies in a `TreeMap`.
- A comparator that returns 0 for different elements makes them one element: `new TreeSet<int[]>((x, y) -> Integer.compare(x[0], y[0]))` keeps only one of `[1, 2]` and `[1, 3]`.
- Removing keys while looping over `tailMap(...).keySet()` throws `ConcurrentModificationException`. Find each key with a fresh `floorEntry` or `ceilingEntry`, as the merge does.

Practise on LeetCode, in this order

☐	LC 729	My Calendar I	Medium
☐	LC 2034	Stock Price Fluctuation	Medium
☐	LC 220	Contains Duplicate III	Hard
☐	LC 352	Data Stream as Disjoint Intervals	Hard
☐	LC 2276	Count Integers in Intervals	Hard

Write the template from memory ☐ read it ☐ +1 day ☐ +3 days ☐ +7 days ☐ +21 days

Costs at a glance

Time and space of every template, as stated in its window. n is the input size; the window names the other letters.

	Pattern	Time	Space	Page
Part 1 · Arrays, hashing and prefix sums				
1.1	Frequency counting	O(n)	O(k)	18
1.2	Seen-before hash map	O(n)	O(n)	21
1.3	Prefix sums	O(n + q)	O(n)	23
1.4	Prefix sum + hash map	O(n)	O(n)	26
1.5	2D prefix sums	O(rc + q)	O(rc)	28
1.6	Difference array	O(n + q)	O(n)	31
1.7	Kadane's maximum subarray	O(n)	O(1)	34
1.8	Boyer–Moore majority vote	O(n)	O(1)	36
1.9	Cyclic sort (index as hash)	O(n)	O(1)	38
1.10	In-place matrix moves	O(r·c)	O(1)	40
Part 2 · Two pointers and sliding windows				
2.1	Opposite-end pointers	O(n)	O(1)	44
2.2	K-sum (sort + two pointers)	$O(n^2)$	O(1)	46
2.3	Read/write pointers (in place)	O(n)	O(1)	49
2.4	Three-way partition (Dutch flag)	O(n)	O(1)	51
2.5	Merge two sorted sequences	O(m + n)	O(1)	53
2.6	Fixed-size sliding window	O(n)	O(1)	55
2.7	Sliding window (variable size)	O(n)	O(k)	58
2.8	Exactly k (two at-most counts)	O(n)	O(k)	60
2.9	Minimum window substring	O(m + n)	O(1)	62
2.10	Sliding window maximum (deque)	O(n)	O(k)	64
Part 3 · Binary search and sorting				
3.1	Lower bound and upper bound	O(log n)	O(1)	68
3.2	Search in a rotated sorted array	O(log n)	O(1)	71
3.3	Binary search on the answer	O(n log R)	O(1)	74
3.4	Binary search on real numbers	O(100 · n)	O(1)	76
3.5	Binary search on a slope	O(log n)	O(1)	79
3.6	K-th smallest by counting	O(n log R)	O(1)	82
3.7	Custom sort and comparators	O(n log n)	O(n)	85
3.8	Merge sort with counting	O(n log n)	O(n)	87
3.9	Quickselect	O(n) expected	O(1)	90
3.10	Counting sort and buckets	O(n + K)	O(n + K)	93
3.11	Coordinate compression	O(n log n)	O(n)	96
Part 4 · Stacks, queues and heaps				
4.1	Stack for matching and nesting	O(n)	O(n)	99
4.2	Expression evaluation	O(n)	O(n)	102

	Pattern	Time	Space	Page
4.3	Monotonic stack: next greater	O(n)	O(n)	105
4.4	Largest rectangle in histogram	O(n)	O(n)	108
4.5	Contribution counting	O(n)	O(n)	111
4.6	Designing with stacks	O(1)	O(n)	113
4.7	Top k with a size-k heap	O(n log k)	O(k)	116
4.8	Two heaps for a running median	O(log n)	O(n)	119
4.9	K-way merge with a heap	O(N log k)	O(k)	121
4.10	Greedy with a heap	O(n log n)	O(n)	124
Part 5 · Linked lists				
5.1	Dummy head node	O(m + n)	O(1)	128
5.2	In-place reversal (prev, cur, next)	O(n)	O(1)	131
5.3	Fast and slow pointers	O(n)	O(1)	134
5.4	Gap pointers and head switching	O(n)	O(1)	137
5.5	LRU cache (hash map + linked list)	O(1)	O(capacity)	139
Part 6 · Intervals and greedy				
6.1	Merge overlapping intervals	O(n log n)	O(n)	143
6.2	Insert and intersect intervals	O(n)	O(n)	145
6.3	Interval scheduling (sort by end)	O(n log n)	O(1)	147
6.4	Sweep line (start and end events)	O(n log n)	O(n)	149
6.5	Jump game (farthest reach)	O(n)	O(1)	152
6.6	Gas station (running balance)	O(n)	O(1)	155
6.7	Two passes: left, then right	O(n)	O(n)	157
Part 7 · Recursion and backtracking				
7.1	Subsets (include or exclude)	$O(n \cdot 2^n)$	O(n)	160
7.2	Skip duplicates at the same depth	$O(n \cdot 2^n)$	O(n)	163
7.3	Permutations (used[] array)	O(n·n!)	O(n)	166
7.4	Combinations from a start index	O(k·C(n, k))	O(k)	169
7.5	Constraint backtracking (N-Queens)	O(n!)	O(n)	172
7.6	Grid DFS with mark and restore	O(mn·3^L)	O(L)	175
7.7	Cut a prefix, recurse on the rest	$O(n \cdot 2^n)$	$O(n^2)$	178
7.8	Meet in the middle	O(2^(n/2)·n)	O(2^(n/2))	181
Part 8 · Trees				
8.1	Preorder, inorder, postorder	O(n)	O(h)	186
8.2	Level order (BFS by levels)	O(n)	O(w)	189
8.3	Return a value up the tree	O(n)	O(h)	191
8.4	Maximum path sum (one arm up)	O(n)	O(h)	194
8.5	Root-to-leaf paths and path sums	O(n)	O(n)	196
8.6	Binary search tree operations	O(h)	O(h)	199
8.7	Lowest common ancestor	O(n)	O(h)	202
8.8	Build and serialize a tree	O(n)	O(n)	205
8.9	General tree from an edge list	O(n)	O(n)	207
8.10	Binary lifting (k-th ancestor, LCA)	O(log n) each	O(n log n)	210

	Pattern	Time	Space	Page
8.11	Tree DP (take or skip a node)	O(n)	O(h)	213
8.12	Rerooting (every node as root)	O(n)	O(n)	216
8.13	Euler tour (subtree = range)	O(n)	O(n)	218
Part 9 · Graphs				
9.1	Graph DFS and connected components	O(V + E)	O(V)	223
9.2	BFS shortest path (unweighted)	O(V + E)	O(V)	226
9.3	Multi-source BFS	O(mn)	O(mn)	229
9.4	BFS over states	O(S·b)	O(S)	232
9.5	Bipartite check (two colours)	O(V + E)	O(V)	235
9.6	Directed cycles with three colours	O(V + E)	O(V)	238
9.7	Topological sort (Kahn's algorithm)	O(V + E)	O(V + E)	241
9.8	DP on a DAG in topological order	O(V + E)	O(V + E)	244
9.9	Union-Find (disjoint set union)	O(α(n)) per op	O(n)	247
9.10	Dijkstra's shortest paths	O(m log n)	O(n + m)	250
9.11	0-1 BFS (deque)	O(V + E)	O(V)	254
9.12	Bellman–Ford and at most k edges	O(n · m)	O(n)	257
9.13	Floyd–Warshall (all pairs)	$O(n^3)$	$O(n^2)$	260
9.14	Kruskal's MST (sort + DSU)	O(m log m)	O(n + m)	263
9.15	Prim's MST (dense or heap)	$O(n^2)$	O(n)	265
9.16	Bridges and articulation points	O(n + m)	O(n + m)	268
9.17	Strongly connected components	O(n + m)	O(n + m)	271
9.18	Euler path (Hierholzer)	O(n + m)	O(n + m)	274
Part 10 · Dynamic programming				
10.1	The DP recipe (memo, table, rolling)	O(n)	O(1)	278
10.2	Linear DP over a sequence	O(n)	O(n)	280
10.3	Grid DP (one row or in place)	O(m·n)	O(n)	283
10.4	0/1 knapsack (reverse loop)	O(n·W)	O(W)	286
10.5	Unbounded knapsack (forward loop)	O(k·A)	O(A)	289
10.6	Longest increasing subsequence	O(n log n)	O(n)	292
10.7	Longest common subsequence	O(m·n)	O(m·n)	294
10.8	Edit distance and alignment	O(m·n)	O(m·n)	297
10.9	Palindrome DP on s[i..j]	$O(n^2)$	$O(n^2)$	300
10.10	Interval DP (last split point)	$O(n^3)$	$O(n^2)$	303
10.11	Stock DP (hold and free states)	O(n·k)	O(k)	306
10.12	Bitmask DP over subsets	$O(2^n \cdot n)$	$O(2^n)$	309
10.13	Digit DP (count numbers ≤ N)	O(L·S·10)	O(L·S)	313
10.14	Partition into k contiguous groups	$O(k \cdot n^2)$	O(k·n)	316
Part 11 · Strings				
11.1	Expand around the centre	$O(n^2)$	O(1)	320
11.2	KMP prefix function	O(n + m)	O(m)	323
11.3	Z-function	O(n)	O(n)	326
11.4	Rolling hash (Rabin–Karp)	O(n log n)	O(n)	329

	Pattern	Time	Space	Page
11.5	Trie (prefix tree)	O(L)	O(26 · N)	332
11.6	Manacher's algorithm	O(n)	O(n)	335
Part 12 · Math and bits				
12.1	Bit tricks: lowest bit, submasks	O(1) per op	O(1)	339
12.2	XOR tricks (pairs cancel)	O(n)	O(1)	342
12.3	Binary trie for maximum XOR	O(n·B)	O(n·B)	345
12.4	GCD, LCM and extended Euclid	O(log n)	O(1)	348
12.5	Fast power and modular inverse	O(log n)	O(1)	350
12.6	Prime sieve and factorisation	O(n log log n)	O(n)	353
12.7	nCr mod p (factorial tables)	O(n) + O(1)	O(n)	356
12.8	Matrix power for recurrences	$O(k^3 \log n)$	$O(k^2)$	359
Part 13 · Range queries and ordered maps				
13.1	Sparse table (static range min)	O(1) per query	O(n log n)	363
13.2	Fenwick tree (binary indexed)	O(log n)	O(n)	366
13.3	Segment tree (point update)	O(log n)	O(n)	369
13.4	Lazy propagation (range updates)	O(log n)	O(n)	372
13.5	Ordered maps (TreeMap, TreeSet)	O(log n)	O(n)	375

Problem index: LeetCode number → pattern

Every practice problem in the book, by LeetCode number. Met a problem and don't know where to start? Look it up here and read that window first.

1 Two Sum → 1.2
2 Add Two Numbers → 5.1
3 Longest Substring Without Repeating Characters → 2.7
4 Median of Two Sorted Arrays → 3.6
5 Longest Palindromic Substring → 11.1
10 Regular Expression Matching → 10.8
11 Container With Most Water → 2.1
15 3Sum → 2.2
16 3Sum Closest → 2.2
17 Letter Combinations of a Phone Number → 7.4
18 4Sum → 2.2
19 Remove Nth Node From End of List → 5.4
20 Valid Parentheses → 4.1
21 Merge Two Sorted Lists → 5.1
22 Generate Parentheses → 7.7
23 Merge k Sorted Lists → 4.9
24 Swap Nodes in Pairs → 5.2
25 Reverse Nodes in k-Group → 5.2
26 Remove Duplicates from Sorted Array → 2.3
27 Remove Element → 2.3
28 Find the Index of the First Occurrence in a String → 11.2
31 Next Permutation → 7.3
33 Search in Rotated Sorted Array → 3.2
34 Find First and Last Position of Element in Sorted Array → 3.1
35 Search Insert Position → 3.1
37 Sudoku Solver → 7.5
39 Combination Sum → 7.4
40 Combination Sum II → 7.2
41 First Missing Positive → 1.9
42 Trapping Rain Water → 6.7
44 Wildcard Matching → 10.8
45 Jump Game II → 6.5
46 Permutations → 7.3
47 Permutations II → 7.3
48 Rotate Image → 1.10
49 Group Anagrams → 1.1
50 Pow(x, n) → 12.5
51 N-Queens → 7.5
52 N-Queens II → 7.5
53 Maximum Subarray → 1.7
54 Spiral Matrix → 1.10
55 Jump Game → 6.5
56 Merge Intervals → 6.1
57 Insert Interval → 6.2
61 Rotate List → 5.4
62 Unique Paths → 10.3, 12.7
63 Unique Paths II → 10.3
64 Minimum Path Sum → 10.3
69 Sqrt(x) → 3.3
70 Climbing Stairs → 10.1
71 Simplify Path → 4.1
72 Edit Distance → 10.8
73 Set Matrix Zeroes → 1.10
74 Search a 2D Matrix → 3.1
75 Sort Colors → 2.4
76 Minimum Window Substring → 2.9
77 Combinations → 7.4
78 Subsets → 7.1
79 Word Search → 7.6
80 Remove Duplicates from Sorted Array II → 2.3
81 Search in Rotated Sorted Array II → 3.2
82 Remove Duplicates from Sorted List II → 5.1
84 Largest Rectangle in Histogram → 4.4
85 Maximal Rectangle → 4.4
86 Partition List → 5.1
88 Merge Sorted Array → 2.5
90 Subsets II → 7.2
91 Decode Ways → 10.2
92 Reverse Linked List II → 5.2
93 Restore IP Addresses → 7.7
94 Binary Tree Inorder Traversal → 8.1
97 Interleaving String → 10.8
98 Validate Binary Search Tree → 8.6
99 Recover Binary Search Tree → 8.1
102 Binary Tree Level Order Traversal → 8.2
103 Binary Tree Zigzag Level Order Traversal → 8.2
104 Maximum Depth of Binary Tree → 8.3
105 Construct Binary Tree from Preorder and Inorder Traversal → 8.8
106 Construct Binary Tree from Inorder and Postorder Traversal → 8.8
110 Balanced Binary Tree → 8.3
112 Path Sum → 8.5
113 Path Sum II → 8.5
114 Flatten Binary Tree to Linked List → 8.1
115 Distinct Subsequences → 10.8
118 Pascal's Triangle → 12.7
120 Triangle → 10.3
121 Best Time to Buy and Sell Stock → 1.7
122 Best Time to Buy and Sell Stock II → 10.11
123 Best Time to Buy and Sell Stock III → 10.11
124 Binary Tree Maximum Path Sum → 8.4
125 Valid Palindrome → 2.1
127 Word Ladder → 9.4
128 Longest Consecutive Sequence → 1.2
129 Sum Root to Leaf Numbers → 8.5
130 Surrounded Regions → 9.1
131 Palindrome Partitioning → 7.7
132 Palindrome Partitioning II → 10.9
133 Clone Graph → 9.1
134 Gas Station → 6.6
135 Candy → 6.7
136 Single Number → 12.2
137 Single Number II → 12.2
139 Word Break → 10.2
140 Word Break II → 7.7
141 Linked List Cycle → 5.3
142 Linked List Cycle II → 5.3
143 Reorder List → 5.3
144 Binary Tree Preorder Traversal → 8.1
145 Binary Tree Postorder Traversal → 8.1
146 LRU Cache → 5.5
148 Sort List → 3.8
150 Evaluate Reverse Polish Notation → 4.2
152 Maximum Product Subarray → 1.7
153 Find Minimum in Rotated Sorted Array → 3.2
154 Find Minimum in Rotated Sorted Array II → 3.2
155 Min Stack → 4.6
160 Intersection of Two Linked Lists → 5.4
162 Find Peak Element → 3.5
164 Maximum Gap → 3.10
167 Two Sum II - Input Array Is Sorted → 2.1
169 Majority Element → 1.8
173 Binary Search Tree Iterator → 8.6
174 Dungeon Game → 10.3
179 Largest Number → 3.7
187 Repeated DNA Sequences → 11.4
188 Best Time to Buy and Sell Stock IV → 10.11
190 Reverse Bits → 12.1
191 Number of 1 Bits → 12.1
198 House Robber → 10.1
199 Binary Tree Right Side View → 8.2
200 Number of Islands → 9.1
203 Remove Linked List Elements → 5.1
204 Count Primes → 12.6
206 Reverse Linked List → 5.2
207 Course Schedule → 9.6
208 Implement Trie (Prefix Tree) → 11.5
209 Minimum Size Subarray Sum → 2.7

210 Course Schedule II → 9.7
211 Design Add and Search Words Data Structure → 11.5
212 Word Search II → 7.6, 11.5
213 House Robber II → 10.2
214 Shortest Palindrome → 11.2
215 Kth Largest Element in an Array → 3.9, 4.7
216 Combination Sum III → 7.4
217 Contains Duplicate → 1.2
218 The Skyline Problem → 6.4
219 Contains Duplicate II → 1.2
220 Contains Duplicate III → 3.10, 13.5
221 Maximal Square → 10.3
224 Basic Calculator → 4.2
225 Implement Stack using Queues → 4.6
227 Basic Calculator II → 4.2
229 Majority Element II → 1.8
230 Kth Smallest Element in a BST → 8.6
231 Power of Two → 12.1
232 Implement Queue using Stacks → 4.6
233 Number of Digit One → 10.13
234 Palindrome Linked List → 5.3
235 Lowest Common Ancestor of a Binary Search Tree → 8.7
236 Lowest Common Ancestor of a Binary Tree → 8.7
238 Product of Array Except Self → 1.3
239 Sliding Window Maximum → 2.10
242 Valid Anagram → 1.1
257 Binary Tree Paths → 8.5
260 Single Number III → 12.2
268 Missing Number → 1.9, 12.2
274 H-Index → 3.10
279 Perfect Squares → 10.5
283 Move Zeroes → 2.3
287 Find the Duplicate Number → 3.6, 5.3
289 Game of Life → 1.10
295 Find Median from Data Stream → 4.8
297 Serialize and Deserialize Binary Tree → 8.8
300 Longest Increasing Subsequence → 10.6
303 Range Sum Query - Immutable → 1.3
304 Range Sum Query 2D - Immutable → 1.5
307 Range Sum Query - Mutable → 13.2
309 Best Time to Buy and Sell Stock with Cooldown → 10.11
310 Minimum Height Trees → 8.12
312 Burst Balloons → 10.10
313 Super Ugly Number → 4.9
315 Count of Smaller Numbers After Self → 3.8, 13.2
322 Coin Change → 10.5
324 Wiggle Sort II → 2.4, 3.9
327 Count of Range Sum → 3.8
328 Odd Even Linked List → 5.1
329 Longest Increasing Path in a Matrix → 9.8
332 Reconstruct Itinerary → 9.18
337 House Robber III → 8.11
338 Counting Bits → 12.1
347 Top K Frequent Elements → 3.10, 4.7
350 Intersection of Two Arrays II → 2.5
352 Data Stream as Disjoint Intervals → 13.5
354 Russian Doll Envelopes → 10.6
357 Count Numbers with Unique Digits → 10.13
365 Water and Jug Problem → 12.4
368 Largest Divisible Subset → 10.6
372 Super Pow → 12.5
373 Find K Pairs with Smallest Sums → 4.9
377 Combination Sum IV → 10.5
378 Kth Smallest Element in a Sorted Matrix → 3.6, 4.9
383 Ransom Note → 1.1
387 First Unique Character in a String → 1.1
392 Is Subsequence → 2.5
394 Decode String → 4.1
401 Binary Watch → 7.4
402 Remove K Digits → 4.3
406 Queue Reconstruction by Height → 3.7
410 Split Array Largest Sum → 3.3, 10.14
416 Partition Equal Subset Sum → 10.4
421 Maximum XOR of Two Numbers in an Array → 12.3
424 Longest Repeating Character Replacement → 2.7
432 All O`one Data Structure → 5.5
435 Non-overlapping Intervals → 6.3
437 Path Sum III → 8.5
438 Find All Anagrams in a String → 2.6
442 Find All Duplicates in an Array → 1.9
443 String Compression → 2.3
448 Find All Numbers Disappeared in an Array → 1.9
449 Serialize and Deserialize BST → 8.8
450 Delete Node in a BST → 8.6
451 Sort Characters By Frequency → 3.10
452 Minimum Number of Arrows to Burst Balloons → 6.3
454 4Sum II → 1.2, 7.8
459 Repeated Substring Pattern → 11.2
460 LFU Cache → 5.5
462 Minimum Moves to Equal Array Elements II → 3.9
473 Matchsticks to Square → 7.5
474 Ones and Zeroes → 10.4
480 Sliding Window Median → 4.8
491 Non-decreasing Subsequences → 7.2
493 Reverse Pairs → 3.8
494 Target Sum → 10.4
496 Next Greater Element I → 4.3
502 IPO → 4.10
503 Next Greater Element II → 4.3
509 Fibonacci Number → 12.8
516 Longest Palindromic Subsequence → 10.9
518 Coin Change II → 10.5
523 Continuous Subarray Sum → 1.4
524 Longest Word in Dictionary through Deleting → 2.5
525 Contiguous Array → 1.4
526 Beautiful Arrangement → 7.3
540 Single Element in a Sorted Array → 3.5
542 01 Matrix → 9.3
543 Diameter of Binary Tree → 8.3
547 Number of Provinces → 9.1
560 Subarray Sum Equals K → 1.4
563 Binary Tree Tilt → 8.3
567 Permutation in String → 2.6
583 Delete Operation for Two Strings → 10.7
611 Valid Triangle Number → 2.2
621 Task Scheduler → 4.10
630 Course Schedule III → 4.10
632 Smallest Range Covering Elements from K Lists → 2.9, 4.9
637 Average of Levels in Binary Tree → 8.2
643 Maximum Average Subarray I → 2.6
645 Set Mismatch → 1.9
646 Maximum Length of Pair Chain → 6.3
647 Palindromic Substrings → 10.9, 11.1
648 Replace Words → 11.5
662 Maximum Width of Binary Tree → 8.2
664 Strange Printer → 10.10
668 Kth Smallest Number in Multiplication Table → 3.6
673 Number of Longest Increasing Subsequence → 10.6
682 Baseball Game → 4.2
684 Redundant Connection → 9.9
687 Longest Univalue Path → 8.4
692 Top K Frequent Words → 4.7
695 Max Area of Island → 9.1
698 Partition to K Equal Sum Subsets → 10.12
699 Falling Squares → 3.11, 13.4
700 Search in a Binary Search Tree → 8.6
701 Insert into a Binary Search Tree → 8.6
703 Kth Largest Element in a Stream → 4.7
704 Binary Search → 3.1
707 Design Linked List → 5.5
714 Best Time to Buy and Sell Stock with Transaction Fee → 10.11
715 Range Module → 13.4
718 Maximum Length of Repeated Subarray → 10.7, 11.4
719 Find K-th Smallest Pair Distance → 3.6
721 Accounts Merge → 9.9
724 Find Pivot Index → 1.3
729 My Calendar I → 6.2, 13.5

731 My Calendar II → 6.4
732 My Calendar III → 13.4
735 Asteroid Collision → 4.1
739 Daily Temperatures → 4.3
740 Delete and Earn → 10.2
743 Network Delay Time → 9.10
744 Find Smallest Letter Greater Than Target → 3.1
746 Min Cost Climbing Stairs → 10.1
752 Open the Lock → 9.4
753 Cracking the Safe → 9.18
757 Set Intersection Size At Least Two → 6.3
763 Partition Labels → 6.1
767 Reorganize String → 4.10
773 Sliding Puzzle → 9.4
775 Global and Local Inversions → 3.8
778 Swim in Rising Water → 9.15
784 Letter Case Permutation → 7.1
785 Is Graph Bipartite? → 9.5
786 K-th Smallest Prime Fraction → 3.4
787 Cheapest Flights Within K Stops → 9.12
795 Number of Subarrays with Bounded Maximum → 4.5
802 Find Eventual Safe States → 9.6
805 Split Array With Same Average → 7.8
813 Largest Sum of Averages → 10.14
821 Shortest Distance to a Character → 6.7
834 Sum of Distances in Tree → 8.12
845 Longest Mountain in Array → 6.7
847 Shortest Path Visiting All Nodes → 9.4
850 Rectangle Area II → 3.11
851 Loud and Rich → 9.8
852 Peak Index in a Mountain Array → 3.5
862 Shortest Subarray with Sum at Least K → 2.10
867 Transpose Matrix → 1.10
871 Minimum Number of Refueling Stops → 4.10
875 Koko Eating Bananas → 3.3
876 Middle of the Linked List → 5.3
879 Profitable Schemes → 10.4
881 Boats to Save People → 2.1
886 Possible Bipartition → 9.5
889 Construct Binary Tree from Preorder and Postorder Traversal → 8.8
895 Maximum Frequency Stack → 4.6
901 Online Stock Span → 4.3
902 Numbers At Most N Given Digit Set → 10.13
904 Fruit Into Baskets → 2.7
905 Sort Array By Parity → 2.4
907 Sum of Subarray Minimums → 4.5
909 Snakes and Ladders → 9.2
912 Sort an Array → 2.4, 3.8
914 X of a Kind in a Deck of Cards → 12.4
918 Maximum Sum Circular Subarray → 1.7
921 Minimum Add to Make Parentheses Valid → 6.6
923 3Sum With Multiplicity → 2.2
930 Binary Subarrays With Sum → 2.8
934 Shortest Bridge → 9.3
935 Knight Dialer → 9.12, 12.8
937 Reorder Data in Log Files → 3.7
943 Find the Shortest Superstring → 10.12
947 Most Stones Removed with Same Row or Column → 9.9
952 Largest Component Size by Common Factor → 12.6
968 Binary Tree Cameras → 8.11
973 K Closest Points to Origin → 3.9, 4.7
974 Subarray Sums Divisible by K → 1.4
977 Squares of a Sorted Array → 2.1
979 Distribute Coins in Binary Tree → 8.3
980 Unique Paths III → 7.6
981 Time Based Key-Value Store → 3.1
983 Minimum Cost For Tickets → 10.2
986 Interval List Intersections → 2.5, 6.2
990 Satisfiability of Equality Equations → 9.9
992 Subarrays with K Different Integers → 2.8
994 Rotting Oranges → 9.3
996 Number of Squareful Arrays → 7.3
1004 Max Consecutive Ones III → 2.7
1008 Construct Binary Search Tree from Preorder Traversal → 8.8
1011 Capacity To Ship Packages Within D Days → 3.3
1012 Numbers With Repeated Digits → 10.13
1022 Sum of Root To Leaf Binary Numbers → 8.5
1024 Video Stitching → 6.5
1035 Uncrossed Lines → 10.7
1039 Minimum Score Triangulation of Polygon → 10.10
1044 Longest Duplicate Substring → 11.4
1047 Remove All Adjacent Duplicates In String → 2.3, 4.1
1048 Longest String Chain → 10.6
1049 Last Stone Weight II → 10.4
1051 Height Checker → 3.10
1052 Grumpy Bookstore Owner → 2.6
1071 Greatest Common Divisor of Strings → 12.4
1074 Number of Submatrices That Sum to Target → 1.5
1079 Letter Tile Possibilities → 7.2
1091 Shortest Path in Binary Matrix → 9.2
1092 Shortest Common Supersequence → 10.7
1094 Car Pooling → 1.6, 6.4
1095 Find in Mountain Array → 3.5
1106 Parsing A Boolean Expression → 4.2
1109 Corporate Flight Bookings → 1.6
1122 Relative Sort Array → 3.7
1123 Lowest Common Ancestor of Deepest Leaves → 8.7
1129 Shortest Path with Alternating Colors → 9.4
1130 Minimum Cost Tree From Leaf Values → 10.10
1137 N-th Tribonacci Number → 10.1, 12.8
1143 Longest Common Subsequence → 10.7
1161 Maximum Level Sum of a Binary Tree → 8.2
1162 As Far from Land as Possible → 9.3
1178 Number of Valid Words for Each Puzzle → 12.1
1186 Maximum Subarray Sum with One Deletion → 1.7
1192 Critical Connections in a Network → 9.16
1202 Smallest String With Swaps → 9.9
1203 Sort Items by Groups Respecting Dependencies → 9.7
1219 Path with Maximum Gold → 7.6
1220 Count Vowels Permutation → 12.8
1234 Replace the Substring for Balanced String → 2.9
1248 Count Number of Nice Subarrays → 2.8
1249 Minimum Remove to Make Valid Parentheses → 4.1
1250 Check If It Is a Good Array → 12.4
1278 Palindrome Partitioning III → 10.14
1288 Remove Covered Intervals → 6.1
1292 Maximum Side Length of a Square with Sum Less than or Equal to Threshold → 1.5
1293 Shortest Path in a Grid with Obstacles Elimination → 9.4
1306 Jump Game III → 9.2
1310 XOR Queries of a Subarray → 12.2
1312 Minimum Insertion Steps to Make a String Palindrome → 10.9
1314 Matrix Block Sum → 1.5
1316 Distinct Echo Substrings → 11.4
1318 Minimum Flips to Make a OR b Equal to c → 12.1
1319 Number of Operations to Make Network Connected → 9.9
1326 Minimum Number of Taps to Open to Water a Garden → 6.5
1331 Rank Transform of an Array → 3.11
1334 Find the City With the Smallest Number of Neighbors at a Threshold Distance → 9.13
1335 Minimum Difficulty of a Job Schedule → 10.14
1345 Jump Game IV → 9.2
1353 Maximum Number of Events That Can Be Attended → 4.10
1358 Number of Substrings Containing All Three Characters → 2.8
1366 Rank Teams by Votes → 3.7

1368 Minimum Cost to Make at Least One Valid Path in a Grid → 9.11
1372 Longest ZigZag Path in a Binary Tree → 8.4
1381 Design a Stack With Increment Operation → 4.6
1392 Longest Happy Prefix → 11.2
1395 Count Number of Teams → 3.11
1409 Queries on a Permutation With Key → 13.2
1425 Constrained Subsequence Sum → 2.10
1434 Number of Ways to Wear Different Hats to Each Other → 10.12
1438 Longest Continuous Subarray With Absolute Diff Less Than or Equal to Limit → 2.10, 13.1
1443 Minimum Time to Collect All Apples in a Tree → 8.9
1449 Form Largest Integer With Digits That Add up to Target → 10.5
1456 Maximum Number of Vowels in a Substring of Given Length → 2.6
1462 Course Schedule IV → 9.13
1466 Reorder Routes to Make All Paths Lead to the City Zero → 8.9
1472 Design Browser History → 5.5
1475 Final Prices With a Special Discount in a Shop → 4.3
1478 Allocate Mailboxes → 10.14
1480 Running Sum of 1d Array → 1.3
1482 Minimum Number of Days to Make m Bouquets → 3.3
1483 Kth Ancestor of a Tree Node → 8.10
1489 Find Critical and Pseudo-Critical Edges in Minimum Spanning Tree → 9.14
1504 Count Submatrices With All Ones → 4.4
1514 Path with Maximum Probability → 9.10
1515 Best Position for a Service Centre → 3.4
1519 Number of Nodes in the Sub-Tree With the Same Label → 8.9
1541 Minimum Insertions to Balance a Parentheses String → 6.6
1547 Minimum Cost to Cut a Stick → 10.10
1552 Magnetic Force Between Two Balls → 3.3
1559 Detect Cycles in 2D Grid → 9.6
1568 Minimum Number of Days to Disconnect Island → 9.16
1579 Remove Max Number of Edges to Keep Graph Fully Traversable → 9.14
1584 Min Cost to Connect All Points → 9.14
1590 Make Sum Divisible by P → 1.4
1593 Split a String Into the Max Number of Unique Substrings → 7.7
1631 Path With Minimum Effort → 9.10
1636 Sort Array by Increasing Frequency → 3.7
1649 Create Sorted Array through Instructions → 13.2
1657 Determine if Two Strings Are Close → 1.1
1671 Minimum Number of Removals to Make Mountain Array → 10.6
1696 Jump Game VI → 2.10
1697 Checking Existence of Edge Length Limited Paths → 9.14
1707 Maximum XOR With an Element From Array → 12.3
1720 Decode XORed Array → 12.2
1721 Swapping Nodes in a Linked List → 5.4
1745 Palindrome Partitioning IV → 10.9, 11.6
1749 Maximum Absolute Sum of Any Subarray → 1.7
1755 Closest Subsequence Sum → 7.8
1765 Map of Highest Peak → 9.3
1793 Maximum Score of a Good Subarray → 4.4
1803 Count Pairs With XOR in a Range → 12.3
1824 Minimum Sideway Jumps → 9.11
1825 Finding MK Average → 4.8
1854 Maximum Population Year → 1.6, 6.4
1856 Maximum Subarray Min-Product → 4.4
1857 Largest Color Value in a Directed Graph → 9.8
1863 Sum of All Subset XOR Totals → 7.1
1879 Minimum XOR Sum of Two Arrays → 10.12
1886 Determine Whether Matrix Can Be Obtained By Rotation → 1.10
1901 Find a Peak Element II → 3.5
1910 Remove All Occurrences of a Substring → 11.2
1911 Maximum Alternating Subsequence Sum → 10.11
1922 Count Good Numbers → 12.5
1923 Longest Common Subpath → 11.4
1926 Nearest Exit from Entrance in Maze → 9.2
1928 Minimum Cost to Reach Destination in Time → 9.12
1960 Maximum Product of the Length of Two Palindromic Substrings → 11.6
1963 Minimum Number of Swaps to Make the String Balanced → 6.6
1969 Minimum Non-Zero Product of the Array Elements → 12.5
1971 Find if Path Exists in Graph → 9.1
1976 Number of Ways to Arrive at Destination → 9.10
1979 Find Greatest Common Divisor of Array → 12.4
1985 Find the Kth Largest Integer in the Array → 3.9
1986 Minimum Number of Work Sessions to Finish the Tasks → 10.12
2009 Minimum Number of Operations to Make Array Continuous → 3.11
2034 Stock Price Fluctuation → 13.5
2035 Partition Array Into Two Arrays to Minimize Sum Difference → 7.8
2044 Count Number of Maximum Bitwise-OR Subsets → 7.1
2049 Count Nodes With the Highest Score → 8.9
2050 Parallel Courses III → 9.8
2062 Count Vowel Substrings of a String → 2.8
2074 Reverse Nodes in Even Length Groups → 5.2
2096 Step-By-Step Directions From a Binary Tree Node to Another → 8.7
2097 Valid Arrangement of Pairs → 9.18
2099 Find Subsequence of Length K With the Largest Sum → 4.7
2100 Find Good Days to Rob the Bank → 6.7
2102 Sequentially Ordinal Rank Tracker → 4.8
2104 Sum of Subarray Ranges → 4.5
2115 Find All Possible Recipes from Given Supplies → 9.7
2127 Maximum Employees to Be Invited to a Meeting → 9.17
2140 Solving Questions With Brainpower → 10.2
2179 Count Good Triplets in an Array → 13.2
2192 All Ancestors of a Node in a Directed Acyclic Graph → 9.7
2203 Minimum Weighted Subgraph With the Required Paths → 9.10
2223 Sum of Scores of Built Strings → 11.3
2246 Longest Path With Different Adjacent Characters → 8.4
2256 Minimum Average Difference → 1.3
2265 Count Nodes Equal to Average of Subtree → 8.3
2276 Count Integers in Intervals → 6.2, 13.5
2281 Sum of Total Strength of Wizards → 4.5
2286 Booking Concert Tickets in Groups → 13.3
2290 Minimum Obstacle Removal to Reach Corner → 9.11
2322 Minimum Score After Removals on a Tree → 8.13
2328 Number of Increasing Paths in a Grid → 9.8
2338 Count the Number of Ideal Arrays → 12.7
2360 Longest Cycle in a Graph → 9.17
2379 Minimum Recolors to Get K Consecutive Black Blocks → 2.6
2381 Shifting Letters II → 1.6

2392 Build a Matrix With Conditions → **9.7**
2398 Maximum Number of Robots Within Budget → **13.1**
2400 Number of Ways to Reach a Position After Exactly k Steps → **12.7**
2406 Divide Intervals Into Minimum Number of Groups → **6.4**
2407 Longest Increasing Subsequence II → **13.3**
2411 Smallest Subarrays With Maximum Bitwise OR → **13.1**
2416 Sum of Prefix Scores of Strings → **11.5**
2426 Number of Pairs Satisfying Inequality → **3.11**
2448 Minimum Cost to Make Array Equal → **3.4**
2472 Maximum Number of Non-overlapping Palindrome Substrings → **11.1**
2477 Minimum Fuel Cost to Report to the Capital → **8.9**
2493 Divide Nodes Into the Maximum Number of Groups → **9.5**
2503 Maximum Number of Points From Grid Queries → **9.15**
2514 Count Anagrams → **12.7**
2521 Distinct Prime Factors of Product of Array → **12.6**
2523 Closest Prime Numbers in Range → **12.6**
2536 Increment Submatrices by One → **1.6**
2538 Difference Between Maximum and Minimum Price Sum → **8.12**
2550 Count Collisions of Monkeys on a Polygon → **12.5**
2569 Handling Sum Queries After Update → **13.4**
2581 Count Number of Possible Root Nodes → **8.12**
2597 The Number of Beautiful Subsets → **7.1**
2646 Minimize the Total Price of the Trips → **8.11**
2719 Count of Integers → **10.13**
2761 Prime Pairs With Target Sum → **12.6**
2780 Minimum Index of a Valid Split → **1.8**
2836 Maximize Value of Function in a Ball Passing Game → **8.10**
2846 Minimum Edge Weight Equilibrium Queries in a Tree → **8.10**
2848 Points That Intersect With Cars → **6.1**
2858 Minimum Edge Reversals So Every Node Is Reachable → **8.12**
2872 Maximum Number of K-Divisible Components → **8.9**
2876 Count Visited Nodes in a Directed Graph → **9.17**
2925 Maximum Score After Applying Operations on a Tree → **8.11**
2932 Maximum Strong Pair XOR I → **12.3**
2935 Maximum Strong Pair XOR II → **12.3**
2959 Number of Possible Sets of Closing Branches → **9.13**
2976 Minimum Cost to Convert String I → **9.13**
3008 Find Beautiful Indices in the Given Array II → **11.3**
3015 Count the Number of Houses at a Certain Distance I → **9.13**
3029 Minimum Time to Revert Word to Initial State I → **11.3**
3031 Minimum Time to Revert Word to Initial State II → **11.3**
3070 Count Submatrices with Top-Left Element and Sum Less Than k → **1.5**
3097 Shortest Subarray With OR at Least K II → **2.9**
3165 Maximum Sum of Subsequence With Non-adjacent Elements → **13.3**
3169 Count Days Without Meetings → **6.1**
3171 Find Subarray With Bitwise OR Closest to K → **13.1**
3286 Find a Safe Walk Through a Grid → **9.11**
3303 Find the Occurrence of First Almost Equal Substring → **11.3**
3306 Count of Substrings Containing Every Vowel and K Consonants II → **2.8**
3327 Check if DFS Strings Are Palindromes → **8.13**, **11.6**
3337 Total Characters in String After Transformations II → **12.8**
3373 Maximize the Number of Target Nodes After Connecting Trees II → **9.5**
3419 Minimize the Maximum Edge Weight of Graph → **9.15**
3453 Separate Squares I → **3.4**
3479 Fruits Into Baskets III → **13.3**
3515 Shortest Path in a Weighted Tree → **8.13**

♥ Made for Purvam Prajapati · 125 patterns · 337 Java templates compiled with javac --release 8 and tested · one window a day ♥